Privacy at your Hands: Efficient Privacy-Enhancing Technologies for the Internet of Things

PROEFSCHRIFT

ter verkrijging van de graad van doctor aan de
Technische Universiteit Eindhoven, op gezag van
de rector magnificus prof. dr. S.K. Lenaerts,
voor een commissie aangewezen door het College
voor Promoties, in het openbaar te verdedigen op
donderdag 30 oktober 2025 om 13:30 uur

door

Dominik Roy George

geboren te Wenen, Oostenrijk

Dit proefschrift is goedgekeurd door de promotoren en de samenstelling van de promotiecommissie is als volgt:

Voorzitter:	prof. dr. B. van Dongen
Promotoren:	prof. dr. S. Etalle
	dr. N. Zannone
Copromotor:	dr. S. Sciancalepore
Leden:	prof. dr. ir. N. Meratnia
	dr. G. Oligeri (Hamad Bin Khalifa University, Doha)
	prof. dr. G. Boggia (Polytechnic University of Bari)

Het onderzoek of ontwerp dat in dit proefschrift wordt beschreven is uitgevoerd in overeenstemming met de TU/e Gedragscode Wetenschapsbeoefening.

INTERSCT.

The work in this dissertation is partially supported by the INTERSCT project, Grant No. NWA.1162.18.301, funded by Nederlandse Organisatie voor Wetenschappelijk Onderzoek (NWO)

Printed by: ADC Nederland - 's-Hertogenbosch

Cover design: Dominik Roy George

A catalogue record is available from the Eindhoven University of Technology Library.
ISBN: 978-90-386-6472-9

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

An electronic version of this dissertation is available at `http://repository.tue.nl` and `http://dominikroy.github.io/`

Dedicated to **വല്യപ്പച്ചൻ**
(Konnackal Ouseph Varkey, grandfather).
His pursuit of knowledge was never-ending. At the age of 84 he began his fourth master's degree in Psychology and at 87, he pursued a PhD in Theology. He was unable to complete these studies, as he passed away at the age of 89 on 24 October 2017.
This work honors his memory and continues his legacy.

Summary

The Internet of Things (IoT) is a rapidly growing ecosystem that encompasses often constrained IoT devices, featuring limited processing power, storage resources, and low energy consumption, allowing them to be deployed for specific use cases, e.g., to automate tasks or collect data. At the same time, fostering constrained devices in different environments may require to execute real-time computations onboard, due to scarce Internet connection and no mobile coverage or real-time constraints, without the possibility to involve trusted third parties (TTPs). Such challenging scenarios require the IoT operator or the user to store sensitive information onboard the IoT devices, e.g., data associated with Intellectual Property (IP) rights or Personal Identification Information (PII). This leads to major security and privacy concerns both for users and Service Providers (SPs), due to the ease of accessing and hijacking IoT devices. An adversary could collect and use the PII to impersonate the user. Furthermore, regulations such as the General Data Protection Regulation (GDPR) and the upcoming Cyber Resilience Act (CRA) mandate that SPs keep user data fully private, making SPs responsible for any unauthorized disclosure.

Privacy-Enhancing Technologies (PETs) are a valuable tool to address such privacy concerns in traditional IT domains. PETs include a wide range of techniques that allow to achieve input privacy, privacy-preserving computation, and output privacy. However, PETs introduce additional computational burden caused by the underlying cryptographic techniques. This overhead is often manageable in the traditional IT domain, while such a burden represents a problem when integrated in the IoT domain. To address privacy concerns in the IoT ecosystem and to avoid the computational overhead of PETs, IoT devices usually delegate complex computations to TTPs, e.g., the Cloud or powerful Edge nodes. However, in scenarios where TTPs are unavailable, it is necessary to understand how to re-design and engineer PETs to enable their usage directly on IoT devices. Note that integrating PETs on IoT devices comes with different challenges and trade-offs, depending on: (i) the environment where IoT devices are deployed, e.g., locations with scarce Internet connection, (ii) the type of data they access, e.g., real-time data evaluation, and (iii) the limited available resources, e.g., battery life. The mentioned challenges lead to the following main research question:

How can we design and deploy Privacy-Enhancing Technologies effectively and efficiently so to be supported on constrained IoT devices and networks?

We answer the question by investigating relevant privacy concerns in real-world IoT use cases. First, we identify privacy concerns emerging when two closely-deployed IoT networks use the same communication technology and working on the same spectrum at the same time leads to interferences, packet losses, and Quality of Service degradation. However, uncontrolled disclosure of the internal channel usage could lead to confidentiality issues. To solve this problem, we design a protocol that allows two networks to discover interfering

operations without revealing any internal confidential details. Another IoT-based application scenario we consider is the one of Autonomous Vehicles (AVs). To guarantee the safe deployment of AVs in daily life, we need to detect in advance mutual occurring collisions in their path. However, identifying collisions requires the exchange of the mutual trajectories, potentially leaking private path information, e.g., the location of storage sites, and may also reveal private users' data. To address the problem, we propose a fully-accurate interactive protocol allowing privacy-preserving trajectory matching among AVs. Another problem we address is the secure attestation for IoT devices that run multiple software for various tasks onboard. Therefore, integrating the Trusted Platform Modules on IoT devices allows to remotely attest the authenticity and integrity of all such programs. Enabling traditional binary remote attestation reveals all the running processes on the system and raises privacy and intellectual property concerns. To address this problem, we propose a remote attestation method with constrained disclosure. Another problem we consider is the shared usage of emerging technologies such as Unmanned Aerial Vehicles (UAVs), a.k.a. drones. Drone-as-a-Service (DaaS) is an increasingly adopted business model, which enables possibly unskilled users with no background knowledge to operate drones and run automated drone-based tasks. Therefore, the resources provided by drones are typically managed by multiple parties, which requires the integration of multi-party access control solutions. In this context, the leakage of the access control policies might disclose confidential information. To address this privacy problem, we propose a privacy-preserving multi-party access control solution tailored to the application scenarios of Third-Party UAV Services. Finally, another real-world IoT-based application we consider is the deployment of drones for goods delivery and humanitarian operations in hard-to-reach areas characterized by scarce or absent Internet connection. Such drones use state-of-the-art biometric face verification techniques to hand out goods securely. However, adversaries active in the field could capture the drone and access all the information stored therein, including sensitive biometric information of the goods' recipient. To address the concern, we propose Obscura, the first solution for privacy-preserving face verification on commercial drones. Overall, this thesis provides manifold contribution: (i) we identify many real-world IoT-related use cases generating privacy concerns; (ii) we identify privacy-enhancing technologies suitable for addressing these challenges; (iii) we design new algorithms and strategies to integrate such PETs on real-world constrained IoT devices while dealing effectively and efficiently with the constraints of the IoT ecosystem; and finally, (iv) we extensively evaluate such solutions on relevant real-world IoT devices to showcase their effectiveness and efficiency.

Contents

1

Introduction

1.1. Scope and Motivation

1.1.1. Context

The concept of IoT was introduced for the first time in 1999 by Kevin Ashton, to describe how "things connecting to the internet" could impact our lives [15]. This paradigm revolutionized academia and industry. It all started by connecting simple Radio-Frequency Identification (RFID) tags, sensors, and actuators based on unique identifiers to communicate with each other to accomplish a common task [16] and since then, over the years, the coverage of the *Internet of Things* paradigm has expanded significantly to cover new emerging domains, e.g., health care, smart cities, and cyber-physical systems, to name a few [29, 123, 152, 245]. We use *Internet of Things* devices extensively in our day-to-day lives, e.g., through smart accessories. Smartwatches may collect heart rate data, smart doorbells provide a video feed, a smart central thermostat regulates the temperature and controls automatic blinds in a preprogrammed way. This showcases how IoT devices collect information and allow us to automate tasks. According to Grand View Research [99] the market size for IoT in 2024 was estimated to be USD 1.39 billion, and is projected to reach USD 2.65 billion by 2030. Moreover, based on the report in [197], by 2030, there will be 40 billion connected IoT devices.

Today, *Internet of Things* devices and technologies are very heterogeneous. Most of the scientific literature considers IoT devices to be resource-constrained, characterized by very limited computational, bandwidth, memory, and energy availability [16]. Such devices are typically connected to a gateway that collects all the sensed data and interacts with the devices to let them perform actuation tasks [6, 91]. However, new emerging technologies such as Autonomous Vehicles (AVs) and Unmanned Aerial Vehicles (UAVs) are today considered part of the IoT ecosystem. Although AVs and UAVs are as computationally powerful as full-fledged laptops, they still feature limited battery life. Since performing more computationally intensive tasks requires more energy, they still need to calibrate the number of computations performed against the available energy budget [46, 54, 113, 203].

Due to the nature of the data collected as part of their intended tasks, IoT devices typically

collect sensitive private information, e.g., data about the user's health, routines, or have access to important configuration parameters of the network they belong to. This information may contain details regarding Personal Identifiable Information (PII) or data associated with Intellectual Property (IP) rights. Although data communication to the intended receiver could be protected through the establishment of secure channels, this is not enough to fully protect information against unauthorized disclosure from legitimate parties or other form of data leakage, e.g., via physical access to the devices [10, 246].

Overall, operating on sensitive information raises privacy concerns for both users and Service Providers (SPs). Users face the threat of an adversary that could collect and use the PII in an unauthorized way, e.g., to infer private data or impersonate them. Looking at SPs, consider that recent regulations such as the General Data Protection Regulation (GDPR) [230] and the upcoming Cyber Resilience Act (CRA) [83] mandate SPs to keep the data provided by the user fully private. Thus, in many scenarios, SPs could be held responsible for any disclosure of such data to unauthorized parties [77].

Therefore, both users and SPs should implement effective measures to fully protect sensitive information from unauthorized use by any party involved in the system.

1.1.2. Privacy-Enhancing Technologies

In the context of traditional IT applications, such privacy concerns are usually addressed successfully through the use of Privacy-Enhancing Technologies (PETs). The concept of PETs was coined in 1995 in reports of the Information and Privacy Commissioner of Ontario and the Dutch Data Protection Authority [48, 108]. In the cryptography community, David L. Chaum paved the way for the first PET-related application, which was published in 1981 [48, 52], by introducing a scheme for email systems to hide the identity of the participant communicating [52]. The scheme achieved anonymity while allowing for observability of the corresponding network traffic [48, 52], and this idea is still used in various domains, albeit in adapted forms. Since then, various PETs have been introduced to achieve two main classes of privacy properties, i.e., input privacy and output privacy. *Input Privacy* ensures that no computing parties can learn or derive input data during the computation from any involved party, including other parties providing input [12, 221]. Techniques that guarantee input privacy include Multi-Party Computation (MPC), Homomorphic Encryption (HE), and Zero-Knowledge Proof (ZKP) [12, 221]. *Output Privacy* ensures that intermediate information and results of the computations do not reveal any input or identifiable data. It also protects from intermediate computations [12, 221]. For example, Differential Privacy (DP) preserves output privacy for statistical results or data aggregation, preventing disclosure of identifiable input data [12, 221]. We summarize below the main PETs.

Trusted Execution Environment Trusted Execution Environments (TEEs) are an isolated and secure area within the memory of a host device or designated area in the main processor, known as *enclave* [91]. TEEs allow the execution of sensitive operations in a secure portion of the host system, while guaranteeing confidentiality and integrity of the code and data in the presence of an adversary or a distrustful host operating system. To this aim, TEE provisions

privacy by providing safe storage and safe code execution, which is already integrated with various chip manufacturers, e.g., Intel SGX or ARM Trustzones [170].

Anonymization and Synthetic Data Anonymization techniques remove identifiable information or attributes from the dataset, so preventing individuals' identification from the (anonymized) dataset [180]. This approach is vulnerable to re-identification attacks through cross-referencing datasets. An alternative technique involves using synthetic data, which generates randomly sampled data that has a distribution similar to the actual dataset. The main challenge is retaining the statistical properties while ensuring its privacy. Meanwhile, the work by Stadler et al. [200] shows that anonymization-based schemes and generative data still leak information, compromising the utility of the data. A promising alternative technique for concealing or masking datasets is Differential Privacy.

Differential Privacy Differential Privacy is a technique that adds noise to the data to prevent identification of the underlying individual record of the dataset while allowing to retrieve meaningful statistical evaluation results [76]. The hardness of Differential Privacy lies in the right choice of the level of noise added to the original dataset while retaining meaningful results. Therefore, this depends on the dataset itself.

Homomorphic Encryption HE is an encryption scheme that allows to performs multiplicative and additive operations on encrypted data (ciphertext). After decryption, we obtain the same results as if we performed the same operations on the underlying plaintext [180]. The key benefit is that we can encrypt the input and forward the ciphertext to another party to perform the operations over the ciphertext. Afterward, the output of the performed operations can be sent to another party or the same party that holds the secret key to understand the result. Meanwhile, the third party who performed the operations does not know the input nor the output. There are three types of HE schemes: (i) Fully Homomorphic encryption, (ii) Partial Homomorphic encryption, and (iii) Leveled Homomorphic encryption.

Fully Homomorphic Encryption (FHE) allows an unlimited number of unrestricted multiplicative and additive operations on encrypted data. This makes it the most powerful form of HE, enabling general-purpose computation without the need for any loss of privacy. However, it comes at a significant computational cost, often making it impractical for many real-world applications. Partial Homomorphic Encryption (PHE) refers to a set of encrypted computation schemes that support only a single type of operation, either addition or multiplication operations, but not both. In the very limited use cases where PHE schemes are applicable, they can be quite efficient. As a middle ground, Leveled Homomorphic Encryption (LHE) enables both addition and multiplication operations on encrypted data, but only up to a predefined multiplicative depth. Beyond this depth, the accumulated noise in the ciphertexts becomes unmanageable. LHE offers more computational flexibility than PHE, while being more efficient than FHE for tasks with known and limited depth of operations.

Secure Multi-Party Computation MPC schemes allow several mutually distrusting parties to jointly compute a function over their private inputs without disclosing their inputs [206]. MPC can be realized with various constructions, e.g., Oblivious Transfer (OT) [174], HE,

garbled circuits [241], or additive secret sharing [163]. Three example schemes are: (i) Private Set Intersection, (ii) Private Information Retrieval, and (iii) Secure Function Evaluation. Private Set Intersection allows set operations between two parties without revealing the non-intersecting elements of their sets. Private Information Retrieval enables querying a database without the database knowing what has been queried or the result of the query. Secure Function Evaluation [97] permits generating circuits that represent the function, and through additive secret sharing or OT, each party can evaluate the circuit and reconstruct the result at the end.

Zero-Knowledge Proofs The concept behind ZKP is that a verifier can verify the authenticity and integrity of the data, as well as the computation over the data by a prover, without disclosing any information regarding the data [180]. ZKP systems are characterized by three properties [98]: (i) *completeness*, i.e., the prover is capable of convincing the verifier that the statement is correct, (ii) *soundness*, i.e., the prover is capable of convincing the verifier that the statement is incorrect, and (iii) *zero-knowledge*, i.e., the prover does not reveal any extra information from the data it wants to hide, except the correctness from the statement. ZKP schemes can be interactive or non-interactive. The non-interactive characteristics allow sending the statement/proof to multiple parties [91].

1.1.3. Research Gap

Despite the privacy benefits and properties discussed in the earlier subsection, PETs also introduce significant computational burden, mainly caused by the underlying cryptographic techniques. Although such a burden typically does not significantly affect the performance of services running over traditional IT networks, it represents a problem when integrated in the IoT domain, where resources are scarce.

Looking at the scientific literature, a few scientific contributions considered integrating PETs in the IoT ecosystem. However, in most of these proposals, IoT devices delegate complex computations to the Cloud or to more powerful Edge nodes [48, 91, 142].

As an example, Bhati et al.[28] recently designed a scheme for an IoT-to-cloud setting for the smart-metering scenario. They assume that the resource-constrained devices record sensitive information, which may expose users' household habits. To address the privacy concern, they designed a lightweight encryption scheme for the IoT device and a distributed decryption protocol via MPC for the Cloud servers. The Cloud servers receive only a secret share of the plaintext, where the information is further processed while preserving the privacy of the data. The final result can only be reconstructed by an authorized party.

At the same time, we observe that many IoT devices, especially in the context of new emerging technologies, operate with a scarce or absent connection to the Internet, or may want to avoid exchanging data with entities available over the public Internet for various reasons. For example, UAVs and AVs often operate in remote and hard-to-reach areas, e.g., deserts, open seas and areas subject to natural disasters, which are characterized by scarce or absent Internet connectivity [101, 145]. Moreover, we should also consider that interaction over the Internet is not always desired by such devices. Many operations, e.g., collision detection,

face recognition, and access control, need to happen possibly in real time, i.e., with minimal latency. By design, the public Internet is best-effort, i.e., it guarantees that packets are delivered to the destination but it does not provide any guarantees in terms of Quality of Service (QoS), including latency and delay [192, 201]. Therefore, relying on the public Internet for real-time services is not desirable, and operations should be executed locally as much as possible.

In this thesis, we identify several privacy concerns related to real-world IoT use cases:

1. We identify privacy concerns emerging when two closely-deployed IoT networks use the same communication technology. Having two networks working on the same spectrum at the same time leads to interferences, packet losses, and Quality of Service degradation. Such situation could be solved by exchanging information, e.g., how many devices are in the network and at what time on which time channel they are communicating. However, the uncontrolled disclosure of such information could lead to confidentiality issues and, in turn, enable targeted attacks.

2. We identify privacy issues connected with the identification of collisions in the path of two Autonomous Vehicles (AVs). To guarantee the safe deployment of AVs in daily life, we need to detect in advance mutual occurring collisions in their path. However, identifying collisions requires the exchange of the mutual trajectories, potentially leaking private path information, e.g., the location of storage sites, and may also reveal private users' data.

3. We identify privacy issues arising when remotely attesting the trustworthiness of proprietary software running onboard IoT devices. Integrating the Trusted Platform Module on IoT devices allows us to remotely attest the authenticity and integrity of all running software. However, deploying traditional binary remote attestation protocols may reveal metadata of all running software, allowing a passive adversary to identify software vulnerabilities based on the metadata to deploy tailored attacks.

4. We identify privacy issues connected with the deployment of multi-party access control models on drones used by several parties, according to the Drone-as-a-Service (DaaS) paradigm. Although these services provide significant advantages, the resources provided by drones are typically owned by multiple parties, requiring the integration of multi-party access control solutions. In this context, the leakage of the access control policies specified by the data owners might disclose confidential information.

5. Finally, we identify privacy issues connected with the adoption of face recognition on drones used for delivering goods and humanitarian operations in hard-to-reach areas characterized by scarce or absent Internet connection. Such drones use state-of-the-art biometric verification techniques to hand out goods securely, e.g., by verifying the face of the recipient of the goods. However, adversaries active in the field could capture the drone and access all the information stored therein, including sensitive biometric information of the goods' recipient.

Use cases 1 and 2 highlight that two IoT devices must identify matches among locally stored data records. Exchanging data records in plaintext reveals sensitive information, e.g., path information or configurations. Private Set Intersection (PSI) (see Sec. 1.1.2) is a technique

used in many other IT-related use cases to address similar privacy concerns. However, off-the-shelf PSI variants seldom consider the challenges that arise within the IoT domain for real-time data evaluation.

As stated in use case 3, traditional remote attestation discloses the measurements of all running software. The challenge is to design an architecture that still preserves the principles of traditional remote attestation while only selectively disclosing measurements and maintaining confidentiality.

Use cases 4 and 5 show that IoT devices might require computing a decision over their private inputs. Secure Function Evaluation (SFE) or HE are techniques guaranteeing input privacy and addressing this need. However, such techniques come with communication and computation overhead, so being not suitable for IoT devices. Thus, it is necessary to investigate optimization strategies within the techniques and how they can be tailored to IoT devices with limited available resources, e.g., battery life.

Overall, integrating PETs on IoT devices comes with different challenges and trade-offs as mentioned before. In many of the aforementioned use cases, IoT devices operate in environments with scarce Internet connections, reducing the possibility of offloading computationally heavy tasks to Trusted Third Parties (TTPs). Therefore, it is necessary to investigate how to compute locally privacy-aware operations without relying on TTPs, while considering the IoT constraints.

The aforementioned privacy issues demonstrate the necessity of deploying PETs on IoT devices while striking a balance between energy consumption and privacy.

1.2. Research Question

The above considerations lead to our main research question:

Main Research Question (MRQ)
How can we design and deploy Privacy-Enhancing Technologies effectively and efficiently so to be supported on constrained IoT devices and networks?

We can further refine such overarching research questions into several sub-questions.

One first major privacy concern we identify is related to IoT devices which need to exchange sensitive information for various purposes, i.e., increasing safety or providing real-time services. Examples include sharing of information about channel access in constrained IoT devices, e.g., to avoid interferences, and sharing of trajectories, to avoid collisions. Therefore, it is necessary to identify solutions that enable finding matches among the sensitive input information while guaranteeing data privacy, leading to the following research question:

SRQ1: *How can we effectively and efficiently identify matches in private input data stored onboard two IoT devices?*

We address the research question above by investigating the mentioned IoT use cases and

devising methods to let IoT devices bilaterally identify matches in their internally stored private inputs, with no cooperation from external entities at runtime. We investigate which PET is applicable, allowing us to disclose only the identified matches in the private input. Apart from understanding the applicable PET, we design auxiliary algorithms which allow IoT devices to efficiently and effectively state if any matches exist.

We also consider that IoT devices often run proprietary software, subject to Intellectual Property (IP) rights. To verify that the device is trustworthy enough to host and run such proprietary software, remote attestation protocols are normally used. However, traditional remote attestation protocols leak confidential information about the running proprietary software, introducing a confidentiality problem. This leads to the following research question:

SRQ2: *How can we remotely attest the trustworthiness of the proprietary software running on IoT devices while preserving its confidentiality?*

To address the question, we investigate the privacy and confidentiality issues of the traditional binary remote attestation. We design an architecture that allows for the selective disclosure of masked measurements. This allows any verifier who verifies the attesting system only to understand the associated measurements. Simultaneously, the verifier can verify the entire chain of trust of all the measurements without understanding the underlying proprietary software.

Another angle we consider is that PETs should run on somehow constrained devices, with less computational and energy availability than traditional IT devices. This leads to the following research question:

SRQ3: *Which design strategies allow reducing the computational overhead while guaranteeing input privacy in IoT devices?*

To address this question, we investigate various protocol design strategies, e.g., using predeployed data to reduce computational overhead while still guaranteeing input privacy.

Finally, we should also consider the networking characteristics of the IoT, e.g., the availability of scarce or even absent Internet connectivity. Thus, it is hard to involve TTPs or any cloud-assisted system in the protocols. This leads to the following research question:

SRQ4: *How can we realize privacy-aware operations on IoT devices without relying on trusted third parties?*

To address this research question, we redesign and engineer several PETs to preserve the privacy of input data without relying on a TTP. However, by doing so, we confront other consequences, e.g., no computational outsourcing capabilities, local computational and communication overhead, and increased energy consumption. Mitigating the impact of such aspects is not trivial, and will be addressed in this thesis through specific application-based design choices.

1

1.3. Thesis Outline and Contributions

This thesis provides answers to the main research question and sub-questions through three main parts:

1. Part I designs and formalizes privacy-preserving protocols to identify matches in private input. This part answers **SRQ1** and **SRQ4**.

2. Part II designs and evaluates a solution for remotely attesting IoT devices while disclosing only necessary information. This part answers **SRQ2**.

3. Part III investigates 2-Party Computation in the context of emerging IoT technologies and addresses the research questions: **SRQ3** and **SRQ4**.

In chapter 2, we propose PRM (an acronym for Private RST Matching), a novel protocol enabling private interference discovery among IEEE 802.15.4 networks to answer the research questions **SRQ1** and **SRQ4**. PRM aims to allow two nodes, e.g., the gateways of two IoT networks, to identify which radio cells are used by both at the same time, without sharing to the other party the whole Radio Scheduling Table (RST). Identifying such cells and resolving the conflict beforehand helps reduce interference, while improving reliability and preserving the secrecy of the radio resources assignment. The underlying PET of PRM is PSI, and we design an auxiliary algorithm called the *Iterative Set Division* strategy to reduce significantly the computational overhead on constrained devices. We also evaluate the performance assessment of PRM on heterogeneous IoT devices.

In Chapter 3, we contribute to answer the research questions **SRQ1** and **SRQ4** by proposing ePPTM, an enhanced interactive protocol for private spatiotemporal trajectory matching among Autonomous Vehicles, i.e., detection of collisions without revealing to the remote party anything else than the colliding points and timestamps. To this aim, ePPTM integrates two main building blocks, i.e., the *Incremental Capsule Matching* algorithm, analyzing co-location between trajectories at an increasing level of granularity, and *privacy-preserving proximity testing*, allowing comparison among private sets without revealing anything more than colliding elements. We also implemented a proof-of-concept and deployed ePPTM on two testbeds characterized by heterogeneous processing capabilities and tested its accuracy and runtime overhead using actual vehicles' trajectories from the city of Porto, Portugal, available at [156]. Our results demonstrate the perfect accuracy of ePPTM (100% collision detection) and the capability of running on relevant devices with reasonable overhead.

In Chapter 4, to answer **SRQ2**, we design a scheme called Remote Attestation with Constrained Disclosure (RACD), enabling traditional binary remote attestation to mask and selectively disclose event log entries. We achieve this by extending the Linux Integrity Measurement Architecture (IMA) concept and adding a non-interactive zero-knowledge (NIZK) proof based on Schnorr signatures. Our RACD approach complies with the third principle of remote attestation—constrained disclosure, which states that a target should be able to enforce policies governing which measurements are sent to each verifier. We also implement a proof-of-concept of RACD and demonstrate the feasibility of running our solution on relevant devices with reasonable overhead through performance measurements.

Chapter 5 addresses the research questions **SRQ3** and **SRQ4** by investigating the feasibility of privacy-preserving multi-party access control schemes for Third-Party UAV Services. Our scheme builds on top of an existing approach based on the SFE paradigm [195], further adapting its architecture to meet the constraints and requirements of the Drone-as-a-Service (DaaS) scenario. Through the proposed scheme, drones can evaluate complex multi-party policies without relying on a persistent Internet connection and without inferring user policies specified by data owners. We deployed a proof-of-concept of the proposed scheme and performed an extensive experimental assessment to evaluate the performance of our approach w.r.t. several key metrics for constrained devices. The results show that our scheme can support privacy-preserving multi-party policy evaluation on a processing-constrained device in realistic configurations while requiring a very limited energy toll.

Chapter 6 tackles the research questions **SRQ3** and **SRQ4** by introducing Obscura, a novel protocol for privacy-preserving face verification on constrained drones. In a nutshell, Obscura allows a drone, potentially deployed in areas with no Internet connectivity, to verify the identity of a person without storing locally any PII, i.e., the facial features provided by such person to the SP. We design Obscura by smartly combining the MPC paradigm with the latest additive secret sharing techniques. Moreover, we provide an extensive performance evaluation of Obscura on a proof-of-concept deployed over two relevant testbeds. The results promise significant savings in terms of execution time and energy consumption.

In Chapter 7, we draw attention to the limitations of our solutions and provide prospects for addressing them while opening new avenues for future research. In particular, we focus on the details of our privacy-preserving protocols underlying the privacy-enhancement technique. Moreover, we explore possible ways to update our schemes to be quantum-safe. Finally, we propose investigating the direction of integrating multiple privacy-enhancing techniques while ensuring they are computationally efficient and addressing privacy concerns.

Finally, Chapter 8 provides answers to the research questions and concludes this work.

1.4. Publications

Our research has led to valuable findings that have been presented in both conference and journal publications, thus contributing to address the identified research gap:

1. **Dominik Roy George**, Savio Sciancalepore: *"Obscura: Privacy-Preserving Face Verification on Constrained Drones"*, Submitted and Under Review.

2. **Dominik Roy George**, Savio Sciancalepore: *"ePPTM-Enhanced Privacy-Preserving Trajectory Matching on Autonomous Vehicles."* IEEE Internet of Things Journal (IoT-J), 2025.

3. Michael Eckel, **Dominik Roy George**, Björn Grohmann, Christoph Krauß: *"Remote Attestation with Constrained Disclosure"*. Annual Computer Security Applications Conference (ACSAC), 2023, pp. 718–731, Austin, TX, USA.

4. **Dominik Roy George**, Savio Sciancalepore, Nicola Zannone: *"Privacy-Preserving Multi-Party Access Control for Third-Party UAV Services"*. ACM Symposium on Access Control Models and Technologies (SACMAT), 2023, pp. 19–30, Trento, Italy.

5. Savio Sciancalepore, **Dominik Roy George**: *"Privacy-Preserving Trajectory Matching on Autonomous Unmanned Aerial Vehicles"*. Annual Computer Security Applications Conference (ACSAC), 2022, pp. 1–12, Austin, TX, USA.

6. **Dominik Roy George**, Savio Sciancalepore: *"PRM - Private Interference Discovery for IEEE 802.15.4 Networks"*. I IEEE Conference on Communications and Network Security (CNS), 2022, pp. 136-144, Austin, TX, USA.

I

Designing Privacy-Preserving Protocols for Mesh Networks and AVs

2

Private Interference Discovery for IEEE 802.15.4 Networks

This chapter is inspired by [93]:
D.R. George, S. Sciancalepore
PRM - Private Interference Discovery for IEEE 802.15.4 Networks
2022 IEEE Conference on Communications and Network Security (IEEE CNS)

In multi-tenant IoT deployments, networks managed by different administrators may share the same spectrum, possibly experiencing packet loss and degradation of the Quality of Service (QoS). As highlighted in Chapter 1.1.3, identifying interferences within the same spectrum raises confidentiality concerns, due to the sensitivity of information connected to the channels assigned to various devices. In this chapter, we address this issue by presenting PRM (Private RST Matching), the first scheme to identify potential interferences among multi-tenant wireless networks in advance, without exposing the entire scheduling information of each network to untrusted parties. This contributes to achieving *input privacy* and addresses **SRQ1** and **SRQ4**.

2.1. Introduction

IEEE 802.15.4 is emerging as the leading communication technology for constrained devices, thanks to the capability of minimizing energy consumption [148]. In a IEEE 802.15.4 network, the time is divided into time-slots, and the nodes communicate deterministically on a given frequency and time slot according to a RST, established through radio scheduling algorithms [107].

However, scheduling algorithms usually apply within IEEE 802.15.4 networks managed by the same network administrator. Due to the mobile and pervasive nature of modern IoT, often, networks managed by different administrators work on the same spectrum, at the same

time. For example, IEEE 802.15.4 networks installed within cars or on-board of drones may need to work in area where another IEEE 802.15.4 network is already deployed. Similarly, two static IoT networks may need to be deployed in proximity, while being managed by different administrators. In such cases, the application of traditional radio scheduling algorithms would require the sharing of the local Radio Scheduling Table (RST) to arrange an optimized communication schedule and prevent interferences, packet losses, and Quality of Service (QoS) degradation. However, sharing the RST might generate significant threats. Indeed, through the RST, a malicious adversary might gain knowledge of the number of devices, the used frequencies, their deployment, and the communication patterns, to name a few. Using such information, the adversary can capture the devices, or launch more advanced attacks, jeopardizing the service provided by the IoT network. Thus, RSTs should remain private, and an interference-free schedule has to be arranged without knowing the details of the scheduling of the remote IoT network.

To the best of our knowledge, no contributions in the scientific literature investigated security and privacy issues in radio scheduling algorithms for IEEE 802.15.4 networks. Indeed, all the solutions currently available focus on the optimization of dedicated metrics, such as the communication overhead [134], the energy consumption [124], and the throughput [129], neglecting security and privacy considerations. At the same time, deploying a solution for *private interference discovery* in IoT networks is challenging. Indeed, solutions for this problem should cope with the intrinsic limitations of IoT devices, in terms of processing capabilities, communication overhead, and most importantly, energy consumption. At the same time, they should discover only *shared* channels, while maintaining the privacy of the whole RST.

Contribution. In this chapter, we propose PRM (an acronym for Private RST Matching), a novel protocol enabling private interference discovery among IEEE 802.15.4 networks. PRM aims to allow two nodes, e.g., the gateways of two IoT networks, to identify which radio cells are used by both at the same time, without sharing to the other party the whole RST. Identifying such cells and resolving the conflict beforehand helps reduce interference, while improving reliability and preserving the secrecy of the radio resources assignment. To cope with the limitations of IoT devices, we design two modes of operation of the protocol, one based on the plain execution of PSI protocols and another one based on an innovative iterative set division strategy that significantly reduces the computational overhead of PSI on constrained devices. Besides providing a formal security analysis of PRM, we also carry out an experimental performance assessment on heterogeneous IoT devices, i.e., the Raspberry Pi and the ESPCopter. Our results show that PRM can always discover shared radio cells, reporting an execution time dependent on the mutual allocation of the occupied cells in the RST. Overall, while more powerful gateways (e.g., the Raspberry Pi) can handle a larger amount of occupied cells in less than 1 sec, more constrained devices (e.g., the ESPCopter) can take more time while still taking advantage of the iterative set division approach. To the best of our knowledge, PRM is the first solution in the literature to address and solve the problem of private interference discovery among IoT networks.

Roadmap. This chapter is organized as follows. Sec. 2.2 presents our scenario and adversary, Sec. 2.3 describes PRM, Sec. 2.4 provides the security analysis, Sec. 2.5 shows the performance of PRM, Sec. 2.6 wraps-up the discussion, and finally, Sec. 2.7 concludes the chapter

and outlines future work.

2.2. Scenario and Adversary Model

In this section, we describe the scenario (Sec. 2.2.1) and report the details of the adversary model (Sec. 2.2.2).

2.2.1. Scenario

In this work, we assume two independent wireless networks, namely A and B, which are using the IEEE 802.15.4 communication technology to exchange wireless messages between member nodes. As such, they use one or more of the 16 available channels in the bandwidth $[2.4 - 2.5]$ GHz, and one or more of the available N time-slots. IEEE 802.15.4 provides low-data-rate wireless communication between constrained devices [148]. To communicate, two nodes need to be synchronized, i.e., to have their radios on at the same frequency, at the same time. In IEEE 802.15.4, time is divided into time-slots, lasting $T = 10$ msec by default. Within a time-slot, based on its status, a node can transmit (receive) a data packet and receive (transmit) the corresponding acknowledgement, if required. We denote the combination of a specific time-slot and frequency as a *cell*, as in [148]. Overall, N time-slots and F frequencies are available, where $F = 16$ and, usually, $N = 101$, according to [3].

Each IEEE 802.15.4 network has a *Coordinator*, i.e., a specific node responsible for maintaining and regulating network connectivity among the devices. The coordinator is often also the *Gateway* of the IoT network, responsible for communicating with other IoT networks managed by a different network administrator, as well as the Internet. As such, the *Coordinator* is usually equipped with larger processing, storage, and energy capabilities than the constrained IoT devices, thus supporting the execution of symmetric and asymmetric cryptography techniques. Without loss of generality, we assume the coordinator of each network is in possession of the information regarding the cells used in the network, where at least one packet can be transmitted by any node. The table containing the information described above is the RST, and it can change periodically based on new cells assignment. It is worth noting that the specific radio scheduling algorithm used by the network is not relevant to our work, and it can be either centralized or distributed [107]. We only assume that the information about the possible usage of a cell is available to the coordinator. Also, we do not care if a specific cell is not always used (e.g., because of lack of traffic). If the IoT network plans to use a specific cell, then we consider it as *occupied* by a communication.

We assume B is a static network, installed in a dedicated location. For instance, B can be the monitoring network of a building, where IoT devices equipped with vibration sensors constantly monitor the stability of the overall construction. At the same time, A can be either a static or a mobile network. For instance, A could be a network installed on a transportation means, such as a car, or be another static network to be deployed in the same area of A.

Using the same bandwidth, when in the same reception radius, A and B are very likely to interfere with each other, causing packet losses and throughput reduction, with a consequent

degradation of the QoS offered to the end-users. A naive solution to this problem would be that the Coordinators of the two networks exchange the respective RST, to identify colliding time-slots and avoid interferences. However, disclosing the specific cells to be used by the IoT devices to communicate would represent a severe threat for the specific network. Indeed, as detailed in Sec. 2.2.2, the leakage of such information could enable easy disruption by a motivated adversary, e.g., enabling effort-less eavesdropping, traffic analysis, and jamming, to name a few. Therefore, we assume that each network would like to avoid interference with the neighboring one, while not revealing sensitive details about its operations, such as the channel allocation (i.e., the RST) and its logical topology. Our solution, namely PRM, is specifically intended to address and solve this problem (see Sec. 2.3).

2.2.2. Adversary Model

We assume an adversary, namely, ϵ, interested in becoming aware of the RST of a given party. The information contained in the RST can be used for multiple malicious purposes. For instance, by knowing the cells used by any two nodes to communicate, ϵ can stealthily listen to the communication as a passive eavesdropper, and acquire sensitive information. Assuming the communication link is encrypted, such information can still be used by ϵ to perform traffic analysis, or to jam the communication link, preventing any communication to occur and potentially causing more severe harm to the system monitored by the IoT network. With reference to our scenario, we assume ϵ is in line with the well-known Honest-but-Curious (HbC) model. As such, the adversary behaves honestly, by following all the steps of our protocol. At the same time, by using the received information, it tries to gain as much knowledge as possible regarding the cell assignment schedule of the victim. Additionally, we assume ϵ is a global eavesdropper, passively listening to the messages exchanged by both parties.

Furthermore, ϵ is a location-bounded adversary, located in the same area of the target network. We presume that ϵ can also feature active capabilities, e.g., inject packets at will, either by replaying existing packets or by inserting ad-hoc messages, to obtain the desired information.

Our protocol, namely PRM, is intended specifically to allow two networks to discover interfering cells in the RST, while preventing the previously-described adversary to gain information about their cell assignment schedule. More details follow in Sec. 2.3.

2.3. PRM Protocol

We introduce in this section PRM. We describe the basic cryptographic primitive (Sec. 2.3.1) and introduce the iterative set division strategy (Sec. 2.3.2). Furthermore, we provide a detailed description of the overall PRM scheme (Sec. 2.3.3).

For the readers' convenience, we report in Tab. 2.1 the overview of our main notation.

Table 2.1: Notation Summary.

Notation	Description
ϵ	Adversary.
A, B	Network Identifiers.
R_A, R_B	Radio Scheduling Tables of the networks A and B.
a_j, b_j	j-th elements/cells of the list of A and B.
r_{A_j}	Individual nonce for each element.
c_j	Computed element of a_j.
C_A	List of computed elements.
$p_{A\|\|B}, q_{A\|\|B}$	Safe primes for each party.
$n_{A\|\|B}$	Size of the modular field for each party.
$H(\cdot)$	Generic hash function.
C_B	List of computed elements ($H(x_j)$) on B.
Y	List of elements send to server to compare.
M	Row count of RST.
N	Column count of RST.
$R_{A,i,k}, R_{B,i,k}$	Split part k of RST at round i.
oc_i	Occupied cells at the round i of PRM.
e_{tx}	Energy of sending packets.
e_{rx}	Energy of receiving packets.
$\rho_{i,k}$	Defines the current state of consumed energy.
$\tau_{A\|\|B}$	Energy threshold.
δ_i	Privacy overhead at the round i.
ν	Privacy threshold.

2.3.1. Private Set Intersection

PSI protocols allow a set of parties to evaluate the intersection of the sets of elements in their possession, by revealing to the other parties only the shared elements [169, 61]. Such protocols have received increasing attention in the last years, since the elements not possessed by both parties are not exposed to the remote party, allowing for enhanced privacy [62, 47].

Many PSI protocols are available in the literature. They differ on several factors, such as in computational overhead, bandwidth, energy consumption, and number of involved parties, to name a few. In this work, we build on the private equality testing approach proposed by Kotzanikolaou et al. [161] because first, both parties become aware of the *shared* input. Hence, both parties can take action to solve the situation. Note that, if only one party has insights regarding the intersecting input, it would be the only one to have such knowledge, enabling several possible threats to emerge (see Sec. 2.2). Second , compared to other schemes in the literature, such a scheme is the most lightweight and efficient. In the IoT context, where processing constraints and energy consumption play a major role, such considerations are crucial. The approach by Kotzanikolaou et al. relies on the well-known prime numbers Factorization Problem (FP) at the basis of the RSA algorithm, and it uses the corresponding security guarantees to build a PSI scheme for discrete, finite, low entropy

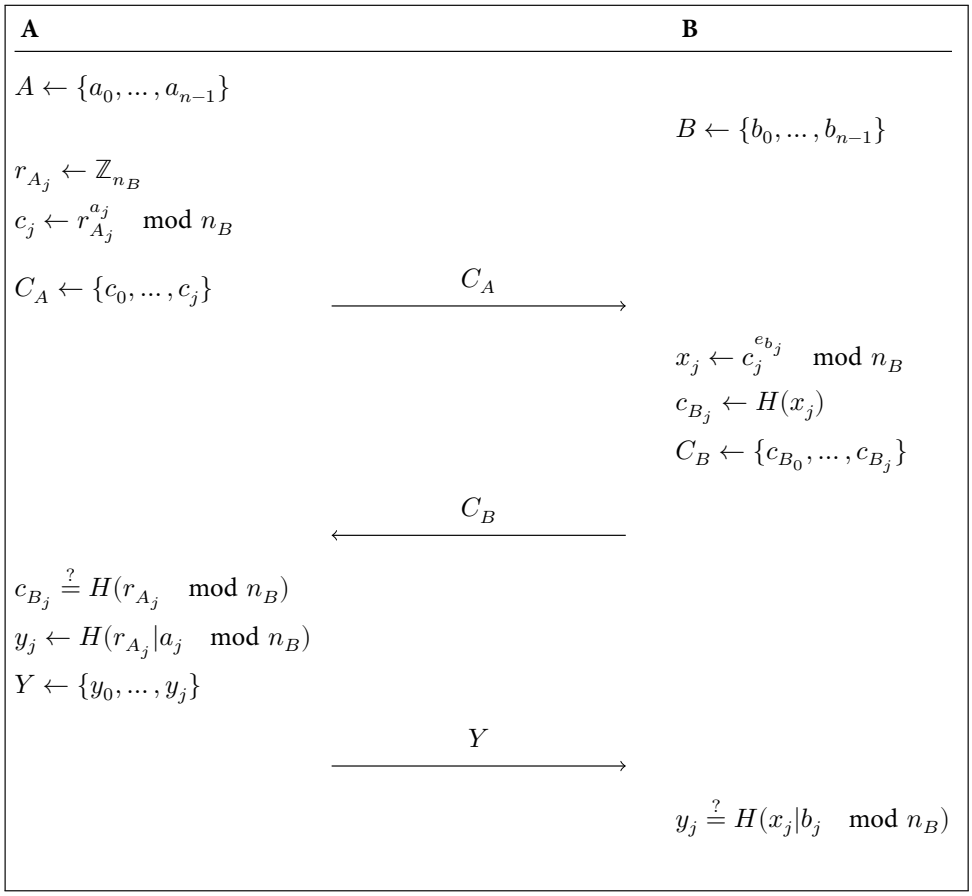

Figure 2.1: FP-PSI protocol.

data. In the following, we describe the adaption of the protocol by [161] to our problem, i.e., considering the comparison of multiple elements in the RSTs of the Coordinators of the IoT networks.

During the setup phase, A and B select two large prime numbers at random, namely, (p_A, q_A) and (p_B, q_B), and use them to compute the values $n_A = p_A \cdot q_A$ and $n_B = p_B \cdot q_B$. Also, for each element in the set, they compute a key pair (e, d), where d is the identifier of the element in the set and $e = \frac{1}{d} \mod \phi(n)$. Figure 2.1 illustrates the steps required by the protocol, while we provide the details below.

1) Denote with a_j and b_j the generic elements of the set possessed by A and B, respectively. A first extracts a vector of J random nonces, namely,
$\mathbf{r_A} = \left[r_{A,1}, r_{A,2}, \dots, r_{A,j}, \dots, r_{A,N_A}\right]$, where N_A is the number of elements in the set of A. Then, A generates a vector of encrypted elements $\mathbf{C_A}$, where each element c_j is

computed according to Eq. 2.1.

$$c_j = r_{A_j}^{a_j} \mod n_B, \tag{2.1}$$

A delivers the vector $\mathbf{C_A}$ to B.

2) At message reception, B takes each element in the vector C_A and, for each element in its own set, namely, b_i, it computes an encrypted response x_j, as in Eq. 2.2.

$$x_j = c_j^{e_{b_i}} \mod n_B. \tag{2.2}$$

Then, B computes the digest of each encrypted response x_j, namely $c_{B,j}$, through the hash function H, as in Eq. 2.3.

$$c_{B,j} = H(x_j). \tag{2.3}$$

Then, B delivers to A the vector $\mathbf{C_B} = \left[c_{B,1}, \dots, c_{B,j}, \dots, c_{B,K}\right]$ (with $K = M * N$).

3) At message reception, A performs the equality check on each element, by applying Eq. 2.4.

$$c_{B_j} \stackrel{?}{=} H(r_{A_j} \mod n_B). \tag{2.4}$$

For all the elements satisfying Eq. 2.4, A computes an encrypted proof, namely y_j, as per Eq. 2.5.

$$y_j = H(r_{A_j} | a_j \mod n_B). \tag{2.5}$$

A delivers back to B the vector of encrypted proofs, namely, $\mathbf{Y_A} = y_1, \dots, y_j, \dots, y_S$ (where S is the number of elements satisfying Eq. 2.5).

4) At message reception, B compares the received proofs with its elements, according to Eq. 2.6.

$$y_j \stackrel{?}{=} H(x_j | b_j \mod n_B), \tag{2.6}$$

thus confirming the intersecting elements in its list.

In summary, at the end of the protocol, both parties know the elements (in our case, the cells) possessed by both of them, requiring a change to prevent interference issues.

However, we notice that the straightforward application of the scheme in [161] to our problem could result in a computationally-intensive protocol, because it requires one exponentiation per cell as described in Eqs. 2.1 and 2.2. Overall, the scheme presented above requires as private input the number of occupied cells (namely, oc), and such number depends on many factors (no. of nodes, type of traffic, etc.). Considering that IoT nodes are usually energy-constrained, such a scheme is hardly applicable efficiently. In the following, we couple the scheme described above with a new algorithm based on multiple iterative set divisions, thus significantly reducing its overhead.

2.3.2. Iterative Set Division Strategy

The rationale behind the iterative set division approach lies in the consideration that, usually, network administrators schedule radio operations in consecutive time-slots and channels, located close each other in the RST. Thus, the core idea of this approach is to identify beforehand collision-prone portions of the RST and to focus the comparison on these portions of the RST, if necessary, instead of comparing each cell singularly. To ease the description of the afore-mentioned approach, we refer to the example in Figure 2.2.

Denote with R_A and R_B the RSTs of A and B, respectively. First, each of them divides its RST in two halves. We denote the two halves of the RST possessed by the entity A at the iteration $i = 1$ as $R_{A,1,0}, R_{A,1,1}$, i.e., $R_{A,i,k}$ (similarly for B). If the RST in possession of A has at least one slot assigned to a communication link in the specific half of the RST, then A is set to be in possession of the element $R_{A,1,0}$; otherwise, A does not possess the element. Similar considerations apply for B. Then, A runs the PSI protocol described in Sec. 2.3.1 to check if any half of its RST is possessed also by B. In the specific example of Fig. 2.2, we can see that the first half of the RST of both parties is occupied by at least a single communication link (marked as green). However, the second half of the RST is marked red, since B does not have any occupied cells here. Thus, at the end of the scheme, A and B know that only the left-most halves of their RST collide. Therefore, they trigger a new round $i = 2$, splitting the colliding half in two new halves, namely $R_{A,2,0}, R_{A,2,1}$ (resp. $R_{B,2,0}, R_{B,2,1}$). In the example in Fig. 2.2, at the end of the round $i = 2$, A and B learn that both portions (halves) of their RSTs could have collisions (second round marked on both portions as green on both parties). Thus, they continue the protocol with the round $i = 3$, further splitting each colliding portion of the RST in two new elements. At the end of this iterative scheme, A and B narrow down the comparisons to only the portion of the RST where a collision really exists, finding the colliding slot.

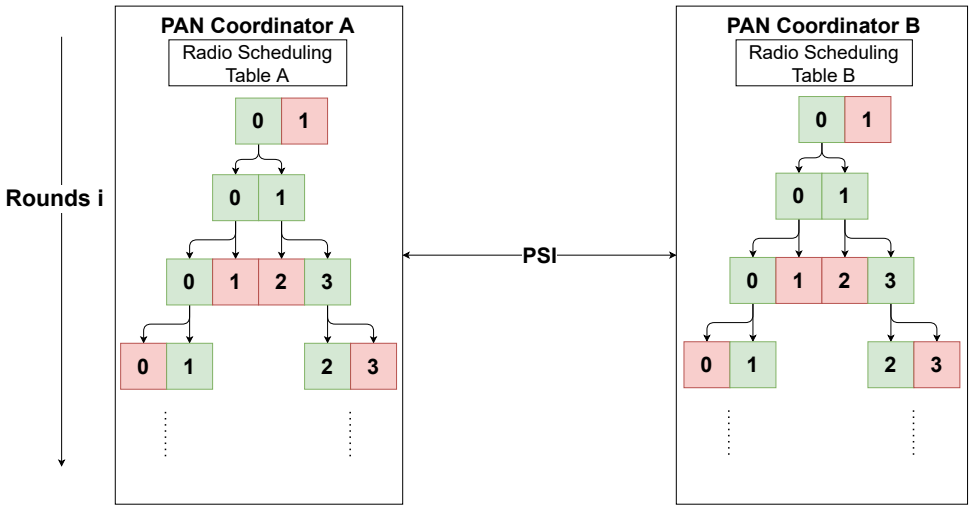

Figure 2.2: Illustration of the iterative set division approach.

We notice that, in some conditions, the iterative set division approach can reduce the computational overhead on involved parties. Indeed, when the two parties have occupied cells in far-away portions of the RST, the approach immediately discovers the absence of colliding cells, requiring a number $oc + \sum_{i=1}^{log_2(N)} 2^i$ exponentiations. Thus, depending on the location of occupied cells, the iterative set division scheme has a computational advantage over the PSI approach in Sec. 2.3.1. In particular, when the entities have occupied cells in far-away portions, the iterative set division approach takes two exponentiations only for each party, while the PSI requires a number of exponentiations equal to the number of all occupied cells oc of both parties.

However, in general, the iterative set division approach is not always more advantageous than the application of the PSI scheme described in Sec. 2.3.1. Indeed, the convenience of the iterative approach depends on the mutual distribution of the occupied cells among the communicating parties. Thus, if the parties have occupied cells in the same portion of the RST, the iterative set division approach ends up increasing the amount of operations necessary to discover collisions, worsening the computational overhead as compared to the application of the PSI scheme. The integration of such considerations at each step of the iterative set division algorithm leads to the full PRM scheme, described below.

2.3.3. The PRM scheme

Fig. 2.3 illustrates the whole PRM protocol, while we describe each step below.

1) PRM is triggered by the client, when it realizes to be in the wireless reception radius of the server.

2) Both parties run the iterative set division approach described in Sec. 2.3.2. Thus, the client and the server first split R_A and R_B on the column dimension, into two halves, namely, $R_{A,1,0}, R_{A,1,1}$ and $R_{B,1,0}, R_{B,1,1}$. Then, A applies the step 1) described in Sec. 2.3.1 considering as *possessed elements* the cell identifiers $R_{A,i,k}$ where occupied cells exist. Thus, first A delivers the challenge vector $\mathbf{C_A}$ to B, and then, B applies the step 2) described in Sec. 2.3.1, over the *possessed elements* and delivers back the response $\mathbf{C_B}$.

3) At message reception, A identifies the colliding elements, running Eq. 2.4, and it computes the vector of encrypted proofs at the round $i = 1$, namely, $\mathbf{Y_{A,1}}$ (see step 3) in Sec. 2.3.1). If no colliding elements are found, A delivers $\mathbf{Y_{A,1}}$, and stops the protocol, concluding that no collisions are present.

4) Conversely, if at least one colliding element is found, A checks if it is convenient to continue further with the iterative set division approach, considering both energy and privacy requirements. We define ν as the *privacy threshold* of the protocol, i.e., the maximum privacy leakage the entity A would like to provide through the protocol. Similarly, we define τ [J] as the *energy threshold* of the protocol, i.e., the maximum amount of energy that A would like to spend for the protocol. First, A computes the

privacy overhead at the current round i, namely $delta_i$, through Eq. 2.7.

$$\delta_i = \frac{1}{2^{M*\frac{N}{2^i}}}, \tag{2.7}$$

where M is the number of rows of the RST, N the number of columns of the RST and 2^i the number of elements in the current round. Note that $delta_i$ considers the probability of guessing the *occupied* cells in possession of A from B. A checks if $\delta_i < \nu$.

If such condition is not satisfied, A switches from the iterative set division approach to the full FP-PSI scheme (see Sec. 2.3.1), considering as occupied cells only the ones in the portion of the RST colliding with B. If, instead, the above-stated condition is verified, A computes the *energy overhead* at the current round i, namely ρ_i, as in Eq. 2.8

$$\rho_i = \sum_{j=1}^{i}(2^i + e_{tx} + e_{rx}) + (oc_{i_{(A)}} + 2 * e_{tx} + e_{rx}), \tag{2.8}$$

where e_{tx} and e_{rx} are the energy to transmit and receive a packet (in J), respectively, while oc_i is the number of elements possessed by A at the round i. Note that ρ_i considers the overall energy spent by A at this time, as well as the worst-case amount of energy spent by A if all the elements in the next round of the iterative set division approach collide with the elements in possession of B. As before, A checks if $\rho_{i,k} < \tau$. If such condition is not satisfied, A switches from the iterative set division approach to the full FP-PSI scheme (see Sec. 2.3.1), considering as occupied cells only the ones in the portion of the RST colliding with B.

Instead, if this condition is verified, A goes ahead with PRM by applying further the iterative scheme.

5) A applies the same procedure of step 2) for the new iteration of the iterative set division approach. Then, A delivers to B the vector $Y_{A,1}$ from the previous round $i = 1$ and the vector of encrypted challenges of the round $i = 2$, namely, $C_{A,2}$.

6) At message reception, B first verifies the matching of the elements at the round $i = 1$, through the application of Eq. 2.6 (step 4) in Sec. 2.3.1) on the input vector $Y_{A,1}$. Then, considering the elements colliding with A, B further divides the colliding elements in two halves, and applies the same procedure described at step 3).

7) The procedure described at step 3) is repeated, until either there are no matches between the elements possessed by A and B or they cannot divide further the elements in two (either because the FP-PSI protocol in Sec. 2.3.1 was run or the iterative set division approach ended up in portions made up of single columns with occupied cells of the RST). After identifying all the colliding cells, the client and server select new channels and time-slots for the colliding cells, choosing into the non-colliding portions. If necessary, they can launch a new instance of PRM to check if these new cells still collide.

A		**B**
$R_A \leftarrow \{a_0, \dots, a_K\}$		$R_B \leftarrow \{b_0, \dots, b_{K-1}\}$
$R_{A,1,0} \leftarrow \{a_0, \dots, a_{\frac{K-1}{2}}\}$		$R_{B,1,0} \leftarrow \{b_0, \dots, b_{\frac{K-1}{2}}\}$
$R_{A,1,1} \leftarrow \{a_{\frac{K-1}{2}+1}, \dots, a_{K-1}\}$		$R_{B,1,1} \leftarrow \{b_{\frac{K-1}{2}+1}, \dots, b_{K-1}\}$
$c_0 \leftarrow r_{A_0}^{R_{A,1,0}} \mod n_B$		
$c_1 \leftarrow r_{A_1}^{R_{A,1,1}} \mod n_B$		
$C_{A,1} \leftarrow \{c_0, c_1\}$	$\xrightarrow{C_{A,1}}$	$x_0 \leftarrow c_0^{e_{B,0}} \mod n_B$
		$x_1 \leftarrow c_1^{e_{B,1}} \mod n_B$
		$c_{B_0} \leftarrow H(x_0)$
		$c_{B_1} \leftarrow H(x_1)$
$c_{B_0} \stackrel{?}{=} H(r_{A_0} \mod n_B)$	$\xleftarrow{C_{B,1}}$	$C_{B,1} \leftarrow \{c_{B_0}, c_{B_1}\}$
$c_{B_1} \stackrel{?}{=} H(r_{A_1} \mod n_B)$		
$y_0 \leftarrow H(r_{A_0} \vert R_{A,1,0} \mod n_B)$		
$y_1 \leftarrow H(r_{A_1} \vert R_{A,1,1} \mod n_B) \quad Y_{A,1} \leftarrow \{y_0, y_1\}$		
if $(\delta_{2,k} < \nu) \wedge (\rho_{2,0,A} < \tau_A \quad \vee \dots \vee \quad \rho_{2,k,A} < \tau_A)$		
$R_{A,2,0} \leftarrow \{a_0, \dots, a_{\frac{K-1}{4}}\}$		
$\vdots$		
$R_{A,2,k=3} \leftarrow \{a_{\frac{3K-1+6}{4}+1}, \dots, a_{K-1}\}$		
$c_0 \leftarrow r_{A_0}^{R_{A,2,0}} \mod n_B$		
$\vdots$		
$c_k \leftarrow r_{A_k}^{R_{A,2,k}} \mod n_B$		
$C_{A,2} \leftarrow \{c_0, \dots, c_k\}$		
else	$\xrightarrow{Y_{A,1}, C_{A,2}}$	$y_0 \stackrel{?}{=} H(x_0 \vert R_{B,1,0} \mod n_B)$
Initiate FP-PSI for all remaining occupied cells		$y_1 \stackrel{?}{=} H(x_1 \vert R_{B,1,1} \mod n_B)$
		if $(\delta_{2,k} < \nu) \wedge (\rho_{2,0,B} < \tau_B \vee \dots \vee \quad \rho_{2,k,B} < \tau_B)$
		$R_{B,2,0} \leftarrow \{b_0, \dots, b_{\frac{K-1}{4}}\}$
		$\vdots$
		$R_{B,2,k} \leftarrow \{b_{\frac{3K-1+6}{4}+1}, \dots, b_{K-1}\}$
		$e_{B,0} \leftarrow \frac{1}{R_{B,i,0}} \mod \phi(n_B)$
		$\vdots$
		$C_{B,2} \leftarrow \{c_{B_0}, \dots, c_{B_k}\}$
if $(\delta_{3,k} < \nu) \wedge (\rho_{3,0} < \tau_A \quad \vee \dots \vee \quad \rho_{3,k} < \tau_A)$	$\xleftarrow{C_{B,2}}$	**else**
continue successive approach		Initiate FP-PSI for all remaining occupied cells
$\vdots$		
else		
$\vdots$	$\vdots$	$\vdots$
Re-position colliding cells		

Figure 2.3: The design of the PRM.

2.4. Security Analysis

We provide here the security analysis of PRM, using both ProVerif (Sec. 2.4.1) and probability theory (Sec. 2.4.2).

2.4.1. Formal Security Analysis via ProVerif

The most relevant security feature of PRM is to guarantee the privacy of the whole RSTs of the participating parties, while allowing to discover interfering cells. Such a property is provided through the integration of the protocol by Kotzanikolaou et al. [161]. In the following, in line with many works in the recent literature [33, 136, 189, 212], we use the automated tool ProVerif to formally verify the security of the single run of our scheme, as well as to confirm that our construction using such a protocol does not break its security.

The output of ProVerif in Lst. 2.1 shows that the private occupied cells *a* and *b* are not exposed to the attacker. In addition, the attacker is not able to guess/brute-force the occupied cells *a* and *b* of the RST of the participating parties. Thus, ProVerif confirms that the single run of PRM protects the confidentiality of the occupied cells of the entities participating in the protocol, while ensuring the identification of interfering cells. In appendix A, we provide details about ProVerif and the definitions necessary to understand the output shown in Lst. 2.1. Interested readers can use the source code we released at [72] to reproduce our results and further extend them in customized use cases.

Listing 2.1: Excerpt of the output of the ProVerif tool.

```
1  Verification summary:
2  Weak secret a is true.
3  Weak secret b is true.
4  Query not attacker(a[]) is true.
5  Query not attacker(b[]) is true.
6  Query not attacker(rA[]) is true.
```

2.4.2. Additional Security and Privacy Considerations

In the previous section, we formally showed the security of the single run of PRM. However, our protocol includes multiple rounds and, at each round, the two communicating entities use as private input element identifiers representative of a reduced number of RST cells.

At each round, PRM leaks some information to the remote party, i.e.: (i) the number of element identifiers possessed by this party, namely k, and (ii) the number of colliding element identifiers. In the following, using sound probability theory, we formally define the guessing probability of the adversary at the round i of PRM, namely $p_c(i)$. We first discuss in Prop. 2.4.1 the case where the iterative approach stops at the round i because no colliding element identifiers are found.

Proposition 2.4.1. *Assume that at the round i of PRM A and B do not find any colliding element identifiers in their private set. Then, the probability for an honest-but-curious adversary to guess the $\tilde{k}$ cells possessed by the remote party is $p_c(i) = \frac{\tilde{\kappa}}{F \cdot 2^{(MAX-i)} \cdot (U_i - \xi_i)}$, being ξ_i the number of challenges sent by the adversary, U_i the number of element identifiers at the round i, F the number of rows in the RST and $MAX = \lceil * \rceil log_2(N)$.*

Proof. Assume that at the round i of PRM A and B do not find any colliding element identifiers in their private set. At this round i, we can define the probability for an honest-but-curious adversary to guess the correct element identifiers possessed the remote party (namely, $p_B(i)$) as in Eq. 2.9.

$$p_B(i) = \frac{\kappa_i}{U_i - \xi_i}, \tag{2.9}$$

where ξ_i is the number of challenges sent by A to B at the round i, κ_i is the number of responses sent back by B to A at the round i, and U_i is the overall number of available element identifiers at the round i. Note that, per construction, $P_B(i) \in [0, 1]$. Recall that at the round i we consider element identifiers representative of a number 2^{MAX-i} of columns in the RST. Therefore, Eq. 2.9 can be casted into Eq. 2.10 to obtain the probability of guessing a single column.

$$p_{col}(i) = \frac{\tilde{\kappa}}{2^{(MAX-i)} \cdot (U_i - \xi_i)}. \tag{2.10}$$

As each column contains F cells, we can resort to Eq. 2.11 to obtain the probability $p_c(i)$ of guessing the possessed cell.

$$p_c(i) = \frac{\tilde{\kappa}}{F \cdot 2^{(MAX-i)} \cdot (U_i - \xi_i)}, \tag{2.11}$$

where F is the number of rows of the RST (i.e., the number of available frequencies), while $\tilde{k}$ represents the number of cells occupied in the RST of the remote party. Note that, as each round of the PRM divides colliding element identifiers in halves, $MAX = \lceil * \rceil log_2(N)$. □

However, PRM could end also because of energy or privacy constraints. In these cases, the following Prop. 2.4.2 apply.

Proposition 2.4.2. *Assume that at the round i of PRM A and B find κ_i colliding element identifiers in their private set, but PRM ends. Then, the probability for an honest-but-curious adversary to guess one of the $\tilde{k}$ cells possessed by the remote party is $p_c(i) = \sum_{j=1}^{\kappa_i} \frac{\tilde{k}_j}{F \cdot 2^{(MAX-i)}}$, with $MAX = \lceil * \rceil log_2(N)$ and $\sum_{j=1}^{\kappa_i} \tilde{k}_j = \tilde{k}$.*

Proof. Assume that at the round i of PRM A and B find κ_i colliding element identifiers in their private set, but PRM ends because of a privacy or energy constraint. Recall that the element identifier at the round i of PRM is representative of a group of 2^{MAX-i} columns. Also, consider that each of the κ_i colliding identifiers is representative of a number $\tilde{k}_j$ of

occupied columns, and that, per definition, $\sum_{j=1}^{\kappa_i} \tilde{k}_j = \tilde{k}$. Thus, the probability $p_{col}(i)$ of guessing an occupied column of the remote party at the round i of PRM is defined by Eq. 2.12.

$$p_{col}(i) = \sum_{j=1}^{\kappa_i} \frac{\tilde{k}_j}{2^{(MAX-i)}}. \tag{2.12}$$

As any of the F rows of a column can be occupied, we can derive the probability of guessing an occupied cell as in Eq. 2.13.

$$p_c(i) = \sum_{j=1}^{\kappa_i} \frac{\tilde{k}_j}{F \cdot 2^{(MAX-i)}}. \tag{2.13}$$

□

2.5. Performance Assessment

In this section, we provide the performance assessment of PRM, through combined experimentation and simulations. Sec. 2.5.1 describes the setup of our experiments, while Sec. 2.5.2 provides the results.

2.5.1. Experiments Setting

To evaluate the feasibility and impact of PRM on real IoT devices, we evaluated the overhead of our scheme through combined simulations and experimentation on real hardware devices. As for the hardware, we selected the Raspberry Pi 3 Model B+ and the ESPCopter. The Raspberry Pi Model B+ is equipped with a Cortex-A53 processor running at 1.4 GHz, 1GB of RAM, and 16GB of storage, and it is often used as a tiny gateway for static IoT deployments [173, 131]. The ESPCopter is a wirelessly networkable small-size programmable mini-drone, powered by the microcontroller ESP8266, equipped with 1 MB of Flash memory and 96KB of RAM [82]. In our setup, it models a resource-constrained IoT device. On the Raspberry Pi, we adapted the code available at [161]. The ESPcopter does not provide Python support. Therefore, we implemented our solution from scratch, leveraging the support provided by the widespread *mbedTLS* library. As for the simulations, we used Matlab R2021a on a Lenovo Thinkpad P1, equipped with two Intel Core i7-8750H processors running at 2.20 GHz, 32GB of RAM, and 1 TB of SDD Storage. For each experiment presented in the next section, we ran 100 simulations, and we reported the average results of our measurements.

Overall, we implemented the operations required by PRM on the afore-mentioned hardware platforms, and we took note of the required time. Then, through simulations, we extracted the average number of protocol instances required in different settings, and we leveraged the previously-noted execution times to derive the overhead of the full scheme. Finally, note that our simulations address the worst case of our scheme, where PRM is executed until it reaches the maximum round of the iterative set division approach.

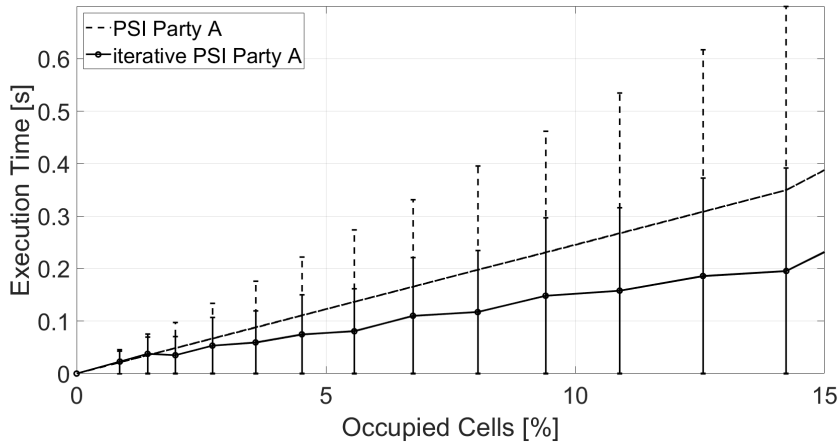

Figure 2.4: Execution time of party A of the PSI-based approach and the iterative scheme, when A has a RSTs filled in the right-upper part with the 95% confidence interval.

2.5.2. Results

In this section, we provide some results and insights into the use of our proposed scheme, looking at its performances with different filling profiles of the RSTs on the involved devices, as well as different hardware platforms. We first investigated the scenario where the RST of the network A contains a given fraction of occupied cells, all grouped in the upper-right part, while the RST of the network B has this same profile, but with a reduced number of occupied cells. Note that the afore-cited configurations are a typical choice for network administrator, especially when cells are assigned starting from the first available frequencies and slots. Fig. 2.4 reports the execution time in such a scenario of both the PSI-based approach and the iterative set division scheme on the Raspberry Pi device, while increasing the number of occupied cells, i.e., cells assigned for radio communications. We report the average results of our experiments.

Overall, we notice that the execution time of both schemes increases with the increase of the filling ratio ($\%$ of occupied cells) of the right upper part of the RST on both parties, with different behaviors. When the occupied cells exceed 1%, the iterative set division approach is faster, as the peers (specifically, the client) take advantage of the distribution of the cells in the RST to perform a reduced number of exponentiations. Using such an approach, shared occupied cells could be localized quicker, leading to reduced execution times.

Figure 2.5 provides the results in the same setup of Fig. 2.4, but using the ESPCopter and focusing on the party A. We notice a similar behavior to Fig. 2.4, but the ESPcopter takes more time to complete the protocol.

When only the 12.5% of the RST is populated, the ESPcopter needs more than one minute to complete PRM (99.6 sec.), which is not acceptable neither feasible. This is the reason why we introduced the *energy threshold* in our protocol. On the one hand, mobile networks

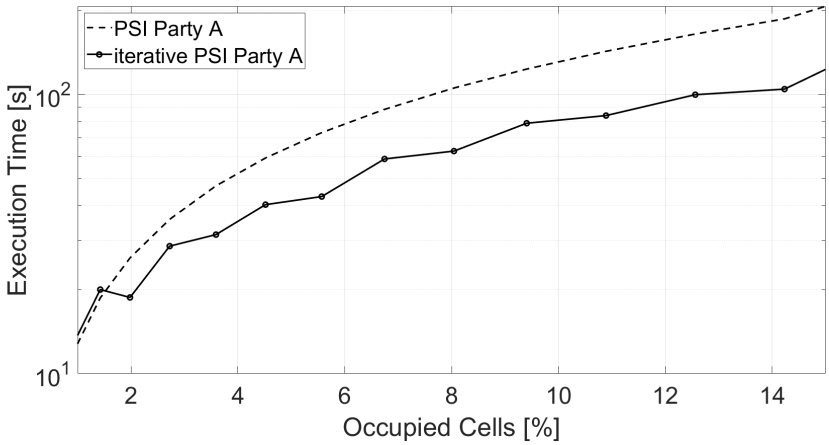

Figure 2.5: Execution time of the two variants of PRM on the ESPCopter, when A has a RSTs filled with occupied cells in the right-upper part.

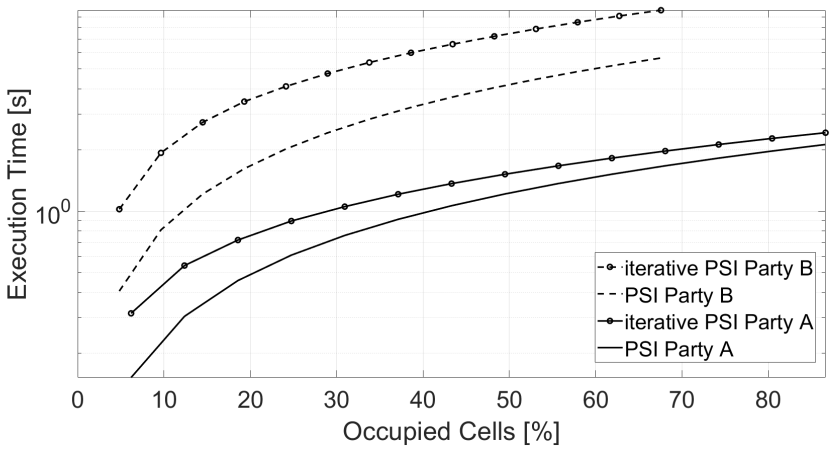

Figure 2.6: Execution time of the two proposed schemes, with an increasing value of the filling ratio of the RST, where occupied cells are extracted uniformly at random.

typically operate with few occupied cells, either for energy saving reasons or with a minimal cell configuration of one active cell [228, 229]. In these cases, PRM can complete even on a very constrained platform in less than 1 sec. On the other hand, when the RST is more populated, more powerful IoT devices are desirable.

The previous results investigated the scenario where occupied cells are localized in a specific part of the RST. Assuming a scenario where the occupied cells are extracted uniformly at random in the whole RSTs, Figure 2.6 shows the execution time of the two proposed schemes, with an increasing value of the filling ratio. When the occupied cells in the RST are dis-

tributed uniformly at random, the plain-PSI approach is the quickest. Indeed, as occupied cells are distributed at random in the RST, grouping operations performed by the iterative approach do not lead to the progressive elimination of multiple unused portions of the RST. Therefore, as the rounds of the iterative approach are executed, multiple additional exponentiation operations are performed, leading to higher execution times. Conversely, the plain-PSI approach takes an execution time that is proportional to the amount of occupied cells (not to their distribution), being more efficient in this specific case. We will discuss these results in the next section.

2.6. Discussion

In this section, we wrap-up on our results and provide some additional considerations on PRM.

Schemes Selection. As shown by the results in Sec. 2.5.2, the iterative set division approach is the most efficient solution when one of the parties in the protocol has occupied cells localized in a given portion of the RST. Conversely, if the distribution of the cells in the RST is random (see Fig. 2.6), the plain-PSI scheme is the best choice. Note that the party initiating the protocol (A) always knows its own cells' distribution. Thus, it can choose the strategy minimizing the overhead.

Set division Strategy. When applying the iterative set division approach, PRM divides the RST progressively into equal-sized portions. In line of principle, multiple different shapes can be chosen when dividing, e.g., rows, squares, or columns. However, the selection of a shape directly affects the privacy leakage of PRM. In our specific case, recall that the number of rows of the RST is $F = 16$, and thus, dividing progressively on rows would cause a significant guessing probability to the remote party. Instead, as the number of columns in the RST is higher ($N = 101$ in our case), dividing over columns only provides enhanced privacy guarantees, always allowing to fulfill privacy constraints (see part below).

Thresholds. As discussed in Sec. 2.3.3, PRM features two thresholds, namely, the privacy threshold ν and the energy threshold τ. We define ν to be in range of $[0, 1]$. When ν is 0, PRM executes the plain-PSI over the entire set of occupied cells. When ν is 0.1, then PRM allows the iterative set division to reach the last possible round, since the highest privacy leakage of such approach is $\frac{1}{2^{16}}$, i.e., the guessing probability of a single cell in one column of the RST. On the one hand, a peer can choose the desired value of ν, having the control on the privacy leakage. On the other hand, the selection of ν impacts on the number of performed exponentiations, and thus, its overhead. Therefore, the peer should always select the privacy threshold trading off between privacy and energy.

2.7. Conclusion

In this chapter, we presented PRM, the first solution for privacy-preserving interference discovery among IEEE 802.15.4 networks. PRM allows the Coordinators of two IEEE 802.15.4 networks to discover shared cells in the Radio Scheduling Tables, causing interference, while

keeping the whole RST private. PRM integrates and smartly combines two dedicated algorithms, namely, the plain-PSI and the iterative set division approach, performing PSI on elements representing a distinctive amount of occupied cells in the RSTs. Besides designing the overall protocol and proving its security, we also provided an experimental performance assessment using real heterogeneous IoT devices, i.e., a Raspberry Pi and an ESPCopter. Our results show that, through a smart choice in the allocation of occupied cells in the RST, the iterative set division strategy can allow even constrained devices (such as the ESPCopter) to perform privacy-preserving interference discovery in a few seconds. On more capable gateways (e.g., the Raspberry Pi), when the percentage of occupied cells is below the 20%, PRM always concludes in less than 1 sec., proving to be a feasible solution. Additionally, we discussed strategies: (i) on scheme selection for PRM which allows to reduce the computation overhead beforehand if the cell occupation is known; (ii) on using variations of shapes to divide the RST, which consequently may affect the privacy leakage, and (iii) on thresholds regarding privacy and energy, depending on the threshold the involved entities can prioritize depending on the situation to sacrifice more energy for privacy or less energy for less privacy.

Future work will consider the comparison among different strategies for the iterative set division approach (columns, rows, squares) and a detailed investigation on the energy consumption of PRM.

3

ePPTM - Enhanced Privacy-Preserving Trajectory Matching on Autonomous Vehicles

This chapter is inspired by [94, 191]:
D.R. George, S. Sciancalepore
ePPTM - Enhanced Privacy-Preserving Trajectory Matching on Autonomous Vehicles
2025 IEEE Internet of Things Journal (IoT-J)
and
S. Sciancalepore, **D.R. George**
Privacy-Preserving Trajectory Matching on Autonomous Unmanned Aerial Vehicles
2022 Annual Computer Security Applications Conference (ACSAC)

The problem considered in the previous chapter involved a small amount of information (the RST) to be verified privately by two parties. However, private information could be larger, e.g., in Autonomous Vehicles (AVs), it could involve spatiotemporal trajectory data, usually exchanged to ensure collision-free paths. Also such information is private, and exchanging it in plain-text could lead to severe privacy issues. Taking into account such considerations, in this chapter, we address the private matching problem of detecting spatiotemporal collisions among AVs in advance to enhance safety and reduce risks, as highlighted in Chapter 1.1.3. We propose ePPTM, a fully-accurate protocol for privacy-preserving trajectory matching among AVs improving over our preliminary work in [191], and we demonstrate its effectiveness with a proof-of-concept evaluation on real-world datasets. This contributes to achieving *input privacy* and addresses **SRQ1** and **SRQ4**.

3.1. Introduction

AVs are gaining increasing momentum in the scientific community and Industry, thanks to the enormous business opportunities they enable in several application domains [150]. Thanks to their flexible and mobile features, AVs are being increasingly deployed on land (autonomous cars and robots) [181, 244], air (autonomous drones and aircraft) [216, 218] and sea (autonomous ships and submarines) [114, 117] for key applications, e.g., surveillance, goods delivery, and search-and-rescue, to name a few. According to relevant business forecasting associations, the estimated autonomous vehicles market size will grow to 888.40 Billion USD by 2028, confirming this general trend [96].

With such an enormous increase in deployed units, risks of collisions of AVs among them and with other (non-autonomous) vehicles and people are even more concrete [56, 233]. Therefore, detecting in advance possible collisions and avoiding them is becoming increasingly crucial to fully unlock the potential behind the deployment of such a new technology [248]. Specifically, discovering collisions among autonomous vehicles in advance could allow re-instantiating vehicles' routes and planning safe collision-free paths, increasing, in turn, the efficiency of the business and the safety of involved vehicles and users [90, 232].

Traditional solutions to detect collisions among various AVs in advance require the mutual exchange of trajectory data, including space and time data [178]. Using such information, one or several AVs could generate collision-free paths and minimize safety risks. However, space and time trajectory data represent private information, whose disclosure to untrusted parties could enable malicious entities to tailor attacks against such vehicles. For example, when autonomous drones are deployed to bring a defibrillator for rescue operations in hard-to-reach areas (e.g., see the application in [20]), an adversary aware of the trajectory of the vehicle could choose the best location where to capture the drone, take it down (e.g., via jamming), and steal the payload [42, 43]. Also, the knowledge of the departure location of the flight could reveal the location of storage sites where resources are stored, while the availability of destination information could reveal sensitive details about the users' location [190].

A few scientific contributions in the last years investigated privacy-preserving collision detection and spatiotemporal matching (see Sec. 3.7 for a detailed overview). However, the solutions currently available hardly run on AVs, e.g., because of reliance on online TTPs, applicability only for simple trajectories and lack of considerations about the travel time. A preliminary solution to the problem of private spatiotemporal trajectory matching was proposed in [191]. The protocol proposed therein, namely PPTM, is designed to allow two autonomous drones to discover collisions at runtime without involving TTPs at runtime. It relies on a dedicated algorithm for trajectory comparison, comparing the trajectories in space and time at an increasing level of granularity. Although the preliminary findings and results of such a proposal are promising, PPTM assumed the availability of fine-grained trajectory data (with interarrival time of a few milliseconds), which is hardly the case in real-world applications and deployments of AVs [138]. Thus, PPTM achieved perfect collision detection only using fine-grained trajectories, and it was tested only via simulations on synthetic data. However, as we demonstrate in this chapter, when trajectory data are sparse (i.e., they are available once every tens of seconds, at least), some of the mathematical simplifications used

by the protocol do not apply anymore. This weakness leads to missed collision detection and significant risks to the integrity of the vehicles and the safety of the people involved.

Contribution. In this chapter, we propose ePPTM, an enhanced interactive protocol for private spatiotemporal trajectory matching among Autonomous Vehicles, i.e., detection of collisions without revealing to the remote party anything else than the colliding points and timestamps. To this aim, ePPTM integrates two main building blocks, i.e., the *Incremental Capsule Matching* algorithm, analyzing co-location between trajectories at an increasing level of granularity, and *privacy-preserving proximity testing*, allowing comparison among private sets without revealing anything more than colliding elements. We also discuss two modes of ePPTM, i.e., the *Truncated Mode* and *Full Mode*, with the former potentially trading off computational and energy overhead with a few *false positives* (non-collisions erroneously detected as collisions), while always ensuring no *false negatives* (i.e., no undetected collisions). ePPTM improves over the current literature by allowing always-accurate collision detection, independently from the shape and the level of granularity of the trajectories of the involved vehicles. Additionally, we revisit the security analysis of our protocol from our previous work in [191], demonstrating formally the security of the single round of our updated ePPTM protocol, and we formally quantify the probability that an honest-but-curious attacker can guess private trajectory points based on the output and information released by our protocol. Moreover, ePPTM can run on any AV, even the ones characterized by strict computational and energy constraints. We implemented an actual proof-of-concept of ePPTM, and released the corresponding code open source at [59]. We also deployed ePPTM on two testbeds characterized by heterogeneous processing capabilities and tested its accuracy and runtime overhead using actual vehicles' trajectories from the city of Porto, Portugal, available at [156, 202]. We compare the accuracy of ePPTM against the earlier solution in [191], showing its superior performance. Our results demonstrate the perfect accuracy of ePPTM (100% collision detection). We discuss relevant aspects of ePPTM w.r.t.: (i) performance differences between colliding and non-colliding trajectories; (ii) trade-off between privacy and overhead; (iii) its capability to run efficiently across multiple instances involving the same devices and various settings, and (iv) collision resolution.

Roadmap. The rest of this chapter is organized as follows: Sec. 3.2 presents the system and the adversary model, Sec. 3.3 describes the overall design of ePPTM, Sec. 3.4 analyzes the security of ePPTM, Sec. 3.5 provides the results of a thorough experimentation of our solution on actual testbeds and vehicles' trajectories, Sec. 3.6 discusses relevant deployment aspects, Sec. 3.7 reviews related work, and finally, Sec. 3.8 concludes the chapter.

Table 3.1: Notation used throughout this chapter.

Notation	Description
ϵ	Adversary.
M, N	Number of entries in the path of AVs A and B.
$\mathbf{V_{j,m}}$	Speed of the AV j in the time-frame $[t_m t_{m+1}]$.
$\mathbf{T_j}$	Trajectory of the j-th AV.
x_j, y_j, t_j	Trajectory point by the AV j, where x and y is the location at timestamp t.
$u_{j,i}$, $v_{j,i}$	Coefficients of the equation of the slope intercept form at round i of ePPTM.
$d_{m,\tilde{y}_{j,i}}$	Distance of the m-th point in T_j from the ref. line eq. $\tilde{y}_j$ at round i of ePPTM.
$dm_{j,i}$	Maximum value between all $d_{m,\tilde{y}_{j,i}}$.
δ	Collision threshold.
σ	Maximum GPS inaccuracy.
$r_{j,i}$	Radius of the capsule of AV j at round i of ePPTM.
$h_{j,i}$	Length of the capsule of AV j at round i of ePPTM.
$d_{j,i}$	Euclidean distance between first and last points in T_j at round i of ePPTM.
$\Theta_{j,i}$	Angle of the capsule of AV j at round i of ePPTM.
$\mathbf{s_i}$	Nonces extracted at the round i of ePPTM.
$\mathbf{O_i}$	Origin points used at the round i of ePPTM.
$\mathbf{x'_j}, \mathbf{y'_j}$	x-y coordinates of roto-translated points of T_j.
$\tilde{\mathbf{x}}_\mathbf{j}, \tilde{\mathbf{y}}_\mathbf{j}$	Position identifiers of T_j.
$\mathbf{sh}_{\mathbf{r_{j,i}}}$	Vector to shift points along x and y axis based on the radius $r_{j,i}$.
$\mathbf{p_j}$	A point of the capsule of the responder AV j.
$\mathbf{p_{j,tip}}$	A point of the capsule's tip of the responder AV j.
$\mathbf{p_{j,base}}$	A point of the capsule's base of the responder AV j.
α	Angle of the rotation Matrix rot.
a_l	Angle steps of the semicircle of the responders' capsule.
η	Circle steps of the semicircle of the responders' capsule.
$\mathbf{p_{j,\zeta}}$	Vector of all the points p_j of the responder's capsule AV j.
$acc(x, d)$	$acc(\cdot, \cdot)$ computes line points starting from point x based on the direction d.
f, g	Functions mapping positions and times to unique ids.
$H(\cdot)$	Generic hashing function.
p_j, q_j	Safe primes of the AV j.
n_j	Size of the modular field of the AV j.
$\Pi_{j,i}$	Ref. capsule of AV j at round i of ePPTM.
$\mathbf{\Gamma_{j,i}}$	Vector of K position identifiers on j at the round i of ePPTM.
$\xi_{j,i}$	Nonce of AV j at round i of ePPTM.
$\mathbf{c_{j,i}}$	Vector of the encrypted challenges generated by j at the round i of ePPTM.
$\boldsymbol{\gamma}_\mathbf{j,i}$	Vector of K cell identifiers intersected by resp. j's cap., at the round i of ePPTM.
$\tilde{\mathbf{c}}_\mathbf{j,i}$	Vector of the encrypted responses generated by j at the round i of ePPTM.
$\mathbf{w_{j,i}}$	Vector of *check values* on j at the step i of ePPTM, containing K elements.
$\tilde{\mathbf{w}}_\mathbf{j,i}$	Encrypted proof of proximity computed by j at the step i of ePPTM.
$\mathbf{t_j}$	Vector of timestamps of colliding points on j.
$\tilde{t}_j$	Time-frame between the first and last timestamp in $\mathbf{t_j}$.
Δ	Time collision threshold.
t_O	Origin Timestamp of the *Time Trajectory Match*.
s'_j	Nonce of the *Time Trajectory Match*.
$\tilde{\tau}_j$	Timestamp id obtained by applying g.

3.2. System Model and Adversary Model

In this section, we introduce the system model (Sec. 3.2.1) and the adversary model (Sec. 3.2.2). For the readers' convenience, we report in Tab. 3.1 the overview of our main notation.

3.2.1. System Model

We consider two autonomous unmanned vehicles, A and B, produced by any manufacturers and operating in any domain. We assume that A and B are deployed for a mission, such as delivering goods or surveillance, and feature trajectories T_A and T_B from starting points T_{A_0},T_{B_0} to destination points $T_{A_{M-1}}$,$T_{B_{N-1}}$, where $M \neq N$. In addition, we consider that the time instant t_{j_m} where each vehicle j occupies the location T_{j_m} is known accurately. At the same time, the time lapse between two known locations T_{A_m} and $T_{A_{m+1}}$ is variable and known. We only consider that the velocity V_{j_m} of the motion from the location occupied at T_{j_m} and $T_{j_{m+1}}$ is linear, with marginal error. In line with the capabilities of real-world Commercial Off-The-Shelf (COTS) devices, we consider vehicles equipped with radio communication modules, allowing them to communicate wirelessly [127]. The specific communication technology depends on the type of AV. Without loss of generality, we consider the usage of WiFi-direct, which allows secure pairwise communication within the ISM band $[2.4 - 2.5]$ GHz, without relying on any third-party infrastructure element [69]. In addition, both parties need to be in radio visibility to establish a connection over WiFi. Note that WiFi-based communication modules used on AVs can enable connection over relatively large ranges (up to 25 km) [210], [74]; thus, mutual radio visibility among devices does not imply the risk of immediate collision. We also assume that both parties deploy a secure peer-to-peer connection, e.g., by taking advantage of the well-known Transport Layer Security (TLS) protocol [7]. Although TLS provides authentication and confidentiality, it does not prevent privacy issues arising from the intentional disclosure of sensitive information.

In this context, we refer to the aforementioned entities as A and B. They are deployed on a mission executed in a three-dimensional spatiotemporal domain, such as aerial, or a two-dimensional spatiotemporal domain, such as ground or maritime. During the mission, both parties aim to detect timely any spatiotemporal collisions in their path, so as to be able to modify their planned route accordingly. At the same time, both parties aim not to disclose their trajectory T_A and T_B, and keep such data private. In [191], we introduced PPTM, a solution tackling this problem. In this work, we extend such a protocol and propose ePPTM, which improves over PPTM regarding collision detection capabilities and performance when dealing with sparse trajectories.

3.2.2. Adversary Model

The adversary assumed in this work, namely ϵ, features both passive and active capabilities. On the one hand, ϵ is a global, spatially-unbounded, and frequency-unlimited eavesdropper, able to detect any Radio Frequency (RF) activity initiated by UAVs. Using such capabilities, ϵ can know the MAC addresses of the AVs and spoof them. On the other hand, ϵ is also

equipped with a radio that can transmit packets on the same communication frequencies of the target AV. Thus, ϵ can create packets, either genuine or forged, and initiate an instance of ePPTM with a target AVs. ϵ can also deploy its own AV and run an instance of ePPTM with a target. The aim of ϵ is to retrieve the trajectory points of the path of a target AVs. The adversary ϵ executes the described protocol as intended and then works offline to retrieve additional private information using the received messages. Private information includes locations on the path and time-related aspects, such as the start and end time of the mission and its duration. The availability of path-and-time-related information to an adversary can lead to severe attacks, e.g., tracking the AV, launching targeted jamming attacks, capturing the AV and any goods it delivers, disrupting its operations and generating significant loss of financial revenues. Therefore, we aim to preserve the privacy of such information for the AVs.

3.3. ePPTM Protocol Design

3.3.1. Plain-text Collision detection Logic

This section explains the rationale of the collision detection logic used in ePPTM, with no privacy considerations in place. We will address trajectory privacy in Sec. 3.3.2 and Sec. 3.3.3.

For simplicity, we describe below the collision detection logic in the 2D-Cartesian-coordinate system (x, y). However, extension to 3-D is straightforward. Consider two AVs in mutual radio visibility, and assume each AV has a predetermined trajectory, namely T_A and T_B, containing the planned locations of the vehicles at a given time. Considering the generic AV j, $T_j = [x_{j,0}, y_{j,0}, t_{j,0}; x_{j,1}, y_{j,1}, t_{j,1}; \cdots, x_{j,J}, y_{j,J}, t_{j,J}]$, being J the number of trajectory points. For enhanced readability, we also denote $\mathbf{p_m} = [x_m, y_m]$ as the location at timestamp t_m. We denote as M and N the number of trajectory points of A and B, respectively, where $M \geq N$.

The proposed collision detection algorithm, namely *Incremental Capsule Matching*, consists of multiple rounds and it ends with a decision if the trajectories of the two AVs collide or not, i.e., *Collision* or *No Collision*. The algorithm is initiated by the vehicle with the higher number of trajectory points, i.e., A. During each round, we can identify three sub-phases: *Reference Capsule Computation*, *Remote Party Mapping*, and *Identifiers Matching*. We describe them in detail below.

Reference Capsule Computation. In the first round ($i = 1$), considering the location points (x_j, y_j) in T_A, A computes a *reference shape* bounding all its future trajectory points. The *reference shape* is a polygon that includes all the trajectory points, each of which has a minimum distance r from the edges of the shape. As shown in Fig. 3.1 (dashed light black line), in the 2-D space, such a polygon is a capsule characterized by two semi-circles (dotted red line) of radius r centered in the initial and final points of the trajectory and a rectangle merging them (dotted gray line). In the 3D spatial domain, one can replace semicircles with semi-spheres and the rectangle with a cylinder, obtaining a 3D capsule. We highlight that, compared to rectangles and circles, the capsule is the shape fitting the above-mentioned condition (dis-

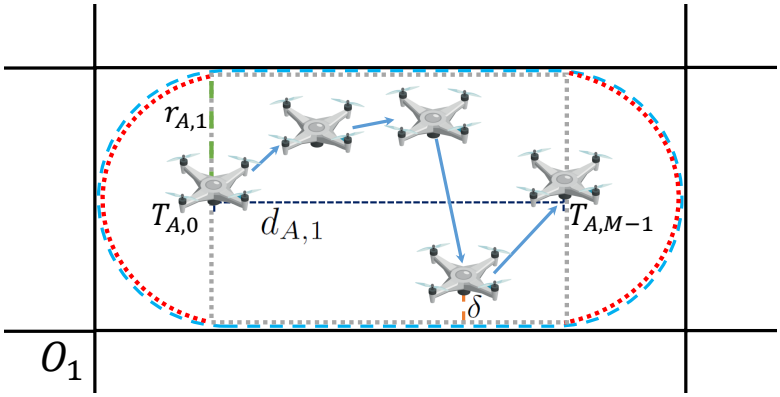

Figure 3.1: Computation of the capsule of A in a reference grid based on the trajectory points T_A.

tance r from the points) while allowing the coverage of the minimum possible additional space around the trajectory points. In contrast, circles and rectangles include further points at the edges, which are not necessary and possibly lead to false positives when applied for collision detection. We define the *reference shape* selected by the initiator of the protocol at a given round as the *reference capsule* at round i. To define the local capsule radius r_A, A computes the line between the first and the last trajectory points, as per Eq. 3.1, where $u_{A,1}$ and $v_{A,1}$ are computed through the well-known slope intercept form of a line.

$$\tilde{y}_{A,1} = u_{A,1} \cdot x_{A,1} + v_{A,1}. \tag{3.1}$$

Next, A computes the distance $d_{m,\tilde{y}_{A,1}}$ between each trajectory points of T_A and the above-mentioned line. Then, A picks the maximum distance as in Eq 3.2.

$$dm_{A,1} = \max_{m} d_{m,\tilde{y}_{A,1}}. \tag{3.2}$$

We define δ as the collision threshold (space guard), i.e., the minimum allowed distance between any points of the trajectories T_A and T_B. We also denote with σ the maximum inaccuracy of the GNSS locations. A computes the reference radius $r_{A,1}$ of the *reference capsule* for round $i = 1$ as in Eq. 3.3, by summing up the cited maximum distance $dm_{A,1}$, the collision threshold δ and the GNSS inaccuracy σ.

$$r_{A,1} = dm_{A,1} + \delta + \sigma. \tag{3.3}$$

A computes the reference length $h_{A,1}$ of the reference capsule in round $i = 1$, by summing up twice the reference radius $r_{A,1}$ to the Euclidean distance between the first and last trajectory point of T_A as in Eq. 3.4.

$$h_{A,1} = d_{A,1} + 2r_{A,1} = ||\mathbf{T_{A,M-1}} - \mathbf{T_{A,0}}||^2 + 2r_{A,1}. \tag{3.4}$$

A computes the angle $\Theta_{j,i} = \Theta_{A,1}$ between the first and last trajectory points of T_A as in Eq. 3.5.

$$\Theta_{A,1} = \arctan\left(\frac{T_{A,y,M-1} - T_{A,y,0}}{T_{A,x,M-1} - T_{A,x,0}}\right), \tag{3.5}$$

where $atan(\cdot)$ refers to the tangent arc operator.

Then, A generates two nonces, for the x- and y-axis respectively, namely $\mathbf{s_i} = \mathbf{s_1} = \left[s_{1,x}, s_{1,y}\right]$. These nonces, together with the angle $\Theta_{A,1}$, are used to roto-translate randomly the trajectory points in T_A. We first generate a random translated origin $O_{1,x}, O_{1,y}$ and then apply the roto-translation via Eq. 3.7.

$$\begin{aligned} O_{1,x} &= x_{A,1} - r_{A,1} - s_{1,x} \cdot h_{A,1}, \\ O_{1,y} &= y_{A,1} - r_{A,1} - s_{1,y} \cdot 2 \cdot r_{A,1}. \end{aligned} \tag{3.6}$$

$$\begin{aligned} \mathbf{x'_A} &= \left[\mathbf{x_A} cos\left(\Theta_{A,1}\right) - \mathbf{y_A} sin\left(\Theta_{A,1}\right)\right] - O_{1,x}, \\ \mathbf{y'_A} &= \left[\mathbf{x_A} sin\left(\Theta_{A,1}\right) + \mathbf{y_A} cos\left(\Theta_{A,1}\right)\right] - O_{1,y}. \end{aligned} \tag{3.7}$$

Note that the *reference capsule* depicted in Fig. 3.1 is in a grid, uniquely identifiable when knowing the origin points and the angle used for the roto-translation. A can compute the identifier of the cell, i.e. the *capsule identifier* (see Fig. 3.3). Following, all the points in $\mathbf{T_A}$ are mapped to trajectory identifiers $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{y}}$, by dividing them for the combination of capsule dimensions, as shown in Eq. 3.8.

$$\begin{aligned} \tilde{\mathbf{x}}_\mathbf{A} &= \lfloor * \rfloor \frac{\mathbf{x'_A}}{h_{A,1}}, \\ \tilde{\mathbf{y}}_\mathbf{A} &= \lfloor * \rfloor \frac{\mathbf{y'_A}}{2 \cdot r_{A,1}}. \end{aligned} \tag{3.8}$$

We highlight that the remote party can generate the grid defined by the initiator (A) with the knowledge of a few values. To this aim, A delivers to B: (i) the origin $\mathbf{O_1} = \left[O_{1,x}, O_{1,y}\right]$; (ii) the capsule length $h_{A,1}$; (iii) the capsule radius $r_{A,1}$; and, finally, (iv) the reference angle $\Theta_{A,1}$.

Remote Party Mapping. B uses the received reference values to map its trajectory points into the space-tessellation provided by A. To this aim, B roto-translates its points using Eq. 3.7. Then, B construct a *local capsule* using Eqs. 3.3, 3.4, and 3.5, as depicted in Fig. 3.2. Here, we denote $r_{B,1}$ as the radius of the responder at round 1 and $\Theta_{B,1}$ as the angle of the responder's *local capsule* at round 1. To ensure the identification of any possible grids of the reference system touched by its local capsule (highlighted through light grey grids in Fig. 3.2), B computes a set of points used to fully define the area covered by its local capsule, as reported in Fig. 3.2. We use the notation $\mathbf{p_{B,\zeta}}$ to denote the list containing all such points. B first computes three points for the semicircle of the tip and three points for the semicircle

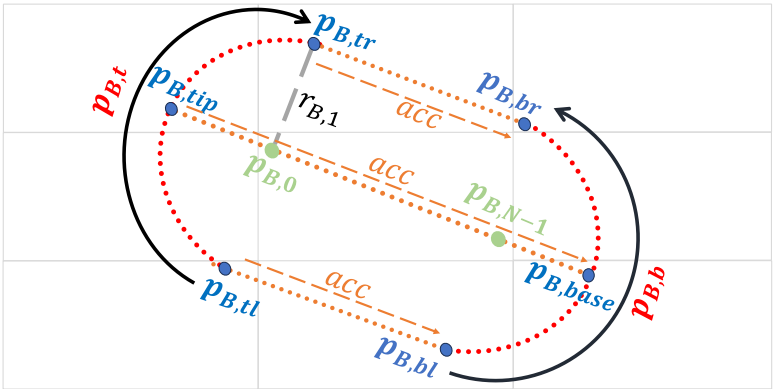

Figure 3.2: Mapping of trajectory points of B in the space tessellation provided by A.

of the base of its local capsule. It first uses Eq. 3.9 to compute the shift along x and y from the first trajectory point to the tip and the last trajectory points to the base of the capsule.

$$\mathbf{sh}_{\mathbf{r_{B,1}}} = [r_{B,1} \cdot sin(\frac{\pi}{2} - \Theta_{B,1}), r_{B,1} \cdot sin(\Theta_{B,1})]. \tag{3.9}$$

Then, B uses Eq. 3.10 and Eq. 3.11, to compute the base and the tip of the capsule. Points at the extreme from the tip of the capsule to the first trajectory point, namely, $p_{B,0}$, and from the base of the capsule to the last trajectory point, namely, $p_{B,N-1}$, respectively (depicted as green dots in Fig. 3.2).

$$\mathbf{p_{B,tip}} = \mathbf{p_{B,0}} - \mathbf{sh}_{\mathbf{r_{B,1}}}. \tag{3.10}$$

$$\mathbf{p_{B,base}} = \mathbf{p_{B,N-1}} + \mathbf{sh}_{\mathbf{r_{B,1}}}. \tag{3.11}$$

The points $\mathbf{p_{B,tip}}$ and $\mathbf{p_{B,base}}$, reported in Fig. 3.2 as black dots, are added to the list $\mathbf{p_{B,\zeta}}$. B also computes some points on the semi-circles at the base and the tip of the capsule. We define the function rot, taking as input the rotation angle α, as defined in Eq. 3.12, where T denotes the transpose.

$$rot(\alpha) = \begin{bmatrix} cos(\alpha) & -sin(\alpha) \\ sin(\alpha) & cos(\alpha) \end{bmatrix}^T. \tag{3.12}$$

B uses Eq. 3.12 to compute a set of points on the semicircles of the local capsule. To this aim, we define the step angles $a_l = -\frac{\pi}{2} + l \cdot \eta \mid l \, \epsilon \, \mathbb{N}, a_l \leq \frac{\pi}{2}$, being η the circle step and we use them as in Eq. 3.13.

$$\begin{aligned} \mathbf{p_{B,t}} &= ((\mathbf{p_{B,0}} - \mathbf{p_{B,tip}}) \times rot(a_l)) + \mathbf{p_{B,0}}, \forall a_l, \\ \mathbf{p_{B,b}} &= ((\mathbf{p_{B,N-1}} - \mathbf{p_{B,base}}) \times rot(a_l)) + \mathbf{p_{B,N-1}}, \forall a_l. \end{aligned} \tag{3.13}$$

The result of Eq. 3.13 is a set of points on the semicircles of the base and the tip of the capsule, represented as red dots in Fig. 3.2. Note that such a set of points also includes the angle points of the rectangle of the capsule. We denote such points as $\mathbf{p_{B,tl}}$, $\mathbf{p_{B,tr}}$, $\mathbf{p_{B,bl}}$, $\mathbf{p_{B,br}}$, reported as black dots in Fig. 3.2. All such points are also added to the list $\mathbf{p_{B,\xi}}$.

Then, B computes a set of points on the perimeter and the bisect of the rectangle of the capsule. We denote $d_B = \mathbf{p_{B,N-1}} - \mathbf{p_{B,0}}$ and define the function *acc* as in Eq. 3.14, where $sgn(d)$ denotes the sign of d and p is the starting point.

$$acc(p, d) = p + (sgn(d) \cdot \mathbf{sh_{r_{B,1}}}). \tag{3.14}$$

Starting from the trajectory point $p_{B,0}$ and the angle points of the rectangle $\mathbf{p_{B,tl}}$ and $\mathbf{p_{B,tr}}$, B sums the value acc to identify points on the capsule, until it reaches the boundaries of such capsule, i.e., $p_{B,N-1}$, $\mathbf{p_{B,bl}}$, $\mathbf{p_{B,br}}$ (orange dots in Fig. 3.2). All such points are also added to the list $\mathbf{p_{B,\zeta}}$.

In addition, if the reference radius is smaller than the radius of the capsule of B, i.e., $r_{A,1} < r_{B,1}$, then there could be other grids that are crossed by the capsule of B. To identify them, B generates additional points as described further. B first computes a local shift $\mathbf{sh_{r_{A,1}}}$ as in Eq. 3.15.

$$\mathbf{sh_{r_{A,1}}} = [r_{A,1} \cdot sin(\frac{\pi}{2} - \Theta_{B,1}), r_{A,1} \cdot sin(\Theta_{B,1})]. \tag{3.15}$$

B uses such a shift to generate additional starting points as $p_{B,tl} + \mathbf{sh_{r_{A,1}}}$ and, for each of them, additional fictitious points $\mathbf{p_{B,tl}} + \mathbf{sh_{r_{A,1}}} + acc$. Note that the computation of the starting points and intermediate points stops at the boundaries of the capsule. All such points are also added to the list $\mathbf{p_{B,\zeta}}$.

Finally, for all the generated points in the list $\mathbf{p_{B,\zeta}}$, B computes the resulted position identifiers $[\tilde{\mathbf{x_B}}, \tilde{\mathbf{y_B}}]$ using Eq. 3.8. B takes all the unique identifiers and delivers them to A.

Identifiers Matching. Finally, A concludes the round $i = 1$ by evaluating if the identifiers of the reference capsule $[\tilde{\mathbf{x_A}}, \tilde{\mathbf{y_A}}]$ match any of the remote identifiers $[\tilde{\mathbf{x_B}}, \tilde{\mathbf{y_B}}]$. If the identifiers do not match, then we are sure that the two trajectories do not collide. Thus, the algorithm terminates with a *No Collision* decision. Otherwise, if any identifiers match, there is a chance of collision between the two trajectories. However, due to the specific construction of the capsules, which led to a *coarse comparison*, there is a chance that trajectories could still not collide.

Incremental Capsule Matching. If there is a match, our algorithm proceeds to the next round $i = 2$ by dividing the colliding trajectories into two partially overlapping subsets, as depicted in Fig. 3.3. We denote this algorithm as *Incremental Capsule Matching*. It is a tree-based algorithm, where in each round, the capsule identifying the colliding trajectories is divided into two subsets. Such sets share only one point, i.e., the last point of the first set and the first point of the second set (see Fig. 3.3). According to the *Incremental Capsule Matching* algorithm, at each round, we compare each capsule of A with each capsule of B. If any capsule of A or B collides with any one of the other parties, we take it to the next round; otherwise, we discard the capsule (and the related points of the trajectory) as not colliding.

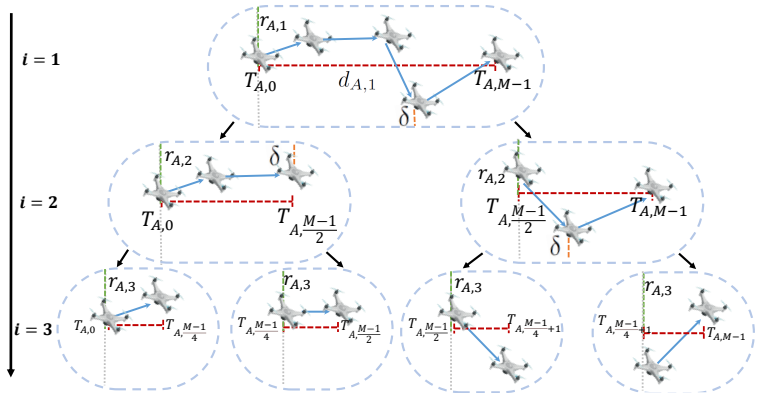

Figure 3.3: Division of trajectory points into smaller capsules according to the *Incremental Capsule Matching* algorithm.

The algorithm terminates when (i) there are no matches between any capsules (No Collision) or (ii) there is a match between capsules (Collision), and there is no set of capsules containing more than two trajectory points, indicating a *Collision event*.

Truncated Mode and Full Mode. The *Incremental Capsule Matching* algorithm features two modes: *Truncated Mode* and *Full Mode*. In the *Truncated Mode*, the *Incremental Capsule Matching* algorithm terminates when the first one of the parties reaches the minimum allowed capsule size for each of the remaining capsules in a specific round, i.e., only two trajectory points. In the *Full Mode*, the algorithm terminates only when both parties reach such a limit. On the one hand, the *Truncated Mode* reduces the number of rounds and, thus, it can potentially reduce the runtime overhead of the protocol while guaranteeing zero false negatives, i.e., no missed collisions. On the other hand, it could provide false positives since the capsules of one party still include more than two trajectory points, leading to an enlarged capsule. We evaluate and discuss such a trade-off in Sec. 3.5. Note that, when working in the *Truncated Mode*, the incremental capsule matching algorithm requires a maximum number of rounds equal to $\lceil * \rceil log_2 N$ $(M \geq N)$. Conversely, the *Full Mode* provides zero false negatives and also zero false positives, i.e., it correctly identifies non-colliding points. However, it requires more rounds, and can potentially incur more overhead. In the worst case, the full mode requires $\lceil * \rceil log_2 M$ $(M \geq N)$ rounds to identify all the colliding points.

We recall that matching between trajectory points does not imply collision. If A and B locations match, timestamps should also match to lead to a collision—the vehicles collide only if they occupy the same location at the same time. Thus, if the output of the *Incremental Capsule Matching* algorithm over the locations is a collision, we also check that the timestamps corresponding to the colliding locations match. If this is the case, we finally declare *collision* between the trajectories. Otherwise, the algorithm outputs *No collision*.

We highlight that, compared to the logic used in [191], our new collision detection logic requires also the responder to build a local capsule. Moreover, on the responder, we use a set of points chosen ad-hoc on the perimeter and inside the capsule to ensure that the

responder identifies all the grids crossed by its local capsule. This amendment ensures that, in all operational conditions, the algorithm has no false negatives.

The algorithm explained above does not take into account any privacy considerations. Indeed, sharing the trajectory identifiers and the timestamps leads A and B to share private data. To overcome the privacy limitation, we merge our collision detection logic with a private set intersection scheme, inspired by the one proposed by Kotzanikolau et al. in [161]. The result is the full privacy-preserving trajectory matching scheme. We discuss the setup phase and runtime phase of the resulting protocol in Secs. 3.3.2 and 3.3.3, respectively.

3.3.2. Setup phase

During the setup phase, both AVs compute cryptography materials necessary for the later *Spatiotemporal Matching* phase. We define the following parameters: (i) Two functions $f, g : G \rightarrow \mathcal{G}$ which map discrete, finite, and low-entropy data G (GNSS locations and timestamp) into equal-size unique values $\mathcal{G}$, representing the corresponding unique identifiers; (ii) a generic hashing function $H(\cdot)$; and (iii) Two prime numbers p'_j and q'_j, used to generate *safe primes* p_j and q_j, i.e., $p_j = 2p'_j + 1$ and $q_j = 2q'_j + 1$. The values of p_j and q_j also define $n_j = p_j \cdot q_j$ and $\phi(n_j) = 4p'_j q'_j$. These values, the hash function $H(\cdot)$ and the modular field n_j are stored on the AV. Note that p_j and q_j are kept secret.

Note: ePPTM relies on the same PSI introduced in Chapter 2 in PRM. We suggest the reader to recall the details of PSI in Sec. 2.3.1. The notation remains unchanged except for two symbols: PRM uses the nonce $r_{A,i}$, whereas ePPTM denotes the corresponding random value by $\xi_{j,i}$.

3.3.3. Spatiotemporal Matching

The *Spatiotemporal Matching* phase of ePPTM is run at runtime, when any two vehicles A and B would like to identify if they collide or not, in a privacy-preserving fashion. This phase includes two sub-phases, namely *Space Trajectory Matching* and *Time Trajectory Matching*, dedicated to identifying matches in the location-related and time-related information of the vehicles. Note that the two vehicles always execute the *Space Trajectory Matching* phase first. Only if there are colliding points, they execute the *Time Trajectory Matching*. This gives the advantage of reducing the computational overhead of the protocol on the AVs. We report below the detailed steps of ePPTM, dividing them into the two phases previously mentioned. Fig. 3.4 provides the protocol flow of the sub-phase *Space Trajectory Matching* and Fig. 3.5 reports the protocol flow of the sub-phase *Time Trajectory Matching*.

1. A and B first establish a secure connection, e.g., using (D)TLS. Then, A and B start the *Space Trajectory Match* sub-phase by exchanging the number of local remaining trajectory points, i.e. M for T_A and N for T_B. We consider $M \geq N$, and thus, A takes the role of the initiator, and B is the responder.

2. At the round $i = 1$, A computes its *reference capsule* $\Pi_{A,1}$ by taking the first trajectory point $T_{A,0}$ and last trajectory point $T_{A,M-1}$ of T_A and applying the logic described in

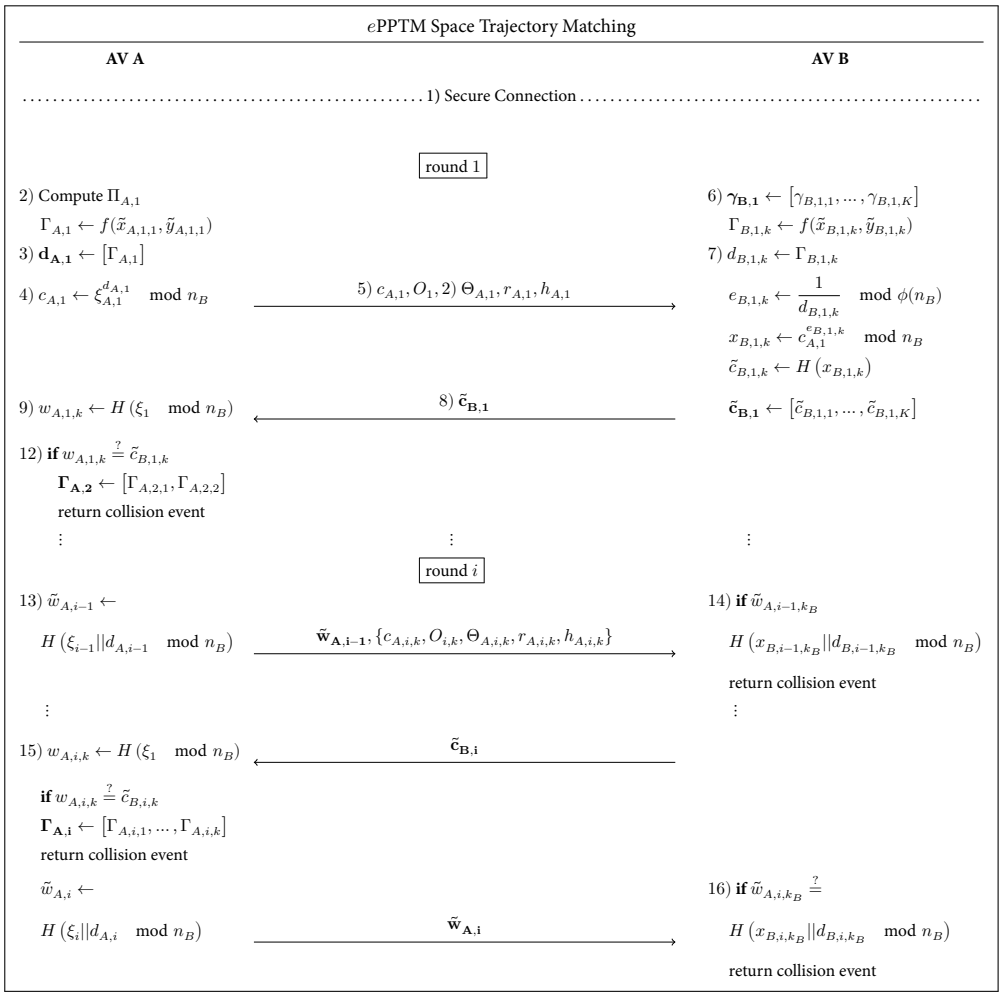

Figure 3.4: Protocol flow of ePPTM - *Space Trajectory Matching* sub-phase.

Sec. 3.3.1. We denote with $\Gamma_{A,1}$ the unique identifier of the capsule $\Pi_{A,1}$, computed through the function f as in Eq. 3.16.

$$\Gamma_{A,1} = f(\tilde{x}_{A,1,1}, \tilde{y}_{A,1,1}) = H(\tilde{\mathbf{x}}_{\mathbf{A},\mathbf{1},\mathbf{1}} || \tilde{\mathbf{y}}_{\mathbf{A},\mathbf{1},\mathbf{1}}) \rightarrow \mathbb{Z}, \tag{3.16}$$

where $\tilde{x}_{A,1,1}$ and $\tilde{y}_{A,1,1}$ are the identifiers of capsule $\Pi_{A,1}$ in 2D (see Eq. 3.8). Note that, per construction, A (initiator) always has a single capsule identifying the reference trajectory T_A. We define r_1, h_1 and Θ_1 as the radius, length, and the angle of the reference capsule $\Pi_{A,1}$, respectively, and we denote with O_1 the random origin point chosen by A.

3. A sets $\mathbf{d}_{\mathbf{A},\mathbf{1}} = \left[\Gamma_{A,1}\right]$.
4. A also extracts a nonce ξ_1 and computes the encrypted challenge $c_{A,1}$ as per Eq. 3.17.

$$c_{A,1} = \xi_{A,1}^{d_{A,1}} \mod n_B. \tag{3.17}$$

5. Then, A delivers a wireless message over the encrypted channel, containing: (i) the encrypted challenge $c_{A,1}$, (ii) the origin point O_1, (iii) the angle $\Theta_{A,1}$ of the reference capsule, (iv) the radius of the capsule $r_{A,1}$, and (v) the length of the capsule $h_{A,1}$.
6. B first uses O_1, $\Theta_{A,1}$, $r_{A,1}$, and $h_{A,1}$ to map its trajectory T_B into the space tessellation provided by A, according to the rationale presented in Sec. 3.3.1. Note that the trajectories of A and B are different, and thus, such a mapping operation performed by B can possibly lead to several grid identifiers. We define $\boldsymbol{\gamma}_{\mathbf{B},\mathbf{1}} = \left[\gamma_{B,1,1}, \dots, \gamma_{B,1,K}\right]$ the set of K cell identifiers obtained by B after mapping its local trajectory T_B. The generic capsule identifier of B at round 1, namely $\gamma_{B,1,k}$, can be computed according to Eq. 3.18, and contains, per construction, at least one point in T_B.

$$\Gamma_{B,1,k} = f(\tilde{x}_{B,1,k}, \tilde{y}_{B,1,k}) = H(\tilde{\mathbf{x}}_{\mathbf{B},\mathbf{1},\mathbf{k}} || \tilde{\mathbf{y}}_{\mathbf{A},\mathbf{1},\mathbf{k}}) \rightarrow \mathbb{Z}. \tag{3.18}$$

7. For each unique $k-th$ cell identifier, B sets $d_{B,1,k} = \Gamma_{B,1,k}$ and computes $e_{B,1,k} = \frac{1}{d_{B,1,k}} \mod \phi(n_B)$ and the encrypted response $x_{B,1,k}$, as per Eq. 3.19, and the digest of each encrypted response $x_{B,1,k}$, namely $\tilde{c}_{B,1,k}$, through the hash function H, as in Eq. 3.20.

$$x_{B,1,k} = c_{A,1}^{e_{B,1,k}} \mod n_B. \tag{3.19}$$

$$\tilde{c}_{B,1,k} = H\left(x_{B,1,k}\right). \tag{3.20}$$

8. B delivers to A the vector of encrypted responses $\tilde{\mathbf{c}}_{\mathbf{B},\mathbf{1}} = \left[\tilde{c}_{B,1,1}, \dots, \tilde{c}_{B,1,K}\right]$.
9. When receiving the message, A compares each encrypted response $\tilde{c}_{B,1,k}$ in $\tilde{\mathbf{c}}_{\mathbf{B},\mathbf{1}}$ with the *check values* $w_{A,1,k}$, obtained via Eq. 3.21.

$$w_{A,1,k} = H\left(\xi_1 \mod n_B\right). \tag{3.21}$$

When $\exists d_{B,1,k}$ s.t. $d_{A,1} = d_{B,1,k}$, then:

$$\begin{aligned} \tilde{c}_{B,1,k} &= H\left(x_{B,1,k}\right) \\ &= H\left(c_{A,1}^{e_{B,1,k}} \mod n_B\right) \\ &= H\left(\left(\xi_1^{d_{A,1}}\right)^{e_{B,1,k}} \mod n_B\right) \\ &= H\left(\xi_1 \mod n_B\right). \end{aligned} \tag{3.22}$$

Therefore, if $w_{A,1,k} \stackrel{?}{=} \tilde{c}_{B,1,k}$, the capsule of A, namely $\Pi_{A,1}$ lies in a cell of the grid where also one of the capsules of B is (see Fig. 3.2). Otherwise, if the identifiers are all different, then $w_{A,1,k} \neq \tilde{c}_{B,1,k}$, $\forall k \in K$, meaning that the set of capsule identifiers of B does not include the (reference) capsule identifier of A.

10. Next, A derives the *encrypted proof of proximity* $\tilde{w}_{A,1}$ using Eq. 3.23.

$$\tilde{w}_{A,1} = H\left(\xi_1 || d_{A,1} \mod n_B\right). \tag{3.23}$$

11. If $w_{A,1,k} \neq \tilde{c}_{B,1,k}$, $\forall k \in K$, the trajectories do not collide. Therefore, A delivers to B only the parameter $\tilde{w}_{A,1}$, allowing B to verify the obtain the same output.

12. If $\exists d_{B,1,k}$ s.t. $d_{A,1} \stackrel{?}{=} d_{B,1,k}$, A initiates the second round $i = 2$ of the protocol by dividing the set of trajectory points into two sets of size $\lceil \frac{M}{2} \rceil$. For each set of points, A generates the capsule and the corresponding identifier, leading to a new set of capsule identifiers $\mathbf{\Gamma_{A,2}} = \left[\Gamma_{A,2,1}, \Gamma_{A,2,2}\right]$.

13. In the generic round i, A executes the operations at the steps (3), (4), and (5) for each capsule $\Gamma_{A,i,k}$. Next, A computes $c_{A,i,k}$ using fresh nonces $\xi_{A,i,k}$, as in Eq. 3.17 and delivers to B $\tilde{\mathbf{w}}_{\mathbf{A,i-1}} = \tilde{w}_{A,1}$ (from the previous round) and a set of tuples $\{c_{A,i,k}, O_{i,k}, \Theta_{A,i,k}, r_{A,i,k}, h_{A,i,k}\}$, representing the new mapping values for B to check further collisions.

14. At message reception, B checks if the cell identifiers computed during the previous round match with the trajectory of A. Thus, $\forall k_B$, B executes Eq. 3.24.

$$\tilde{w}_{A,i-1,k_B} \stackrel{?}{=} H\left(x_{B,i-1,k_B} || d_{B,i-1,k_B} \mod n_B\right), \tag{3.24}$$

where the values of $x_{B,i-1,k_B}$ can be immediately plugged into the equation as being previously computed. If $\nexists k$ satisfying Eq. 3.24, then ePPTM terminates with the output *No Collision*, and the vehicles can continue their mission with no adjustments to the trajectory.

15. If $\exists k$ s.t. Eq. 3.24 holds, B takes the correspondent set of colliding coordinates and, for each k, it executes the operations described at the steps (6) to (8).

16. ePPTM continues iteratively until (i) there are no colliding points, and so the protocol ends with the output *No Collision*, or (ii) one of the two vehicles reaches a maximum capsule size of two trajectory points. In the *Truncated mode*, if still $\exists k$ s.t. Eq. 3.24

holds, this phase of ePPTM ends, and the protocol continues with the *Time Matching Phase* (described below). In the *Full Mode*, this phase of ePPTM continues by letting the other party continue dividing the set of colliding points in halves until also such party reaches a maximum capsule size of two trajectory points.

If the *Space Trajectory Match* sub-phase ends with no collisions, ePPTM also ends with the output *No Collision*. Otherwise, for any colliding points in the trajectory, A and B initiate the *Time Trajectory Match* sub-phase, described below. Note that the *Time Trajectory Match* sub-phase uses a logic similar to the previously-described *Space Trajectory Match* sub-phase, with the notable difference that it applies to (unidimensional) trajectory data.

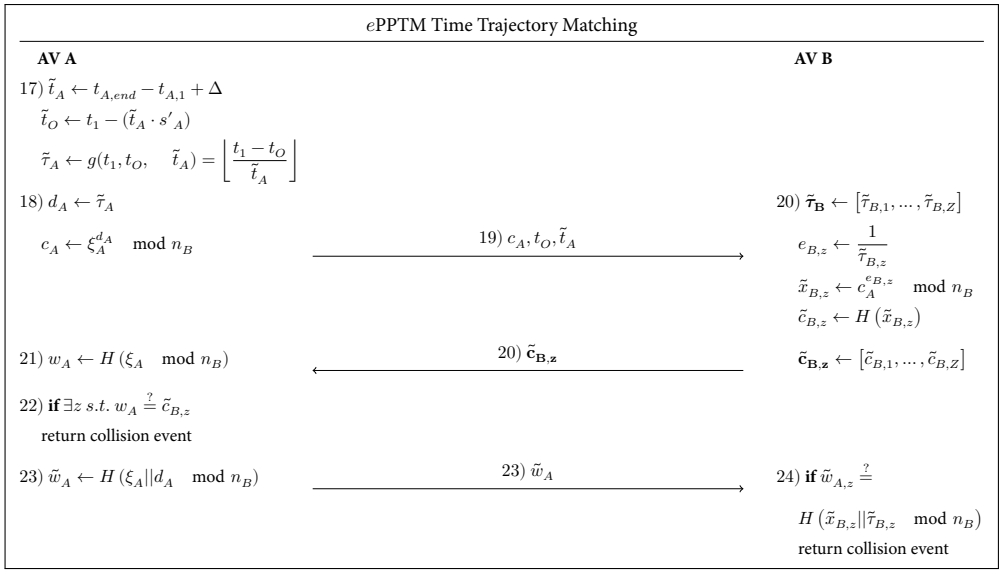

Figure 3.5: Protocol flow of ePPTM - *Time Trajectory Matching* sub-phase.

17. We define $\mathbf{t_A}$ as the vector of timestamps of the colliding points of A as output from the space trajectory match sub-phase. A computes the time difference $\tilde{t}_A = t_{A,end} - t_{A,1} + \Delta$, where $t_{A,1}$ and $t_{A,end}$ are the starting and end timestamp of the first point and last point in the set, respectively, and Δ defines the (configurable) minimum allowed threshold for the time collision. A computes the *origin timestamp* t_O as in Eq. 3.25.

$$t_O = t_1 - (\tilde{t}_A \cdot s'_A), \tag{3.25}$$

where s'_A is a nonce used to randomize the origin timestamp (so as to preserve the privacy of timestamps). Using t_O, A shifts the timestamps and computes the identifier $\tilde{\tau}_A$ by using the function g, as in Eq. 3.26.

$$\tilde{\tau}_A = g(t_1, t_O, \tilde{t}_A) = \left\lfloor \frac{t_1 - t_O}{\tilde{t}_A} \right\rfloor. \tag{3.26}$$

18. Similarly to the *Space Trajectory Match* sub-phase, A sets $d_A = \tilde{\tau}_A$. It extracts a random nonce ξ_A and computes the *encrypted challenge* c_A, as in Eq. 3.27.

$$c_A = \xi_A^{d_A} \mod n_B. \tag{3.27}$$

19. Then, A delivers to B the encrypted challenge c_A, the *origin timestamp* t_O, and the *time division* $\tilde{t}_A$.

20. When receiving the message, B uses the *origin timestamp* t_O to re-scale its local timestamps. The re-scaled values are divided by the *time division* $\tilde{t}_A$ and floored to obtain the local list of time identifiers $\tilde{\boldsymbol{\tau}}_{\mathbf{B}} = \left[\tilde{\tau}_{B,1}, \ldots, \tilde{\tau}_{B,Z}\right]$. For each of them, B generates the parameter $e_{B,z} = \frac{1}{\tilde{\tau}_{B,z}}$ and the *encrypted response* $\tilde{x}_{B,z}$, as in Eq. 3.28.

$$\tilde{x}_{B,z} = c_A^{e_{B,z}} \mod n_B. \tag{3.28}$$

Next, B computes the digest of each encrypted response $\tilde{x}_{B,z}$, namely, $\tilde{c}_{B,z}$, as in Eq. 3.20.

$$\tilde{c}_{B,z} = H\left(\tilde{x}_{B,z}\right). \tag{3.29}$$

Finally, B delivers to A $\tilde{\mathbf{c}}_{\mathbf{B},\mathbf{z}} = \left[\tilde{c}_{B,1}, \ldots, \tilde{c}_{B,Z}\right]$.

21. At the reception of the message, A generates the *check value* w_A as per Eq. 3.23.

$$w_A = H\left(\xi_A \mod n_B\right). \tag{3.30}$$

22. A checks if $\exists z$ s.t. $w_A \stackrel{?}{=} \tilde{c}_{B,z}$. If the check is successful for any $\tilde{c}_{B,z}$, then A and B are going to be *close* (less than the time guard δ) at the same time, thus colliding also in time. Therefore, they can take action to re-adjust their trajectories (see Sec. 3.6 for a discussion about collision resolution). Otherwise, A and B are going to be at the same place but at different time instants, not risking collision. Therefore, ePPTM can safely end with the output *No Collision*.

23. B verifies matches in time, by sending back the *encrypted proof of proximity* $\tilde{w}_A$, computed using Eq. 3.31.

$$\tilde{w}_A = H\left(\xi_A || d_A \mod n_B\right). \tag{3.31}$$

24. To check for time overlap, $\forall z$, B executes Eq. 3.32.

$$\tilde{w}_{A,z} \stackrel{?}{=} H\left(\tilde{x}_{B,z} || \tilde{\tau}_{B,z} \mod n_B\right), \tag{3.32}$$

If $\exists$, z s.t. Eq. 3.32 is satisfied, A and B are in close proximity simultaneously, leading to the output *Collision*. Otherwise, the output is *No Collision*.

We note that ePPTM should be executed completely every time two previously unseen vehicles would like to verify any match of their trajectories in space and time. When two vehicles already executed ePPTM in the past, they could reuse previous values and accelerate new protocol instances. We discuss such an aspect in Sec. 3.6.

3.4. Security analysis

3.4.1. Formal Security Analysis via ProVerif

The most relevant security feature offered by ePPTM is *Trajectory Privacy*. Indeed, ePPTM utilizes privacy-preserving proximity testing, and in particular, a customization of the solution proposed by the authors in [161], to identify precisely collisions in two trajectories without releasing to the other party anything else than the colliding points and timestamps. In line with other works in the literature [235, 213, 211], we use ProVerif to formally verify the security of the single round of our scheme, so confirming that our construction does not reveal any secrets and neither breaks the security of its building blocks. In appendix A, we provide details about ProVerif and the definitions necessary to understand the output shown in Fig. 3.6. Based on our formal definition of ePPTM for a single round, Fig. 3.6 shows the output provided by Proverif. In our case, the variables dA_i and dB_i_k are not exposed to the adversary. They represent the locations $d_{A,i}$ and $d_{B,i,k}$ at round i in the *Space Trajectory sub-phase* of ePPTM (see steps 3 and 7) in Fig. 3.4), and equivalently, dA and $\tilde{\tau}_{B,z}$, in the *Time Trajectory sub-phase*.

As shown in Fig. 3.6, the attacker cannot guess/brute-force such locations and timestamps despite their low entropy in ePPTM at round i. Based on the output in Fig. 3.6, ePPTM keeps changing the location identifiers in every round i while providing strong secrecy. Thus, ProVerif confirms that the single steps of ePPTM achieve the desired properties while ensuring collision discovery. Interested readers can find the source code of ePPTM in ProVerif at [58].

```
Verification summary:
Weak secret dA_i is true.
Weak secret dB_i_k is true.
Query not attacker(dA_i[]) is true.
Query not attacker(dB_i_k[]) is true.
Non-interference dB_i_k is true.
Non-interference dA_i is true.
```

Figure 3.6: Results of the security analysis of ePPTM using ProVerif (code available at [58]).

3.4.2. Privacy Analysis

In the previous section, we formally showed through ProVerif that the *Trajectory Privacy* is guaranteed for a single round of ePPTM. However, the collision detection logic of ePPTM involves multiple steps, and at each step, the communicating entities use trajectory identifiers for a group of trajectory points as private input.

We formally define $p_c(i)$ as the guessing probability at round i of ePPTM by the remote party, i.e., the probability of guessing correctly a location traversed by the remote party by taking advantage of the information exchanged on the communication channel at the round

i of ePPTM. We remark that ePPTM terminates in two cases: (i) no colliding trajectory identifiers are found at the round i, or (ii) a number κ_i of colliding trajectory identifiers are found at the round i, but the protocol stops, e.g., because of its application in the *Truncated Mode* or other energy constraints. We discuss the two cases below in Props. 3.4.1 and 3.4.2, respectively.

Proposition 3.4.1. *Consider that ePPTM terminates at round i due to non-colliding trajectory identifiers. In this case, the probability for an honest-but-curious adversary to guess trajectory identifiers of radius δ traversed by the remote party at round i is $p_c(i) = \frac{\tilde{\kappa}}{2^{(\lceil * \rceil log_2(M)-i)} \cdot (U_i - \xi_i)}$, where $\tilde{k}$ is the number of trajectory identifiers used by the remote party, ξ_i the number of challenges sent by the adversary, U_i the overall number of element identifiers at round i, and M is the total number of trajectory points.*

Proof. Let us assume that ePPTM terminates at round i due to of A and B not having any colliding trajectory identifiers. The probability $p_B(i)$ for an honest-but-curious adversary to guess the correct trajectory identifiers of the remote party at round i follows Eq. 3.33.

$$p_B(i) = \frac{\kappa_i}{U_i - \xi_i}, \tag{3.33}$$

where ξ_i is the number of encrypted challenges sent by A to B at round i, κ_i is the number of encrypted responses sent back by B to A at round i, and U_i is the overall number of available trajectory identifiers at round i. Note that, per construction, $p_B(i) \in [0, 1]$ and, depending on which party takes the role of the responder, the number of encrypted responses could be higher in ePPTM than in the solution in [191] because of the fictitious points that the responder computes. Thus, $p_B(i)$ could be slightly higher in ePPTM than in [191].
We can associate $p_B(i)$ to $p_C(i)$, denoting the probability of guessing a specific location in the trajectory of the target entity. At the round i, the trajectory identifiers of ePPTM identify 2^{MAX-i} distinct trajectory points, being $MAX = \lceil * \rceil log_2(M)$. Also, we recall that, at the last round MAX, the trajectory identifiers (capsules) have a radius δ. Denoting with κ_i the colliding trajectory identifiers at round i and with $\tilde{\kappa}$ the location identifiers in the trajectory of the remote party, we can obtain the probability $p_c(i)$ of guessing a specific trajectory point according to Eq. 3.34.

$$p_c(i) = \frac{\tilde{\kappa}}{2^{(\lceil * \rceil log_2(M)-i)} \cdot (U_i - \xi_i)}. \tag{3.34}$$

□

Proposition 3.4.2. *Consider the scenario where ePPTM finds κ_i colliding element identifiers in the private set of A and B at round i, but the entities stop ePPTM. In this case, the probability for an honest-but-curious adversary to guess one of the $\tilde{k}$ location identifiers possessed by the remote party and used at round i of the protocol is $p_c(i) = \sum_{j=1}^{\kappa_i} \frac{\tilde{k}_j}{2^{(MAX-i)}}$, with $MAX = \lceil * \rceil log_2(M)$ and $\sum_{j=1}^{\kappa_i} \tilde{k}_j = \tilde{k}$.*

Proof. Consider the case where ePPTM identifies κ_i colliding identifiers at round i, and consider that the trajectory identifier of ePPTM at step i represents a group of 2^{MAX-i} identifiers of radius δ, where δ is the safety radius and $MAX = \lceil * \rceil log_2(M)$. We can also

recall that each colliding identifier κ_i includes $\tilde{k}_j$ *occupied trajectory identifiers* of radius δ, and that $\sum_{j=1}^{\kappa_i} \tilde{k}_j = \tilde{k}$. Thus, similarly to the analysis in [191], we can define $p_c(i)$ as in Eq. 3.35.

$$p_c(i) = \sum_{j=1}^{\kappa_i} \frac{\tilde{k}_j}{2^{(MAX-i)}}. \tag{3.35}$$

Thus, the higher the number of colliding identifiers, the higher the probability that the adversary can guess correctly a point of the trajectory of the remote party. □

3.5. Performance Analysis

3.5.1. Proof-of-Concept Implementation

To provide insights into the accuracy and performance of ePPTM, we implemented an actual proof-of-concept using the programming language Python. Compared to the earlier version of ePPTM published in [191], we deploy an actual client-server architecture on relevant hardware devices executing the enhanced version of ePPTM discussed in Sec. 3.3.

To characterize the performance of our solution on heterogeneous devices and networks, we consider two different testbeds, namely *Testbed 1* and *Testbed 2*.

Testbed 1 (T1). We consider two Raspberry PI 3 Model B+ devices [176]. This is a well-known medium-range embedded platform equipped with a Cortex-A53 processor running at 1.4 GHz, 1GB of RAM, and 16 GB of storage, running the OS Raspbian Bullseye 64-bit. As acknowledged by several contributions in the literature such as [95, 144, 210], the computational and bandwidth resources offered by the Raspberry PI are comparable to the ones provided by low-end drones, and there are even commercial drones integrating the Raspberry PI as an on-board CPU [75, 210].

Testbed 2 (T2). We consider one laptop Lenovo Thinkpad P1, equipped with two Intel Core i7-8750H processors running at 2.230 GHz, 32GB of RAM, and 1 TB SDD, running Arch Linux, acting as the responder of ePPTM. For the initiator, we consider one standalone PC DELL Precision-T7600, equipped with 12 Intel Xeon E5-26300 processors running at 2.30GHz,15.5 GB of RAM, and 1.1 TB SDD, running Ubuntu 20.04.6 LTS. This device acts as the initiator of ePPTM. *Testbed 2* allows us to emulate vehicles characterized by more significant computational power, like high-end ones.

For communication purposes, we set up an ad-hoc IEEE 802.11 network working at the operating frequency 2.4 GHz in a regular office environment. This choice is made on purpose to let our proof-of-concept share the same spectrum of other communications and possibly experience interferences and packet losses. This scenario aligns with the expected deployment of ePPTM in crowded urban scenarios and allows us to test our solution in realistic network conditions.

For running our experiments, we (i) enabled remote communication capabilities using communication sockets secured through a TLS connection using the Python package *pyOpenSSL*,

(ii) integrated within ePPTM the PSI building block proposed by the authors in [161], (iii) organized code into several processes, and (iv) used real-world GPS datasets, which we converted into the Cartesian coordinate system.

Dataset. To validate our solution in realistic conditions, we feed our implementation with a dataset of real-world trajectories. We consider the dataset released in [156], and available for download at [202].The dataset includes the real-world trajectories of several taxis in the city of Porto (Portugal), acquired in the period between July 2013 and June 2014.
For our performance assessment, we investigate separately the situations where the trajectories of the AVs collide and do not collide. To this aim, we manually processed the dataset through a self-implemented tool checking if a given pair of trajectories collide or not. Based on the output of such a tool, we defined two different sub-datasets, namely, *Colliding (C)* and *Non-Colliding (NC)*, characterized by colliding and non-colliding trajectory pairs, respectively.

The resulting dataset *C* contains 87 pairs of colliding trajectories, while the dataset *NC* contains 3,228 pairs of non-colliding trajectories. Each trajectory contains a variable number of points, approximately between 3 and 100 trajectory points. The inter-arrival time between any two consecutive trajectory points in every trajectory is exactly 15 seconds, as provided by the reference dataset. Although data augmentation strategies could be used to obtain additional trajectory points, we opted intentionally not to do that, so to consider actual trajectories, characterized by a sparse structure rather than a more high-resolution one. Additionally, we acknowledge that the trajectory data is based on a road-bounded environment of Porto. Although ePPTM is designed for free space trajectories, ePPTM can be applied in rule-bounded environments w.r.t. trajectory data. In this case, the dataset visually allows us to check the correctness of ePPTM on a map and identify edge cases in collisions, which lead to introducing additional operations in ePPTM as stated in Sec. 3.3.1 and illustrated in Fig. 3.2. We note that ePPTM has the capability to identify collisions in free-space and road-bounded environments, whereas in road-bounded environments, the traffic rules generally prevent collisions.

Finally, to enable the experimental comparison of ePPTM with the proposal in [191], we also implemented such a protocol in Python. We deployed both ePPTM and the solution in [191] on the same testbed (T2) to evaluate their performance and accuracy differences on real-world datasets. Our proof-of-concept and the dataset are available on GitHub at [59], allowing interested readers to reproduce our results as well as to use them as a ready-to-use basis for further development.

3.5.2. Accuracy Analysis

We first verify the correctness of ePPTM, i.e., the capability of our protocol to identify correctly all colliding and non-colliding trajectories in the two considered datasets. To this aim, we report in Tab. 3.2 the number of identified collisions and non-collisions between trajectory pairs when ePPTM runs over the two considered datasets, i.e., NC and C, using the Truncated Mode and Full Mode, respectively.

We highlight that the *Full mode* of ePPTM correctly identifies all the colliding trajectories

		ePPTM Truncated Mode		ePPTM Full Mode	
		NC	C	NC	C
Ground Truth	NC	3178	50	3228	0
	C	0	87	0	87

Table 3.2: Accuracy of ePPTM considering actual colliding (C) and non-colliding (NC) trajectories.

and non-colliding trajectories. There are 0 cases where non-colliding trajectories (NC) are identified as colliding (C), namely zero False Positives (FPs) and vice versa, i.e., zero False Negatives (FNs).

The *Truncated Mode* of ePPTM also provides zero FNs, i.e., it never misses possible collisions (zero cases of output NC when the ground truth is C). This finding aligns with our design criteria, as safety always comes first. However, it introduces some FPs, i.e., cases of output C when the ground truth is NC. Such false positives are 50 over 3228, for an overall False Positive Rate (FPR) of only 0.0155% for the considered dataset. Thus, in line with our design criteria, the Truncated Mode of ePPTM trades-off accuracy (FPs) with reduced protocol overhead (see Sec. 3.5.3).

We also take a step further, and we investigate the extent of the *privacy leakage* incurred by the *Truncated Mode* of ePPTM, i.e., the amount of non-colliding trajectory points leaked to the other party. We report the results of our investigation in Tab. 3.3, considering the initiator and responder of ePPTM for the Truncated and Full modes separately.

ePPTM Phases	Initiator		Responder	
	NC	C	NC	C
Space Trajectory	50 (28.7 %)	18 (4.5 %)	48 (53.7%)	6 (8.7%)
Time Trajectory	7 (45.21%)	36 (5.2%)	7 (75.9%)	25 (9.4%)

Table 3.3: Percentage of trajectory points leaked by the *Truncated Mode* of ePPTM for the datasets NC and C.

With the dataset NC, we observe that the initiator leaks on average 28.7% of private trajectory points, less than the responder (53.7%). Although the specific values depend on the used dataset, the general result is consistent with the logic of our protocol. Indeed, in the *Truncated Mode*, when the protocol ends, the initiator is using smaller capsules (made up of 2 trajectory points) than the responder. Accordingly, the probability of leaking information on the initiator is less than on the responder, which is using larger capsules possibly including more trajectory points.

When considering the colliding dataset C, the leakage caused by the *Truncated Mode* is 4.5% on the initiator and 8.7% on the responder. While the general trend mentioned above for the initiator and responder is confirmed, we also notice that the leakage for colliding trajectories is less than for non-colliding trajectories. Indeed, colliding trajectories are more similar; thus, when the truncated mode ends, there are fewer remaining points not fully checked for collisions on the responder. We notice similar trends also considering the *Time Trajectory* sub-phase, as shown in the second row of Tab. 3.3.

We stress that when full accuracy is required, the *Full Mode* of ePPTM guarantees zero FP and zero FN, while fully preserving the privacy of non-colliding trajectory points. Conversely, the Truncated Mode trades off accuracy with protocol overhead (see below).

3.5.3. Protocol Overhead Analysis

In this section, we investigate experimentally the overhead incurred by ePPTM on relevant devices, considering the testbeds T1 and T2 described in Sec. 3.5.1.

Experiment Settings. We evaluate the overhead of ePPTM considering the spatiotemporal matching phase (Sec. 3.3.3) and four relevant metrics, i.e. the execution time, memory footprint (specifically, RAM consumption), bandwidth, and energy consumption on both the initiator and responder devices. For measuring the execution time, we measure the time taken by each protocol round on both the initiator and the responder until the output of the final decision about the collision or non-collision status of the considered input trajectories, in milliseconds. For measuring the RAM consumption, we consider the maximum instantaneous RAM consumption of the relevant process running on the device (in kilobytes), using the Linux tool "/usr/bin/time -v". For measuring the bandwidth overhead, we consider the application bytes generated and delivered by the process of our protocol, using the Python library *sys*, independently of the meta-data added by lower-layer protocols. Finally, for estimating the energy consumption of ePPTM, we use the tool *powertop*, a software-based energy estimation tool. *Powertop* provides an estimate of the power (in Watt) of a running process. We compute the energy consumption of the process by multiplying the power estimate for the execution time, as: $E = P \times \Delta t$, where E is the estimated energy in Joule, P the power (in Watt) estimated by *powertop* and Δt the duration of the spatiotemporal matching phase of ePPTM.

For all tests, we fixed the value of some specific protocol parameters. In particular, we set the collision threshold $\delta = 50\ meters$, the GNSS inaccuracy $\sigma = 50\ meters$ and the circle steps of the responder $\eta = \frac{\pi}{20}$, in line with realistic values and use cases for the deployment of ePPTM.

We ran our tests considering the datasets NC and C, separately. We report all results using the Empirical Cumulative Distribution Function (ECDF) of the particular investigated metric. We recall that, given a generic stochastic process X, $ECDF(x) = y$ implies that $p(X \leq x) = y$. For convenience of reporting, for the testbed T2, we consider the PC as the initiator and the laptop as the responder.

Non-colliding Trajectories. Fig. 3.7(a), (b), (c), (d), (e), and (f) report the performance of ePPTM over the non-colliding (NC) dataset in terms of execution time on the initiator (a) and responder (b), bandwidth overhead (c), RAM consumption (d) and energy consumption (e) (f). We compare the performance between initiator and responder in Fig. 3.7 (c), (d), and (e), since we include additional operations on the responder side to improve the accuracy of ePPTM as stated in Sec. 3.3.1 and illustrated in Fig. 3.2. We further discuss the results between initiator and responder in T1 in Sec. 3.6. From Fig. 3.7 (a) and (b), we observe that, with this dataset, there is a small performance gain due to the introduction of the *Truncated Mode*. The quantile 0.95 of the execution time of the initiator in T2 is 1.596 s for the *Full*

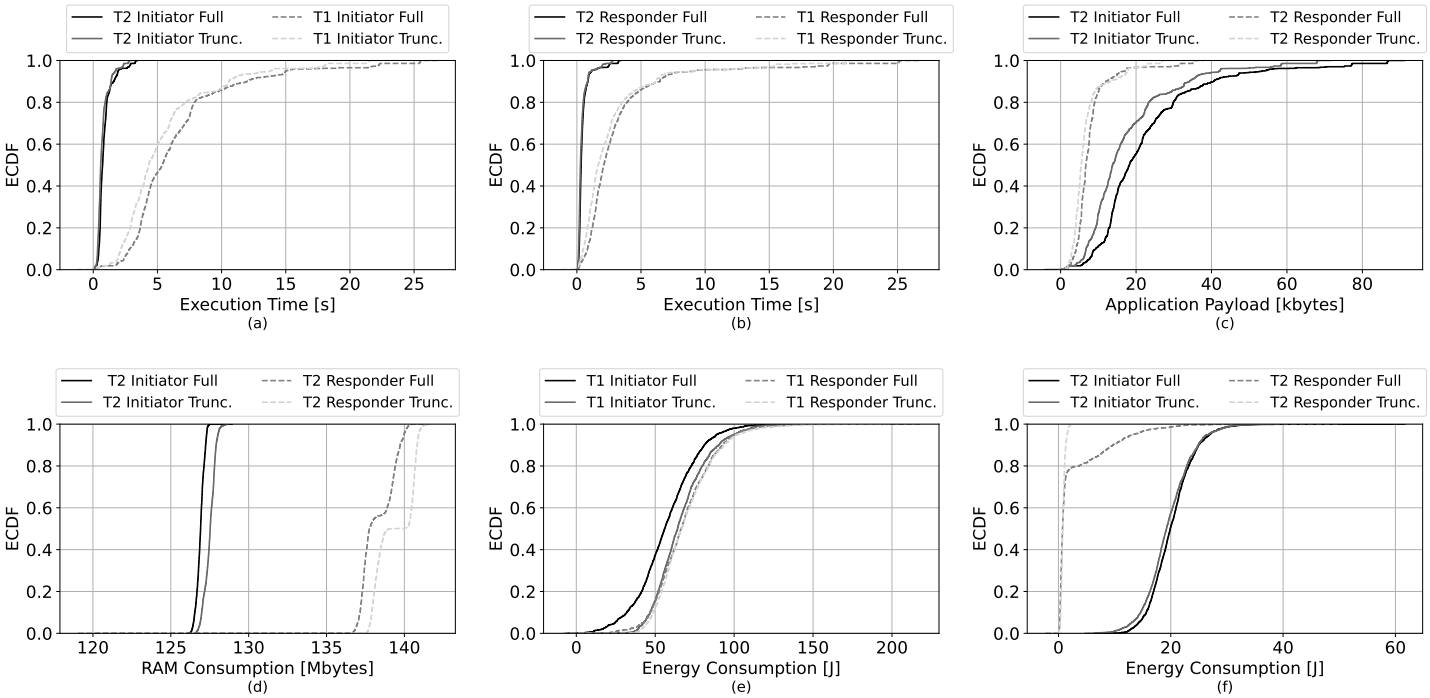

Figure 3.7: Performance evaluation of ePPTM on the non-colliding (NC) dataset, in terms of (a) execution time on the initiator, (b) execution time on the responder, (c) application payload on testbed T2, (d) RAM consumption on testbed T2, (e) energy consumption on testbed T1, and (f) energy consumption on testbed T2.

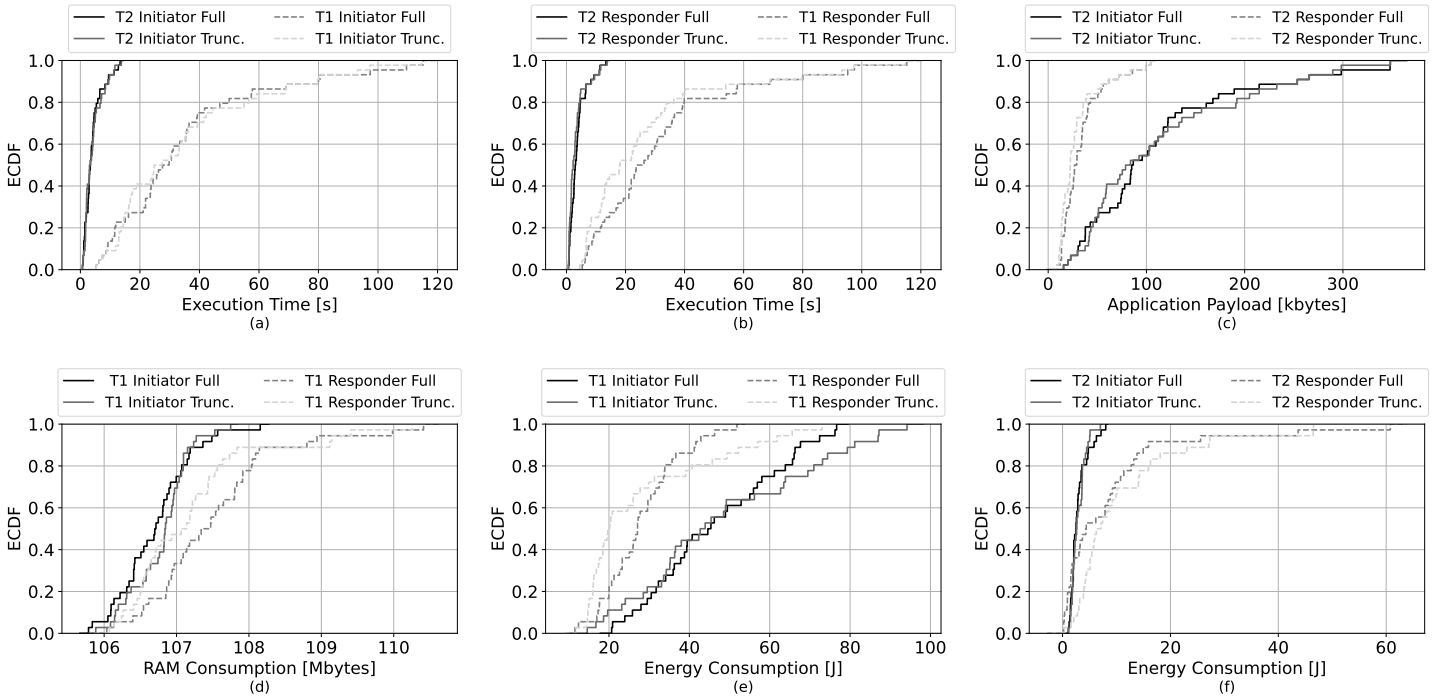

Figure 3.8: Performance evaluation of ePPTM on the colliding (C) dataset, in terms of (a) execution time on the initiator, (b) execution time on the responder, (c) application payload on testbed T2, (d) RAM consumption on testbed T1, (e) energy consumption on testbed T1, and (f) energy consumption on testbed T2.

mode and 1.386 seconds for the *Truncated mode*, while for T1, in the same setup, the quantile 0.95 of the execution time on the initiator is 11.989 seconds in *Full mode* and 10.556 s in *Truncated mode*. Thus, the truncated mode is 13.16% and 11, 95% faster than the *Full Mode* in T1 and T2, respectively. On the responder, the performances of the two modes of ePPTM are more similar. Indeed, the quantile 0.95 of the execution time of the responder in T2 is 7.730 seconds in *Full mode* and 7.885 seconds in *Truncated mode*, while for T1, in the same setup, the quantile 0.95 of the execution time on the responder is 65.780 seconds in *Full mode* and 64.423 seconds in *Truncated mode*. Such variability is due to the larger overhead incurred by the responder compared to the initiator. Indeed, in line with the results shown later for the collision (C) dataset, the responder takes more time than the initiator due to the additional computations (e.g., fictitious trajectory points) and bandwidth overhead (see Fig. 3.8 (c)). We provide more insights on these findings in Sec. 3.6.

The results for the RAM consumption reported in Fig. 3.7 (d) show that the initiator requires much less RAM than the responder, e.g., the quantile 0.95 of the RAM of the initiator in T2 with Full mode is 127.297 Mbytes, while for the responder, in the same setup, this is 139.972 Mbytes. There is a negligible difference between the truncated and full modes, i.e., 128.024 Mbytes of RAM consumption in T2 with Truncated mode, indicating that RAM consumption is not affected by the specific operational mode of ePPTM.

As for the energy consumption, Fig. 3.7 (f) shows that for T2, in over 30% of the cases, the energy consumed on the responder for running ePPTM in the *Full Mode* is significantly higher than the energy required for the *Truncated Mode*. Indeed, the quantile 0.95 of the energy consumption on the responder is 1.550 J when using the *Truncated Mode* and 13.200 J when using the *Full Mode*, which represents an increase of over 751%. This result clearly shows the advantage of using the *Truncated Mode* over the *Full Mode* for some trajectory pairs. For the initiator, the difference in the energy consumption between the two modes is less evident, i.e., 26.617 J and 26.923 J for the *Truncated Mode* and *Full Mode*, respectively. We remark that, for the specific case of T2, we cannot compare the energy consumption of the initiator to the one of the responder, since they are different types of devices. This is possible only for T1, as shown in Fig. 3.7 (e). Note that the energy consumption of the *Full Mode* of ePPTM in such a testbed is nearly equivalent on the initiator and responder, with the quantile 0.95 for both of them being 88.695 J and 100.053 J, respectively. The energy consumption of the *Truncated Mode* of ePPTM on the initiator and responder are nearly equivalent. Indeed, the quantile 0.95 for the initiator is 92.767 J and for the responder is 101.825 J. We highlight that the initiator in *Full Mode* takes less energy consumption than the *Truncated Mode*. Indeed, the *Truncated Mode* finds collision in the *space* domain while in time it does not find any collision, so requiring slightly more energy.

Colliding Trajectories. Fig. 3.8(a), (b), (c), (d), (e), and (f) report the performance of ePPTM over the colliding (C) dataset in terms of execution time on the initiator (a) and responder (b), bandwidth overhead (c), RAM consumption (d) and energy consumption (e) (f). Similarly, as in non-colliding trajectories, Fig. 3.8 (c), (d), and (e) compare the performance between the initiator and responder based on the above-mentioned statement.

Considering the execution time, the devices in T1 are faster than the ones in T2, consistently with the computational capabilities of the involved devices. Considering the quantile

0.95 of the ECDF, the initiator in T2 takes 11.080 s to execute ePPTM in *Full mode* and 11.030 s in *Truncated mode*, while the initiator in T1 takes 94.870 seconds for the *Full mode* and 91.217 seconds for the *Truncated mode* (see Fig. 3.8(a)). Similarly for the responder, in *T1*, it takes 92.964 seconds in *Full mode* and 91.238 seconds in *Truncated mode* (quantiles 0.95), while in T2 it takes 10.850 seconds in *Full mode* and 10.996 seconds in the *Truncated mode* (quantiles 0.95), (see Fig. 3.8(b)). We first notice that, independently from the testbed, the execution time of the initiator is slightly higher than that of the responder, due to the higher number of messages (and information) delivered by the initiator. We can confirm this finding through the results of the bandwidth analysis, reporting, e.g., a quantile 0.95 of overhead for the responder in full mode (T1) of 83.739 KB and 293.460 KB for the initiator. We also see that the *Truncated Mode* reduces more significantly the execution time on the responder rather than on the initiator, especially during the *Time Trajectory Matching*. An interesting result we observe with dataset C (as opposed to the NC dataset) is that there is no big difference between the *Truncated Mode* and *Full Mode* on both testbeds. Indeed, when trajectories collide, trajectories are more similar. Thus, any computational overhead saved by the *Truncated Mode* during the Space Trajectory Match sub-phase is compensated by a similar (sometimes, even higher) overhead incurred during the Time Trajectory Match sub-phase, not leading to reduced execution time.

Considering RAM consumption, the initiator requires slightly less RAM than the responder, e.g., the quantile 0.95 of the RAM of the initiator in T1 in *Full Mode* is 107.508 Mbytes, while for the responder, in the same setup, this is 109.201 Mbytes. There is a negligible difference between the two modes, i.e., 107.336 Mbytes of RAM consumption in T1 with the *Truncated Mode*.

Finally, as for the energy consumption, Fig. 3.8 (e) shows that, considering the testbed *T1*, the responder consumes less energy than the initiator. Also, in line with the analysis of the other performance metrics, the *Truncated Mode* does not decrease the overhead of the protocol compared to the *Full Mode*. With the *Full Mode*, the initiator in the testbed T1 consumes up to 73.373 J (quantile 0.95), while the responder in the same setup consumes up to 43.717 J (quantile 0.95). At the same time, in the same setup, considering the *Truncated Mode*, the initiator and the responder consume 87.0256 J and 62.769 J (quantiles 0.95), respectively. As for the energy consumption, Fig. 3.8 (f) reports the energy consumption of the initiator and responder in both modes of ePPTM considering the testbed *T2*. Recall that the energy consumption for the initiator and the responder cannot be compared when considering this testbed, since they run on different hardware and the power estimation is calibrated differently. Despite such limitation, our tests show that the difference between the energy consumption of the *Full Mode* and the *Truncated Mode* is negligible. The initiator in the testbed T2, with the *Full Mode*, consumes up to 6.488 J (quantile 0.95), while it consumes up to 5.045 J with the *Truncated Mode*. Similarly, the responder in the same setup consumes up to 30.109 J (quantile 0.95) when it runs in *Full Mode* and 32.093 J (quantiles 0.95) when it runs in the *Truncated Mode*, confirming our claims above.

Comparison. Finally, to position ePPTM in the current state of the art and quantify its improvements, we compare its performance against the protocol proposed in [191]. We first consider the accuracy of the protocols, i.e., the capability of correctly identifying colliding and non-colliding trajectories. We report the results of our investigation in Tab. 3.4, con-

		PPTM Full Mode C		ePPTM Full Mode C	
Ground Truth	NC	7	8%	0	**0%**
	C	80	92%	87	**100%**

Table 3.4: Accuracy of Full modes of PPTM [191] and ePPTM considering actual colliding (C).

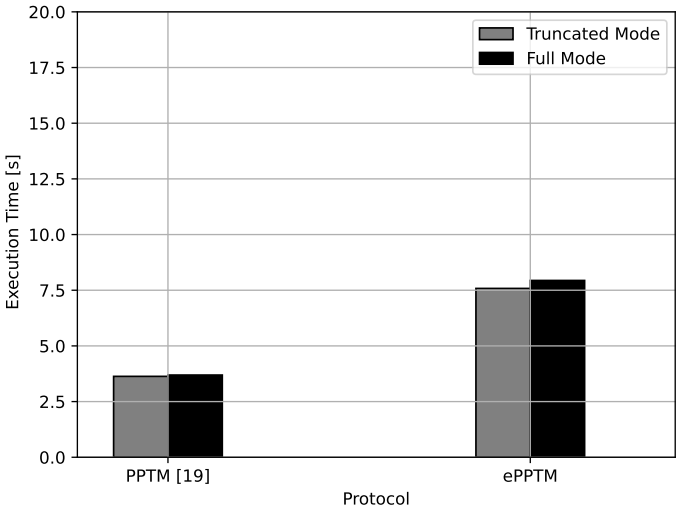

Figure 3.9: Overall execution time of the solution in [191] and ePPTM on T2.

sidering the dataset C and the *Full Mode*. Note that the accuracy of the solution in [191] is much worse than ePPTM when it comes to more sparse trajectories, in line with the features of real-world trajectories. The solution proposed in [191] achieves an overall accuracy in identifying colliding trajectories of only 0.916, not being able to identify all colliding trajectories. Such accuracy becomes even lower considering the colliding points on sparse trajectories. Specifically, the solution proposed in [191] reports 7 FNs, i.e., collisions erroneously classified as non-collisions, representing 8.44% of our dataset. Such misclassifications are due to the inaccuracy of the assumptions made by the proposal in [191] on the density of the trajectory points—which are sparse in the real-world dataset, instead. In such scenarios, our solution achieves zero FNs and zero FPs, while posing a reasonable and limited toll on the involved AVs. We also evaluate the execution time of both solutions, considering the same setup and the testbed T2. We report the results of our investigation in Fig. 3.9. The solution in [191] is quicker than ePPTM, as the quantile 0.95 of the truncated mode and full mode is 8.545 s and 9.379 s, respectively, compared to 21.984 s and 22.124 s of ePPTM in the truncated mode and full mode, respectively. Such a difference is mostly due to the computations occurring on the responder, e.g., the computation of the fictitious trajectory

points. The solution in [191] is very efficient when trajectory points are dense. However, as shown in Tab. 3.4, it does not provide the accuracy (and thus, safety) required by real-world applications for sparse trajectories, not being safe to use in most realistic situations.

3.6. Discussion

We briefly discuss below some relevant aspects emerging from our performance assessment, as well as some practical considerations that a system administrator can apply when deploying ePPTM in the real world.

Colliding vs Non-Colliding Trajectories. Comparing the results for non-colliding trajectories (Fig. 3.7) and colliding trajectories (Fig. 3.8), we notice that ePPTM requires much less overhead in terms of execution time and energy consumption when the trajectories of the involved devices do not collide. Indeed, when trajectories do not collide and are significantly different, the *Incremental Capsule Matching* algorithm of ePPTM identifies the absence of collisions earlier, so requiring much fewer rounds to complete. At the same time, we notice that the *Truncated Mode* brings more significant savings in terms of execution time and energy consumption when trajectories do not collide, rather than when they collide. Indeed, when using the *Truncated Mode* with colliding trajectories, more trajectory points are marked as *colliding* at the end of the *Space Trajectory Match* sub-phase and are provided to the *Time Trajectory Match* sub-phase. Such points cause the *Time Trajectory Match* sub-phase to generate an additional overhead, which outweighs the performance gains obtained earlier. In many cases, the resulting overhead of the *Time Trajectory Match* sub-phase makes the overall execution time and energy consumption of ePPTM higher in the *Truncated Mode* than in the *Full Mode*. In Fig. 3.7 and Fig. 3.8, we compare the initiator and responder in *T1* to understand the computational overhead of the operations on the responder side as illustrated in Fig. 3.2. The additional operations that improve the accuracy require more RAM consumption, as shown in Fig. 3.7(d) and Fig. 3.8 (d). Meanwhile, the energy consumption is higher for the initiator than for the responder in the colliding dataset, which may be the cause of keeping the communication socket open for multiple rounds. Whereas, in the non-colliding dataset, the difference between the initiator and responder is not drastic, which implies that, in case more rounds need to be executed, the more energy the initiator requires to send packets to the responder, which is shown in Fig. 3.7(c).

Trade-off between Privacy and Overhead. Enhanced trajectory privacy comes at the cost of higher protocol overhead, especially when the two trajectories have many points in common. Let us consider the example shown in Fig. 3.10, considering the two trajectories identified by green and red points, respectively. We notice that there are many collisions among the points of the two trajectories (precisely, 46 on the initiator and 45 on the responder), leading to the necessity of narrowing down the comparison among them to the level of the single points to identify them all with certainty. For this particular case, we break down in Tab. 3.5 the time required to execute ePPTM until the round i and the additional privacy leakage of the protocol in case the initiator would stop the protocol at that round. We first notice that the first rounds of ePPTM take less time than the latest rounds. Indeed, in earlier rounds, there are fewer capsule identifiers to compare, leading to less overhead than latest rounds.

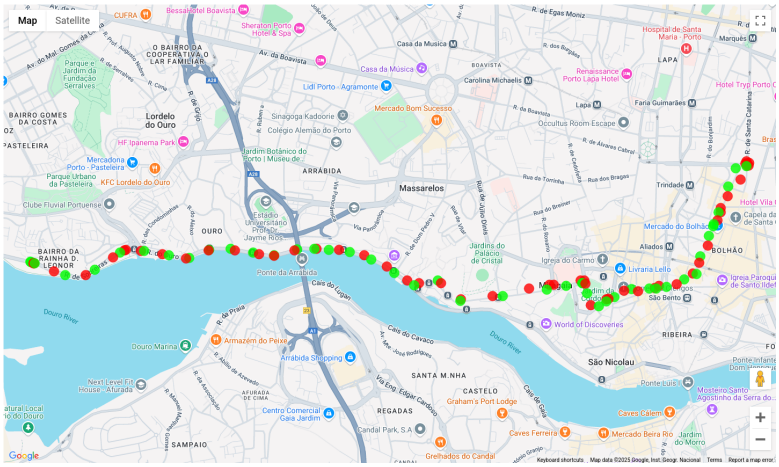

Figure 3.10: Example of colliding trajectories whose comparison could require significant time. Red and green dots refer to the trajectories of the initiator and responder, respectively.

For this specific case, stopping the protocol at any round does not cause any additional privacy leakage to the initiator. Indeed, since the two trajectories are very similar to each other, all colliding points are immediately identified (at round 1), and they do not change in the following rounds. Although this is not always the case, this example is indicative of the ample possibilities that involved devices have to stop protocol execution earlier. For example, when the initiator realizes that the local set of colliding points is not changing after the first four rounds of the protocol, it could stop execution earlier, by saving over 90% of the computation time (from 10.278 s to 1.007 s). This choice comes at the risk of disclosing more points than the ones colliding and giving up *full trajectory privacy*. In this case, there is no additional privacy leakage, but it is not always the case. In real-world scenarios, where the energy on AVs could be at budget, the vehicle could define an *energy threshold*, i.e., the maximum amount of energy that can be spent per protocol execution. Note that the vehicles do not know in advance how many trajectory points do not collide with the other party — this depends on the relationship between the two trajectories, which are always kept private. Stopping the protocol based on a local (earlier) termination criterium allows for preserving energy, computational overhead, memory, and communication while potentially incurring just a little privacy leakage.

Multiple Protocol Instances among same AVs. Consider the case where two AVs which already ran ePPTM in the past (independently from the outcome) come again in wireless visibility after some time. If local and remote trajectory points are unchanged since the last run of ePPTM with the remote device, they could simply skip protocol execution. If any points are changed, in principle, the devices should run ePPTM once again from scratch. However, we notice that a change in one or more points of the trajectory does not necessarily lead to the change of the boundaries of the capsules used in the *Incremental Capsule Matching* protocol, especially in the earlier rounds. For example, consider Fig. 3.11, where the dark trajectory represents the trajectory of a vehicle at a time t, and the dark grey trajec-

Round identifier	Time to complete up to Round N	Additional Leakage
1	0.0901 s	0%
2	0.167 s	0%
3	0.470 s	0%
4	1.007 s	0%
5	2.416 s	0%
6	5.559 s	0%
7	10.278 s	0%

Table 3.5: Time taken to complete ePPTM until round i with the two trajectories in Fig. 3.10, and percentage of additional trajectory points leaked (false positives).

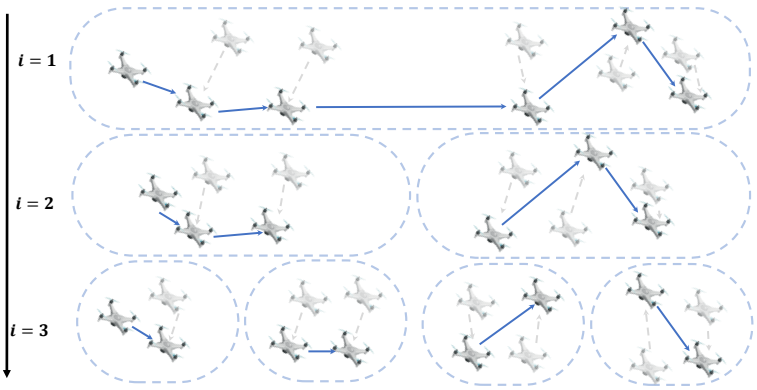

Figure 3.11: Example of change of trajectory points (light grey trajectory) compared to the earlier one (dark trajectory), which does not generate a change in the capsules created during the earlier phases of the *Incremental Capsule Matching* algorithm ($i = 1$, $i = 2$ and $i = 3$), but only at later phases ($i > 3$).

tory represents the trajectory of the same vehicle after some time, e.g., $t + T$. We notice that the changes in point coordinates do not affect the capsules created at earlier rounds ($i = 1$, $i = 2$, and $i = 3$) but only the capsules created at later rounds ($i > 3$). Thus, the involved entities can save the data related to the previous ePPTM instance with a specific remote device and reuse them without modifications, saving computational and energy resources at the cost of some memory overhead. When changes in the trajectories are slight and the trajectories are different enough, a (relatively small) change in the trajectory points does not result in additional overhead. Conversely, when new trajectory points introduce significant changes in the overall shape of the trajectory, the two AVs should run ePPTM once again from scratch.

Direction Disclosure. Although ePPTM can provide full trajectory privacy, it does not protect the information about the direction of the travel of the initiator. Indeed, such information is encoded in the value of the angle Θ disclosed by the initiator at each round and is immediately available to the responder. We do not see any relevant real-world use case where this information leads to significant privacy leakage, without any location information associated. However, if the protection of travel direction is required, a way to protect it

is to use spheres rather than capsules, using a sphere radius equal to the larger value between the capsule radius r and the capsule length h. However, using spheres rather than capsules introduces false positives and, thus, a possible privacy leakage on the trajectory points disclosed to the remote party.

Scalability. As discussed in Sec. 3.2.1, our system model considers two AVs which would like to discover spatiotemporal collisions in their trajectories without disclosing any other information. Applying ePPTM in a context with multiple autonomous vehicles would require a linear overhead, as every time the protocol would have to be run once again. We see opportunities to reduce the overhead significantly. For example, if the initiator has to run the protocol at the same time with N non-colluding parties, it could re-use the *encrypted trajectory identifiers* computed at each protocol round, with no privacy risks when N is significantly smaller than the size of the space used for generating nonces. We will investigate this in our future research.

Collision Resolution. Finally, we highlight that ePPTM is meant to detect collisions among AVs, while collision resolution is, in general, out of scope. To mention a possible resolution strategy, once collision has been detected through ePPTM, the vehicles can solve the situation by choosing new trajectory points to replace the colliding ones and ensuring that the new points are different and do not lead to collisions. A reasonable solution is to agree to such new points with the other party (no privacy), since safety takes priority over privacy. When privacy takes priority, the two entities can run ePPTM once again only on the newly chosen trajectory points to verify that they do not collide anymore. However, we stress that collision resolution is out of the scope of this manuscript.

3.7. Related Work

In this chapter, we tackle the problem of privacy-preserving trajectory matching on AVs, i.e., the identification of any possible collision among planned trajectories of two vehicles. Such a problem shares similarities with *collision detection* and *collision avoidance*, but also notable differences. Collision detection and avoidance schemes discover incumbent collisions between two vehicles based on their current location and (an estimate of) future movements. This objective can usually be achieved through sensors and various (communication) technologies integrated onboard, e.g. cameras, microphones, LIDARs, and RADARs. However, such technologies only optimize route planning and improve logistic efficiency, and do not achieve trajectory matching. In addition, in this chapter we would like to identify intersections in a privacy-preserving fashion, i.e., disclosing to the other party only colliding trajectory points and time. This objective has been considered only by a few contributions, as summarized in Tab. 3.6.

Overall, many different PETs and helping vector reduction schemes have been used in the literature for providing privacy to trajectory data. We see PETs like OT ([64]), Garbled Circuits (GCs) ([234]), HE ([88]), secret sharing ([67]), k-anonymity ([207]), and PSI ([191, 204, 214]). We notice that only a few approaches are dedicated to privacy-preserving matching of the overall trajectory of the moving entities, i.e., [64, 67, 88, 191, 205, 243]. Other approaches only consider the current location of the devices, or a limited portion of the fu-

Table 3.6: Qualitative comparison of privacy-preserving trajectory matching schemes. The symbol ● indicates that the feature is fulfilled, while the symbol ○ indicates that the feature is not fulfilled.

Contribution	Cryptography Technique	Full Trajectory Matching	Spatiotemporal Matching	No TTP Online at Runtime	Applicable with Complex Trajectories	Accurate with Sparse Trajectories
Ye et al. [243]	l-diversity	●	○	○	●	●
Svaigen et al. [207]	k-Anonym.	○	○	○	○	○
Dehghan et al. [64]	Gröbner basis + OT	●	●	●	○	○
Nkenyereye et al. [159]	CLSC + BP	○	○	○	○	○
Liang et al. [247]	CP-ABE	○	○	○	○	○
Wei-jiang et al. [234]	GC	○	○	●	○	○
Frikken et al. [88]	HE	●	○	●	○	○
Anushka et al. [67]	SSS	●	○	●	○	○
Tedeschi et al. [213]	El-Gamal	○	●	●	○	○
Sun et al. [205]	PSI+BF	●	○	●	○	○
Tedeschi et al. [214]	PSI	○	○	●	○	○
Sciancalepore et al. [191]	PSI	●	●	●	●	○
ePPTM	PSI	●	●	●	●	●

ture path based on such a current location, such as [213]. Many of the proposals in the literature mostly consider the location of the devices when aiming to identify collisions, with no considerations about time. This is the case, e.g., of the proposal by the authors in [67, 88, 159, 205, 207, 214, 234, 243, 247]. This occurs either because the proposed scheme is conceived to identify collisions in the present moment, or because the vehicles are assumed to be synchronized, which is almost never the case. We also notice that some proposals, such as the ones in [159, 207, 243, 247], rely on TTPs available online at runtime for trajectory matching. Although such TTPs could be easily accessible via the public Internet in many cases, AVs often operate in environments characterized by intermittent or scarce Internet connectivity. In those environments, the mentioned proposals cannot be used reliably. Some solutions for full trajectory matching, such as [64], assume the vehicles do not take complicated trajectories, but mostly follow paths described through polynominals. This is clearly a limiting assumption, which does not apply for vehicles traveling over the air (drones) or sea (ships), as well as vehicles in crowded urban areas. Finally, as mentioned in the introduction and demonstrated experimentally in Sec. 3.5, our previous solution proposed in [191] exhibits reduced performance when trajectory data are sparse, i.e., available with coarse granularity. Thus, in such conditions, it could generate false negatives, i.e., missed collisions, threatening the safety of the involved vehicles.

In this context, the ePPTM protocol proposed in this contribution extends and improves our previous conference paper in [191] by providing a fully privacy-preserving mechanism guaranteeing zero false negatives (i.e., no risks of collisions) even with sparse trajectories, independently from the level of details of the available trajectories. This is achieved through an improved detection logic, relying on a new strategy for computing capsules on the responder side.

3.8. Conclusion

In this chapter, we presented ePPTM, an accurate and energy-aware solution for privacy-preserving spatiotemporal trajectory matching on autonomous vehicles. ePPTM efficiently

combines two dedicated algorithms, i.e., the *Incremental Capsule Matching* algorithm, detecting collisions based on capsules generated over various consecutive trajectory points, and a privacy-preserving proximity testing algorithm, specifically dedicated to detecting co-locations while preserving the privacy of trajectory points. We presented two modes of ePPTM, i.e., the *Truncated Mode* and *Full Mode*, with the former potentially allowing to decrease computational overhead at the expense of little false positives, while always ensuring zero false negatives (no missed collisions). We implemented an actual proof-of-concept of ePPTM, released the code open source, and used such proof-of-concept to experimentally test the accuracy and overhead of ePPTM using actual vehicles' trajectories on two test environments containing multiple heterogeneous devices. Our performance evaluation demonstrates that ePPTM provides 100% accuracy in *Full Mode*, and it can be computationally feasible and energy-friendly on modern autonomous vehicles especially when trajectories have just a few colliding points. We also demonstrate the enhanced accuracy of our solution compared to the most relevant competing solution available in the literature, at the cost of potentially higher overhead. Future work will cover more complex scenarios with several AVs and the development of a fully geometrical approach, so as to evaluate further the pros and cons compared to our approach.

II

Selective Disclosure in Attestation Protocols

4

Remote Attestation with Constrained Disclosure

This chapter is inspired by [78]:
M. Eckel, **D.R. George**, B. Grohmann, and C. Krauß
Remote Attestation with Constrained Disclosure
2023 Annual Computer Security Applications Conference (ACSAC)

Remote attestation based on the Trusted Platform Module (TPM) enables verification of the authenticity and integrity of software running on a computer system or IoT devices. However, as briefly outlined in Chapter 1.1.3, traditional binary remote attestation requires the disclosure of all details about the executed software from all vendors, e.g., proprietary code and sensitive configuration details, posing confidentiality concerns. To solve privacy and intellectual property concerns and to address **SRQ2**, in this chapter, we introduce a privacy-preserving remote attestation approach that allows executing traditional binary remote attestation while selectively disclosing software details and providing *input privacy*.

4.1. Introduction

Ensuring computer system trustworthiness is crucial for system security. Trusted Computing, remote attestation, and measured boot are established technologies for verifying software integrity and authenticity. Measured boot computes deterministic hash values to measure the integrity of all loaded software components on a target system, the attester Measurements are stored in an event log in the kernel space of the operating system (OS), securely anchored in a TPM [227] using a cryptographic folding hash. The Linux IMA extends measured boot to the OS and running applications. A verifier can use remote attestation to verify claimed software on the attester.

Most software systems today use software from different sources and vendors, and compart-

mentalization — e.g., containerization or virtualization—is a widely used security practice. This enables developers to bundle their software and all its dependencies into a singular package that can be deployed and executed on different computing environments. Furthermore, system administrators can utilize security policies to safeguard the host system and other software from potentially malfunctioning or malicious compartments.

However, traditional binary remote attestation faces several issues:

1. Measuring and reporting all events from all (sub) components in the same place makes it hard for a remote verifier to disentangle individual (sub)component log entries, especially with virtual machines (VMs) and containers.

2. Software vendors are unwilling to reveal file names or hash values of various binary versions and files included in a software package for privacy and intellectual property reasons. Passive attackers can infer running software versions based on hash values, enabling targeted attacks based on known vulnerabilities. Only vendors know which file names and hash values are valid and trusted, making them responsible for vouching for corresponding log entries' validity.

In this chapter, we present a novel approach called Remote Attestation with Constrained Disclosure (RACD), which allows for traditional binary remote attestation to mask and selectively disclose event log entries. We achieve this by extending the Linux IMA concept and adding a NIZK proof based on Schnorr signatures. Our RACD approach complies with the third principle of remote attestation—constrained disclosure, which states that a target should be able to enforce policies governing which measurements are sent to each verifier. Our contributions in this chapter are:

1. Introducing our RACD approach, which enables constrained disclosure of event log entries for binary remote attestation.

2. Providing a security and privacy analysis, including formal verification with ProVerif, to ensure that our solution does not compromise existing security properties.

3. Developing a PoC implementation of our RACD approach, based on the Linux IMA concept, and making the source code publicly available.

4. Demonstrating the feasibility of our approach through performance measurements based on our PoC implementation.

The remainder of this chapter is organized as follows: In Section 4.2 provides an overview of Trusted Computing and remote attestation. In Section 4.3, we present our system model, use cases, and our threat model. We outline requirements for our proposed approach in Section 4.4. Section 4.5 details our RACD concept. In Section 4.6 we provide the implementation details. To justify the efficacy of our solution in terms of security and privacy, we evaluate it in Section 4.7. Performance measurements and results of the formal verification with ProVerif are also discussed in this section. We analyze related work in Section 4.8. In Section 4.9, we conclude the chapter and suggest potential future directions for our research.

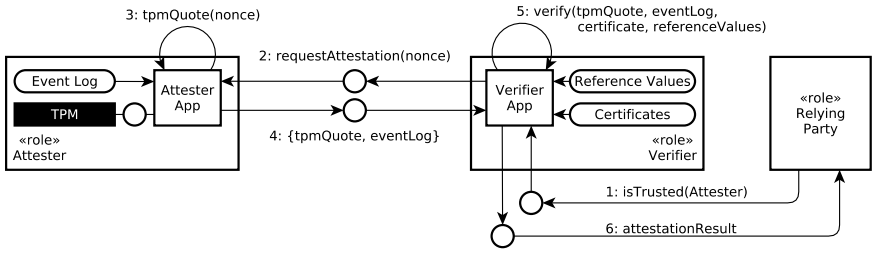

Figure 4.1: Traditional remote attestation process with the roles Attester, Verifier, and Relying Party (cf. IETF RATS [30]).

4.2. Background

Our RACD approach relies on a TPM and utilizes a modified measured boot process and remote attestation to ensure security. Measured boot, as defined by the Trusted Computing Group (TCG), enables a system to maintain an integrity-protected event log of all executed boot components, including the BIOS or UEFI, bootloader, and OS kernel [135, 37]. Each log entry identifies a component, e.g., "UEFI" or "Linux Kernel", along with a cryptographic hash digest known as the *measurement* of the software binary.

To protect the log entries, they are anchored in a tamper-evident manner in a TPM with multiple Platform Configuration Registers (PCRs). PCRs implement a cryptographic folding hash function called *extend*, and they only allow data to be read from or extended to the PCR. It is not possible to set a PCR to an arbitrary value.

In our RACD approach, the TPM is utilized due to its robust resistance against physical attacks, making tampering impractical for attackers. Being passive, every component must have the TPM measurement functionality integrated into its logic. Thus,

1. components execute their logic,
2. measure the next component in the boot order,
3. append the measurement to the boot event log and the TPM, and
4. pass control to the next component.

It is crucial that components are measured before execution, to ensure potentially malicious components are recorded before they become active. At system power-on, the first component to activate is the Core Root of Trust for Measurement (CRTM), which is typically immutable and must be trusted implicitly.

In a remote attestation process, a target system, the attester, provides verifiable evidence to a remote verifier (see Figure 4.1). Based on this evidence, the verifier appraises whether it is trustworthy, i.e., runs only authentic software. For this purpose, remote attestation relies on measured boot and the produced boot event log. When a communication partner, also known as a *relying party* in IETF Remote Attestation Procedures (RATS) terminology [30], needs to determine the trustworthiness of the attester, it requests an attestation from the

verifier (Figure 4.1, step 1). The verifier requests an attestation from the attester (step 2) by providing a random nonce that is used for freshness and to counter replay attacks. With a *TPM Quote* operation (step 3), the attester instructs the TPM to create a digital signature over its internal state, i.e., its PCRs, incorporating the nonce. The key that is used to sign the internal state of the TPM is only known to the TPM. The attester provides the TPM Quote and the event log as cryptographically signed evidence of its operational state to the verifier (step 4). The verifier assesses the validity of the digital signature (step 5) by using the corresponding public key or digital certificate. Then, it compares the entries in the event log against a whitelist that contains *known good* component hashes. Finally (step 6), the verifier communicates the attestation result securely to the relying party, indicating whether the attester is trustworthy or not.

Linux IMA extends measured boot into the OS [252, 251]. Thereby, it measures and logs all native application binaries *before* they are loaded into memory for execution, irrevocably anchoring them in the TPM. Further, all files accessed by user *root* are measured [186, 185]. This way, IMA augments the remote attestation process with the IMA log, providing measurements of OS binaries and files.

4.3. System Model

In this section, we use the terminology from the IETF RATS architecture [30] to describe the components in our system model. We focus on the roles, their interactions, and involved data (artifacts) to describe relevant processes. Involved roles are 1. the *attester*, that produces evidence to be appraised by the verifier, 2. the *verifier*, who appraises the validity of evidence from the attester and produces attestation results, 3. the *relying party*, that consumes attestation results from the verifier. Involved artifacts are 1. *evidence*, which are (digitally signed) claims about the platform configuration, including measurements of software binaries and files, such as TPM quotes and event logs, 2. the *attestation result*, that is produced by the verifier and that includes information about the trustworthiness of the attester, 3. the *reference values*, that the verifier uses as a whitelist of known good measurements of software binaries and files to appraise evidence.

Relying parties must ensure the integrity of a target system (attester) before entrusting it with the computation of sensitive or confidential data. For that purpose, relying parties consume attestation results from a verifier to determine the trustworthiness of an attester. We adapt the traditional remote attestation architecture from Figure 4.1 and introduce the role of partial verifiers, which appraise masked event logs with only partially disclosed entries (cf. Figure 4.2). In this way, we can partition the event log and disclose only log entries that are associated with a particular third party. To mask the event log and disclose only chosen entries, we use non-deterministic hashes based on Schnorr signatures. The original responsibility of the (main) verifier is distributed among several partial verifiers. The main verifier's new task is to assess if every entry in the event log has been verified by at least one partial verifier. This is because we assume that a system can only be considered trustworthy if all entries in the event log have been verified with a positive result, and no log entries remain unchecked.

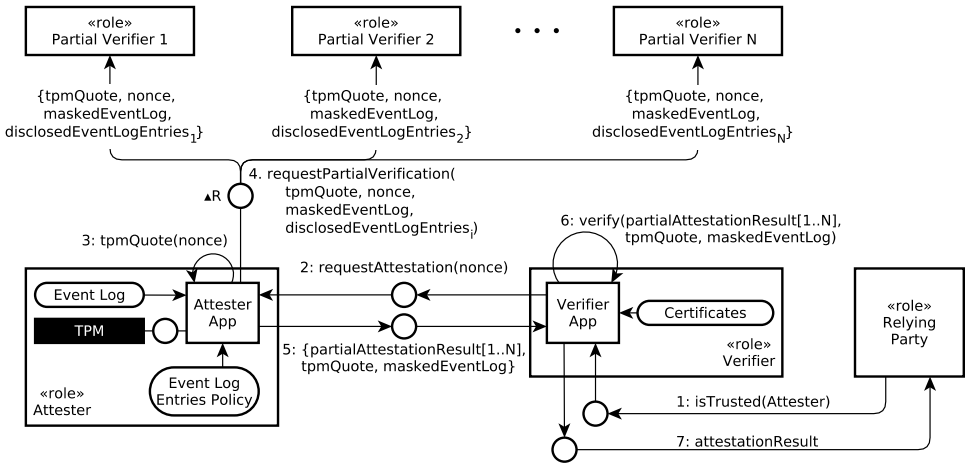

Figure 4.2: Remote Attestation with Constrained Disclosure (RACD) process with the roles Attester, Verifier, Partial Verifier, and Relying Party (cf. IETF RATS [30]).

Whenever a relying party needs to know if the attester is *trusted*, it contacts the verifier (cf. Figure 4.2, step 1). The verifier requests an attestation from the attester (step 2), which performs a *TPM Quote* operation (step 3). Up to this point there is no difference to the traditional remote attestation. However, instead of returning the TPM quote and the event log to the verifier, the attester now requests partial verifications from the partial verifiers (step 4). For this purpose, the attester sends the TPM quote, the nonce, the masked event log (as created by our adapted measurement process, using non-deterministic hashes), and the disclosed event log entries to the partial verifiers. Based on an *event log entries policy*, the attester knows which entries must be appraised by which partial verifier. The event log entries policy constitutes a lookup table, and may be implemented as one or more files, or a database. This lookup table can, for instance, be installed and maintained by the system operator, or distributed as part of the installed software packages, containers, or VMs.

Each partial verifier 1. verifies the TPM Quote signature, 2. appraises the integrity of the masked event log against the TPM quote, 3. appraises the disclosed event log entries based on our NIZK proof, and 4. matches the disclosed log entries to known-good reference values (whitelist). As a result, partial verifiers reply to the attester with a signed *partial attestation result*, containing the nonce and a map of matched entries (the corresponding non-deterministic hashes) from the masked event log, associated with an indicator whether the entry is authentic/trusted.

After receiving all partial attestation results, the attester forwards them to the main verifier, together with the TPM quote and the masked event log (step 5). In step 6, the main verifier 1. verifies the TPM Quote signature and the nonce, 2. validates the masked event log against the TPM quote to verify its integrity, 3. verifies the signature of all partial attestation results (using trusted certificates), and then 4. checks if each entry in the masked event log is hit by at least one (trusted) entry in the partial attestation results. As a response to the relying party (step 7), the main verifier sends a signed attestation result about the trustworthiness of the

attester, exactly as in the traditional remote attestation process.

Note that in our RACD approach, the main verifier no longer needs to maintain reference values. In contrast, traditional remote attestation requires maintaining a whitelist, which discloses all running events on the attester system. Further, note that the RACD approach is still based on roles, i.e., attester, verifier (split into partial verifiers and main verifier), and relying party. One entity may implement several roles, e.g., a system with a TEE can implement both a (co-located) verifier in a TEE and an attester in the Rich Execution Environment (REE).

With US President's Executive Order (EO) 14028 on "Improving the Nation's Cybersecurity" [85], the Executive Office of the President mandates software vendors to provide customers with Software Bills of Materials (SBOMs) [217]. A SBOM is a manifest for a software package and contains information such as file paths, hashes, and metadata supplied as part of the software package in a machine-readable format. They are vendor-specific and can perfectly serve as lookup tables for the event log entries policy. Moreover, SBOMs can serve as reference values for a verifier in a remote attestation process. Microsoft [68, 17, 153] and Adobe [182], among others, ship their products with SBOMs.

4.3.1. Use Cases

Our RACD approach has several real-world applications, ranging from IoT and other embedded Linux-based systems to desktop PCs and even cloud scenarios. The integrity and authenticity of the software is verified by several provider-specific partial verifiers, each one responsible for verifying a set of software components, represented as entries in the measurement log. With our RACD approach, we are able to disclose only entries that belong to a partial verifier, blinding all others. For instance, a system with the OS Ubuntu (Canonical), Microsoft Teams, and JetBrains IntelliJ IDEA installed, would require partial verifiers from Canonical, Microsoft, and JetBrains. This motivates us to provide a scheme with capabilities of traditional remote attention while providing privacy with constrained disclosure.

As long as there is software from multiple sources or providers involved, including containers and VMs, RACD is applicable. This applies to more or less managed systems, such as home energy management systems and network equipment (routers, switches, etc.) as well as interactive systems where users are allowed to install apps, such as automotive head unit systems, Linux-based smartphones like Android, and cloud appliances. In the following, we consider three concrete use cases.

Use Case 1: Charging of AVs

An AV (e.g., UAVs, cars, boats, to name a few) wants to recharge the battery at a charging station and, in addition to the usual authentication with username and password—must confirm that its system is running the latest and uncorrupted version of the charging software. For this purpose, the charging station requests an attestation from the customer system. The AV system acts as an attester that triggers a TPM quote, and requests attestation results from partial verifiers for the installed software. In this case, the charging station itself is also a partial verifier, as it is the provider of the charging software, and consequently has the authority

to appraise the relevant disclosed event log entries. The charging station fulfills the roles of the relying party, the main verifier, and (a) partial verifier.

Use Case 2: Android Smartphone and VPN

An employee wants to use his or her smartphone to access his or her workplace virtual private network (VPN). In addition to the 802.1X and IPsec authentication methods, the VPN gateway requires proof that only authentic software is running on the smartphone, so as not to risk espionage or targeted attacks by malicious apps on the smartphone. For this purpose, the VPN gateway requests an attestation from the smartphone. The smartphone implements the role of an attester, that triggers a TPM quote, and requests attestation results from partial verifiers, e.g., app stores and the smartphone vendor, for the installed apps and system software. The VPN gateway acts as verifier, and grants access to the VPN based on the trustworthiness of the attester, i.e., the smartphone.

Since TPMs are not available per se in mobile phones, the TCG provides the TPM 2.0 Mobile Reference Architecture Specification [222] and the TPM 2.0 Mobile Common Profile [223] that describe how to implement a TPM 2.0 in software in a TEE as available in mobile phones. Further, Chakraborty et al. [49] describe a TPM 2.0 based on Subscriber Identity Module (SIM) cards: "simTPM: User-centric TPM for Mobile Devices".

Use Case 3: Containerized Edge Node

Containerization in edge computing involves packaging applications and their dependencies into containers for deployment on edge devices. Containers provide a consistent environment, ensuring applications run uniformly across diverse edge infrastructures. They offer efficiency, portability, and scalability benefits. In edge scenarios, containers are often managed using orchestration tools, allowing for automated deployment, scaling, and operations of application containers across clusters of edge devices. This setup streamlines application updates, minimizes overhead, and tackles the unique challenges of edge computing, like intermittent connectivity and limited resources. Containers share the same underlying OS kernel, which lets Linux IMA within that kernel "see" and measure containerized applications. Unlike hypervisor-based virtualization that emulates an entire machine with its guest OS and kernel, the host OS kernel remains unaware of applications running in the guest OS.

Security is crucial in edge containerization, as vulnerabilities could compromise the host system or expose sensitive data, necessitating robust isolation and container management practices. Using our RACD approach, edge nodes can selectively verify the trustworthiness of containers by disclosing only log details specific to each container. RACD allows all containers to log into a single TPM PCR. With the event log entries policy—that holds information about containerized software—, RACD can disentangle and partition the Linux IMA event log, unmasking a separate set of entries for each of the containers without revealing entries from other containers or software.

In general, our use cases could benefit from TEE technologies such as Intel Software Guard Extensions (SGX) [120] or AMD Secure Encrypted Virtualization (SEV) [5]. However, these technologies are only available on Intel or AMD machines, not on mobile phones. Further,

Intel discontinued SGX technology for Core processors [220, 175] in favor of Trust Domain Extensions (TDX) [121].

4.3.2. Threat Model

In traditional remote attestation, an attester reveals the entire list of running software and verifiers know about all software running on attester systems. A (partial) verifier being an *HbC* adversary, can leverage this information and query the Common Vulnerabilities and Exposures (CVE) database to identify and exploit vulnerable software. The HbC verifier will follow the attestation protocol, without deviating from the protocol's procedure while trying to learn as much information as possible. Even though the attester sends only (deterministic) hashes of binaries along with the file path, an HbC verifier can use or produce a reverse lookup table to match up the hashes to corresponding binaries. Consequently, the HbC verifier has the ability of knowing if attesters run versions of software with unpatched vulnerabilities to compromise their systems by launching targeted attacks. In the real-world scenario, it would lead to a threat in network equipment that complies with the Bell-LaPadula integrity model, as addressed in TCG Trusted Attestation Protocol (TAP) specifications [224, 225].

The Internet Engineering Task Force (IETF) Supply Chain Integrity, Transparency, and Trust (SCITT) working group addresses EO 14028 requirements further and features transparency of evidence, including remote attestation data and event logs. According to the requirements, the traditional remote attestation does not provide user privacy or vendor privacy. Hence, based on our use case 1 (see Section 4.3.1), the bank knows about all installed programs on the target system (user privacy issue). In addition, the partial verifiers are capable of understanding the programs of other partial verifiers (vendor privacy issue).

4.4. Requirements

Our goal is to introduce the technical means to enable constrained disclosure for TPM-based remote attestation. Based on our system model (cf. Section 4.3), and especially our threat model (cf. Section 4.3.2), we define the following security requirements:

- R_1^S Integrity and authenticity of the attester's operational state must be assured. A TPM-anchored event log of all loaded software must be maintained.
- R_2^S Confidential communication must be established between attester and (partial) verifiers in order to prevent *passive man-in-the-middle* attacks.
- R_3^S Mutual authentication must take place between attester and (main/partial) verifier to ensure the identity of each party.
- R_4^S A suitable elliptic curve must be used that guarantees no information is leaked to attackers. It must be efficient and have advantageous security properties for the integration with the RACD protocol based on a NIZK proof.
- R_5^S The attester must be able to (securely) disclose any subset of event log entries to several

partial verifiers in a remote attestation process, while maintaining the existing (TPM-rooted) security properties integrity and authenticity (cf. [57]).

R_6^S Partial verifiers must not be able to collude with each other, nor must they know each other. Otherwise, they would have the power to merge constrained disclosures.

4.5. Remote Attestation with Constrained Disclosure

In this section, we introduce our RACD concept that enables binary remote attestation to selectively disclose event log entries from the attester to partial verifiers, as described in Section 4.3. Section 4.5.1 outlines our adapted measurement process. In Section 4.5.2, we describe our adapted remote attestation process. We use the notation depicted in Table 4.1.

Table 4.1: Notation Summary.

Notation	Description
$h_T(\cdot)$	Template hash function
$H(\cdot)$	Cryptographic hash function
$\varphi(\cdot)$	Injective function
$h_{eventhash}(\cdot)$	Generates an *event hash*
$\mathcal{X}$	Template hash domain
$\mathcal{Y}$	Integer domain
g	Generator of cyclic group G
x_i	i-th element of the template hash domain
r_i	Randomly chosen value for i-th element in $\mathcal{X}$
v_i	Randomly chosen value for x_i from the integer domain $\mathbb{Z}$
g_i	Generator of the i-th element
t_i	Computed scalar for the i-th element/entry
c_i	Computed challenge for the i-th element/entry
(c_i, s_i)	Computed scalars as pairs of the i-th element
t_i'	Computed scalar from the partial verifier for the i-th element/entry
c_i'	Computed challenge from the partial verifier for the i-th element/entry
L	Prime order group of Curve25519

4.5.1. Measurement Process

We adapt the IMA measurement process as well as the binary remote attestation process to allow selective disclosure of IMA log entries. For that purpose, we replace the deterministic hashes in the log with non-deterministic hashes based on a NIZK proof (Schnorr signature) per log entry. This allows us to disclose only corresponding entries to a partial verifier during remote attestation, while keeping intact the TPM-based integrity protection of the event log as a whole.

The measurement process of our RACD approach follows the same initial steps as the measured boot sequence. The chain of trust starts with the immutable Root of Trust for Measure-

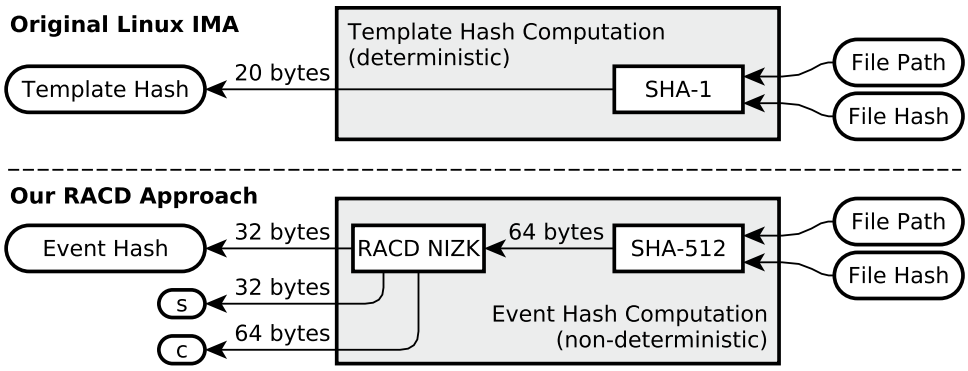

Figure 4.3: Computation of the *template hash* from Linux IMA and the *event hash* from our RACD approach.

4

ment (RTM) and continues up to the OS. The difference in our approach lies in the Linux IMA measurement process. The original implementation of IMA measures software using a configurable deterministic hash algorithm, e.g., SHA-256, resulting in a file hash (cf. diagram at the top of Figure 4.3). The file hash and the file path of the measured file are hashed together with SHA-1, resulting in the *template hash*, according to IMA terminology. This *template hash* is then used to *extend* the configured TPM PCR (by default PCR 10).

The deterministic *template hash* of a software binary is overly informative. An attacker can deduce details about a specific binary using just the *template hash*, given that software binaries are typically stored in known file paths and a limited number of versions exist with their corresponding hashes. To avoid this issue, we employ a non-deterministic hash, called *event hash*, in our RACD method. The term *event* aligns with TCG terminology. A diagram of the *event hash* computation is shown at the bottom of Figure 4.3.

The event hash basically is the cryptographically blinded *template hash*, constructed by applying a NIZK signing process known as Schnorr signature [105, 34]. First, the *template hash* function is defined as $h_T : \mathcal{X} \to \mathcal{Y}$ (we use SHA-512). Let $\mathcal{X}$ be a set of *template hashes* and function h_T transforms them into the range $\mathcal{Y}$. In order to generate the *event hash*, a generator g of a (cyclic) group G needs to be selected (generator g in the case of this work is the base point of an appropriate elliptic curve over a finite field). Second, the function for the *event hash* is defined as $h_{eventhash} : \mathbb{Z} \times \mathcal{X} \to G$, which is computed as

$$h_{eventhash}(r, x) := g^{r\varphi(h_T(x))}. \tag{4.1}$$

The first step of the computation is to blind the *template hash* by generating a random integer $r \in \mathbb{Z}$. x is an element of the *template hash* domain ($x \in \mathcal{X}$). The injective function $\varphi : \mathcal{Y} \to \mathbb{Z}$ maps $\mathcal{Y}$ into the integer domain $\mathbb{Z}$. The *event hash* (instead of the *template hash*) is stored in the IMA log and extended into the TPM PCR. Note: The scalar r is not saved in the IMA log.

PCR	Event Hash	IMA Template	File Hash	File Path	c	s
10	c4456…331	ima-cd	65727…12b	boot_aggregate	b4612…1ac	3cbd2…a47
10	73c9b…eff	ima-cd	153f3…c95	/lib64/ld-linux-x86-64.so.2	9ce45…2bf	feb14…476
10	9b0ed…c09	ima-cd	4ebaf…c9a	/lib/x86_64-linux-gnu/libc.so.6	2bb78…100	2199e…fd5
10	fef56…71b	ima-cd	64743…f69	/bin/dash	5234d…edd	06069…e68
10	ee599…667	ima-cd	fc66b…cef	/bin/mkdir	de567…4bd	7fc90…a2b
10	089c4…4bf	ima-cd	2f43e…699	/bin/ln	7d400…031	6b1a7…e90
10	78bd2…14c	ima-cd	e46fd…b48	/bin/mount	067d…93d	d7fc6…70c
⋮	⋮	⋮	⋮	⋮	⋮	⋮

Table 4.2: Constrained disclosure enhancements to the IMA log ($\text{eventlog}_{\text{ima-cd}}$)

Non-Interactive Zero-Knowledge Proof Generation

Once the *event hash* is created, the attester must produce a (non-interactive) "proof of knowledge" based on the well-known Schnorr signature and Fiat-Shamir heuristic, which is then presented to the partial verifier during the remote attestation process. The NIZK proof must be generated for each measured binary in the IMA log. The attester (IMA) computes for every measured entry i a generator $g_i := g^{\varphi(h_T(x_i))}$. Next, it chooses a random integer $v_i \in \mathbb{Z}$ and computes the value $t_i := g_i^{v_i}$. Afterwards, the challenge $c_i := H(g_i, t_i, h_{eventhash}(r_i, x_i))$ is generated, where H is a cryptographic hash function. Finally, the attester computes the scalar $s_i := v_i - c_i r_i \pmod{|G|}$. The pair (c_i, s_i) is the "extra data" (cf. Figure 4.3) for each log entry that is necessary for the partial verifier to verify the *event hash* since it has zero knowledge of the key r_i. The computational steps are based on the standard Schnorr NIZK proof over an elliptic curve [105].

Adapted IMA Log for RACD

We introduce a new IMA log format where we replace the column *template hash* with *event hash* to prevent confusion, since the *event hash* is the cryptographically blinded *template hash*. The *IMA Template* column holds our new IMA template identifier *ima-cd* ("constrained disclosure"). Again, for each entry i in the IMA log, an element $r_i \in \mathbb{Z}$ is randomly chosen and used to compute the hash value $h_{eventhash}(r_i, x_i)$. x_i, an element of the domain $\mathcal{X}$ of *template hashes*, is a concatenation of *File Hash* and *File Path* for each entry i in the IMA log (the integer i is the index of a log entry). Besides, the columns c and s are known as *extra data* (cf. Table 4.2). The pair (c_i, s_i) is necessary for the partial verifier to verify the Schnorr signature (the "proof of knowledge").

4.5.2. Remote Attestation Process

Figure 4.4 visualizes our RACD process with focus on the partial verifiers. The (main) verifier sends an attestation request to the attester, including a nonce for freshness and against replay attacks . The attester performs a TPM quote and reads the event log. Then, the attester requests partial verifications of the event log from all partial verifiers, passing the TPM quote, the nonce, the masked event log—i.e., the entire *Event Hash* column $\Pi_{(\text{eventHash})}(\text{eventLog}_{\text{ima-cd}})$

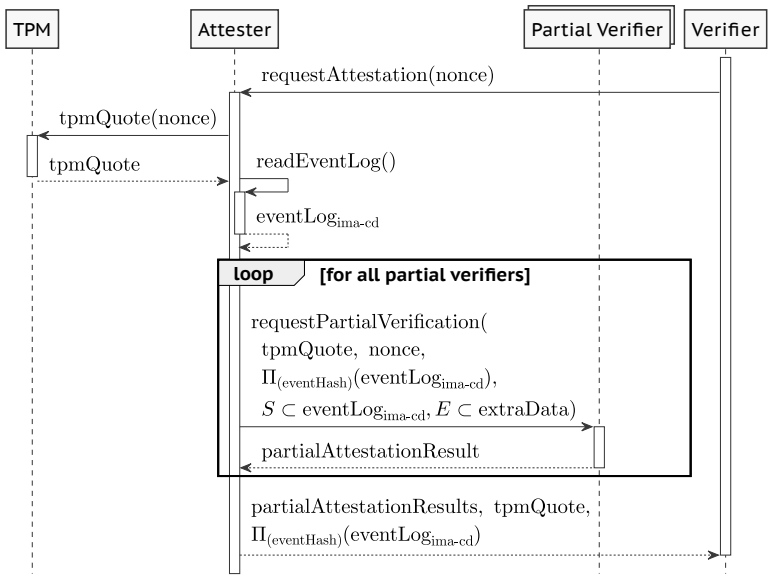

Figure 4.4: Remote Attestation with Constrained Disclosure process.

— as well as the corresponding disclosed log entries $S \subset \text{eventLog}_{\text{ima-cd}}$ and $E \subset \text{extraData}$. $E \subset \text{extraData}$ defines the subset of corresponding pairs (c_i, s_i) of the disclosed log entries. Based on the *event log entries policy*, the attester knows which event log entries must be verified by which partial verifier. Note that the *Event Hash* column contains only the non-deterministic event hashes. Partial verifiers run through the following verification process:

1. *Verify the overall event log integrity* of the masked event log by simulating the TPM PCR *extend* operation over all (non-deterministic) *event hashes*, thus, recomputing the expected TPM PCR value. Then, the partial verifier compares the recomputed PCR value against the actual one from the TPM quote. Only if they match, the log is considered.

2. *Verify the log entry integrity* of all disclosed entries by verifying the Schnorr signature. For this purpose, the partial verifier computes the generator g_i for each of the corresponding entries, which is the same as in the *measurement procedure*. Further, the scalar $t'_i := g_i^{s_i} \cdot h_{eventhash}(r_i, x_i)^{c_i}$ is computed, which is used in the cryptographic hash function H in order to compute $c'_i := H(g_i, t'_i, h_{eventhash}(r_i, x_i))$. Note that the corresponding *event hash*, c_i, and s_i can be directly read from the log entry. Finally, the partial verifier accepts the proof of knowledge, if and only if $c_i = c'_i$. This verification process allows the attester to prove to the partial verifier that the *event hash* is the *masked template hash*, without revealing the key. By applying this procedure, the partial verifier has zero knowledge of the integer r_i. Figure 4.5 illustrates in mathematical notation the adapted version of the NIZK proof [34, 105].

3. *Appraise the content of log entries* by matching the *File Hash* and *File Path* of each log

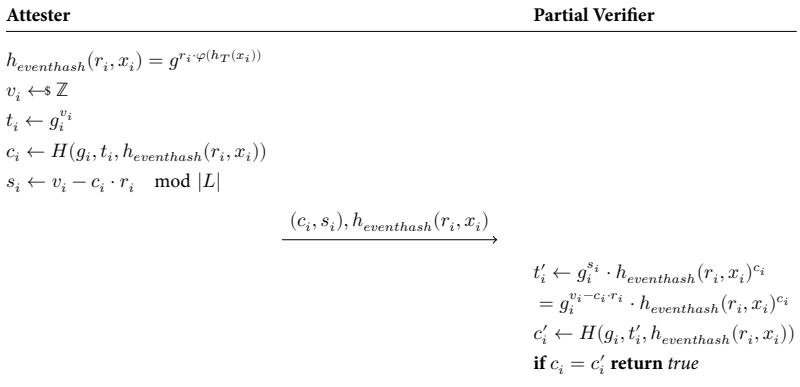

Figure 4.5: Adapted Schnorr Non-Interactive Zero-Knowledge Proof.

4

entry with known-good reference values from a whitelist.

Partial verifiers return a signed partial attestation result to the attester. This includes the nonce and a list of verified entries in the form $(eventhash, result)$. The *eventhash* corresponds to a disclosed log entry and the *result* is either *trusted* or *not trusted*. Finally, the attester replies to the verifier with all partial attestation results, the TPM quote, and the masked event log $\Pi_{(\text{eventHash})}(\text{eventLog}_{\text{ima-cd}})$.

The (main) verifier appraises all received partial attestation results. Only if attestation results for *all* entries in the masked event log are present and *trusted*, the entire attester system is considered trusted. The attestation result is then sent to the relying party.

There is a chance that entries are verified by more than one partial verifier. Shared libraries, such as the *libc*, are used by several applications or containers, and consequently may be verified by multiple partial verifiers. The following is a more formal presentation of the partial verification process: Let E be the set of all entries of an event log, and $e \in E$ be a single entry. Further, let $d_i \subset E$ be disclosed entries from an attester to a partial verifier i, and n be the number of partial verifiers needed to verify all entries at least once. Then, the (main) verifier needs $\bigcup_{i=1}^{n} d_i \geq E$ verified entries to ensure verifications for all entries ($\forall e \in E$) are present.

4.6. Implementation

We conduct performance measurements using a Raspberry Pi 3 Model B V1.2 running Raspberry Pi OS Lite in version *bullseye* from February 21, 2023. A LetsTrust TPM with an Infineon Optiga™ SLB 9670 TPM 2.0 is attached to the GPIO ports. The proof-of-concept (PoC) implementation is based on CHARRA [87] and [92]. It is written in portable C99 and the source code is available on GitHub at `https://github.com/DominikRoy/RACD`. Our concept builds upon Linux IMA and remote attestation, but without modifying the kernel source code. Instead, we develop userspace applications based on the IMA principle and log format, implementing the RACD protocol. To obtain more meaningful results,

we evaluate the performance by running both attester and verifier applications on separate Raspberry Pis, enabling a direct comparison of attestation and verification processes on the same hardware.

We implement the Schnorr NIZK proof using *Ed25519* [105], an elliptic curve that with a maximum size of 32 bytes, as restricted by the hardware TPM on our target platform. It supports SHA-256 as the strongest hash algorithm for event data in a *TPM PCR Extend* operation. We realize our NIZK logic with Libsodium [66] that employs the Ristretto technique. We establish mutually-authenticated encrypted TLS 1.2 channels between the attester and the verifier, as well as between the attester and all partial verifiers, using mbed TLS [14]. For wire-encoding, we use Concise Binary Object Representation (CBOR) with the QCBOR library [146]. To communicate with the TPM, we use the TPM2-TSS library [89, 226].

To generate the Schnorr signature, Libsodium's *crypto_core_ristretto255_scalar_reduce()* function uniformizes input values for arithmetic operations. The input consists of 512 bits (64 bytes), while the output is 256 bits (32 bytes). We use SHA-512 to hash the file hash and path, which produces a 64-byte digest. This digest serves as the input for the Libsodium function (cf. Algorithms 1 and 2). The resulting 32-byte output is our *event hash*.

Algorithm 1 provides pseudocode for Schnorr NIZK signing. The input is an *event*, i.e., a file path and file hash. The algorithm generates the *template hash* on line 2 and reduces it into a scalar on line 4 to fall within the range of 0 to L, where $L = 2^{252} + 27742317777372353535851937790883648493$. Ristretto scalars are designed to stay within the range of 0 and L for each arithmetic operation. Line 7, 9, and 12 check the validity to ensure that the resulting value is still a valid point on the elliptic curve after scalar multiplication, and uses mathematical steps similar to those in Figure 4.5. Line 8 creates a new generator that is accepted if it is a valid point on the curve, and can be used in place of the basepoint if the output is a valid point on the curve. Lines 4, 5, 14, 15, and 16 use the modulo operation to keep scalars in the valid range. Eventually, line 17 returns the *event hash*, and the scalars c and s.

Our NIZK verification (Algorithm 2) takes an *event record* (c, s, and event hash) as input, and outputs *true* if the verification succeeds, *false* otherwise. Lines 3–6 create the generator g_i. Line 12 adds the two valid curve points to obtain ${t_i}'$, which determines if the proof of knowledge is successful. Success is achieved only if c' equals $eventrecord.c$ (see Section 4.5).

4.7. Evaluation

In this section, we evaluate our RACD approach. In Section 4.7.1, we provide a security and privacy analysis. We demonstrate the feasibility of our approach with a performance evaluation in Section 4.7.2. Section 4.7.3 discusses several aspects of our approach.

4.7.1. Security Analysis

Traditional binary remote attestation provides integrity and authenticity of event logs, protected in hardware by the TPM. The *Template Hash* is anchored in the TPM using the folding

Algorithm 1: NIZK Signing

Input: $event$
Output: $event_{hash}, c, s$
Function `nizksign(`$event$`)`**:**
  $templatehash \longleftarrow h_T(event)$;
  $r \longleftarrow random()$;
  $reduced_{digest} \longleftarrow templatehash \pmod{L}$;
  $r_h \longleftarrow r \cdot reduced_{digest} \pmod{L}$;
  $event_{hash} \longleftarrow g^{r_h}$;
  if $event_{hash}$ *is not a valid point* **then return** error;
  $g_i \longleftarrow g^{reduced_{digest}}$;
  if g_i *is not a valid point* **then return** error;
  $v \longleftarrow random()$;
  $t_i \longleftarrow g_i^v$;
  if t_i *is not a valid point* **then return** error;
  $c \longleftarrow H(g_i \| t_i \| event_{hash})$;
  $reduced_c \longleftarrow c \pmod{L}$;
  $r_c \longleftarrow r \cdot reduced_c \pmod{L}$;
  $s \longleftarrow v - r_c \pmod{L}$;
  return $event_{hash}, c, s$;
End Function

Algorithm 2: NIZK Verification

Input: $eventrecord$
Output: $valid$
Function `nizkverify(`$eventrecord$`)`**:**
  $valid \longleftarrow$false;
  $templatehash \longleftarrow h_T(eventrecord.event)$;
  $reduced_{digest} \longleftarrow templatehash \pmod{L}$;
  $g_i \longleftarrow g^{reduced_{digest}}$;
  if g_i *is not a valid point* **then return** error;
  $gi_s \longleftarrow g_i^{eventrecord.s}$;
  if gi_s *is not a valid point* **then return** error;
  $reduced_c \longleftarrow eventrecord.c \pmod{L}$;
  $eventhash_c \longleftarrow eventrecord.event_{hash}^{reduced_c}$;
  if $eventhash_c$ *is not a valid point* **then return** error;
  $t_i' \longleftarrow gi_s \cdot eventhash_c$
  $c' \longleftarrow H(g_i \| t_i' \| event_{hash})$;
  if $c' == eventrecord.c$ **then** $valid \longleftarrow$true;
  return $valid$;
End Function

hash function *PCR Extend*, providing integrity protection for the entire event log. Further, attestation involves sending the log and a digital signature over the TPM's internal state (TPM quote) to a verifier to ensure both authenticity and integrity. This fulfills requirement R_1^S.

Our RACD method improves binary remote attestation by adding the security property of selective confidentiality regarding running software. This allows an attester system to disclose log entries while preserving the unlinkability between non-deterministic event hashes and the actual software binary, which satisfies requirement R_5^S. The fully masked and signed event log, as well as the signed partial attestation results, are provided to the main verifier, which can then verify their integrity and authenticity to assess the attester's trustworthiness, without knowing the actual running software.

We can disclose all entries to one verifier just like with traditional remote attestation, but this is in contrast with our use case and does not require our RACD approach at all. Our approach only allows communication between the attester and the main verifier, as well as between the attester and partial verifiers. Partial verifiers are not allowed to communicate with each other as per requirement R_6^S.

Threat Defense

We counteract threats identified in our threat model (cf. Section 4.3.2). To prevent passive man-in-the-middle attacks, we use encrypted communication (TLS) between attester and all verifiers (requirement R_2^S). This encrypted channel thwarts adversaries from reading the event log and combining subsets of disclosed data. Before starting the attestation process, we require mutual authentication between the attester and all verifiers (requirement R_3^S). RACD uses NIZK as a sub-protocol, so we utilize the existing NIZK proof system (with adaptations) to achieve constrained disclosure (and privacy) while preserving the overall integrity and authenticity of the event log. We rely on the pre-existing properties of NIZK and informally demonstrate their validity in the following.

Completeness With the proof of knowledge, the attester convinces partial verifiers that the event hash is the blinded template hash. In other words, if the verification is valid ($c_i = c_i'$), the partial verifier accepts the proof of knowledge over the relevant event hash for the provided values $h_{eventhash}(r_i, x_i)$, c_i, and s_i.

Soundness The attester needs the secret to blind the template hash and to convince partial verifiers of the correctness of the statement. If the attester fails to provide the secret, the partial verifier cannot use the "extra data" for non-associated event hashes, making the proof of knowledge verification invalid. This means the attester cannot convince the partial verifier of a false statement.

Our approach involves providing the partial verifier with the subset of entries of the event log and the proof of knowledge over the associated template hashes. We generate a random value (secret) r_i for each measurement and proof of knowledge v_i. We assume the use of a secure random number generator and the recomputation of event hashes on each boot cycle. If the same r_i and v_i are used for each NIZK generation, the partial verifier can retrieve the secret. Thus, we use secure random generation and avoid using the same r_i and v_i during the

measurement process. If the secret can be extracted, the event hash would no longer blind the template hash. If given two valid tuples with the same secret, the verifier can compute the secret r_1:

$$\frac{s_1 - s_2}{c_2 - c_1} = \frac{v_1 - r_1 \cdot c_1 - v_1 + r_1 \cdot c_2}{c_2 - c_1} = \frac{r_1 \cdot (c_2 - c_1)}{(c_2 - c_1)} = r_1. \tag{4.2}$$

Zero-Knowledge To prove knowledge without revealing the secret used to blind the template hash, we pass only the extra data (c_i, s_i) to the associated event hash $h_{eventhash}(r_i, x_i)$, allowing partial verifiers to confirm the statement's accuracy using a NIZK proof.

$$\begin{aligned} t'_i &= g_i^{s_i} \cdot h_{eventhash}(r_i, x_i) \\ &= g^{\varphi(h_T(x_i))\cdot(v_i - c_i \cdot r_i)} \cdot g^{r_i \cdot \varphi(h_T(x_i))\cdot c_i} \\ &= g^{\varphi(h_T(x_i))\cdot v_i - \varphi(h_T(x_i))\cdot c_i \cdot r_i} \cdot g^{r_i \cdot \varphi(h_T(x_i))\cdot c_i} \\ &= g^{\varphi(h_T(x_i))\cdot v_i - \varphi(h_T(x_i))\cdot c_i \cdot r_i + r_i \cdot \varphi(h_T(x_i))\cdot c_i} \\ &= g^{\varphi(h_T(x_i))\cdot v_i} = g_i^{v_i} = t_i. \end{aligned} \tag{4.3}$$

Since we rely on scalar multiplication, it is not possible for adversaries or verifiers to infer the template hash from Equation (4.1). The Elliptic Curve Discrete Log Problem (ECDLP) makes it impractical to retrieve the secret used in the computation from publicly known values. Our method assumes a properly chosen elliptic curve and a collision-resistant cryptographic hash function used to compute the challenge c. The security of the NIZK depends on the ECDLP, the properties of zero-knowledge proofs, a secure random number generator, and a collision-resistant cryptographic hash function H.

Elliptic Curve Analysis

Curve25519 meets our requirements for size and security and supports the ladder technique for fast scalar multiplication, making it a suitable elliptic curve for our purposes. We based our selection on the elliptic curve analysis by Tanja Lange and D. J. Bernstein, who maintain a website listing secure elliptic curves [27]. However, for our implementation of the Schnorr signature scheme, we use the twisted Edwards curve Ed25519, which provides faster scalar arithmetic and equivalent security to Curve25519 for signature schemes [25, 128]. Additionally, Ed25519 is already used for signature schemes in existing literature [26], making it advantageous for our approach.

Curve25519 and Ed25519 have an order of $8L$, where L is the large prime number $2^{252} + 27742317777372353535851937790883648493$. However, the Schnorr signature scheme requires establishing prime order groups [40], which is more complicated with non-prime order groups such as Curve25519 and Ed25519 due to their complex group structure. To resolve this issue, we employ the Ristretto technique [102, 103], which constructs prime order elliptic curve groups and eliminates the cofactor. Ristretto extends existing systems using Ed25519 signatures and ensures no further cryptographic assumptions are required. We need this for our work and thus map Edward points to Ristretto points and validate their canonical encoding.

Protocol Verification with ProVerif

Our protocol selectively discloses log entries while maintaining integrity and authenticity. We use ProVerif to guarantee the secrecy of non-relevant log entries in a single round of our scheme. In appendix A, we provide details about ProVerif and the definitions necessary to understand the output shown in Lst. 4.1. We provide the ProVerif code on GitHub at `https://github.com/DominikRoy/RACD`. We verify the secrecy of x_i with ProVerif and ensure it cannot be retrieved by the verifier or extracted from the randomly generated variables r_i, v_i. In our scheme, we define three events: 1. *verifiedAttestationResult* confirms the partial verifier has verified the disclosed events, 2. *sendAttestationResult* allows the attester to collect results from all partial verifiers and send them to the verifier, and 3. *trustable* indicates the attester device is trustworthy if all partial verifier attestation results are valid.

Listing 4.1: Verification results of ProVerif on RACD.

```
1 Weak secret x_i is true.
2 Query not attacker(x_i[]) is true.
3 Query not attacker(r_i[]) is true.
4 Query not attacker(v_i[]) is true.
5 Query event(trustable) ==> (event (sendAttestationResult
    (tpmQuote_signed,partialAttestationresults)) ==>
    event(verifiedAttestationResult (n,c_i',event_hash,result)))
    is true.
```

Listing 4.1 shows ProVerif's formal verification outcome of our protocol. The values of x_i, r_i, and v_i are not retrievable by the attacker (lines 1–4). Only the vendor knows the associated value x_i. The formal verification guarantees that the non-vendor event hashes do not reveal information to non-associated vendors (partial verifiers). ProVerif ensures that the event *verifiedAttestationResult* occurs before *sendAttestationResult* for the nonce, *eventhash*, and result, confirming the authenticity of *partialAttestationResults*. The verifier confirms the trustworthiness of the attester system only if all *partialAttestationResults* are valid. The order of events is crucial to ensure ProVerif's confirmation of the authenticity of the results and extend the trust base with the attester's device.

4.7.2. Performance Evaluation

We conducted experiments comparing our RACD approach with traditional remote attestation by running both attester and verifier applications on two Raspberry Pis (see Section 4.6 for details). Our aim was to measure the execution time of the attestation and verification processes on the same hardware platform. We focused exclusively on the incremental impact of RACD verification compared to traditional remote attestation process, as our main contribution and the most expensive operation is the Schnorr NIZK proof verification, which is performed for each entry in an event log.

Our experiments involved running 50 iterations with an event log size of 50 entries. The attestation process for both schemes was similar, with RACD taking on average 224 ms and traditional remote attestation taking 218 ms. The slight increase in RACD execution time

was due to additional processing of the event log, such as creating the masked event log and identifying disclosed entries. However, the RACD verification process took much longer than traditional remote attestation, on average 178 ms compared to 13 ms. This was because the verifier had to verify the NIZK proof for each entry, which required significantly more computational steps.

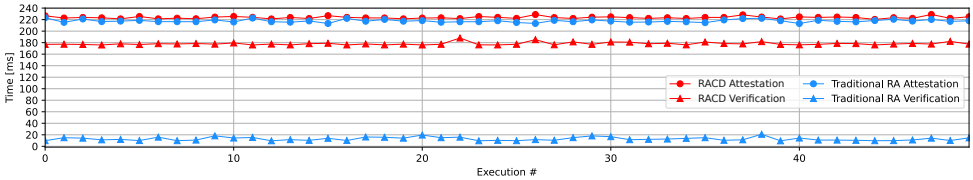

Figure 4.6: Execution time of attestation and verification processes for our RACD approach and traditional remote attestation approach, using 50 iterations and 50 log entries.

Although TPM-based remote attestation with constrained disclosure comes at a cost, our approach enables scalability by allowing verifications to be distributed and performed simultaneously by multiple partial verifiers. As the number of partial verifiers increases, the time required for an entire verification process can decrease significantly. We did not consider communication costs in our experiments, as we did not observe a significant difference. Our results are depicted in Figure 4.6, where the x-axis contains the respective iteration of the execution and the y-axis indicates the execution time in milliseconds.

4.7.3. Discussion

While initially using a nonce (as a salt or freshness key) might seem simpler and more elegant compared to our NIZK method, it comes with limitations. For instance, all verifiers must be online during the (measured) boot process to supply the nonce. Additionally, if the nonce is not selected carefully, it becomes vulnerable to rainbow table attacks. On the other hand, our NIZK approach offers the advantage of transferability, eliminating the need for verifiers to be online during boot-up. NIZK also securely proves the validity of the data without disclosing the underlying secret and allows for the verification of the obfuscated hash.

The computational burden associated with using NIZK for information security and limited disclosure is acknowledged. We also concur that increasing the volume of log entries would proportionally elevate the processing time. However, our analyses demonstrate that when parallelized across 1 to 50 partial verifiers—each handling 50 unique log entries—the computational time remains largely consistent. For example, Figure 4.6 on the x-axis depicts a scenario involving 50 partial verifiers, each managing 50 individual log entries, culminating in a total of 2500 log entries. This is comparable to a basic Ubuntu system, which has between 600 and 800 files and binaries measured by Linux's IMA, not counting additional vendor applications. Jäger et al. [122] designed a software defined networking (SDN) node for Industrial Internet of Things (IIoT) applications. Their node features approximately 600 entries for Xen's Dom0 and about 1700 entries for Xen's DomU in the Linux IMA logs. These figures support the validity of our study, showing that our 2500 log entry evaluation closely aligns with or even replicates real-world configurations.

4.8. Related Work

Debes et al. [63] present an approach called Oblivious Remote Attestation (ORA) to achieve zero-knowledge proof of integrity conformance. The authors emphasize the importance of privacy of the prover system's configuration and scalability in a multi-domain setting. To achieve this, they propose two additional protocols for provisioning and configuration updates, which are essential to ORA functioning. They also mandate the use of a TEE, an TPM NV Extend Index, and TPM enhanced authorization (EA) policies, as they state these measures are necessary to ensure a privacy-friendly attestation. Specifically, they limit access to an attestation key with TPM EA policies. In contrast, our RACD approach does not expand the Trusted Computing Base (TCB) by mandating the use of a TEE, TPM NV Extend Indexes, or TPM EA policies. Our protocol attains an equivalent level of security, without requiring the use of these supplementary measures, while extending its applicability to a broader range of devices. Furthermore, our protocol executes approximately 50 ms faster than theirs.

Sadeghi et al. [184] introduce property-based attestation (PBA) to address the privacy deficiency of binary attestation. The idea is to use a TTP to map platform configurations—i.e., binary measurements—to (security) properties, such as "booted up correctly" or "IDS is running". In contrast to our approach, a TTP is required. Chen et al. [53] extend the PBA protocol with a detailed design in theory that does not require a TTP. They apply ring signatures and commitments. Compared to our work, they rely on a commitment scheme and require more computational steps. Further, their concept has two privacy limitations: 1. If the set of configurations is small, the verifier can guess configurations, and 2. multiple executions of the protocol with different configurations allows to find an intersection. Both [184] and [53] suggest designs but do not implement them. In contrast, we design a practical approach, implement a PoC, and evaluate its security and performance.

Luo et al. [147] designed a privacy-preserving integrity measurement approach for containers. For each container, they measure all processes and generate a secret for each measurement (*PCR.secret*), and the secret is stored in the event logs. Each container log entry is *XOR'ed* with a corresponding *PCR.secret*. All *XOR'ed* values are folded into a folding hash that is extended into a specific PCR in the TPM. During remote attestation, the verifier receives the *PCR.secret*s for the corresponding container and the accumulated hash of a PCR signed by the TPM. The verifier applies the same procedure described above and compares the actual value with the received one. The state of the other containers and the host system is in the accumulated hash signed by the TPM. However, the verifier can only check the value of the corresponding container. Jie et. al [125] have a similar approach. Instead of the *XOR* operation, they concatenate the *template hash* with a random factor (*shielded factor*) that is transmitted in plain to the verifier. Both methods transfer their secrets to the verifier. If one binary measurement is sent to multiple verifiers, the uniqueness of the secret is not given. However, our work does not reveal the secret but proves that we have knowledge about it. Undeniably, approaches that use symmetric primitives offer performance benefits, but they also come with aforementioned downsides, which we account for in our approach.

Lauer et al. introduce a formal model and logic for containerized systems and their interac-

tions [141]. Based on these, they design and verify a secure integrity measurement system for containerized systems. Since Linux IMA is incapable of creating domain/container specific integrity reports, they extend IMA in theory to achieve this functionality and gain additional desirable properties, such as measurement log stability and constrained disclosure for multi-domain systems. However, their approach requires each container (or domain) to be measured into a separate TPM PCR. Since a TPM typically has 24 PCRs—from which only 2–12 are available, depending on the platform configuration—, the number of containers that can be run and measured separately is very limited. For example, measured boot uses PCRs 0–7 (and also 8–9, depending on the configuration), Linux IMA by default uses PCR 10, Intel Trusted Execution Technology (TXT) [119] occupies PCR 17 for its Dynamic Root of Trust for Measurement (DRTM), and PCRs 16 and 23 are debug PCRs, i.e., they are intended for development and can be reset during runtime. The work from Lauer et al. can greatly benefit from our work by measuring multiple containers into a single TPM PCR and achieve the same properties for measurement logs: log stability and constrained disclosure.

Our RACD approach targets the secrecy of *binary measurements* during remote attestation. There exists the Direct Anonymous Attestation (DAA) protocol [41] to keep the *identity* of a TPM secret. However, DAA, Enhanced Privacy ID (EPID), etc., are orthogonal to our approach and can be used in conjunction with it.

4.9. Conclusion

In this chapter, we showed how to realize constrained disclosure for TPM-based binary remote attestation data by applying a non-interactive zero-knowledge (NIZK) system using Schnorr signatures.

We thoroughly analyzed security properties informally as well as formally with ProVerif. Based on our PoC implementation, we evaluated the performance and demonstrated its feasibility. The performance results showcase that RACD requires, on average, 6 ms longer than the traditional remote attestation, which is negligible. However, the verification process of RACD takes much longer than the traditional process, due to the necessity to verify each NIZK proof of each entry to guarantee *constrained disclosure*. The number of entries is influenced by the number of IMA logs in a real system, which we discussed in this chapter. We compared our approach with existing work and highlighted its uniqueness. As future work, we intend to further research the application of our RACD approach in the context of VMs and software containers, and plan to integrate it into IMA in the Linux kernel.

III

Secure Two-Party Computation for UAV Services

5

Privacy-Preserving Multi-Party Access Control for Third-Party UAV Services

This chapter is inspired by [95]:
D.R. George, S. Sciancalepore, N. Zannone
Privacy-Preserving Multi-Party Access Control for Third-Party UAV Services
2023 ACM Symposium on Access Control Models and Technologies (SACMAT)

Third-Party Unmanned Aerial Vehicle (UAV) services, a.k.a. *Drone-as-a-Service (DaaS)*, are an increasingly adopted business model which enables possibly unskilled users with no background knowledge to operate drones and run automated drone-based tasks. As briefly highlighted in Chapter 1.1.3, due to the shared ownership of the involved data, Third-Party UAV services require the adoption of multi-party access control solutions, possibly leaking confidential information about the access control policies specified by the data owners. In this chapter, we propose a privacy-preserving multi-party access control solution tailored to the application scenarios of Third-Party UAV Services. This contributes to achieving *input privacy* and addresses **SRQ3** and **SRQ4**.

5.1. Introduction

The increasing availability on the market of UAVs, namely, *drones*, is making it even more appealing to use them in various domains [23]. At the same time, regulations applying to drone flights and professional certifications, required to fly such devices, require users to spend significant time and money to use drones safely [235]. In this context, *Third-Party UAV Services*, also referred to as DaaS, represents an emerging business model where Service Providers (SPs) lease ready-to-use drones, as well as certified pilots, customizable to the

needs of any application scenario [24]. Today, DaaS providers are appearing on the market at a high pace, and recent reports show this trend will increase in the next years [45].

Although the DaaS paradigm significantly reduces the time and effort to integrate drones into business, it also introduces security and privacy concerns. Drones are managed by untrusted SPs, and they access resources owned by multiple parties. Thus, DaaS scenarios should use multi-party access control solutions, which can regulate access to resources through policies provided by multiple parties [199]. At the same time, such policies might contain sensitive information, e.g., relationships among Data Owners (DOs). If leaked, such information might raise privacy issues and, possibly, jeopardize business. Therefore, we need to adopt privacy-preserving multi-party access control solutions for DaaS scenarios.

To the best of our knowledge, no solutions are available today for privacy-preserving multi-party access control for constrained devices. Some solutions available in the literature cover traditional domains (see Sec. 5.6), but their usage within drones is not straightforward. Indeed, drones feature several constraints in terms of processing, communication, storage, and energy. At the same time, drones frequently operate in remote areas where Internet connectivity is scarce and no other entities are available to assist the processes. Therefore, currently-available solutions need to be re-engineered to fit the requirements of the DaaS scenario.

Contribution. Our work aims to investigate the feasibility of privacy-preserving multi-party access control schemes for Third-Party UAV Services. Our scheme builds on top of an existing approach based on the SFE paradigm [195], further adapting its architecture to meet the constraints and requirements of the DaaS scenario. Through the proposed scheme, drones can evaluate complex multi-party policies without relying on a persistent Internet connection and without inferring user policies specified by data owners. We deployed a proof-of-concept of the proposed scheme on a Raspberry PI 3 and performed an extensive experimental assessment to evaluate the performance of our approach w.r.t. several key metrics for constrained devices. The results show that our scheme can support privacy-preserving multi-party policy evaluation on a processing-constrained device in realistic configurations while requiring a limited energy toll.

Roadmap. The remainder of the chapter is organized as follows. Sec. 5.2 introduces our scenario, use cases, and requirements. Sec. 5.3 provides the details of our solution, and Sec. 5.4 discusses security considerations. Sec. 5.5 reports an experimental evaluation of our solution using a real proof-of-concept implementation. Sec. 5.6 discusses related work. Finally, Sec. 5.7 concludes the chapter.

5.2. Scenario, Use Cases, and Requirements

In this section, we introduce our reference scenario and adversary model (Sec. 5.2.1). We also provide practical use cases (Sec. 5.2.2), which are used to extract the main requirements for achieving privacy-preserving multi-party access control in DaaS scenarios (Sec. 5.2.3).

5.2.1. Scenario and Adversary Model

The scenario considered in this work is based on the *Third-Party UAV service* architectural paradigm, referred to as DaaS. The DaaS paradigm provides advantages in several application domains, such as surveillance, farming, and construction [2], where individuals and companies cannot afford to buy drones, maintain them, and/or acquire the licenses necessary for flying UAVs.[1]

Without loss of generality, the DaaS paradigm enables a SP to lease one or more drones to its customers to achieve specific tasks, e.g., the inspection of specific equipment or areas [45, 236, 242]. The SP provides the drones and equips them with the necessary tools, sensors, and software applications, depending on the specific purpose the drone is intended for. Moreover, the SP can provide a professional pilot to run operations on the field. The drones and their sensors, hereby referred to as *resources*, are made available for usage and access to several Data Consumers (DCs). To interact with the drone and access resources, DCs use *controllers*, i.e., software applications that can be installed on the user equipment (such as a smartphone) and that allow interacting with the drone, in real-time, through a dedicated (secure) network connection.

Access to the resources provided by drones is regulated through access control policies specified by the DOs. Specifically, DCs can access data collected by the drone and integrated sensors only if the access control policy associated with the resources allows for it (see Sec. 5.2.2 for practical examples). In specific situations, the DC can also be a DO. DaaS applications may also include other entities, such as the Manufacturer, i.e., the entity that produced the drone, which can be different from the SP. However, given that such entities are not directly involved in the use cases tackled in our work, we do not focus specifically on their functions.

In this work, we focus on resources owned by multiple DOs, each with a different level of authority on the resource. Examples of such resources include data on areas affected by natural disasters, agriculture fields, harbors, and buildings under construction (see Sec. 5.2.2 for more details). In the IT domain, the protection of such resources is usually achieved through *multi-party access control* [162]. In this paradigm, each DO can specify access constraints on its resources (hereafter referred to as *user policy*), which define under which conditions a subject is allowed to access a given resource. When a DC requires access to a resource, the multi-party access control mechanism accounts for the user policies of all the owners of that resource to determine whether the authorization should be granted or denied, while handling possible conflicts among the DOs' user policies, e.g., based on their level of authority on the resource.

While the protection of co-owned resources is of utmost importance, *user policies* may also be sensitive. If revealed, these policies can leak confidential information, e.g., about the relationship among different DOs, the resource, and the specific DO [195]. The above considerations are even more relevant in the DaaS domain. Indeed, drones might be more easily hijacked and captured than traditional IT systems. Moreover, manufacturers often keep remote access to the deployed vehicles, e.g., for maintenance, and might stealthily access such

[1] Several countries have regulation in place (e.g., FAA regulations in the US [1]) requiring individuals and companies using UAVs to maintain a valid drone pilot license.

policies when performing regular maintenance operations. Thus, the policy content should be protected, resulting in the need to deploy a privacy-preserving multi-party access control solution.

While solutions supporting privacy-preserving multi-party access control have been proposed for traditional IT systems [195, 196], their application and contextualization in the DaaS scenario poses additional challenges. Indeed, drones frequently operate in remote locations where connectivity is scarce or not available at all. Thus, drones cannot simply outsource operations to more powerful servers available on the Internet or other network nodes, but they have to perform all computations offline. Moreover, drones are energy-constrained devices, which should preserve energy as much as possible so as not to consume all the available energy too quickly, before the expected end of their planned mission.

5.2.2. Use Cases

This section presents concrete use cases to illustrate the need for privacy-preserving multi-party access control in DaaS applications. From such use cases, we elicit the requirements that such a solution should meet (cf. Sec. 5.2.3).

Disaster Management

Disaster management plays a major role nowadays, and the DaaS paradigm is increasingly used in related rescue operations [11, 165].

Application Scenario. Assume a disaster scenario, e.g., a landslide or flooding, where several local forces (e.g., the local police, fire force, and municipality) and state forces (e.g., the military) are involved in rescue operations. Using a drone, such entities can better orchestrate their interventions and optimize the distribution of forces on the field. However, the relatively short duration of the operation might not justify the costs needed to buy and maintain a drone; rather, rescue forces can subscribe to a shared drone through the DaaS paradigm to coordinate their interventions. In this context, the military, local police, fire force, and municipality are both DOs and DCs. Moreover, other entities may require access to the drone and its resources, thus acting as DCs, to participate (in various capacities) in rescue operations, e.g., volunteers, paramedics, civil protection, and news channels. The resources to be accessed include readings of the surrounding environment, e.g., the camera (video feed), heat sensors (sensor data), and environmental sensor measurements, to identify victims and monitor the status of flooding or landslide.

Problem. Assume that, during the operations, the drone flies over a military base near the area where the natural disaster occurred. In such a situation, the military, acting in the capacity of DO, would like to deny access to the video feed and heat map to rescue forces, as they could reveal sensitive information about the base. Therefore, the military force might deploy an access control policy on the drone, explicitly denying access to the video feed to any other entity when the drone flies in the proximity of the military base. However, this policy might contain sensitive information, such as the location of the areas to be protected, which should not be disclosed to entities with access to the drone, including the SP.

As another example, consider the case where the drone discovers dead bodies. For privacy and safety purposes, the rescue forces (police and fire forces, which are DOs) want to deny the news channels access to the video stream. Indeed, not only private images could be leaked, but also news-media operators might crowd the area, obstructing rescue operations. At the same time, the knowledge about an access control policy enforcing such a condition on the drone would immediately reveal to the news channel why they cannot access the video stream.

Challenge. Enforcing privacy-preserving multi-party access control in this use case is particularly challenging. Natural disasters might occur in remote and temporarily hard-to-reach locations, where Internet connectivity is limited. Therefore, the drone cannot rely on a persistent Internet connection, e.g., to delegate policy evaluation to Cloud-based systems. As a result, access control policies should be evaluated in a private way offline, involving only the drone and, if present, locally-available user-owned resources, such as a controller, while dealing with a resource-limited environment.

Smart Transport/Vessel Inspection

The DaaS paradigm can also support port authorities to optimize their inspection procedures while minimizing related operation and deployment costs.

Application Scenario. A harbor has port authorities responsible for handling incoming and outgoing ships, e.g., harbormaster, immigration bureau, and coast guard. We assume that the port authorities (DOs) feature a drone, operated according to the DaaS paradigm, used for inspection activities and other tasks, e.g., validating the crew's identity, inspecting shipment documents, and checking compliance with shipping regulations. In particular, the drone can be used by the harbormaster to coordinate the traffic of vessels and perform vessel inspections. Similarly, the immigration bureau can use the drone to acquire and process the visa documents and passports of crew members. At the same time, we assume that the coast guard runs periodic surveillance operations in the harbor area. To prevent disclosure of the activities performed during these operations, the coast guard may specify an access control policy that restricts the access to the video stream of the drone when it flies in the proximity of the operation area.

Problem. When the drone approaches a vessel for inspection, the harbormaster and the immigration bureau (DCs) might not be able to access the video stream of the drone, as the trajectory of the drone might cross the area currently patrolled by the coast guard (DO). By looking at the policies deployed by the coast guard on the drone, others DOs and the SP might learn sensitive information about the operations run by the coast guard, such as the date, time, and exact area in which such operations are performed.

Challenges. Vessel inspections typically occur some kilometers offshore, where persistent Internet connection is not available. Therefore, the drone should be able to make access decisions offline (i.e., while not being connected to the public Internet), resorting to the aid of only locally-available entities (e.g., a controller), while not featuring connection to any additional network element. In this setting, policy evaluation should not drain too much

energy from the drone's battery, as returning to the harbor for battery recharge would result in high costs and delays in the harbor operations.

Smart Construction

In smart construction, we envision that different companies (subcontractors) can rely on the DaaS paradigm to automatize work and reduce costs [126, 166].

Application Scenario. Assume the owner of a construction site contracts multiple subcontractors, specialized in specific areas of the site (e.g., plumbing, installing electricity lines). The site manager can provide the subcontractors with a drone, e.g., to install electricity lines. Additionally, for privacy reasons, the drone only features a connection to the private network of the construction site owner, thus minimizing the information transferred through the public network. Due to safety concerns, the contractor also enables external parties, such as the power line authority, to regulate the usage of some resources. For instance, the power line authority (DO) can specify access requirements to limit power usage by elements carried by the drone. Indeed, abusing electricity usage during line installation operations could lead to a power shortage or a blackout for the entire area. At the same time, the construction site manager (DO) can authorize external entities to access the real-time video feed of the drone. For instance, during the electrical installation, the authority can access the video streaming to oversee if the installed electricity lines comply with regulations. Thus, specific installation operations could be forbidden to the power line operator due to the constraints imposed by the authority regulating the power grid.

Problem. If access control policies are loaded and used in clear on the drone, all parties interacting with the drone can see the information contained in the user policies. At the same time, an adversary capturing the drone might access these policies. Thus, they would know why a specific entity using the drone cannot install the electricity lines, potentially leaking sensitive information. For instance, knowledge of constraints expressed by the power grid authority might be misused to achieve a blackout or shortage in the region.

Challenge. While performing the planned tasks, the drone can only connect to the construction site owner's company network, which might offer limited services. Also, connecting to such a network might require a VPN connection, which is typically hard to support on the drone because of the significant communication, processing, and energy resources it requires [9]. Therefore, to save energy and increase the drone's battery lifetime as much as possible, policy evaluation should be executed locally, either onboard or through the assistance of locally-available users' equipment, while preserving the confidentiality of user policies.

5.2.3. Requirements

The use cases in the previous section highlight the need for the adoption of a privacy-preserving multi-party access control scheme in the DaaS scenario. On the one hand, multiple entities (DOs) share ownership of resources and can specify access constraints to regulate their access (*Support for Multi-party Data Governance,* **R2**). On the other hand, to enforce access control policies, the drone needs access to the DOs' policies and the information therein. Such

policies, however, can be sensitive. If leaked, unauthorized entities might infer sensitive information about, e.g., the identity of the entities using the drone and their relations. For instance, unauthorized entities might retrieve the location of military bases (use case 5.2.2), spatio-temporal information about joint military operations (use case 5.2.2), and the restrictions on the usage of power lines (use case 5.2.2). Thus, DaaS use cases require an access control solution that allows the privacy-preserving evaluation of DOs' policies (*Policy Confidentiality*). In particular, the access control mechanism should preserve the confidentiality of the access constraints in the user policies (*Attribute Confidentiality*, **R3**) while remaining capable of collaborative decisions. Moreover, it should prevent someone from linking the access decision to a specific user policy, which can be exploited to derive the user policies (*Decision Untraceability*, **R4**).

However, the design of such an advanced security mechanism in DaaS applications comes with several constraints. First, drones involved in DaaS applications might not feature persistent Internet connection. Even when equipped with a SIM providing mobile Internet connection, the drone might operate in areas where Internet connectivity is temporarily out of service (e.g., soon after a natural disaster occurred, as in use case 5.2.2), is not available (e.g., offshore, as in use case 5.2.2), or where there are restrictions on the type of connection to be used (as in use case 5.2.2). As a result, drones cannot reliably outsource access control operations to the Cloud or any fully-trusted edge devices. Conversely, they need to evaluate access control policies and enforce access decisions offline, at most, with the assistance of a semi-trusted party available in the field (*Outsourced Third Party*, **R1**).

In addition, although significant processing and storage resources might be available, drones are typically energy-constrained devices, which should be able to complete all planned tasks before battery exhaustion. For instance, interrupting their tasks to re-charge the battery might cause loss of lives as in use case 5.2.2, economic losses as in use case 5.2.2, and construction delays as in use case 5.2.2. Therefore, the enforcement of privacy-preserving multi-party access control should be computationally *lightweight*, i.e., it should not have a large impact on the time necessary to access resources and the overall lifetime of the drone (*Constrained Environments*, **R5**).

We summarize the aforementioned requirements in Tab. 5.1.

5.3. Privacy-preserving multi-party access control in DaaS

A main problem of existing multi-party access control solutions is that they typically do not account for the protection of the policies, whose disclosure can leak confidential information as well. This problem is addressed in [195], which proposes a framework for the privacy-preserving evaluation of multi-party access control policies. In this work, we rely on that framework for the specification of multi-party access control policies and for the cryptographic primitives for their secure evaluation; based on it, we propose a privacy-preserving multi-party access control architecture for the DaaS scenario. We first present an overview of the framework in [195] (Sec. 5.3.1); we then describe the architecture of our solution

Table 5.1: Privacy-preserving multi-party access-control requirements.

ID	Requirements	Description
R1	**Outsourced Third Party**	The scheme allows the assistance of a semi-trusted party.
R2	**Multi-party Data Governance**	The scheme allows to specify access constraints to co-owning resources.
R3	**Attribute Confidentiality**	The scheme should preserve the confidentiality of the access constraints in the policies.
R4	**Decision Untraceability**	The scheme should prevent someone from linking the access decision to a specific user policy.
R5	**Constrained Environments**	The scheme should be computationally *lightweight*.

(Sec. 5.3.2) and the protocol flow (Sec. 5.3.3).

5.3.1. Privacy-Preserving Policy Evaluation via Secure Function Evaluation

Our work is built on top of the framework for privacy-preserving multi-party access control proposed in [195]. This framework allows users to provide their policies, expressed in Attribute-Based Access Control (ABAC), in private form and provides secure computation protocols for evaluating multi-party policies, preserving the confidentiality of the user policies forming the multi-party access control policy.

The authors of [195] realized their framework and secure protocol using the SFE paradigm [65]. SFE allows several parties to compute a function on their private inputs without revealing any information besides the result of the function. Given two parties p_1, p_2 and their corresponding inputs x and y, each party creates secret shares for their input, represented as $\langle x \rangle_1, \langle x \rangle_2$ and $\langle y \rangle_1, \langle y \rangle_2$ respec., such as $\langle x \rangle_1 \oplus \langle x \rangle_2 = x$ and $\langle y \rangle_1 \oplus \langle y \rangle_2 = y$. Each party shares one of the input shares with the remote party and evaluates a given function f using the shares in its possession. The actual result of the function ($z = f(x, y)$) can then be reconstructed by combining the shares of the result, $\langle z \rangle_x$ and $\langle z \rangle_y$, obtained by applying f to the input shares, i.e., $z = \langle z \rangle_x \oplus \langle z \rangle_y$.

By leveraging the SFE paradigm, the work in [195] devises secure computation protocols for target evaluation and policy composition of ABAC policies. These protocols are used for the privacy-preserving evaluation of *multi-party access control policies*. Intuitively, *multi-party policies* are created from the shares of *user policies*, which specify the DOs's access constraints for their resources, according to a *policy template* defining how user policies should be combined based on the relationship between the DOs and the resource. When receiving an access request, the server and a Semi-Trusted Party (STP) evaluate the request, each using its share of the multi-party policy. The access decision is obtained by combining the shares of the decision computed by these entities on their shares of the multi-party policy and request.

Although the proposal in [195] addresses privacy concerns in multi-party settings, it has not been considered or applied in constrained scenarios, where the involved entities are characterized by computational, communication, storage, and energy constraints, and feature a non-persistent Internet connection. We take such a step ahead through our work, as described in the rest of the section.

5.3.2. Architecture Overview

Fig. 5.1 presents the architectural view of the proposed privacy-preserving multi-party access control system for the DaaS scenario. The architecture comprises five main entities: the DOs, the SP, the STP, the controller, and the drone.

- The *DOs* are the entities that own the resources in the system. We specifically consider a *multi-owner* scenario, where multiple owners might share possession of a specific resource. In this context, each DO defines *user policies* (in the form of ABAC policies), specifying the access constraints a requester should fulfill to access the protected resource. We assume that DOs are known in advance.
- The *SP* is the entity responsible for the services offered by the drone and for the enforcement of DOs' access constraints.
- The *STP* is a semi-trusted party that assists the SP in the privacy-preserving evaluation of multi-party access control policy.
- The *controller* runs on a device possessed by the DC, equipped with (wireless) communication capabilities, e.g., a mobile phone. Within the controller, we identify two main independent and isolated logical *spaces*: the *client space* and the *STP space*. The client space encompasses the applications used by the DC to access the resources on the drone. The STP space is controlled by the STP and comprises a Policy Decision Point (PDP) aimed to support the privacy-preserving evaluation of multi-party policies.
- The *Drone* is the entity possessing the resource(s) requested by the DC client. When deployed for the planned mission, the drone communicates persistently with the controller, and it does not feature a persistent Internet connection. To protect its resources and services, the drone includes a *Access Control Module* (ACM), which comprises a *Policy Enforcement Point (PEP)*, a *PDP*, a *Policy Information Point (PIP)*. The PEP is responsible for the interaction with the DC's application and enforcing the access decision; the PDP evaluates the DC's requests (with the assistance of the PDP in the STP space); the *PIP* gathers environmental attributes (e.g., from sensors) needed for policy evaluation.

The DOs, STP, and SP interact offline during the *setup phase* to configure the access control policies for the protection of the drone's resources, such as data and physical capabilities provided by the drone. The DOs create two shares of their *user policy* (denoted as $\langle s \rangle_x$ and $\langle s \rangle_y$ in Fig. 5.1) and provide one share to the SP and the other to the STP. From these shares, the SP and STP separately construct a *multi-party policy*, specifying the permissions to be enforced on drone resources, based on a predefined *policies templates*.[2] The shares of the *multi-party policy* are then deployed by the SP and STP into the drone and STP space on the

[2]As in [195], we assume that the policy template is defined by the SP, in agreement with the DOs, based on their relationship with the resources to be protected.

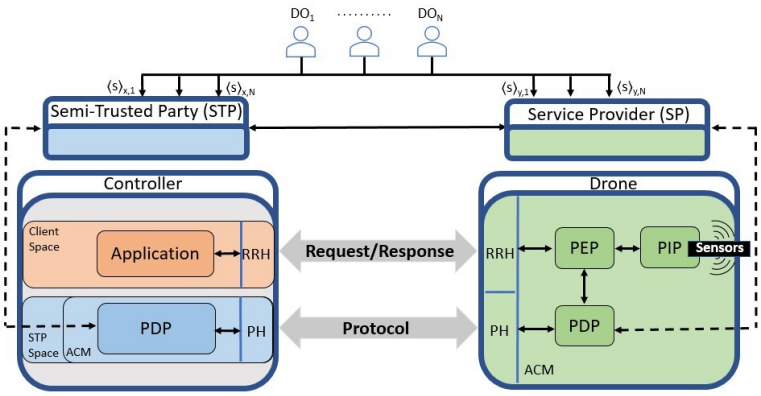

Figure 5.1: Architecture of the Privacy-Preserving Multi-Party Access Control scheme for the DaaS scenarios.

controller, respectively.

When operating in the field, a DC might request access to resources or services offered by the drone (through the applications installed in the controller). Access to these resources and services is determined *at runtime* in a privacy-preserving way through the interaction of two entities: the controller and the drone. In particular, the PDPs in the drone and in the STP space of the controller evaluate the request against the share of the multi-party policy, independently. The drone recombines the computed shares of the decision to reconstruct the final access decision to be enforced. Note that the whole protocol flow is executed locally between the controller and the drone, without an online connection to the public Internet.

The communication between the controller and the drone is managed through two interfaces. The Request/Response Handler (RRH) manages the interaction between the application in the client space and the drone, whereas the Protocol Handler (PH) enables the interactions between the drone and the STP PDP for privacy-preserving policy evaluation.

Security Assumptions. We assume an *honest-but-curious* threat model, where the entities behave according to the protocol specification, but they are interested in obtaining more information from their input, intermediary messages and output. In addition, we assume that the SP and the STP are independent entities, which do not collude. We will discuss the implications of collusion attacks in Sec. 5.4. We also assume that the *client space* and the *STP space* deployed on the controller are independent and do not interact with each other. This can be achieved, e.g., by using containerization techniques that provide built-in isolation among processes.

5.3.3. Protocol Details

Our scheme includes two phases: *setup phase* and *online phase*. A sequence diagram representing the schema is presented in Fig. 5.2.

Setup Phase. This phase, executed offline before the deployment of the DaaS, involves the

following steps:

(1) The DOs generate secret shares of their user policies, i.e., $\langle s \rangle_x, \langle s \rangle_y$.
(2) Using a secure connection over the public Internet, the DOs provide one secret share, i.e., $\langle s \rangle_x$, to the SP.
(3) At the same time, the DOs provide the other secret share, i.e., $\langle s \rangle_y$, to the STP using another secure connection.
(4) Using a secure channel, the SP sends to the STP the *policy template* used to construct the *multi-party policy* using DOs' shares.
(5) The STP deploys the multi-party policy into the controller.
(6) Similarly, the SP deploys the multi-party policy into the drone.

Online Phase. In this phase, the DC's controller and the drone interact to determine whether access to the requested resources should be granted. This phase involves the following steps:

(1) The application hosted in the *client space* of the controller sends a request to the drone for accessing a resource.
(2) At reception of the request, the *Drone PEP* requests the *Drone PIP* to provide environmental attributes for policy evaluation.
(3) The *Drone PIP* retrieves the environment attributes from the drone's sensors and sends them to the *Drone PEP*, which augments the access request with those attributes.
(4) The *Drone PEP* forwards the request to the *STP PDP*, which generates two shares of the request, $\langle q \rangle_x$ and $\langle q \rangle_y$, using a share generation algorithm. One share, $\langle q \rangle_x$, is kept on the drone; the other, $\langle q \rangle_y$, is meant for remote evaluation on the STP.
(5) The drone forwards the share $\langle q \rangle_y$ to the *PDP* in the STP space.
(6) The *Drone PDP* and the *PDP* in the STP space evaluate their shares of the request using their shares of multi-party policy, resulting in the shares of the decision, namely, $\langle d \rangle_x$ and $\langle d \rangle_y$ resp.
(7) The *STP PDP* sends its share of the decision, i.e., $\langle d \rangle_y$, to the *Drone PDP*.
(8) The *Drone PDP* re-combines the shares $\langle d \rangle_x$ and $\langle d \rangle_y$, obtaining the final decision d.
(9) The final decision d is sent to the *Drone PEP* for its enforcement.
(10) If the decision is *permit*, the drone grants access to the resource(s). Otherwise, if the decision is *deny*, the drone notifies the requesting application that access is denied.

5.4. Security Considerations

In the following, we discuss the security properties of the proposed scheme. Overall, the discussed approach is secure against an *honest-but-curious* adversary. This means that the involved parties (DC, DOs, STP, and SP) follow the protocol steps and, at the same time, aim to obtain sensitive information from the exchanged messages. In this context, we can see that none of the involved entities can learn information about the *user policies*. Specifically:

- The SP cannot derive the DOs' *user policies*. This is because the SP knows only one share of the *user policies*, which is insufficient to obtain the user policies thanks to the SFE properties [65]. At the same time, the SP is the entity deploying and managing the drone. Thus, it knows the final decision and the specific request issued by the *application*. This is necessary

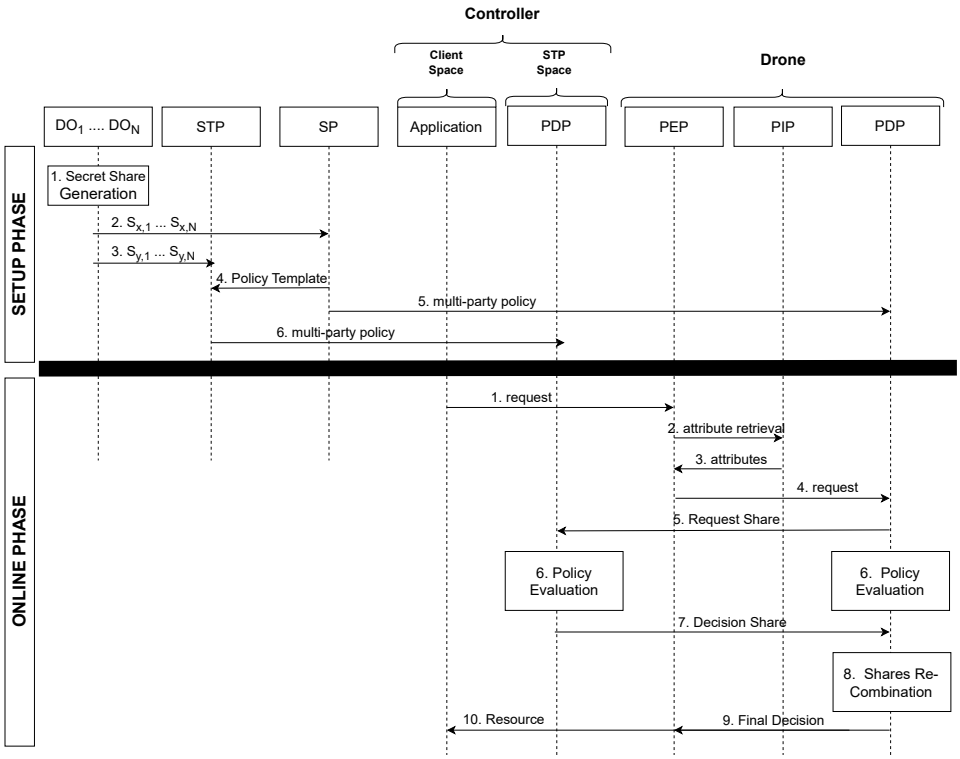

Figure 5.2: Sequence diagrams of the *Setup Phase* and *Online Phase* of the proposed scheme.

as the drone needs to provide access to the requested resource. However, this information is insufficient to derive the individual *user policies*.

- The STP cannot derive the *user policies* nor the final access decision. As we assume the *STP space* and the *Client space* in the controller are isolated, the STP has only one share of the *user policy* and the *policy template*, not enough to infer the *user policies*.
- On the controller, the *application* knows the final decision. However, the *application* cannot infer the individual *user policies* from it as it does not have any share. However, we highlight that the *application* might attempt to reconstruct the *multi-party policy*, e.g., by issuing all possible requests. However, we highlight that, in our scenario, issuing all possible requests might be difficult, if not impossible, as attributes are retrieved by the PIP at run time.

It is also worth discussing the impact of collusion attacks on the security of our scheme.

Collusion between SP and DOs. Some DOs might collude with the SP to learn the policies of the other DOs. However, the DOs and the SP combined have only one share of the non-colluding DOs' *user policies*. Thus, they cannot derive any information about these policies from intermediary messages.

Collusion between STP and DOs. Even in the case where all but one DOs collude with the

STP, they cannot infer the *user policies* of the honest DO, as the STP does not know the final decision. Thus, the STP and the colluding DOs cannot compare such a decision with the known *user policies* to derive the unknown ones.

Collusion between SP and *application*. Neither the SP nor the *application* has access to the other share of DOs' user policies. Thus, even if they collude, they cannot learn them. Both entities know the final decision; thus, neither of them learns new information.

Collusion between STP and *application*. Similarly to the previous case, the STP and *application* do not have both the shares of the *user policies*. Thus, they cannot infer them. By colluding with the *application*, the STP can learn the final decision. Yet, the STP cannot derive DOs' user policies from it.

5.5. Evaluation

We evaluated our scheme, presented in Sec. 5.3, based on its performance and feasibility in the DaaS scenario. Specifically, we are interested in answering the following three research questions:

- **CRQ1:** How the level of policy granularity in the scenario affects the communication, processing, and energy cost of the proposed privacy-preserving multi-party policy evaluation scheme on a constrained drone and controller?
- **CRQ2:** How the number of stakeholders in the scenario affects the communication, processing, and energy cost of the proposed privacy-preserving multi-party policy evaluation scheme on a constrained drone and controller?
- **CRQ3:** How does the proposed scheme perform compared to alternative privacy-preserving solutions available in the literature?

DaaS scenarios are typically highly dynamic. Thus, they require solutions able to adapt to different contexts. In the field of access control, this is usually achieved by employing fine-grained policies, which give users more control over the data. However, supporting a fine granularity often requires access requests to include a large number of attributes for their evaluation, which might impact system performance. **CRQ1** investigates the impact of policy granularity (represented as the number of attributes to be evaluated) on the performance of the proposed scheme in terms of processing, communication, and energy costs. Moreover, the number of stakeholders in the scenario (e.g., DOs) may impact system performances, which is the focus of **CRQ2**. The number of stakeholders managing the resources is typically reflected in the complexity and size of the access control policy, where larger and more complex policies might require higher execution time, communication overhead, and energy cost for policy evaluation. For both **CRQ1** and **CRQ2**, we are interested in such performance costs on both the constrained drone and the controller hosting the STP. Knowing beforehand the protocol's demands on both entities can help manufacturers and SPs deploy devices with the necessary processing, communication, and energy capabilities to achieve the planned tasks and minimize deployment costs. In this work, we employ SFE as the underlying privacy-preserving technique for the proposed scheme. **CRQ3** aims to assess this

choice by comparing our solution against the use of other privacy-preserving techniques available in the literature, applied in traditional IT domains.

5.5.1. Proof-of-Concept Implementation and Deployment

To answer our research questions, we implemented a proof-of-concept of our proposed solution.[3] For our implementation, we got inspiration from the work in [195], which presents a prototype implementation of their approach for the privacy-preserving evaluation of multi-party access control policies. Their prototype relies on the ABY framework [65] as the main building block. In a nutshell, the ABY framework provides basic modules to design secure two-party computation protocols using Arithmetic, Boolean circuits, and Yao's garbled circuits. However, the prototype in [195] was used to evaluate the performance of the approach using a fully local simulation running on a single machine. In this respect, it does not support the pre-sharing of the user policies and remote connections, as required by the DaaS scenario.

For our proof-of-concept, we followed the same logic of [195] but provided additional functionalities to support a network environment compliant with the DaaS scenario. In particular, we (i) organized the code into several processes, (ii) distributed such processes on several hardware platforms, and (iii) enabled remote communication capabilities using communication sockets. In this context, a major improvement of our implementation is the adoption of the *PutINSharedGate* available from the ABY framework instead of using the *PutINGate* as in [195]. Specifically, using the *PutINSharedGate* allows us to compute and pre-deploy the shares of the user policies during the setup phase rather than providing these policies in plaintext during the online phase, as was done in [195]. To achieve privacy-preserving multi-party policy evaluation, we used 32-bit Boolean circuits and configured the ABY framework to provide a security level of 114 bits.

To answer CRQ3, we additionally implemented another proof-of-concept based on the Partially Homomorphic Encryption (PHE)-based protocol proposed in [195]. In this regard, we also considered using Fully Homomorphic Encryption (FHE) in the benchmark protocol. At a first glance, applying FHE might appear a suitable choice for our scenario due to the well-known properties of FHE to carry out both additions and multiplications in the encrypted domain. However, we discarded such an option as, to preserve policy confidentiality, an equivalent scheme based on FHE would also require the deployment of an online third party (not to let the drone know the user policies). When such a third party is involved in the protocol, a solution based on FHE would require more computational overhead than a solution based on PHE due to the well-known high processing toll of FHE schemes. For implementing PHE-based operations, we used the GMP multiprecision library, in the C++ programming language. To enable a fair comparison with our approach, we improved the original implementation using actual remote sockets. Thus, the results reported in Sec. 5.5.5 consider the same deployment of our proposed approach and the competing solution in terms of hardware, software, and communication setup.

We deployed our proof-of-concept implementations on a real heterogeneous network. To

[3]The source code is available at `https://github.com/DominikRoy/PP-DaaS`.

emulate the drone, we used a Raspberry PI 3 Model B+ [176]. This is an embedded platform equipped with a Cortex-A53 processor running at 1.4 GHz, 1GB of RAM, and 16GB of storage. As acknowledged in the literature [144, 139], the computational and bandwidth resources offered by the Raspberry PI are comparable to the ones provided by low-end commercial drones, and, recently, some commercial drones integrating a Raspberry PI as an onboard CPU also became available on the market [75]. Thus, using the Raspberry PI allows us to evaluate our solution in a realistic setting.

On the other hand, we considered two deployment setups for the controller hosting the STP. In the first setup, we used a regular laptop Lenovo Thinkpad P1, equipped with two Intel Core i7-8750H processors running at 2.20 GHz, 32GB of RAM, and 1 TB of SDD Storage, while in the second we used a Raspberry PI 3 Model B+. Hereafter, we refer to the two setups as *Testbed 1* and *Testbed 2*, respectively. These testbeds allow us to investigate the system performance when heterogeneous hardware is used as a controller. When run on the laptop, the STP executes in a Docker virtual environment, while it executes as a native process when run on the Raspberry Pi. From the communication perspective, the Raspberry Pi modelling the drone and the STP (running either on another Raspberry Pi or on the laptop) communicate over an ad-hoc WiFi network.

5.5.2. Evaluation Metrics

To assess the practical feasibility of our scheme for the DaaS scenario, we evaluated our proof-of-concept implementations in terms of processing, communication, and energy costs.

We measured the processing costs for evaluating an access request in terms of the execution time of the setup phase (in milliseconds), the execution time of the online phase (in milliseconds), and RAM occupancy (in kilobytes). To carry out time measurements, we leveraged the integrated measurement system provided by the ABY framework. The ABY framework provides built-in benchmarking routines to measure the execution time of its various phases (setup, circuit generation, garbling, OTextension, online). However, these phases do not precisely map to the phases of our scheme. Thus, for measuring the execution time of the setup phase of our scheme, we considered the combined time required for the initialization of the ABY framework (setup), the circuit generation, and the garbling (encryption of the circuit). For the online phase of our scheme, we considered the sum of the time to execute the online phase of the ABY framework and *OTExtension*. Note that the *OTExtension* falls into the online phase of our scheme since this process concerns the generation and distribution of the shares of the access request. To measure RAM occupancy, we used the Linux tool "*/usr/bin/time -v*", which provides the highest instantaneous RAM consumption of a given process. Such a metric allows us to determine the maximum RAM demands of the proposed approach, thus providing an indication of the necessary RAM to be deployed on the involved devices.

We evaluated communication costs by measuring the communication overhead introduced by our scheme (in bytes). To this end, we leveraged the integrated measurement system provided by the ABY framework, which reports the summation of bytes sent and received during the setup phase, online phase, garbling (encryption of the circuits), and OTtransfer,

for both the client and server.

We also measured the energy consumption (in milliJoule) required by the drone and the STP to evaluate an access request. To this end, we used *powertop*, a software-based energy estimation tool provided by the Linux OS. *Powertop* estimates the power (in Watt) of a running process. Then, to compute the energy consumption of the process, we multiply such a power estimate for the (measured) duration of the online phase, as: $E = P \times \Delta t$, being E the estimated energy in Joule, P the power (in Watt) estimated by *powertop* and Δt the duration of the online phase.

5.5.3. Experiment 1: Impact of the Request Size

This experiment aims to answer **CRQ1** by evaluating the impact of policy granularity (represented in terms of the number of attributes) on the relevant system performance metrics described in Sec. 5.5.2 on both the drone and the controller.

Settings. For this experiment, we fixed the policy size to 10 and increased the request size (i.e., the number of attributes provided in the access request) from 1 to 20. For each request size, we generated 200 different access requests. In the results, we report the average for every performance metric.

Results. Fig. 5.3 reports the average execution time of the setup and online phases, communication overhead, and RAM required by the proposed scheme using the log scale on the y-axis while increasing the request size (x-axis). We also show the 95% confidence interval of the measurements through shaded areas. The figure is organized in two rows and four columns, where the top row refers to the results obtained by running the STP as a docker instance on the laptop (*Testbed 1*), and the bottom row refers to the measurements obtained by running the STP on a Raspberry PI (*Testbed 2*).

We observe that increasing the request size results in an increase of all performance metrics, except for the execution time of the *setup phase*, which remains almost constant regardless of the request size. Specifically, the *setup phase* takes between 6 and 20 ms on the Raspberry Pi, whereas it takes between 0.15 and 0.17 ms on the laptop.

The execution time of the *online phase* increases with the request size for both testbeds. On *Testbed 1*, the drone takes, on average, 482 ms for completing the *online phase* with a request size of 1, and the execution time grows to 3,730 ms for requests of size 20. On the other hand, the STP takes, on average, 1,322 ms for completing the *online phase* with requests of size 1 and 4,604 ms for requests of size 20. We believe this result, which might appear surprising, is due to the working logic of Docker containers, which cannot efficiently allocate enough resources to the process. The drone and the STP require the same amount of communication overhead and RAM, ranging from 0.14 and 26 Mbytes, for a request size of 1 to 5.8 and 162 Mbytes, respectively, for a request size of 20. On *Testbed 2*, the drone and the STP take approximately the same time for completing the *online phase*, ranging from 410 ms for requests of size 1 to 7,045 ms for requests of size 20. Moreover, the drone and the STP require the same amount of communication overhead and RAM required in *Testbed 1*. Comparing Fig. 5.3(b) with Fig. 5.3(f), we notice that the drone in *Testbed 2* always requires

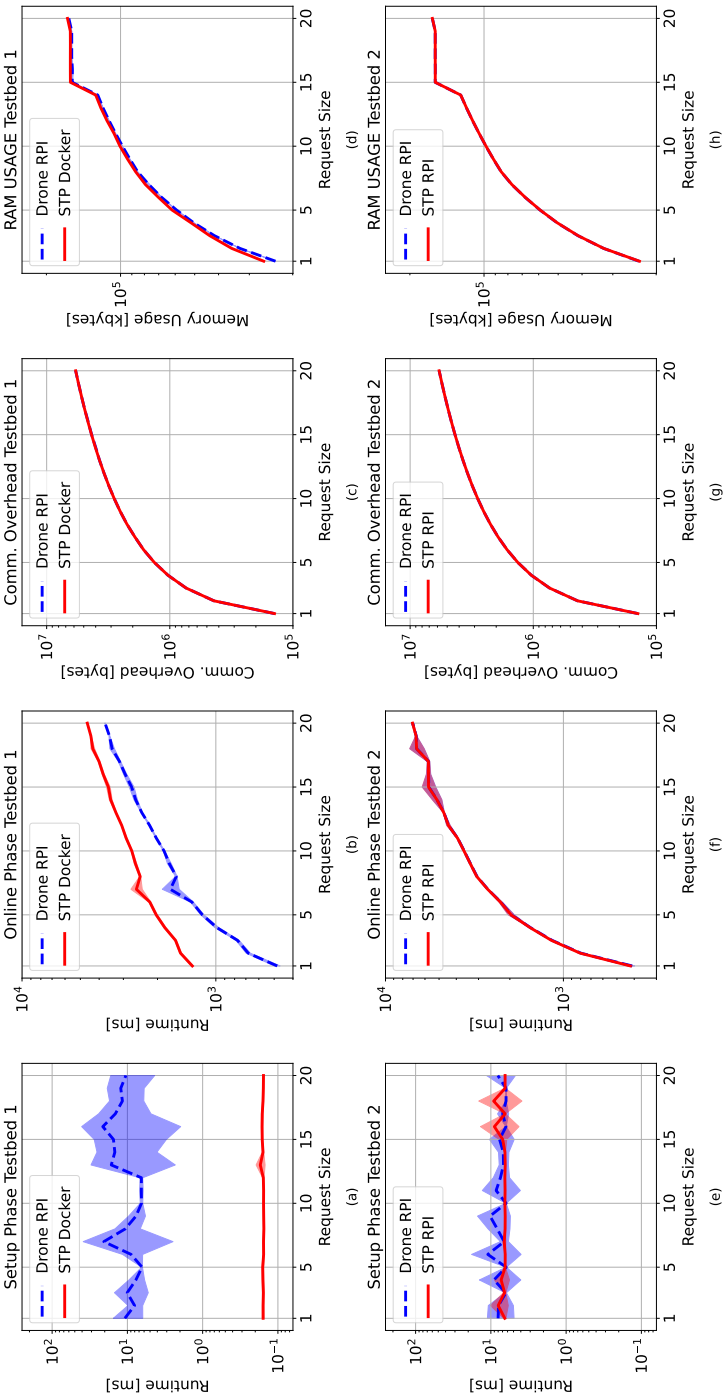

Figure 5.3: Experiments results for *Testbed 1* (top row) and *Testbed 2* (bottom row) varying the request size

5

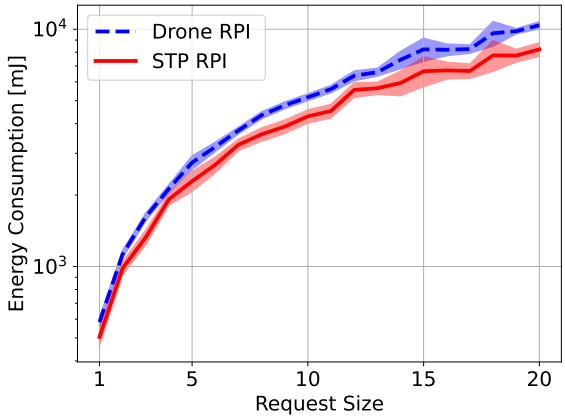

Figure 5.4: Experimental energy consumption of the controller and drone in *Testbed 2*, varying the request size.

more time than in *Testbed 1*. Conversely, the STP in *Testbed 2* requires less time than the STP in *Testbed 1* when the request size is lower than 5, but its performance degrades faster for higher values of the request size.

We also measured the energy consumption of our scheme for *Testbed 2* varying the request size.[4] The results are reported in Fig. 5.4. In line with the results in Fig. 5.3(f), the energy consumption of the involved entities increases with the increase of the request size. To provide a few examples, for requests of size 8, the drone requires approx. $4,338.71$ mJ, while the STP consumes $3,603.17$ mJ. Such a consumption on the drone translates into 0.158 mAh. Considering that an actual drone such as the DJI Mini 2 features a battery with a capacity of $5,200$ mAh [70], such a consumption corresponds to approx. 0.003% of the overall drone battery capacity, with minimal impact on the drone's lifetime.

5.5.4. Experiment 2: Impact of Policy Size

The number of stakeholders involved in the management of resources influences the size and complexity of the policies to be evaluated and, thus, system performance. This experiment aims to investigate such impact by assessing how the increase in policy size affects the relevant system performance metrics described in Sec. 5.5.2 on both the drone and controller.

Settings. We fixed the request size to 10 and increased the policy size (i.e., the number of atomic access constraints in the policy) from 1 to 50. For each policy size, we generated 200 different policies. In the results, we report the average for every performance metric.

Results. Fig. 5.5 reports the average time of the *setup* and *online* phases, communication overhead, and RAM consumption of the proposed solution when increasing the policy size

[4]For *Testbed1* we did not measure the energy consumption, because *Docker* does not has the kernel access for *powertop* to read the energy estimates.

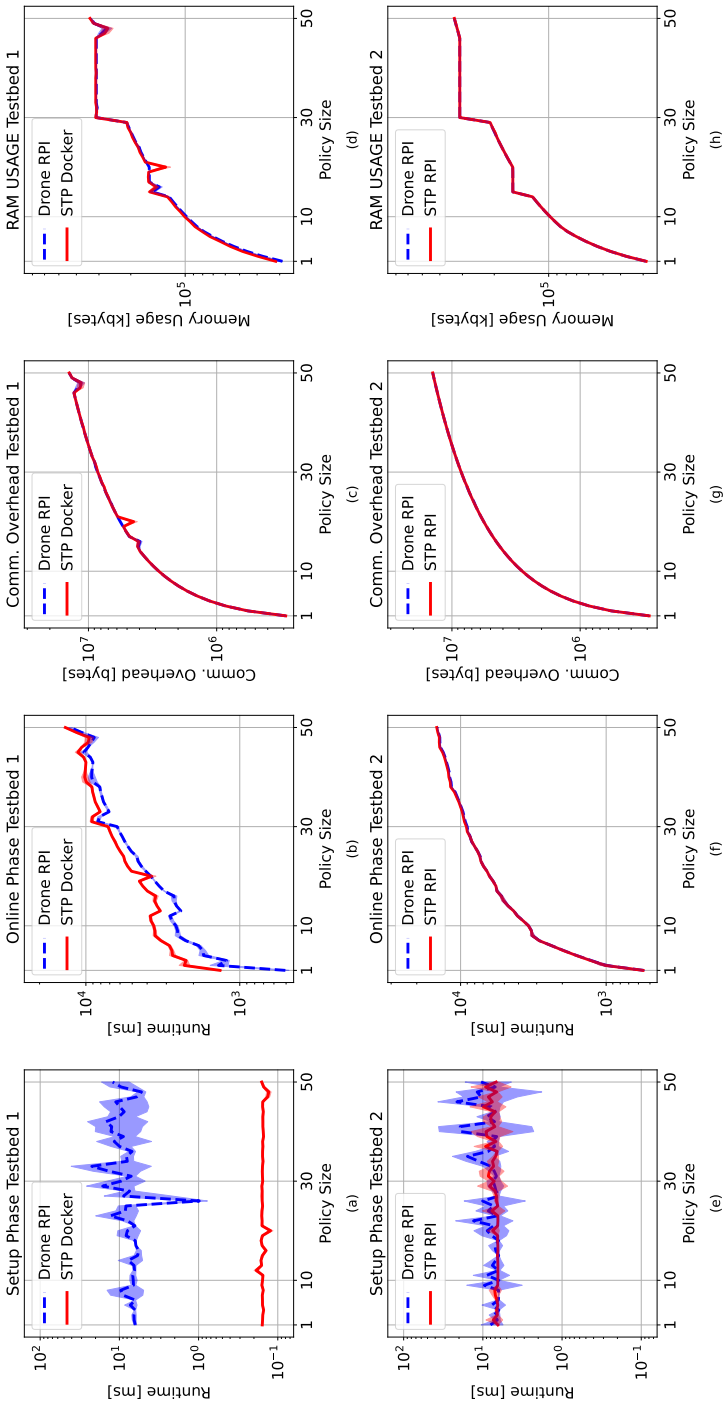

Figure 5.5: Experiments results for *Testbed 1* (top row) and *Testbed 2* (bottom row) varying the policy size

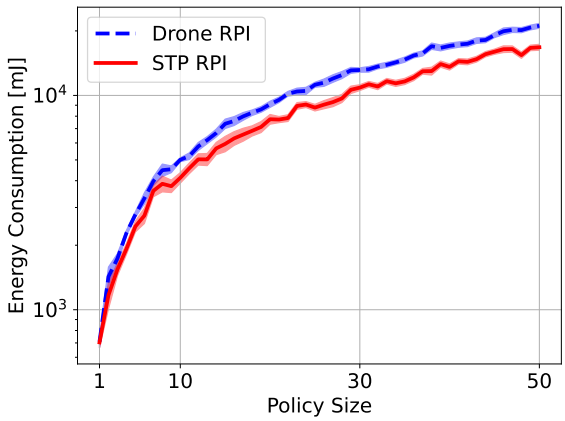

Figure 5.6: Experimental energy consumption of the controller and drone in *Testbed 2*, varying the policy size.

in *Testbed 1* (top row) and *Testbed 2* (bottom row). As for the previous experiments, we also report the 95% confidence interval of all measurements.

Overall, Fig. 5.5 shows a similar trend compared to the results reported in Fig. 5.3. All the metrics under investigation increase with the increase of the policy size, but the time for the *setup phase*, which remains constant in the range between 6 and 20 ms.

As for the execution time of the *online phase*, we notice that it increases faster for policies of size between 1 and 10 compared to larger policies. On *Testbed 1*, the drone takes, on average, 1,849 ms for evaluating a policy size of 6 while requiring a communication overhead of 1.69 Mbytes and 70.4 Mbytes of RAM. At the same time, the STP takes, on average, 2,864 ms, in line with the findings of Fig. 5.3. On *Testbed 2*, the drone and the STP take, on average, 2,378 ms for evaluating a policy size of 6 while requiring the same amount of communication overhead and RAM of *Testbed 1*. As for the previous experiment, we believe these results are acceptable for applying our solution in real settings, as they do not degrade too much the experience of the users and the quality of the offered services. On the other hand, considering the worst-case of a policy size of 50, the *online phase* of the scheme requires an execution time of 12,619 ms on the drone and 13,595 ms on the STP (*Testbed 1*).

We also measured the energy consumption of our scheme for *Testbed 2* varying the policy size. The results are reported in Fig. 5.6. Similarly to the results in Fig. 5.4, the energy consumption increases with the increase of the policy size. With a policy of size 6, the drone requires approx. 3,308.98 mJ. Yet, considering the DJI Mini 2, such a consumption corresponds to 0.0023% of the overall drone battery capacity. Such a limited energy toll confirms the minimal impact of our solution on the drone's lifetime.

5.5.5. Experiment 3: Comparison

In this experiment, we compare the solution described in Sec. 5.3 against the approach based on PHE proposed in [195].

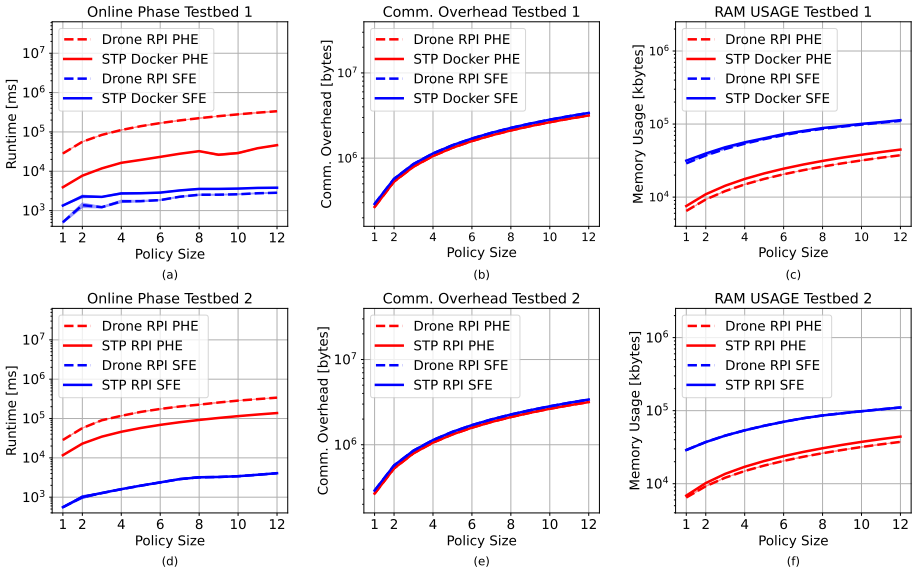

Figure 5.7: Experimental results of our approach and the solution in [195] for *Testbed 1* (top row) and *Testbed 2* (bottom row), varying the policy size.

Settings. In this experiment, for each of the two selected approaches, we fixed the request size to 10 and increased the policy size from 1 to 12. For each policy size, we generated 200 different policies. In the results, we report the average values of each measurement together with the 95% confidence intervals, depicted through shaded areas.

Results. Fig. 5.7 reports the results of our investigation, focusing on the time required in the online phase by the two approaches, the communication overhead and RAM occupancy.

The results show that the approach proposed in [195] requires, on average, two orders of magnitude more time than our solution. At the same time, the solution in [195] requires less communication overhead than ours, especially on the drone. This is due to the communication overhead introduced by the *OTExtension*, which is used by our scheme during the online phase to generate and exchange the shares of the circuit between the involved entities. Finally, we notice that our solution also requires more RAM than [195]. Indeed, building, maintaining, and executing operations on circuits requires more RAM than performing additions in the encrypted domain.

5.5.6. Discussion

The results of our experiments show that system performance degrades when the size of requests and policies increases. Although in extreme cases (request size 20 and policy size 50) the privacy-preserving evaluation of multi-party access control policies on constrained devices is challenging, the experiments demonstrate that the proposed scheme is feasible in

Table 5.2: Comparison of privacy-preserving access control approaches.

	Crypt. Tech.	Outsourced Third Party		Support for Multi-party Data Governance	Attribute Confidentiality	Decision Untraceability	Constrained Environments
		Fully-Trusted	Semi-Trusted				
Kominos et al. [137]	CP-ABE	●	○	○	●	○	○
Nishide et al. [158]	CP-ABE	●	○	○	◐	○	○
Zhang et al. [249]	CP-ABE	●	○	○	●	○	○
Fan et al. [86]	CP-ABE	●	○	○	●	○	○
Kan et al. [240]	CP-ABE	●	○	○	●	○	○
Ming et al. [154]	CP-ABSC	●	○	○	●	○	○
Xu et al. [239]	HE	○	●	○	●	○	○
Kapadia et al. [133]	HE	○	●	○	●	○	○
Alishahi et al. [195, 196]	HE; SFE	○	●	●	●	●	○
Our scheme	SFE	○	●	●	●	●	●

real-world DaaS scenarios when a reasonable number of attributes needs to be evaluated and when reasonably complex policies are deployed. For instance, the drone takes approximately $1,599$ ms to complete the online phase for a request size of 8 and a policy size of 10 (cf. Fig. 5.3). In the same settings, our solution requires approx. 2.23 Mbytes of communication overhead and 85 Mbytes of RAM space, being tolerable for an actual commercial drone. We highlight that in such configurations the consumed energy is approx. 0.003% of the overall battery capacity, not requiring to equip the drone with large batteries.

By comparing *Testbed 1* with *Testbed 2*, we also notice differences in the impact of the request size on the execution time of the online phase on different devices. By comparing Figs. 5.3(b) and (f) for the STP running in Docker, the time is lower for *Testbed 1* for requests of size 5 or less; for larger requests, lower values are observed in *Testbed 2*. Such an unexpected behavior is caused by the *OTextension* of the ABY framework, as more communication overhead is required between Docker and the host system for larger requests. This is confirmed by comparing Figs. 5.5(b) and (f), where we do not observe this behavior. Indeed, the request size for such experiments is the same, not affecting the execution time.

Finally, the results reported in Fig. 5.7 confirm that our approach requires much less time than the PHE-based solution in [196], being more suitable for DaaS scenarios. The reduction in the execution time comes at the cost of an increased communication overhead and RAM. However, these values remain reasonably low to be tolerable on modern commercial drones.

In Sec. 5.6, we evaluate to what extent the solutions available in the current literature consider and address the requirements mentioned in Sec. 5.2.3, motivating the need and novelty of our solution.

5.6. Related Work

Several works investigated privacy-preserving access control solutions in the IoT domain. We summarize existing proposals w.r.t. the requirements identified in Sec. 5.2.3 in Table 5.2.

Ciphertext-Policy Attribute-Based Encryption (CP-ABE) schemes are often proposed to protect resources from unauthorized access in the IoT domain. However, these schemes typically do not protect policy confidentiality. Several works have addressed this gap and pro-

posed schemes that extend CP-ABE with other techniques to also ensure the confidentiality of the policies used to protect the resources. For instance, Kominos et al. [137] propose a privacy-preserving attribute-based encryption scheme that combines CP-ABE with pseudonyms to anonymize identifying attributes in the policy. Nishide et al. [158] combine CP-ABE with wildcards to protect attribute confidentiality. In this scheme, an encryptor uses wildcards to hide attributes that are not relevant to the ciphertext policy. This way, the decryptor does not learn the entire policy but only the attributes that are not "hidden behind the wildcards", thus only partially achieving attribute confidentiality. Zhang et al. [249] combine CP-ABE with hashing to hide the attributes defined in the policy. Other works propose schemes that combine CP-ABE with Linear Secrete Sharing Scheme (LSSS) [86, 240]. For instance, the scheme in [86] uses a one-way anonymous key agreement protocol to anonymize the attributes in the policy and stores the attribute mapping in an LSSS matrix. The matrix is then secretly shared with different parties. An authorized DC needs at least a given number of shares to reconstruct the matrix, and thus, to securely authenticate the attributes if the anonymized ones are present in the matrix. The DC can decrypt the ciphertext only if the anonymized attributes contained in the matrix are securely authenticated, without knowing the entire set of anonymized attributes in the LSSS matrix. A similar solution is proposed by Kan et al. [240], using a Bloom Filters (BF) to mask the attributes in the LSSS matrix, so that the decryptor is not able to infer the policy used to protect the resource. To achieve attribute confidentiality, Yang et al. [154] propose a solution based on Ciphertext-Policy Attribute-Based Signcryption (CP-ABSC), an attribute-based encryption and signing scheme, in combination with LSSS and Cuckoo Hashing (CH).[5] This solution allows the DO to sign and encrypt a resource using two different policies defined over different sets of attributes hidden in the LSSS matrix using CH.

Although these approaches protect the attributes in the policy attached to the encrypted resource, they are not fully suitable for DaaS applications. First, they can only deal with resources that are disclosed to the DC and do not support the protection of resources that can be only accessed on the drone (e.g., sensors). Moreover, the cryptographic operations underlying CP-ABE and CP-ABSC (e.g., pairing-based decryption, attribute generation, and certificate management) are computationally expensive and, thus, existing schemes tailored to constrained devices typically propose to outsource their computation to a third party. Such outsourcing, however, might not be possible in DaaS applications since persistent Internet connectivity is often unavailable. In addition, the third party is required to be fully trusted. By contrast, in our context, drones operate in unknown and untrusted environments, where they cannot rely on any fully-trusted party. Such schemes also assume that the resources to be protected are governed by a single entity. Although multi-authority-based encryption schemes are available [50, 51], they do not support the protection of co-owned resources. To the best of our knowledge, there are no CP-ABE schemes supporting multi-party data governance.

Another body of research investigated the problem of privacy-preserving policy evaluation using HE. For instance, Kapadia et al. [133] use HE together with proxy encryption to preserve attribute confidentiality. The data owner encrypts the attributes in the policy with its

[5]CH is similar to BF, which supports the dynamic insertion and removal of elements in a bit array, thus being more efficient and with a lower false-positive rate.

public key by using the ElGamal cryptosystem. The server checks whether the attributes of the data consumer match with the encrypted ones and "homomorphically multiplies" the matching encrypted attributes with random values, which are disclosed along with the ciphertext to the client. The DC decrypts the attributes to decrypt the ciphertext without learning the individual values of the attributes associated with the ciphertext. Xu et al. [239] propose a privacy-preserving ABAC scheme based on the ElGamal cryptosystem to protect user attributes. This scheme allows a DC to disclose its sensitive attributes in an encrypted form, which are evaluated by the server against an ABAC policy using a privacy-preserving policy matching component without gaining any information about the DC's attribute. Although this scheme does not require a trusted third party to protect user attributes, it only supports the privacy-preserving evaluation of clauses containing only one comparison operator. These proposals are not suitable for the DaaS scenario because they assume resources

owned by a single entity. To the best of our knowledge, the only privacy-preserving solutions based on HE supporting a multi-party governance model are the ones in [195, 196]. Alishahi et al. [196] propose a framework for the privacy-preserving evaluation of multi-party access control policies. The framework allows every party to independently specify policies regulating the access to their resources, which are deployed in the resource server in an encrypted form. The (encrypted) policies are combined according to a predefined governance model and evaluated under encryption, thus preserving the confidentiality of local policies and ensuring decision untraceability. The work in [195] extends [196] by supporting the privacy-preserving evaluation of multi-party access control policies expressed in ABAC. The works in [195, 196] also investigate the use of SFE as an enabler for the privacy-preserving evaluation of multi-party access control policies, showing that SFE provides a more efficient solution compared to HE both in terms of computation time and bandwidth usage. Differently from schemes based on CP-ABE and CP-ABSC, which require a fully-trusted third party when operating on constrained devices, solutions based on HE and SFE only require a semi-trusted party to decouple access control operations, thus preserving policy confidentiality. Overall, as shown in Table 5.2, the work that fulfills the most requirements is [195]. However,

such a proposal cannot be directly integrated into the DaaS scenario. Such a scheme was proposed to address privacy issues in Online Social Networks (OSNs), where persistent Internet connectivity is always available, and computational capabilities are always at the disposal of both the client and the server. Conversely, in the DaaS scenario, the drone should make access control decisions offline, without connecting to the Internet or featuring a persistent reliable connection to a server in the Cloud. At the same time, the drone should also minimize energy consumption to increase lifetime. Our solution addresses these considerations, contextualizing the solution proposed by [195] in the DaaS scenario and mitigating its limitations in terms of processing and suitability for constrained environments (non-persistent Internet connection and energy limitations). We realized a prototype of our solution and performed a thorough experimental performance evaluation, showing its viability in the DaaS domain.

5.7. Conclusions

In this work, we proposed a privacy-preserving multi-party access control solution for emerging Third-Party UAV Services, a.k.a., Drone-as-a-Service. As a distinctive advantage, our solution allows multiple DOs to provide their policies in a private form. These policies are evaluated in a privacy-preserving way by relying on the SFE paradigm, thus protecting sensitive information therein. We deployed a proof-of-concept of our solution on two testbeds, emulating the drone through a Raspberry PI 3 Model B+ device. We showed that our solution achieves privacy-preserving multi-party policies evaluation for system configurations typical of real-world use cases in a reasonable time. We also evaluated communication overhead, RAM occupancy, and energy consumption, showing the suitability of the proposed approach for Drone-as-a-Service scenarios. Overall, our work makes a step further in demonstrating the viability of privacy-preserving techniques on commercial drones. Future work is needed to test our solution on real drones.

6

Obscura: Privacy-Preserving Face Verification on Constrained Drones

This chapter is inspired by:
D.R. George, S. Sciancalepore
Obscura: Privacy-Preserving Face Verification on Constrained Drones
2025 Submitted and Under Review

Commercial drones used nowadays for goods delivery in many urban and remote areas increasingly execute critical operations onboard without relying on the public Internet, so to provide reliable services also when the public Internet is congested or not available at all. Such drones use state-of-the-art biometric verification techniques to hand out goods securely, e.g., by verifying the face of the recipient of the goods. As briefly highlighted in Chapter 1.1.3, on-device processing of sensitive biometric information creates a serious privacy risk. Indeed, when an adversary seizes the drone, they can extract the stored biometric data, posing privacy risks for the users. In this chapter, we propose *Obscura*, the first solution for privacy-preserving face verification onboard of commercial drones. This contributes to achieving *input privacy* and addresses **SRQ3** and **SRQ4**.

6.1. Introduction

UAVs, a.k.a. *drones*, are increasingly used nowadays for various automated tasks in several application domains, including goods delivery, search-and-rescue, and patrolling [23]. The increasing popularity of drones is also confirmed by leading market analysis agencies, projecting the only drone delivery market to grow to 4.6 USD billion by 2030 [151].

Commercial drones deployed nowadays usually rely on Cloud servers available on the public Internet to perform the most computationally intensive and energy-consuming tasks [46, 113]. However, due to the real-time nature of many operations and QoS considerations, e.g.,

collision avoidance and smooth interaction with people, reliance on the public Internet is often problematic. In crowded urban areas, wireless links over WiFi and cellular networks often become congested, significantly increasing the time to receive service [187, 198]. Internet is also *best-effort* by definition, i.e., it does not provide any guarantee that data are delivered effectively or that the delivery meets any QoS requirement [192, 201]. Moreover, drones often have to operate in scenarios where connection to the public Internet is unreliable or completely absent, e.g., remote or hard-to-reach areas [101, 145]. To execute operations reliably in the above-described scenarios, drones need to operate without communicating to any Internet-connected entity, while storing onboard all information necessary to complete the planned tasks. When considering goods delivery via drones, such considerations apply, among others, also to biometric face verification techniques used to verify the identity of the consignee of the goods. As of today, all drones using biometric face verification techniques onboard for goods delivery verify the identity of the consignee on the spot, via a camera and a face verification algorithm [100, 130, 193].

However, when operating beyond the line-of-sight of the operator in an unattended way, drones are naturally exposed to physical and cyber-attacks by attackers active in the area of operation [209]. Such attackers could capture the drone and, beyond stealing transported goods, also access the content of their memory, causing the leakage of any stored information, including potentially private users' data [19, 172, 219]. When the drone is captured and the stored data are leaked, private users' information stored onboard, provided at registration time to the SP, become available to the adversary. Such data leakage constitutes an issue both for the user and for the SP. As for the user, the adversary could use such biometric features to impersonate the user [194]. Moreover, regulations such as the GDPR and the upcoming Cyber Resilience Act (CRA) mandate the SP to keep data provided at registration time fully private, and thus, the SP would be held responsible for any disclosure of such data to unauthorized parties [77, 83]. To protect against such threats, we need a solution allowing a drone, featuring no Internet, to securely and accurately verify the identity of the goods' recipient without storing any sensitive users' information onboard. At the same time, we should take into account that drones are energy-constrained devices, which should preserve energy as much as possible so as not to consume all the available energy too quickly, before the expected end of their planned mission.

To the best of our knowledge, no solutions are available in the literature to solve this problem. Some solutions allow to execute privacy-preserving face recognition and verification (see Sec. 6.8), but they consider different domains, e.g., biometric verification to an online SP or face verification at the gate of an airport. Thus, the straightforward usage of such solutions in the (constrained) scenario discussed above brings significant challenges.

Contribution. In this chapter, we introduce Obscura, a novel solution for privacy-preserving face verification on constrained drones. In a nutshell, Obscura allows a drone, deployed in area with no reliable Internet connectivity, to verify the identity of a person without storing locally any PII, i.e., the facial features provided by such person to the SP. We design Obscura by smartly combining the MPC paradigm with the latest additive secret sharing techniques. We describe the details of our solution and we prove its security against an adversary accessing the drone at deployment time. Moreover, we provide an extensive performance evaluation of Obscura on a proof-of-concept deployed over two relevant testbeds, using as devices

the Raspberry Pi Model 3B+ and the Intel NUC7i5BNK. On the most constrained device (Raspberry Pi Model 3B+), Obscura requires approx. 27 ms to verify the face of a user, while also keeping the verification accuracy intact compared to the plain-text version of the inner face recognition algorithm (up to 97.2%). We also compare our solution to the closest related work (adapted to solve this problem), showing significant savings (more than 50%) in terms of execution time and energy consumption. Finally, we release the source code of Obscura as open-source at [73], allowing the scientific community to replicate our results and encouraging its deployment and further extension.

Overall, our work contributes to highlight the increasingly important role of PETs in the deployment of secure and privacy-aware IoT systems, motivating the development and engineering of new solutions in this domain capable of running on constrained devices without affecting their usability and lifetime.

Roadmap. This chapter is organized as follows. Sec. 6.2 introduces preliminaries, Sec. 6.3 describes the considered scenario, adversarial model, use case and requirements, Sec. 6.4 provides the details of Obscura, Sec. 6.5 discusses the security of Obscura, Sec. 6.6 provides the extensive experimental evaluation of Obscura, Sec. 6.8 reviews related work and qualitatively compares Obscura to the literature and finally, Sec. 6.9 concludes the chapter.

6.2. Preliminaries

In this section we introduce preliminary concepts necessary to fully understand our contribution, i.e., secure MPC (Sec. 6.2.1), optimized additive secret sharing (Sec. 6.2.2) and cosine similarity (Sec. 6.2.3). Tab. 6.1 summarizes the main notation used in the chapter.

6.2.1. Secure Multi-Party Computation

Secure MPC techniques allow several parties to compute the result of a shared function without revealing their private inputs [84]. In this work, we consider MPC techniques allowing two non-colluding parties to compute a function by preserving their *input privacy*. Moreover, we leverage the *offline-online* paradigm of MPC [81]. According to such functions, during the *offline phase*, a trusted dealer or an offline interaction are used by the parties to generate input-independent randomness. During the *online phase*, the parties use the randomness with the private inputs to compute the function. The offline-online paradigm is particularly beneficial in scenarios with computational constraints, e.g., on IoT devices and for real-time computation, to name a few. To generate input-independent randomness, several algorithms can be used, including the well-known Beaver triples [21] or YAO's garbled circuits [241]. To allow for shared computation of the mathematical function, secure multiparty computation techniques represent the function through Boolean or arithmetic circuits. These circuits allow to construct linear and non-linear operations. Linear operations such as addition and multiplication over a large rings can be executed efficiently. However, the execution of non-linear operations, e.g., integer comparison, division, and square root, could lead to computational overhead. In the following, we discuss the input-independent

Table 6.1: Notation used throughout the chapter.

Notation	Description
ϵ	Adversary.
$\mathcal{S}$	Simulator.
$\mathbf{x}$	Reference template of the user.
$\mathbf{y}$	Live template.
x_i, y_i	Components of vector $\mathbf{x}$ and $\mathbf{y}$ with index i.
d	Dimension (length) of the vector $\mathbf{x}$ and $\mathbf{y}$.
P_j	Generic party, with $j \in \{0, 1\}$.
τ	Publicly known threshold for face verification.
$Sign(\cdot)$	Sign function.
$\wedge$	Logic AND operator.
$\leftarrow\!\!\$$	Uniformly at random allocated value.
l	Bit length.
$\mathbb{Z}_{2^l}^d$	Ring for the secret sharing.
$\boldsymbol{\delta}_{\mathbf{x}}, \boldsymbol{\delta}_{\mathbf{y}}$	Values masking $\mathbf{x}$ and $\mathbf{y}$.
$\boldsymbol{\delta}_{\mathbf{xy}}, \boldsymbol{\delta}_{\mathbf{xx}}$	δ-shares for the inner product.
$\boldsymbol{\delta}_{\mathbf{x_j}}, \boldsymbol{\delta}_{\mathbf{y_j}}$	Shares of $\boldsymbol{\delta}_{\mathbf{x}}, \boldsymbol{\delta}_{\mathbf{y}}$.
$\boldsymbol{\delta}_{\mathbf{xy_j}}, \boldsymbol{\delta}_{\mathbf{xx_j}}$	Shares of $\boldsymbol{\delta}_{\mathbf{xy}}, \boldsymbol{\delta}_{\mathbf{xx}}$.
$\boldsymbol{\Delta}_{\mathbf{x}}, \boldsymbol{\Delta}_{\mathbf{y}}$	Masked values of $\mathbf{x}$ and $\mathbf{y}$.
$\langle\cdot\rangle$	Π-secret sharing.

randomness generation and which mathematical functions we represent as an arithmetic circuit.

6.2.2. Optimized Additive Secret Sharing

Secret sharing techniques allow to break up a secret into multiple pieces (namely, shares) distributed to several entities, such that none of these entities have complete knowledge of such a secret [163, 84]. In this work, we use an Optimized Additive Secret Sharing technique inspired by the works in [163, 115], which is particularly efficient for 2-party computation. According to such a technique, a trusted party holding a secret value x can distribute (securely) pieces of such a secret to two parties P_j ($j \in \{0, 1\}$). The parties do not know the secret value, but they can perform operations over such value. To this aim, the semi-trusted party generates a random value δ_x, and uses it to compute the mask value $\Delta_x = x + \delta_x$. Then, it generates two shares of δ_x, namely $\delta_{x_0}, \delta_{x_1}$, with $\delta_x = \delta_{x_0} + \delta_{x_1}$, and distributes (δ_{x_0}, Δ_x) to P_0 and (δ_{x_1}, Δ_x) to P_1. Using the shares $\delta_{x,j}$ and the mask Δ_x, the parties perform secure function evaluation, i.e., operations over the secret x without knowing its value.

For addition and subtraction, both parties compute locally an intermediate result of the operation based on the given input shares. Then, they exchange the shares resulting from the computation to reconstruct the result of the secure addition or subtraction. The arithmetic

operation for addition is defined as in Eq. 6.1, as per [163]:

$$\begin{aligned} z &= x + y \\ &= (\Delta_x - \delta_x) + (\Delta_y - \delta_y) \\ &= (\Delta_x + \Delta_y) - (\delta_x + \delta_y) \\ &= \Delta_z - \delta_z \\ &= (\Delta_x + \Delta_y) - (\delta_{z_j} + \delta_{z_{1-j}}) \\ &= (\Delta_x + \Delta_y) - ((\delta_{x_j} + \delta_{y_j}) + (\delta_{x_{1-j}} + \delta_{y_{1-j}})) \\ &= (\Delta_x - \delta_x) + (\Delta_y - \delta_y) \\ &= x + y, \end{aligned} \tag{6.1}$$

where $\delta_{x_j}, \delta_{y_j}, \delta_{x_{1-j}}, \delta_{y_{1-j}}$ are δ-shares which are uniformly randomly generated by the parties from $\mathbb{Z}_{2^l}$. Indeed, if the function f contains only linear operations, then no communication between the parties is required except for the reconstruction of the shares. However, to perform secure multiplication, communication between the parties is necessary. Additionally, such operation requires function-dependent precomputed uniformly random δ-shares known as Beaver's triple [21]. To compute $z = x \cdot y$ securely, the semi-trusted party can generate the Beaver's triples during the setup phase, i.e., $\delta_x, \delta_y, \delta_{xy}$, where $\delta_{xy} = \delta_x \cdot \delta_y$ and distribute the shares of the triples to the parties P_j. To perform secure multiplication, the steps in the following Eq. 6.2 are executed.

$$\begin{aligned} z_j &= j \cdot \Delta_x \Delta_y - \Delta_x \delta_{y_j} - \Delta_y \delta_{x_j} + \delta_{xy_j} \\ z_{1-j} &= (1-j) \cdot \Delta_x \Delta_y - \Delta_x \delta_{y_{1-j}} - \Delta_y \delta_{x_{1-j}} + \delta_{xy_{1-j}} \\ z &= x \cdot y \\ &= z_j + z_{j-1} \\ &= \Delta_x \Delta_y - \Delta_x (\delta_{y_j} + \delta_{y_{1-j}}) - \Delta_y (\delta_{x_j} + \delta_{x_{1-j}}) + (\delta_{xy_j} + \delta_{xy_{1-j}}) \\ &= \Delta_x \Delta_y - \Delta_x \delta_y - \Delta_y \delta_x + \delta_{xy} \\ &= \Delta_x \Delta_y - \Delta_x \delta_y - \Delta_y \delta_x + \delta_x \delta_y \\ &= \Delta_x (\Delta_y - \delta_y) + (\delta_y - \Delta_y) \delta_x \\ &= y \Delta_x - y \delta_x = y (\Delta_x - \delta_x) \\ &= x \cdot y. \end{aligned} \tag{6.2}$$

In this chapter, we use the technique provided by the authors in [163], improving the multiplication gate to reduce the communication overhead. Using such a technique, only one communication round is required to reconstruct the result of the inner product ($\sum_{i=1}^{d} (x_i \cdot y_i)$), independently of the vector dimension d.

6.2.3. Cosine Similarity

The cosine similarity is a well-known distance metric between two vectors $\mathbf{x}$ and $\mathbf{y}$, quantifying the angle between such vectors, as in Eq. 6.3 [31].

$$cos(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^{d} x_i \cdot y_i}{\|\mathbf{x}\| \cdot \|\mathbf{y}\|}. \tag{6.3}$$

In this work, we use the cosine similarity metric for biometric verification. Consider an user submitting the biometric reference template $\mathbf{x}$ during enrollment and the live biometric template $\mathbf{y}$ during verification. The cosine similarity between the two vectors indicates how much similar the two vectors are. The user is successfully verified if the value of the cosine similarity between the two vectors is greater than or equal to a known threshold τ. Otherwise, the user is not verified. For this work, we use an adapted version of the cosine similarity, $\mathcal{F}_{cosVer}(\mathbf{x}, \mathbf{y}, \tau) = Sign(cos(\mathbf{x}, \mathbf{y}, \tau))$, inspired by the recent contribution by Cheng et al. [55]. Such computation strategy does not use non-linear operations, thus being particularly suitable for implementation through a circuit for MPC. The functionality $\mathcal{F}_{cosVer}(\mathbf{x}, \mathbf{y}, \tau) = c = c_1 \wedge c_2$ is defined as in Eq. 6.4:

$$\begin{aligned}
c_1 &= \text{Sign}\left(\sum^{d}(x_i \cdot y_i)\right), \\
c_2 &= \text{Sign}\left(\frac{1}{\tau^2}\left(\sum^{d}(x_i \cdot y_i)\right)^2 - \left(\sum^{d}(x_i \cdot x_i)\right) \cdot \left(\sum^{d}(y_i \cdot y_i)\right)\right), \\
c &= \begin{cases} c_1 \wedge c_2 & \text{if } \tau > 0, \\ c_1 & \text{if } \tau = 0, \\ c_1 \vee \neg c_2 & \text{if } \tau < 0, \end{cases}
\end{aligned} \tag{6.4}$$

where $\tau \in [-1, 1]$ is the threshold. For executing them securely, we represent the inner products as circuits and compute them securely, while performing the remaining operations in plain-text.

6.3. Scenario, Adversary Model and Requirements

6.3.1. Scenario

We consider the general scenario of a drone operating without reliable Internet connection, which needs to verify the identity of a person to hand out a package. We identify four entities: (i) the SP, (ii) the drone, (iii) the user, and (iv) the Wireless Device (WD).

The SP is an organization that conducts business or distributes sensitive content via drone delivery, e.g., a logistics company, a pharmacy, a medical supplier, the military, or a non-profit

organization, to name a few. The SP is the entity deploying drones for providing services in any areas, as requested by users. When not deployed for missions, drones are hosted at dedicated parking spots provided by the SP. Here, the drone has reliable Internet connectivity to communicate with the SP servers, e.g., for regular maintenance, updates, and to receive information for the next mission. When deployed for a mission, the drone may operate in areas that are characterized by unreliable, intermittent or absent Internet connectivity, e.g., congested places (events full of people) and hard-to-reach areas. Thus, we consider the worst-case scenario where, when deployed on a mission, the drone cannot connect to other remote services over the Internet. Note that, even if Internet connection may be possible, e.g. via satellite communication services, we consider that the drone cannot reliably use such connection, as it provides no QoS guarantees. In line with many commercial products, we consider a drone equipped with medium-range communication technology, such as WiFi, used for telemetry and delivery of the video feed to a remote controller [132]. At the same time, the drone can use the WiFi communication technology to establish secure communication links on the fly with nearby WiFi-enabled entities without relying on any third-party, e.g., through WiFi-direct [69]. The drone also has energy constraints, i.e., it operates on battery power, and thus, it possesses limited energy available for flying and executing computations. Even though drones are becoming increasingly powerful in terms of computational capabilities, they still require significant processing power to handle complex tasks like autonomous navigation, object detection, and real-time data processing [167]. Further, we consider drones equipped with a camera to verify the identity of the user to whom the payload is delivered. Thus, the drone delivers the payload to a user only if the picture taken at delivery time matches an expected template information available for the particular user. To aid biometric verification, the drone could communicate with other entities, e.g., the WDs possessed by users. WDs are personal devices possessed by the user, e.g., a smart card or an IoT device like a smart-watch, equipped with wireless connection capabilities via WiFi. We consider that the WD does not feature a camera, in line with the capabilities of most portable IoT devices (smart watches, smart cards). Also the WD possessed by the user features no reliable Internet connection when interacting with the drone (for the same reasons discussed above). However, it can connect to the drone and exchange information using the dedicated ad-hoc wireless connection provided by the drone. Note that any authentication process involving the drone and the WD is not enough to verify the identity of the person to whom the goods are delivered. Indeed, WDs may be stolen and used by unauthorized people. Thus, verifying users' biometrics provides enhanced authentication guarantees—the same as authentication via *something you are* provides more guarantees than authentication via *something you own* [35].

The *User* in our scenario is a person or organization requiring a service, e.g., military personnel, paramedics, law enforcement, or aid recipients, to name a few. In some cases, the user may be affiliated with the SP. To utilize the delivery service offered by the SP, the user preliminarily registers with the SP, providing the necessary PII, e.g., biometric information, to allow for the verification of the user's identity at runtime. Such a registration occurs preliminarily to delivery operations, where the user is physically present to register at the SP and register their WD to the SP, e.g., through the MAC address of such device. Our system design does not require the SP to be persistently available over the Internet, but only to be reachable for allowing users to register for service. This allows us not to rely on trusted

third parties or require online connections while verifying the user's identity efficiently and privately — even in constrained environments.

6.3.2. Adversary Model

The adversary considered in this work, namely ϵ, takes control of the drone during its deployment in the field for the mission. The overall aim of the the adversary ϵ is to retrieve the PII of the user to which the delivery is destined, i.e., the biometric information provided by the user to the SP at registration time.

To achieve this objective, ϵ features passive and active capabilities. More in detail, ϵ is a global, spatially-unbounded, and frequency-unlimited eavesdropper, able to detect any RF activity initiated by the drone. Using such capabilities, ϵ can passively listen all the communications between the drone and the WD, e.g., capturing the packets sent between the entities.

ϵ can physically capture the drone during the *deployment phase* to access and extract the *biometric templates*. This enables ϵ to use popular data retrieving tools like the one in [44] to retrieve any information previously stored on the drone that was deleted, discarded, or overwritten. We also consider that the adversary ϵ only has control of the drone and not of the WD of the user (see Sec. 6.5 for a discussion about this scenario). Also, Denial of Service is out of scope. In this context, the aim of our solution is to perform face verification of the user while guaranteeing the privacy of the private information available on the drone, i.e., the PII of the user, so that any attempt to steal such information by tampering with the drone is unsuccessful.

6.3.3. Use case

As a practical use case of our scenario, we consider a drone used to distribute aid in remote and hard-to-reach areas, as recently considered by Kasra et al. in [79]. Such a drone has to travel through underdeveloped and scarcely populated areas, where it cannot rely on stable and persistent Internet. At the same time, face verification allows fairness in aid distribution through drones since humanitarian organizations typically have limited resources [79]. During registration in a central distribution hub, humanitarian organizations can also easily distribute cheap WDs to aid recipients, for example, smart cards with readers or restricted wireless devices with a WiFi module [231].

In this scenario, the drone can be targeted by attacks, where malicious groups could steal the transported medical supplies and capture the drone. This groups could further sell the drone to more expert groups or even state-sponsored actors, which have the tools to extract the biometric templates stored onboard and use them further in an unauthorized way, e.g., to sell them on the black market. When such a drone stores biometric information of the user onboard, such information could be extracted and made available to unauthorized parties. Without a solution in place, the loss of the PII can lead to lawsuits for the delivery services and to an overall decrease in the trust of the citizens to the technology, with consequent significant loss of revenues. The aid recipients could suffer an even greater loss, since such

organizations typically do not want their identity to be exposed either, which leads to reduced trust towards humanitarian organizations.

Therefore, in such a scenario, it is crucial to deploy a solution which can protect the (private and sensitive) information of the users stored onboard even when the drone is captured by the adversary, while guaranteeing timely and efficient users' face verification.

6.3.4. Requirements

From the previous discussion, we can extract several system and security requirements that a suitable solution should require.

First, our solution should consider that the drone and the user, including the WD possessed by such a user, do not have a reliable Internet connection at delivery time. Therefore, any computation necessary for face verification needs to be done while being *offline* (**R1**). Moreover, both devices cannot rely on any *TTP* during face verification (**R2**). Our use case also requires biometric verification since aid recipients may not have valid IDs. Thus, the drone should be equipped with a camera instead of a fingerprint or microphone sensor, and our solution necessitates biometric templates based on the users' facial features (**R3**). We also require a solution for face verification characterized by high *accuracy* (**R4**) in person identification, so to effectively fulfill the purpose of reliable and correct goods delivery. Furthermore, to protect the biometric template of a person against stealing from an adversary who gains possession of the drone, the biometric template of the user should not be stored on the drone (**R5**). Since the adversary could access all the content available on the drone, we should also avoid storing cryptography materials onboard leading to retrieving sensitive information, e.g., secret keys, whose leakage would give the adversary the possibility to obtain PII (**R6**). We also require that, at the end of the protocol, the drone should be able to take the final decision about the matching of the user's identity with the intended recipient, so to hand out the package, or not (**R7**). Finally, we should consider the computational and energy constraints of the drone, which could also have to identify multiple users. Therefore, the solution to be designed should be computationally lightweight and energy-friendly, so being suitable for deployment on *constrained devices*, such as drones (**R8**). To the best of our knowledge, the current literature does not provide solutions fulfilling all the above requirements (see Sec. 6.8). We summarize the aforementioned requirements in Tab. 6.2.

6.4. Obscura

6.4.1. Rationale of our Solution

To protect against the adversary discussed in Sec. 6.3.2 and fulfill the requirements laid down in Sec. 6.3.4, we need to carry out face verification on the drone without storing locally sensitive information. To this aim, we propose Obscura, a privacy-preserving face verification protocol conceived specifically for such scenarios. Obscura includes two phases: the Setup Phase (Sec. 6.4.2) and the Deployment Phase (Sec. 6.4.3).

Table 6.2: Requirements table with summarized description.

ID	Requirements	Description
R1	**Offline Computation**	The computation on the drone and the WD is done *offline*, because the Internet connection is unreliable.
R2	**No TTP**	Neither the drone nor the WD rely on a TTP during face verification.
R3	**Face Biometrics**	The scheme uses facial features (camera-based) instead of fingerprints or voice.
R4	**High Accuracy**	The scheme must achieve high *accuracy* for face verification .
R5	**No Onboard Templates**	The drone must not store raw reference biometric templates.
R6	**No Onboard Secrets**	The drone must not store any cryptographic secrets that could reveal PII.
R7	**Onboard Decision**	The drone must take the final match/no-match decision to release the goods.
R8	**Constrained Environments**	The scheme must be lightweight and energy-friendly for *constrained devices*.

During the setup phase, the user subscribes to a delivery service offered by the SP. To this aim, the user connects to the SP through a regular Internet connection, secured via common transport-layer security protocols, like TLS. Over such a secure connection, the user provides a reference image of itself, used by the SP to extract the user's facial properties for biometric face recognition, namely, the user's *face embeddings*, forming the *reference template*. Our solution uses a secure two-party computation technique based on the *offline-online* model. We use the ABY 2.0 sharing technique [163], a.k.a, Π — secret-sharing [115], which allows to generate the δ-shares for the multiplication and the masks for the face-embeddings (see Sec. 6.2.2). The shares and the masked embeddings generated through such a technique are then distributed to the two entities which, during the deployment phase, are active on behalf of the SP and the user, i.e., the drone and the user's WD, respectively. Note that pre-sharing the δ-shares allows us not to store sensitive information (i.e., the user's PII reference biometric template) on the WD and the drone. Also note that, during the setup phase, the drone is hosted at the SP location, featuring persistent Internet connection.

During the deployment phase, the drone is deployed by the SP for a mission provided by the user, e.g., transporting goods like food, medicine, and medical kits, to a place with scarce or absent Internet connection. To hand over the goods securely, the drone needs to verify the user's *live biometric template* against the one provided at setup time, without storing any sensitive information. To this aim, the drone captures the live image of the user through the onboard camera, extracts the corresponding image embeddings, applies the technique described in Sec. 6.2.2 to mask the embedding of the collected live biometric template and

forwards the masked values to the user's WD. To compare the two embeddings and verify the user's biometric template, the drone and WD uses their shares to evaluate privately a shared function, i.e., the inner product between the two vectors. After evaluation, the WD sends the result to the drone, which obtains the final result of the comparison operation. If the reference template and live biometric template are similar enough (based on a threshold), the verification is successful and the drone hands over the package to the user. Otherwise, the drone does not hand over the package, and may move away.

Note that, if the adversary discussed in Sec. 6.3.2 captures the drone, it learns no sensitive information about the user to which the goods are destined. We discuss this further in Sec. 6.5. Also note that our protocol reuses and extends some protocols initially proposed by the authors in [55, 115], and provided below as Protocol 2, 3,and 4. As discussed more in detail in Sec. 6.8, the authors in [55, 115] consider a problem which shares some similarities with the one considered in this chapter. However, compared to their solutions, the protocol described below executes some operations in plain-text, due to the different problem, adversary model and architecture that we consider. As shown by the results in Sec. 6.6, this contributes to make our solution more lightweight and ideal for running on constrained devices.

6.4.2. Setup Phase

The *Setup Phase* of Obscura aims to register all the actors to the SP and to equip them with the necessary cryptography material (δ-shares) to run Obscura. This phase is executed once, before the deployment of the drone for the mission, through a secure channel, e.g., TLS. Fig. 6.1 illustrates the steps required as part of the *Setup Phase* by the user, SP, WD, and drone, as discussed below.

- The user registers to the SP over a secure channel, providing to the SP a photo of itself. The SP extracts from such a photo a vector $\mathbf{x}$, namely the face embeddings, using commonly-applied Machine Learning (ML) techniques like Deepface [208]. We refer to the vector $\mathbf{x}$ as the *reference template* of the user. (see Fig. 6.1, step ①).
- The SP uses the *Setup* Protocol provided in Protocol 1, inspired by the protocol used by the authors in [115], to generate the δ-shares of the user's *reference template* x, namely $\boldsymbol{\delta}_{\mathbf{x}_0}, \boldsymbol{\delta}_{\mathbf{x}_1}, \boldsymbol{\delta}_{\mathbf{x}}$ (see Sec. 6.5 for more details). Then, it distributes the cryptography materials necessary to run the following phases of the protocol to the drone and to the WD of the user. As shown in Fig. 6.1 (step ②), following the execution of Protocol 1, the drone receives a share of the random value for the reference template $\boldsymbol{\delta}_{\mathbf{x}_0}$, the random value $\boldsymbol{\delta}_{\mathbf{y}}$ used to mask the *live template* $\mathbf{y}$, a share of the random value $\boldsymbol{\delta}_{\mathbf{y}}$, namely $\boldsymbol{\delta}_{\mathbf{y}_0}$. At the same time, the user's WD receives the other share of the random value for the reference template $\boldsymbol{\delta}_{\mathbf{x}_1}$, the other share of $\boldsymbol{\delta}_{\mathbf{y}}$, namely, $\boldsymbol{\delta}_{\mathbf{y}_1}$. As stated in [115] to realize the Protocol 1, the SP executes the Protocol 1 in a TEE to compute the preprocessing material for the inner-product, but in our case the drone and WD are leveraging 2-Party computation technique based on OT or HE. This allows the drone to have two auxiliary shares $\boldsymbol{\delta}_{\mathbf{xy}_0}$ and $\boldsymbol{\delta}_{\mathbf{xx}_0}$ necessary to execute the inner-product operation on the local shares of $\mathbf{x}$ and $\mathbf{y}$ at runtime. Similarly, the WD will have two auxiliary shares $\boldsymbol{\delta}_{\mathbf{xy}_1}$ and $\boldsymbol{\delta}_{\mathbf{xx}_1}$ necessary to execute the inner-product operation on the local shares

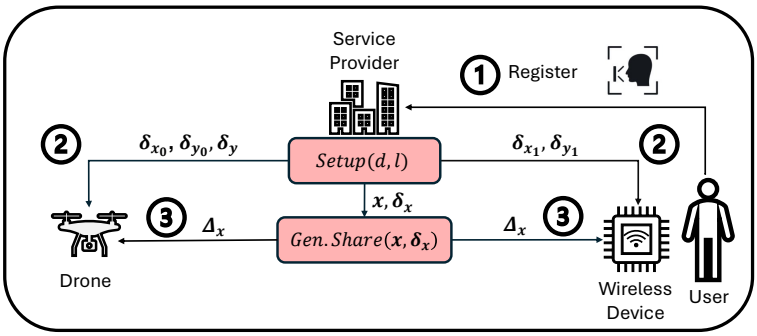

Figure 6.1: Setup Phase of Obscura.

of $\mathbf{x}$ and $\mathbf{y}$ at runtime. For details on how to obtain the δ-shares for the inner product through OT and HE, we refer to Sec. 3.1.3 in [163].

- The SP executes *Protocol* 2 (step ③) using as input the face embedding $\mathbf{x}$ of the user's biometric template and the generated randomness $\boldsymbol{\delta}_{\mathbf{x}}$, obtaining the masked value $\boldsymbol{\Delta}_{\mathbf{x}}$. Finally, the SP distributes $\boldsymbol{\Delta}_{\mathbf{x}}$ to the drone and the WD.

Note that, after the completion of the *Setup Phase*, neither the drone nor the WD need any further communication with the SP.

Protocol 1: $Setup(d, l) \rightarrow \langle \boldsymbol{\delta}_{\mathbf{x}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{y}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{xy}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{xx}} \rangle$

Parties: *Service Provider*.
Input: d (dimension of the input vector), l (bit length), $\mathbb{Z}_{2^l}$ (secret sharing ring).
1: $\boldsymbol{\delta}_{\mathbf{x_0}} \leftarrow\$ \mathbb{Z}_{2^l}^d, \boldsymbol{\delta}_{\mathbf{y_0}} \leftarrow\$ \mathbb{Z}_{2^l}^d$
2: $\boldsymbol{\delta}_{\mathbf{x_1}} \leftarrow\$ \mathbb{Z}_{2^l}^d, \boldsymbol{\delta}_{\mathbf{y_1}} \leftarrow\$ \mathbb{Z}_{2^l}^d$
3: $\langle \boldsymbol{\delta}_{\mathbf{x}} \rangle \equiv (\boldsymbol{\delta}_{\mathbf{x_0}}, \boldsymbol{\delta}_{\mathbf{x_1}}), \boldsymbol{\delta}_{\mathbf{x}} \leftarrow \boldsymbol{\delta}_{\mathbf{x_0}} + \boldsymbol{\delta}_{\mathbf{x_1}}$
4: $\langle \boldsymbol{\delta}_{\mathbf{y}} \rangle \equiv (\boldsymbol{\delta}_{\mathbf{y_0}}, \boldsymbol{\delta}_{\mathbf{y_1}}), \boldsymbol{\delta}_{\mathbf{y}} \leftarrow \boldsymbol{\delta}_{\mathbf{y_0}} + \boldsymbol{\delta}_{\mathbf{y_1}}$
5: $\boldsymbol{\delta}_{\mathbf{xy_0}} \leftarrow\$ \mathbb{Z}_{2^l}^d$
6: $\boldsymbol{\delta}_{\mathbf{xy_1}} \leftarrow (\boldsymbol{\delta}_{\mathbf{x_0}} + \boldsymbol{\delta}_{\mathbf{x_1}}) \cdot (\boldsymbol{\delta}_{\mathbf{y_0}} + \boldsymbol{\delta}_{\mathbf{y_1}}) - \boldsymbol{\delta}_{\mathbf{xy_0}}$
7: $\langle \boldsymbol{\delta}_{\mathbf{xy}} \rangle \equiv (\boldsymbol{\delta}_{\mathbf{xy_0}}, \boldsymbol{\delta}_{\mathbf{xy_1}})$
8: $\boldsymbol{\delta}_{\mathbf{xx_0}} \leftarrow\$ \mathbb{Z}_{2^l}^d$
9: $\boldsymbol{\delta}_{\mathbf{xx_1}} \leftarrow (\boldsymbol{\delta}_{\mathbf{x_0}} + \boldsymbol{\delta}_{\mathbf{x_1}}) \cdot (\boldsymbol{\delta}_{\mathbf{x_0}} + \boldsymbol{\delta}_{\mathbf{x_1}}) - \boldsymbol{\delta}_{\mathbf{xx_0}}$
10: $\langle \boldsymbol{\delta}_{\mathbf{xx}} \rangle \equiv (\boldsymbol{\delta}_{\mathbf{xx_0}}, \boldsymbol{\delta}_{\mathbf{xx_1}})$
Output: $(\langle \boldsymbol{\delta}_{\mathbf{x}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{y}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{xy}} \rangle, \langle \boldsymbol{\delta}_{\mathbf{xx}} \rangle)$

6.4.3. Deployment Phase

During the *Deployment Phase*, the drone verifies securely the face of the user and, if such verification is successful, it delivers the goods. Throughout the phase, the drone and the WD do not have access to any Internet connection. Thus, they work *offline* through a dedicated

Protocol 2: Gen.Share$(v, \delta_v) \rightarrow (\Delta_v)$

Parties: *Service Provider, Drone.*
Input: $\mathbf{v} \in \{\mathbf{x}, \mathbf{y}\}$ $\mathbf{x}, \mathbf{y}$(input vectors), $\boldsymbol{\delta}_\mathbf{v}$ (shares of the mask values).
1: $\boldsymbol{\Delta}_\mathbf{v} \leftarrow (\mathbf{v} + \boldsymbol{\delta}_\mathbf{v})$
Output: $\boldsymbol{\Delta}_\mathbf{v}$

WiFi ad-hoc network whose access point is hosted by the drone. We illustrate the operations executed during the *deployment phase* by the involved entities in Fig. 6.2, and we describe such operations more in details in the following.

1. The protocol is triggered when the drone reaches the destination location, and needs to verify the user's identity to hand over the package. As shown in Fig. 6.2, step ④, the drone captures the live image and extracts the corresponding embedding $\mathbf{y}$. Then, it deletes the live image.

2. The drone needs to compare securely the captured live image $\mathbf{y}$ with the reference template $\mathbf{x}$, to verify the user's identity. To this aim, it needs to compute securely the scaled cosine similarity according to Eq. 6.5, without having access to the plaintext embeddings $\mathbf{x}$.

$$result = \text{Sign}\left(\sum^{d}(x_i \cdot y_i)\right) \wedge \text{Sign}\left(\frac{1}{\tau^2}\left(\sum^{d}(x_i \cdot y_i)\right)^2 - \left(\sum^{d}(x_i \cdot x_i)\right) \cdot \left(\sum^{d}(y_i \cdot y_i)\right)\right). \tag{6.5}$$

   To this aim, the drone uses the masked value of the embedding of the live image $\mathbf{y}$, namely $\boldsymbol{\Delta}_\mathbf{y}$, according to protocol 2 introduced in the *Setup Phase*. Then, it delivers $\boldsymbol{\Delta}_\mathbf{y}$ to the WD (see Fig. 6.2, step ⑤). After delivering $\boldsymbol{\Delta}_\mathbf{y}$, the drone deletes the embeddings of the live image $\mathbf{y}$, e.g., by replacing them with random values.

3. At message reception, the WD computes the *inner product* operations in Eq. 6.5 on its local shares, i.e., between $\boldsymbol{\Delta}_\mathbf{x}$, $\boldsymbol{\Delta}_\mathbf{y}$, $\boldsymbol{\delta}_{\mathbf{x}_1}$, $\boldsymbol{\delta}_{\mathbf{y}_1}$, $\boldsymbol{\delta}_{\mathbf{x}\mathbf{x}_1}$ and $\boldsymbol{\delta}_{\mathbf{x}\mathbf{y}_1}$, by executing Protocol 3. The same operations are executed also by the drone over its local shares $\boldsymbol{\Delta}_\mathbf{x}$, $\boldsymbol{\Delta}_\mathbf{y}$, $\boldsymbol{\delta}_{\mathbf{x}_0}$, $\boldsymbol{\delta}_{\mathbf{y}_0}$, $\boldsymbol{\delta}_{\mathbf{x}\mathbf{x}_0}$ and $\boldsymbol{\delta}_{\mathbf{x}\mathbf{y}_0}$ (see Fig. 6.2, step ⑥).

4. At the completion of the local inner product operations, the WD sends the resulting values o_{xx_1} and o_{xy_1} to the drone (Fig. 6.2, step ⑦).

5. At message reception, the drone reconstructs the result of the overall inner product operation by following the steps 1 and 2 of Protocol 4. The drone uses further the result of step 1 and 2 in Protocol 4 to compute the scaled cosine similarity (Eq. 6.5), using the intermediate results o_{xx} and o_{xy}, as shown in Protocol 4. Based on the result, the drone takes the *decision* if to release the goods to the user (Fig. 6.2, step ⑧). This is established based on the threshold τ in step 4 in Protocol 4. Finally, the drone and

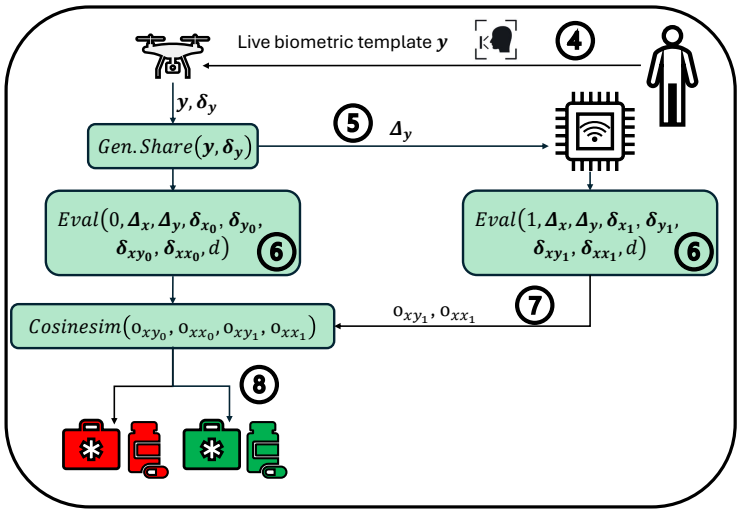

Figure 6.2: Deployment Phase of Obscura.

WD discard all the δ-shares and Δ values used for the biometric verification, e.g., by replacing them with random values.

Protocol 3: $Eval(j, \mathbf{\Delta_x}, \mathbf{\Delta_y}, \langle \boldsymbol{\delta}_\mathbf{x} \rangle, \langle \boldsymbol{\delta}_\mathbf{x} \rangle, \langle \boldsymbol{\delta}_\mathbf{xy} \rangle, \langle \boldsymbol{\delta}_\mathbf{xx} \rangle, d) \rightarrow (\langle o_{xy} \rangle, \langle o_{xx} \rangle)$

Parties: *Drone, Wireless Device.*

Input: $j \in \{0, 1\}$ (being P_0 the drone and P_1 the wireless device), $\mathbf{\Delta_x}$, $\mathbf{\Delta_y}$ (masked values of $\mathbf{x}, \mathbf{y}$), $\langle \boldsymbol{\delta}_\mathbf{x} \rangle, \langle \boldsymbol{\delta}_\mathbf{x} \rangle, \langle \boldsymbol{\delta}_\mathbf{xy} \rangle, \langle \boldsymbol{\delta}_\mathbf{xx} \rangle$ (δ- shares of the input and the inner product), d (dimension of the vector).

1: $o_{xy_j} \leftarrow \sum^d [j \cdot \Delta_x \Delta_y - \Delta_x \delta_{y_j} - \Delta_y \delta_{x_j} + \delta_{xy_j}]$

2: $o_{xx_j} \leftarrow \sum^d [j \cdot \Delta_x \Delta_x - \Delta_x \delta_{x_j} - \Delta_x \delta_{x_j} + \delta_{xx_j}]$

Output: $(\langle o_{xy} \rangle, \langle o_{xx} \rangle)$

6.5. Security Analysis

In this section, we discuss and prove the security of our proposed Obscura protocol against the adversary introduced in Sec. 6.3.2.

As discussed, the adversary ϵ captures the drone during the *Deployment Phase*. Therefore, ϵ has access to the memory of the drone before and after our protocol has been executed. In the following, we first show that by simply listening passively to the communication channel where the drone and the WD exchange information, the adversary cannot achieve its objective, i.e., retrieving the reference template $\mathbf{x}$. Following, we show that an attacker with access to all information collected, received, and stored on the drone, cannot obtain the reference

Protocol 4: Cosinesim($\langle o_{xy} \rangle, \langle o_{xx} \rangle$) $\rightarrow (result)$

Parties: *Drone.*
Input: $\langle o_{xy} \rangle, \langle o_{xx} \rangle$ — $o_{xy_0}, o_{xy_1}, o_{xx_0}, o_{xx_1} \in \mathbb{Z}_{2^l}$ (share of the parties P_0 and P_1)).
Wireless Device P_1 sends o_{xy_1}, o_{xx_1} to the drone.
1: $o_{xy} \leftarrow (o_{xy_0} + o_{xy_1})$
2: $o_{xx} \leftarrow (o_{xx_0} + o_{xx_1})$
3: $c_1 \leftarrow Sign(o_{xy})$
4: $c_2 \leftarrow Sign(\frac{1}{\tau^2} \cdot (o_{xy})^2 - (o_{xx} \cdot \sum^d (\mathbf{y_i y_i})))$
5: $result \leftarrow c_1 \wedge c_2$
Output: (result)

template $\mathbf{x}$. Since such adversarial behavior is equivalent to a drone passively executing the protocol, for such a proof, we consider ϵ as an HbC adversary, which has knowledge of all the information on the drone during the *Deployment Phase* and tries to use such information to obtain $\mathbf{x}$. Finally, considering an active behavior of ϵ, we show how our protocol protects against re-verification attacks.

Passive Behavior. As described in Sec. 6.4.3, during the deployment phase, the drone delivers $\mathbf{\Delta_y}$ to the user's WD and the user's WD delivers to the drone o_{xx_1}, o_{xy_1}. $\mathbf{\Delta_y}$ is the masked value of $\mathbf{y}$, computed as $\mathbf{\Delta_y} = \mathbf{y} + \boldsymbol{\delta}_\mathbf{y}$. As per its definition, it is a random value, and it does not reveal anything regarding the live template $\mathbf{y}$ and the reference template $\mathbf{x}$. o_{xx_1}, o_{xy_1} are the results of the inner product based on the δ-shares of the WD. The computation of such values is based on the masked values possessed by the WD, i.e., $\boldsymbol{\delta}_{\mathbf{x_1}}, \boldsymbol{\delta}_{\mathbf{y_1}}, \boldsymbol{\delta}_{\mathbf{xx_1}}, \boldsymbol{\delta}_{\mathbf{xy_1}}$ and $\mathbf{\Delta_x}, \mathbf{\Delta_y}$, which are random values, that by definition, do not allow to retrieve the plain-text data. Therefore, ϵ cannot retrieve the reference template $\mathbf{x}$ based on the information eavesdropped on the wireless channel.

The following definition and proofs are partly inspired by the works of [55, 115], which consider tools similar to the one at the roots of our solution.

Definition 6.5.1 (Security of Obscura)**.** For the Party P_0 (drone), there is a PPT algorithm $\mathcal{S}$ (simulator) s.t. $\forall \mathbf{x}, \mathbf{y} \in \mathbb{Z}_{2^l}^d$ and every function $\mathcal{F}_{cosVer}(\mathbf{x}, \mathbf{y}, \tau)$: $\mathbb{Z}_{2^l}^d \rightarrow \mathbb{Z}_2$, S simulates the ideal functionality $\mathcal{F}_{cossim}$ (Fig. 6.3) and its behavior is computationally indistinguishable from a real-world execution of protocols 2, 3, 4 in presence of the semi-honest adversary ϵ.

Theorem 6.5.1. *In the $\mathcal{F}_{Setup}$-hybrid model, protocols 2,3,4 securely realize the functionality $\mathcal{F}_{cossim}$ (Fig. 6.3) during the deployment phase.*

Proof. The semi-honest adversary ϵ corrupts Party P_0 (the drone) and has access to such party during the sequential execution of protocols 2, 3, and 4. The $\mathcal{S}$ executes the circuit evaluation assuming that the circuit inputs of the WD P_1 are 0. In protocol 3, $\mathcal{S}$ adjusts the results of the shares of $\langle o_{xy} \rangle, \langle o_{xx} \rangle$ on behalf of P_1, so allowing the adversary ϵ to see the same protocol flow as in the real-world protocol.

Ideal Functionality $\mathcal{F}_{cossim}$

$\mathcal{F}_{cossim}$ interacts with the Drone (P_0) and the wireless device (P_1) and the adversary $\mathcal{S}$ and is parametrized by
the publicly known function $\mathcal{F}_{cosVer}(\mathbf{x}, \mathbf{y}, \tau)$, τ is the threshold.
Input: $\mathcal{F}_{cossim}$ receives the pre-computed inputs
$\Delta_x, \Delta_y, \delta_{x_j}, \delta_{y_j}$ from the parties P_0, P_1.
Computation:
$\mathcal{F}_{cossim}$ reconstructs:
$\mathbf{x} = \Delta_x - (\delta_{x_0} + \delta_{x_1})$,
$\mathbf{y} = \Delta_y - (\delta_{y_0} + \delta_{y_1})$, and
computes $d = \mathcal{F}_{cosVer}(\mathbf{x}, \mathbf{y}, \tau)$.
Output: Send d to Drone.

Figure 6.3: Ideal Functionality.

Protocol 1 During the offline phase, the ideal functionality $\mathcal{F}_{setup}$ generates the necessary cryptographic elements for generating the δ-shares, as shown in Protocol 6.1. As in [115], we use Protocol 1 as a black-box access which simulates the security of the underlying primitives, i.e., OT or HE for the setupMULT in [163] for generating the cryptographic materials.

Protocol 2 During the deployment phase, the drone P_0 is the owner of the input vector $\mathbf{y}$. Therefore, $\mathcal{S}$ does not need to simulate anything since the adversary ϵ is not receiving any messages.

Protocol 3 During the evaluation of the circuit, $\mathcal{S}$ follows the steps of protocol 3 honestly using the preprocessing data obtained from the setup phase. Protocol 3 executes two inner products, where the element-wise multiplication relies on the cryptographic materials received during the setup phase, as per Protocol 1. Meanwhile, the addition of the dimension d in the inner product does not require any interactive steps. Therefore, the simulation is not necessary.

Protocol 4 $\mathcal{S}$ receives $\langle o_{xy} \rangle, \langle o_{xx} \rangle$ for reconstruction from ϵ, which is the output of ϵ. $\mathcal{S}$ uses $\langle o_{xy} \rangle, \langle o_{xx} \rangle$ to compute $o_{xy_0} = o_{xy} - o_{xy_1}, o_{xx_0} = o_{xx} - o_{xx_1}$, where o_{xx_1}, o_{xy_1} are from P_1. $\mathcal{S}$ sends o_{xy_0}, o_{xx_0} to ϵ, where $\mathcal{S}$ simulates in ideal world the behavior of P_1 sending the share. This ensures the adversary ϵ has a similar view as in the real world with P_1. Therefore, $\mathcal{S}$ receives o_{xy_0}, o_{xx_0} from ϵ on behalf of P_1 (real world). The sequential execution of protocols 2-3-4 securely realizes the functionality $\mathcal{F}_{cossim}$ with negligible probability from disclosing the real interaction apart from the ideal interaction in the presence of ϵ and $\mathcal{S}$. □

The protocols in the real world and the ideal world are thus indistinguishable. We see that

the information available on the drone related to the reference template $\mathbf{x}$ is Δ_x, and δ_{x_0}. Per definition, it is not possible to obtain $\mathbf{x}$ by knowing only Δ_x and δ_{x_0}—subtracting δ_{x_0} from Δ_x leads to $x + \delta_{x_1}$, which is still a random value due to δ_{x_1}. Therefore, by executing the protocol as intended, the adversary on the drone does not get any information on the reference template $\mathbf{x}$, proving the security of Obscura.

Re-Verification Attacks. ϵ can also use active attacks to retrieve the reference template $\mathbf{x}$. One of the most popular active attack against biometric verification is *re-verification*, where ϵ forces several consecutive executions of the protocol to obtain several instances of the masked information. By possibly correlating such instances, ϵ can use them for obtaining sensitive protected information. There could be also non-malicious cases where the drone needs to carry out face verification multiple times, i.e., re-identify the user, e.g., due to sub-optimal weather conditions, gesture movements, or motion blur, to name a few, motivating the deployment of multiple instances of the cryptographic materials onboard. In such a scenario, ϵ could exploit the legitimate repeated usage of the reference template $\mathbf{x}$ in our protocol to extract its plaintext value. As shown in Eq. 6.6, and in particular in the underbrace at the last line of such equation, by forcing multiple executions of Obscura with the same input values, ϵ could obtain a system of linear equations to reconstruct $\boldsymbol{\delta}_{\mathbf{x_1}}$, which is one of the secrets used to mask $\mathbf{x}$—the drone has $\boldsymbol{\delta}_{\mathbf{x_0}}$ and $\boldsymbol{\Delta}_{\mathbf{x}}$, and with $\boldsymbol{\delta}_{\mathbf{x_1}}$ it could obtain $\boldsymbol{\delta}_{\mathbf{x}}$ as $\boldsymbol{\delta}_{\mathbf{x}} = \boldsymbol{\delta}_{\mathbf{x_0}} + \boldsymbol{\delta}_{\mathbf{x_1}}$ and then $\mathbf{x} = \boldsymbol{\Delta}_{\mathbf{x}} - \boldsymbol{\delta}_{\mathbf{x}}$.

$$
\begin{aligned}
o_{xy} &= \sum^{d}[\Delta_x\Delta_y - (\Delta_x\delta_{y_0} + \Delta_x\delta_{y_1}) - (\Delta_y\delta_{x_0} + \Delta_y\delta_{x_1})] + \\
&\quad \sum^{d}[(\delta_{xy_0} + \underbrace{\qquad\qquad\qquad}_{(\delta_{x_0}+\delta_{x_1})\cdot(\delta_{y_0}+\delta_{y_1})-\delta_{xy_0}} \delta_{xy_1} \qquad\qquad)] = \\
&= \sum^{d}[\Delta_x\Delta_y - \Delta_x\delta_{y_0} - \Delta_y\delta_{x_0} + \delta_{xy_0} - \Delta_x\delta_{y_1} + \delta_{x_0}\delta_{y_0} + \delta_{y_1}\delta_{x_0}] \\
&\quad + \underbrace{\sum^{d}[\delta_{x_1}\delta_{y_1} + \delta_{x_1}\delta_{y_0} - \Delta_y\delta_{x_1}]}_{\sum^{d}[y\delta_{x_1}]}.
\end{aligned}
\tag{6.6}
$$

To find the unknown value of $\boldsymbol{\delta}_{\mathbf{x_1}}$, the drone needs to solve a system of $m = d$ linear equations, being m the number of re-identification sessions and d the size of the embeddings. Indeed, the space to find the unknown value $\boldsymbol{\delta}_{\mathbf{x_1}}$ is $(2^l)^{(d-m)}$. Obscura features multiple ways to protect against such attacks. First, the application on the WD can have a fail-safe, i.e., the maximum number of allowed verifications. Setting such a maximum number to $m < d$ prevents the adversary from generating d linear equations. Indeed, if $(m < d)$, the adversary does not have enough knowledge to retrieve $\mathbf{x}$ by using Eq. 6.6. Second, the WD could use new cryptography materials for every re-identification session. In particular, different sets of cryptography materials $\delta_x, \delta_{x_0}, \delta_{x_1}, \Delta_x, \delta_y, \delta_{y_0}, \delta_{y_1}$, can be pre-computed by the SP and deployed on the WD and the drone during the setup phase, to be used for the identification of a particular reference template $\mathbf{x}$ during the following *Deployment Phase*. When a re-identification request is issued by the drone, the WD could use every time a different set

of values for such cryptography materials. Since the adversary has control over the drone but not the WD, it cannot force the WD to re-use the same cryptography material for each new identification session. Since each set of values is independent and extracted from a variable distributed uniformly at random, such values allow to provide freshness across the different re-identification session, preventing the adversary to build up the system of linear equations in Eq. 6.6. Note that impersonation of the WD by the adversary, to be successful, requires knowledge of the correct shares possessed by the WD ($\delta_x, \delta_{x_1}, \delta_{y_1}$), which is impractical. Further protection against impersonation is also provided through peer authentication at the establishment of the WiFi connection between the two parties, via traditional WiFi authentication protocols like WPA3 [8], so this attack is also not viable. The only limitation of this defense technique is the little additional storage space required on the WD and the drone, which is however minimal. Also note that at the deployment time the compromised drone cannot communicate with the SP to get additional fresh values.

Note that the attack discussed above is the only viable option for the adversary to obtain $\mathbf{x}$. Other attacks, possibly exploiting interaction with the SP, are not possible in our system model.

Summary. As discussed above, at any point in the deployment phase and with any behavior, capturing the drone does not lead the adversary to compromising the user's reference template $\mathbf{x}$. However, we acknowledge that an adversary capturing the drone before the package is delivered to the user could access the live template $\mathbf{y}$ captured at verification time by the drone. Also, if the adversary can recover previously-deleted (or overwritten) information as discussed in Sec. 6.3.2, recovering the live template $\mathbf{y}$ is possible also after package delivery. Since drones currently deployed always have a camera onboard to take the live photo, differently than any user device, there is no way in our current system model to protect from such a leakage, independently of the adopted privacy-enhancing technology. However, in the scenario of interest for this chapter, we remark that our solution aims to protect the reference template, and not the live template. Indeed, since there is no way for the adversary to recover the reference template $\mathbf{x}$, the SP cannot be held responsible for the leakage of any sensitive information provided at registration time.

6.6. Performance Evaluation

In this section, Sec. 6.6.1 introduces the details of the implemented proof-of-concept testbeds of Obscura, Sec. 6.6.2 describes the experiment settings and finally, Sec. 6.6.3 presents the experimental results.

6.6.1. Testbeds Details

We implemented a proof-of-concept of our protocol using the programming language Python. To characterize the performance of our solution on heterogeneous devices and networks, we consider two different testbeds, namely *Testbed 1* and *Testbed 2*.

Testbed 1 (T1). In this testbed, we consider the drone and the WD to be on Raspberry PI 3 Model B+ devices [176]. This is a well-known low-cost embedded platform equipped with a Cortex-A53 processor running at 1.4 GHz, 1GB of RAM, and 16 GB of storage, running the operating system Raspbian Bullseye 64-bit. As acknowledged by several contributions in the literature such as [95, 144, 210], the computational and bandwidth resources offered by the Raspberry PI Model 3 B+ are comparable to the ones provided by low-end drones, and there are even commercial drones integrating the Raspberry PI as an on-board CPU [75].

Testbed 2 (T2). For this testbed, we consider the drone and the WD to be equipped with larger computational resources. We use two embedded devices Intel NUC7i5BNK [118], equipped with two Intel Core i5-7260U processors running at 2.20 GHz, 16GB of RAM, and 256GB SDD, running Ubuntu 20.04.6 LTS. Compared to *Testbed 1*, *Testbed 2* allows us to emulate drones characterized by more significant computational power, like high-end ones. For communication, we set up an ad-hoc IEEE 802.11 network working at the operating frequency 2.4 GHz in a regular office environment. This choice is on purpose to let our proof-of-concept share the same spectrum of other communications and possibly experience interferences and packet losses, so to execute realistic tests.

For executing our experiments using Obscura, we (i) enabled remote communication capabilities using communication sockets using the Python package *tno.mpc.communication*, (ii) integrated within Obscura the Π-*Secret Sharing* building block proposed by the authors in [55, 163], (iii) organized the code into several processes, and (iv) used as database for face verification a dataset of actual faces.

6.6.2. Experimental Settings

We evaluate the overhead of Obscura considering the deployment phase (Sec. 6.4.3) and four relevant metrics, i.e. the execution time, memory footprint (specifically, RAM consumption), bandwidth, and energy consumption on both the initiator (drone) and responder (WD) devices. For measuring the execution time, we measure the time taken by each protocol instance on both the initiator and the responder, in milliseconds. For measuring the RAM consumption, we consider the maximum instantaneous RAM consumption of the relevant process running on the device (in kilobytes), using the Linux tool *"/usr/bin/time -v"*. For measuring the bandwidth overhead, we consider the application bytes generated and delivered by the process of our protocol, using the Python library *tno.mpc.communication*, independently of the meta-data added by lower-layer protocols. Finally, for estimating the energy consumption of Obscura, we use the tool *powertop*, a software-based energy estimation tool. *Powertop* provides an estimate of the power (in Watt) of a running process. We compute the energy consumption of the process by multiplying the power estimate for the execution time, as: $E = P \times \Delta t$, where E is the estimated energy in Joule, P the power (in Watt) estimated by *powertop* and Δt the duration of the deployment phase of Obscura.

Datasets. We validate our solution with two datasets, i.e., (i) *Labeled Faces in the Wild (LFW)*, available at [110] and denoted as D1, and (ii) *DroneFaces*, available at [109] and denoted as D2. D1 is a general-purpose dataset of real-faces, and it is selected since it has been used by many contributions in the literature dealing with privacy-preserving face recogni-

tion [115, 250]. It includes $13,233$ images of $5,749$ unique identities. For our performance assessment, we set up a balanced dataset of $1,000$ images, balancing out positive with negative face verification operations. Specifically, out of $1,000$ images, 500 images lead to a *true positive*, i.e., the user is successfully verified, and the remaining 500 images lead to a *true negative*, i.e., the user is correctly not verified. The dataset D2 includes $1,364$ frontal face images and 73 portrait images of 11 unique identities, and it is used to evaluate face recognition techniques on drones. We evaluate the performance of our solution with this datasets using two balanced sub-datasets: (i) D2-1 ($1,364$ images), which keeps probes shot at the easier heights of 3–4 m and at 2.5–16 m distance from the target, and (ii) full D2, which includes all probes taken from 1.5–5 m at the same distance range. This allows us to study how performance varies with the drone's angle of depression (as discussed in [109]).

We also use the well-known model *SFace* [38] to extract face embeddings from the dataset, so working with vectors of dimension $d = 128$. Finally, to enable the experimental comparison of Obscura with the closest proposal in [55], we deployed both Obscura and the solution in [55] on the testbeds T1 and T2, and evaluate the performance and accuracy of such solutions on the datasets discussed above. Note that we adapted the proposal in [55] to fit in our scenario so that we can provide a fair comparison. Our proof-of-concept and datasets are available at [73].

For all the tests, we report the results of our experiments using the average and the 95% confidence intervals. For such tests, we set the value of some specific protocol parameters. In particular, we fixed the input size of the δ-shares to 64 bits and the vector dimension $d = 128$, in line with used ML model for the generation of face embeddings.

6.6.3. Results

We consider the effectiveness and overhead of our proposed Obscura solution on the two testbeds introduced in Sec. 6.6.1.

Face Verification Performance. We first verify the accuracy of our solution, i.e., the capability of correctly verifying the identity of individuals, without degrading performances compared to the equivalent non-private solution. To this aim, we used our datasets (see Sec. 6.6.1) and compare the performance of the plain-text scaled cosine similarity algorithm, our solution and the proposal by Cheng et al. in [55]. We report the results of our analysis in Tab. 6.3, on all datasets. For D1, all three approaches report 97.2% of accuracy, confirming that our proposed Obscura solution guarantees the same face verification performance of the plain-text face verification approach and the state-of-the-art. For the dataset D2-1 all three approaches report 78.15% of accuracy, while we achieve 77.46% accuracy for the entire dataset D2, for all three approaches. Therefore, we can conclude that Obscura achieves the same performance of plain-text non private approaches, while fully preserving privacy.

Experiments on T1. We investigate the overhead of Obscura with testbed T1, i.e., the most constrained setup. Fig. 6.4 reports the results for the execution time required by Obscura and the solution by Cheng et al. [55] when executed on T1, with dataset D1. We divide the overall execution time into the time required for (wireless) communication (light grey bars in Fig. 6.4) and the time required for computation (dark grey bars in Fig. 6.4), for each party.

Dataset	Plain-text Face Recognition	Cheng et al. [55]	**Obscura**
D1 [110]	97.2%	97.2%	97.2%
D2-1 [109]	78.15%	78.15%	78.15%
Full D2 [109]	77.46%	77.46%	77.46%

Table 6.3: Face verification accuracy of the plain-text algorithm, the solution in [55] and Obscura, with all datasets.

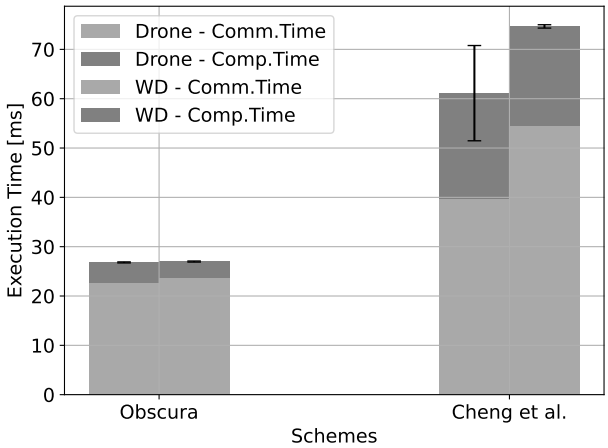

Figure 6.4: Execution time of Obscura and the solution by Cheng et al. [55] on the testbed T1.

The results show that the overall execution time of Obscura is significantly lower than the competing solution, on both parties involved in the protocol. Overall, on the drone Obscura requires on average 26.810 ms, which is 56.14% less than the solution by Cheng et al., which requires on average 61.124 ms. We also notice that, with both schemes, the WD takes more time than the drone to complete the protocol. However, with Obscura, such a difference is very little, making the computational burden of the protocol almost the same for both involved parties. We also notice that the reduced execution time of Obscura comes from both reduced computation and communication time. The solution by Cheng et al. requires more computations than Obscura. Regarding communication overhead, we show in Tab. 6.4 the overall number of bytes delivered by both entities with Obscura and the solution by Cheng et al. in [55]. We notice that, with the chosen configuration, Obscura requires the drone to

Cheng et al. [55]		**Obscura**	
Drone	WD	Drone	WD
1930 bytes	766 bytes	1216 bytes	80 bytes

Table 6.4: Average communication overhead of Obscura and the solution in [55] during the *Deployment Phase*.

send $1,216$ bytes, including the messages to establish the connection and $\mathbf{\Delta_y}$. In contrast, the work in [55] requires the drone to send $1,930$ bytes, including $\mathbf{\Delta_y}$ and the shares of the

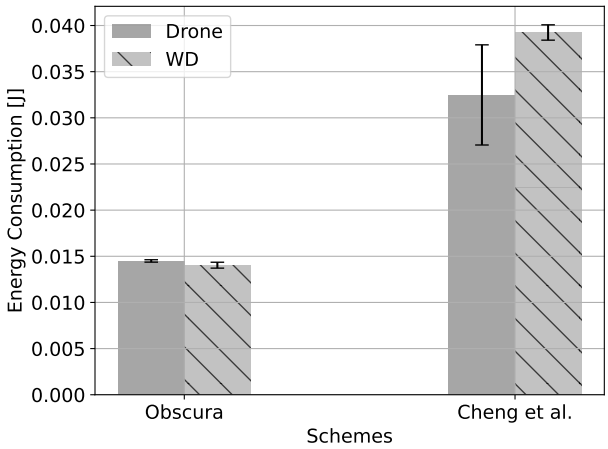

Figure 6.5: Energy consumption of Obscura and the solution by Cheng et al. [55] on the testbed T1.

inner product. Besides, the WD requires on average 80 bytes in Obscura and 766 bytes for the proposal in [55]. The increased overhead of the proposal in [55] is due to the delivery of the shares of the inner product and the share of the function *Condeval* (see [55] for more details). Instead, with Obscura, the WD delivers only o_{xx_1}, o_{xy_1}.

Using the same setup, we investigate the average energy consumption required by Obscura and the competing solution by Cheng et al. [55]. Fig. 6.5 summarizes the results our analysis. Overall, Obscura is significantly more energy efficient than the solution in [55]. Our proposal consumes on average 0.0145 J, which is 55% less than the solution of Cheng et al. when applied to our problem. The energy consumption of Obscura on the drone translates into a power consumption of 0.0008 mAh. Considering that an actual drone, such as the DJI Mini 2, features a battery with a capacity of 5, 200 mAh [70], the power consumption of Obscura corresponds to approx. 0.00002% of the overall drone battery capacity, confirming its minimal impact on the drone's lifetime. We also notice that the energy consumption of the WD with Obscura is less than the one of the competing solution. When using the protocol of [55], the WD consumes on average 0.03924 J, while such consumption reduces to 0.01404 J with Obscura, i.e. 64.2% less.

We also investigate the RAM consumption of both approaches with T1, and report our results in Fig. 6.6. In terms of RAM consumption, the two proposals are approximately equivalent. On the drone, Obscura requires 80.226 MB of RAM, while it requires 80.435 MB on the WD. We attribute such a RAM consumption to the use of process *python3.9* for both schemes. Usually, such a RAM consumption is not a problem for drones, which feature enough RAM [71].

Experiments on T2. We also investigate the overhead of Obscura using the testbed T2. Fig. 6.7 reports the results for the execution time required by Obscura and the solution by Cheng et al. [55]. Similarly to Fig. 6.4, we report through light gray bars the time required for communication and through dark grey bars the time required for computation, for each

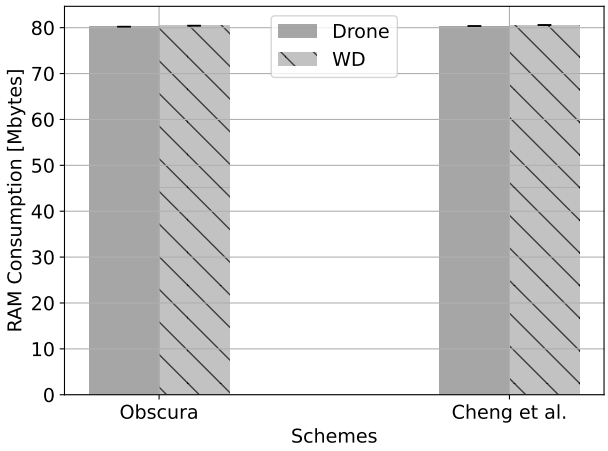

Figure 6.6: RAM consumption of Obscura and the solution by Cheng et al. [55] on the testbed T1.

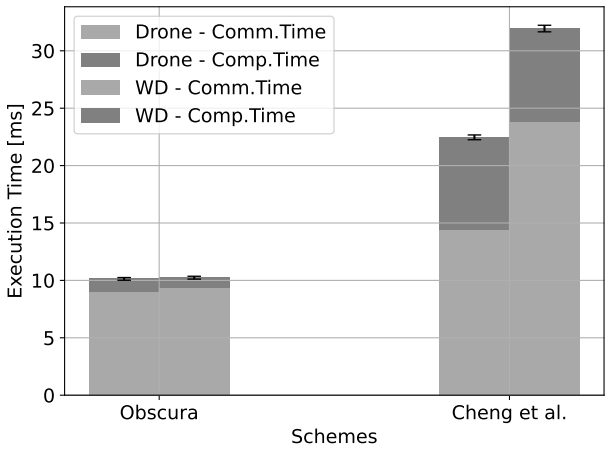

Figure 6.7: Execution time of Obscura and the solution by Cheng et al. [55] on the testbed T2.

party (the drone and the WD). We notice that the results follow the same trend in Fig. 6.4, with our solution being significantly faster than the competing approach by Cheng et al. in [55]. However, being the equipment used for T2 more powerful than the equipment used for T1, the overall execution times are even lower than what reported earlier. Obscura requires 8.984 ms to complete on the drone and 9.414 ms to complete on the WD, decreasing further by approx. 67% the execution time on the testbed T1. Also for T2, Obscura is quicker than the solution by Cheng et al., in this case by 37.6% on the drone. Finally, we report our results for the RAM consumption on T2 in Fig. 6.8. As found for the testbed T1 (Fig. 6.6), Obscura and the competing solution consume the same RAM, i.e., approx. 43.683 MB for T2. Although we do notice that the RAM consumption for T2 appears to be lower than

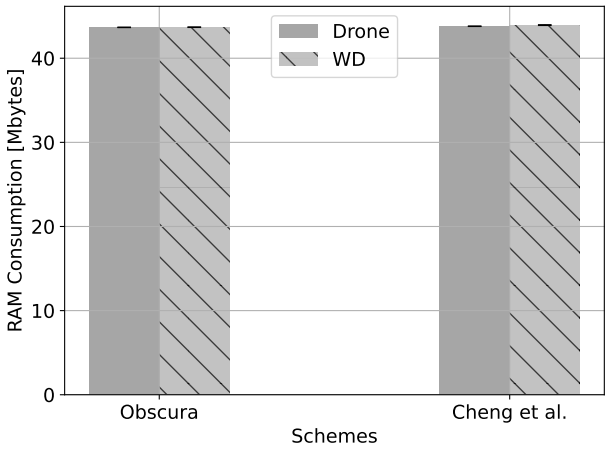

Figure 6.8: RAM consumption of Obscura and the solution by Cheng et al. [55] on the testbed T2.

the one for T1, we recall that a direct comparison between the two testbeds for RAM consumption is not possible, since the underlying system architectures and memory allocation procedures are different.

6.7. Discussion

We briefly discuss below some relevant aspects connected with the deployment of Obscura.

Alternatives to the Trusted Dealer. During the *Setup-Phase*, we consider that the SP plays the role of the trusted dealer. An alternative option is to use OT or HE to generate each party's multiplicative shares of δ. We leave the implementation of such alternative options as future work.

Leakage of the Live Template. An adversary capturing the drone after goods delivery may derive private information from the live template possibly collected by the drone at runtime. To prevent this, the drone could additionally include a TEE, and store the live template therein, as in [115, 168]. Specifically, once the live template is collected, it could be temporarily stored in the TEE. Once $\mathbf{\Delta}_y$ is computed, the TEE could reveal only the masked value and its shares, while protecting the original information. Since TEEs are tamper-resistant and inaccessible, the adversary could not retrieve the live template. Note that setting up the entire face verification process (including also the reference template) in the TEE would enlarge the enclave significantly, while also increasing side-channel exposure and computational overhead. Instead, in line with recent literature [112, 237], by storing limited information (only the live template) in the enclave, the TEE could be kept small and efficient, while offloading to Obscura the protection of the reference template.

6.8. Related Work

In this chapter, we consider the problem of privacy-preserving face recognition onboard drones, i.e. verification of users' identities for delivering goods. To the best of our knowledge, to date, there is no scientific contribution dealing with this problem, making both our problem and solution novel. A few contributions in the literature consider privacy-preserving biometric verification, although working with different assumptions and requirements than the ones considered here. We summarize the main features of such works in Tab. 6.5, with reference to our requirements laid down and motivated in Sec. 6.3.4. Overall, many different PETs-based schemes have been used for executing privacy-preserving face recognition. We see solutions based on FHE (e.g., [36, 111, 238]), Quadratic Homomorphic Encryption (QHE) and Additive Homomorphic Encryption (AHE) ([116, 183]), Paillier encryption ([80, 149, 160]) and Π-Secret Sharing with Function Secret Sharing (FSS) ([55, 115]). We also notice that many approaches rely on a TTP, which is used for storing the plain-text biometric template of the user and for delivering cryptography materials related to such a template during the face verification task. As discussed in Sec. 6.3.4, our use case considers drones deployed in areas with unreliable, scarce or unavailable Internet connection, ruling out the possibility to interact online with such a TTP (see reqs. **R1** and **R2**). Moreover, on top of the previous limitations, some schemes based on FHE, e.g., [36, 80, 111, 149, 238], require the storage of a decryption key onboard of the device ultimately verifying the match of the live template with the reference template. This is not compatible with our requirements, since an adversary taking control of the drone would have access to such a key, so being able to compromise the confidentiality of the PII stored onboard (**R6**).

Scientific contributions that do not require TTPs, i.e., the solutions by [115, 116, 160, 183], do not allow the client (i.e., the network entity initiating the connection) to take the final decision about face verification, not fulfilling our requirement (**R6**). This is a relevant difference compared to our system model, which also prevents us to carry out an experimental comparison of such solutions with our work. Moreover, such schemes are not designed for and not even tested on constrained devices like drones.

At the same time, several template protection techniques have been proposed to protect the reference template's confidentiality against stealing, linkage, and misuse. Many of these techniques, such as salting or token-based transformation techniques, often depend on a token or key stored locally or on a TTP's server, and thus, they do not fulfill our requirements [140]. Biometric cryptosystems such as Fuzzy Vault and Fuzzy Commitment are known to be vulnerable against key recovery attacks, error correction codes attacks, and correlation attacks [4, 177]. Finally, cancelable biometric techniques are well-known to be prone to inversion attacks, and they use complex transformation which could lead to degraded recognition performance [22, 140]. Therefore, cancelable biometric solutions do not fulfill the requirements **R4** and **R6**. The closest work in terms of drone-side deployment is by Pillai et al. [168]. However, their work focuses on the user-centric privacy by classifying if the delivery drone recorded the user. This allows the user to query the footage without disclosing it's location at that time.

Overall, although not fulfilling all our requirements and focusing on a different problem

(optimization of cosine similarity implementation with audio traces for voice recognition), the contribution by Cheng et al. [55] is the only one that can be framed to be comparable with our solution. As shown in Sec. 6.6.3, besides being the only one to fulfill all our requirements, Obscura is more lightweight and energy-friendly than the approach in [55].

In summary, our work is the first to consider the combination of all requirements motivated and discussed in Sec. 6.3.4, allowing to address the problem of privacy-preserving face verification on constrained devices.

Table 6.5: Qualitative comparison of state-of-the-art privacy-preserving face recognition schemes and our work. The symbol ● indicates that a particular feature is supported, while the symbol ○ indicates that the feature is not supported.

	Cryptography Technique	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
Xiang et al. [238]	FHE	○	○	●	○	●	○	○	○
Ma et al. [149]	Paillier	○	○	●	●	●	○	○	○
Im et al. [116]	QHE	●	●	●	●	○	○	○	○
Erkin et al. [80]	Paillier+DGK [60]	○	○	●	●	○	○	○	○
Sadeghi et al. [183]	AHE+GC	●	●	●	○	●	○	○	○
Osadchy et al. [160]	Paillier	●	●	●	●	●	○	○	○
Boddeti [36]	FHE	○	○	●	●	●	○	○	○
Huang et al. [111]	FHE+GC	○	○	●	○	○	●	○	○
Pillai et al. [168]	GC/SS	●	●	○	○	○	●	○	●
Ibarrondo et al. [115]	Π-SS + FSS	○	●	●	●	●	●	○	○
Cheng et al. [55]	Π-SS + CondEval	○	●	○	●	●	●	○	○
Obscura	Π-SS	●	●	●	●	●	●	●	●

6.9. Conclusion

In this chapter, we proposed Obscura, a novel scheme based on Multi-Party Computation allowing to carry out privacy-preserving face verification onboard commercial drones. Through Obscura, commercial drones deployed in areas characterized by unreliable or scarce Internet connectivity can execute face verification fully onboard without storing onboard any sensitive private user's information provided at registration time. This strategy protects the privacy of users and helps the Service Provider (SP) deploying the drone to keep compliance with privacy regulations, even in the presence of adversaries capturing the drone during the delivery of the package. We describe Obscura and prove its security against an adversary capturing the drone at deployment time. Moreover, we implemented a proof-of-concept of Obscura, released the code as open source, and used such a proof-of-concept to experimentally test the accuracy and overhead of our solution on two test environments. Our performance evaluation demonstrates that Obscura is very accurate and lightweight, as it requires approximately 27 ms and 0.0145 J on the most constrained device tested, more than 50% less than the closest competing approach. Future work is needed to extend the scope of our solution further to protect also the live picture of the user acquired during goods delivery.

7

Discussion

The prior chapters highlight privacy concerns in the IoT domain, and how we can address them by applying and adapting various PETs. In this chapter, we draw attention to the limitations of our solutions and provide prospects on how to address them, while opening new avenues for future research.

We divide the chapter into eight sections: Sec. 7.1 discusses the limitations of pre-processing strategies and how to improve them; Sec. 7.2 emphasizes scalability concerns; Sec. 7.3 addresses various adversarial settings and output privacy guarantees; Sec. 7.4 introduces strategies to improve the efficiency of MPC techniques in the IoT domain, based on hardware accelerations or manual circuit optimization; Sec. 7.5 discusses the reduction of sizes of NIZK proofs through the use of different signature schemes which allow updatable proofs; Sec. 7.6 discusses how to define thresholds for PRM and ePPTM; Sec. 7.7 elaborates on how to further harden our works in preparation for the upcoming quantum era; Sec. 7.8 gives insights on the future directions regarding PET composition.

7.1. Pre-Processing

Many of the solutions described in this thesis, i.e., Obscura (Chapter 6) and DaaS (Chapter 5), rely on the pre-processing model, also known as the *offline-online* paradigm. Pre-Processing is a model that stems from the MPC domain and enables MPC-based protocols to achieve efficient computation for real-world applications [206]. The pre-processing model is split into two phases: (i) the offline phase, which computes the function-dependent cryptographic material, and (ii) the online phase, which computes the actual function over the private input and the pre-computed values. The *offline-online* paradigm shifts the computationally intensive operations into the offline phase to reduce computation and communication overhead in the online phase. This paradigm is beneficial in the IoT domain, due to shifting the pre-computations into the setup phase, since during the deployment the IoT devices are confronted with limited computational power or real-time constraints. However, the offline/setup phase requires a trusted dealer or TTP to generate and distribute the pre-computed values to ensure that the values were not tampered with. In the following, we

discuss how pre-processing has been integrated into our protocols and its limitations and practical implications in the IoT domain.

In Chapters 2 and 3, we discussed how to deploy multiple pre-computed moduli for different sessions during the setup phase. In Chapter 5, we rely on the trusted dealer (the DOs) to pre-share attributes of the policy via additive secret sharing. Chapter 6 applies the *offline-online* paradigm, where we precompute the necessary values of the *obscura* protocol, i.e., the Beaver triple for secure multiplication. During the online phases of such protocols, the involved parties use the precomputed values to execute computations to achieve the respective privacy goals.

The prior chapters highlighted the scenarios where pre-processing is useful, e.g., real-time computations or scarce Internet availability during the deployment. In such scenarios, the IoT devices cannot access a TTP to obtain fresh precomputed values. At the same time, the freshness of the cryptographic materials is necessary to mitigate attacks, e.g., replay attacks or the reference template attacks mentioned in Chap. 6. To guarantee freshness, a naive strategy is to precompute a set of cryptographic materials and use a new subset at the beginning of each new execution of the online phase of the protocol. However, this may increase the storage requirements on the device. Note that the amount of cryptographic materials required depends on the application. In MPC, the number of AND gates in a Boolean circuit or multiplication gates in an arithmetic circuit determines the amount of preprocessed values. Thus, both AND and multiplication gates require multiplication triples [65, 163]. The number of AND gates in Chap. 5 depends on the policy size, leading to increased storage requirements. These considerations are similarly applicable to the multiplication gates of the arithmetic circuits.

We can reduce the storage requirements for the pre-processing phase by letting the parties leverage the *silent OT extension* based on pseudorandom correlation generator (PCG) instantiated with Learning Parity with Noise (LPN) assumptions [39]. During the preprocessing phase, the parties may agree on a seed. When the IoT device is deployed, during the online phase, the parties can expand this seed to correlate the necessary tuples for secure function evaluation. This could allow for minimizing the amount of preprocessed values, and multiple seeds can be stored for multiple sessions — providing the necessary freshness. However, integrating such techniques in the designed protocols likely involves careful protocol re-design, thus requiring further research.

7.2. Scalability

In IoT deployments, ensuring scalability of the proposed solutions is essential for allowing such solutions to accommodate increasing and heterogeneous IoT devices. Designing scalable protocols allows us to expand the number of participants without significantly modifying the underlying protocol. Depending on the use case or scenario, MPC-based protocols offer scalability. However, this depends on the underlying cryptographic techniques and whether the function can be computed over multiple private inputs. We discuss below which use case require scalability.

The work in Chapter 5 and Chapter 6 is meant for a two-party setting, and does not scale for more than two parties. However, scalability considerations are critical in the scenarios considered in PRM (Chap. 2) and ePPTM (Chap. 3), e.g., to identify collision among AVs or interference among multiple distinct mesh network gateways. Thus, PRM and ePPTM require redesigning to be scalable. In this context, the first step could be to redesign PRM and ePPTM into an MPC setting. This choice would allow us to consider schemes based on the adversarial threshold setting (see Sec. 7.3.2). However, the increase of the number of parties leads to a corresponding increase of the communication overhead of our protocols, linearly ($O(n)$) in the star topology and quadratically in a mesh topology ($O(n^2)$) [155] and a tree topology ($O(\log n)$). Therefore, additional research is necessary to identify how to update PRM and ePPTM when considering several involved parties while keeping the communication overhead limited.

7.3. Active Adversaries

All the protocols considered in this thesis take into account two parties and a semi-honest or honest-but-curious adversary model. The honest-but-curious adversary model considers an adversary which does not deviate from the protocol flow, while attempting to learn the private information, but it is considered as the weakest model. However, in practice, IoT devices can be entirely compromised, allowing an adversary to provide arbitrary input, or deviate from the protocol, i.e., play the role of an *active adversary*. It is reasonable to assume that an active adversary is present during the *online-phase* of the protocols. During the execution of such phase, the adversary possibly deviates from the protocol flow to retrieve sensitive information from the other party. Therefore, a direct and relevant research opportunity is to extend our protocols to thwart active attackers.

We discuss in Sec. 7.3.1 possible strategies for active security in our existing protocols, which consider the two-party setting. We describe possible directions on expanding certain protocols for the n-*Party setting* against malicious adversaries in Sec. 7.3.2. Finally, we elaborate on how to guarantee *input verifiability* in Sec. 7.3.3.

7.3.1. Two-Party Setting

The two-party setting stems from the MPC domain, and we borrow the term for any protocol between two parties to accomplish a secure computation or task. We discuss below how we could harden our protocols against active adversaries, in cases where an IoT device is compromised during the *online phase* based on the underlying scheme.

To harden the protocols in Chapter 2 and Chapter 3 against active adversaries, we could replace the current *privacy-preserving proximity testing* [161] scheme with a construction that is provably secure against a malicious adversary that deviates from the protocol. Another option is to consider Verifiable Computation (VC) so that each party can verify that the identifiers are based on their actual trajectory. We classify this approach as *input verifiability* during the *online phase*. We provide more details in Sec. 7.3.3.

Given the high chance that PRM and ePPTM need to be redesigned, they can be mapped in the two-party setting of MPC to identify collisions at the exact location of their trajectory (or, equivalently, interferences in the radio scheduling table). By doing so, they can use techniques for authenticating the computations, e.g., via Message Authentication Code (MAC) [55]. Similar considerations apply to make the protocols discussed in Chapter 5 and Chapter 6 robust against active adversaries. However, this implies incurring additional computational overhead, which is a significant consideration in the IoT domain.

7.3.2. n-Party Setting

We have discussed in Sec. 7.2 that another valuable research direction is to extend some of the protocols considered in this thesis to deal with multiple parties, specifically the protocols in Chapter 2 and Chapter 3. However, increasing the number of parties involved in the protocols also requires us to take into account new possible adversarial models.

Allowing untrusted IoT devices to compute over a function with their shared input may not guarantee the intended output. To deal with such issues, we could consider MPC schemes that assume various adversarial threshold settings. These include *honest-majority* ($t < \frac{n}{2}$), *dishonest-majority* ($t < n$) or *two-thirds honest-majority* ($t < \frac{n}{3}$) [81], with t being the number of parties at-most corruptible by the adversary and n being the total number of parties. Depending on the adversarial setting, we could design MPC protocols to ensure these *output guarantees* [81]: (i) *guaranteed output delivery*, ensuring that all participants receive the results in the presence of corrupted parties; (ii) *fairness*, ensuring that all parties acquire the result or that every party aborts computation so that the corrupted party does not acquire the output, and (iii) *security with abort*, ensuring that input privacy is upheld for all parties, even if an abort is triggered by an adversary that has already learned the output. As stated in [81], output guarantees vary based on the active adversary threshold setting. With the *two-thirds honest majority*, all output guarantees could be ensured; in *honest-majority*, *fairness* and *security with abort* could be ensured (as stated in [81]). When extending PRM and ePPTM to the multiparty setting, we could explore which active adversary threshold best fits the given scenario. For example, ePPTM performs safety-related operations, which require guaranteed output delivery; this objective can only be achieved by the two-thirds honest majority setting in case of active security. The aforementioned examples showcase how to map the vast settings of MPC into the context of IoT. Additionally, a strategy that has been investigated concretely in the domain of Privacy-Preserving Machine Learning is how the efficiency and throughput may be improved based on the number (three and four) of involved parties, e.g., by the work of Harthkitzerow et al. [106]. Such considerations could be feasibly adapted for the IoT landscape.

7.3.3. Input Verifiability

The previous sections addressed active adversary models in two-party settings and n-party settings in MPC, which are considered during the *online phase*, where the parties securely evaluate a function. When considering schemes based on MPC, e.g., *Obscura*, we should take into account that MPC schemes guarantee *input privacy* but not *input verifiability* against ma-

licious intended inputs on a compromised IoT device. Thus, MPC strategies do not protect against malicious input but only against malicious adversaries that deviate from the protocol flow. We address here an interesting research direction on how to guarantee *input verifiability* during the *setup phase* and *online phase*.

In Chap 7.1, we discussed various preprocessing strategies involving dealers distributing shares during the *setup phase*. In Chap. 5 and Chap. 6, we use trusted dealers during the *setup phase* to distribute shares. However, in the case of a malicious dealer, distributing compromised shares can lead to incorrect results during the *online phase*. This may lead to severe safety concerns in the IoT domain. To deal with these threats, the literature provides solutions for publicly verifiable secret shares [18]. Such schemes allow all involved entities to confirm the validity of the received shares before evaluating the shares over the function securely during the *online phase*. However, publicly verifiable secret sharing leads to additional computational costs to guarantee integrity.

Other solutions discussed in this thesis, e.g., ePPTM and PRM, are not based on MPC and operate in a two-party setting. We can use VC, to verify in each round that the capsule identifiers are bound to the actual trajectory (ePPTM) or that the portion identifiers are bound to the RST (PRM). This prevents a malicious party from injecting malformed identifiers. Through ZKP gadgets, we can bind the trajectory to the capsule computation, e.g., via a *commit-and-prove* scheme. The *commit-and-prove* scheme ensures that the parties commit to an input and that computations are based on these committed values, which are proven via ZKP. However, the *commit-and-prove* paradigm will be invoked in each round of ePPTM and PRM. Therefore, to reduce the communication and computation overhead of these schemes, a possible direction is to explore the succinctness of generating proofs and batch verification in the context of the IoT landscape.

The prospect is to design a lightweight input verification stage. To this aim, the IoT device validates the input, which prevents IoT devices from spending energy on a computation that may be resource-intensive. Therefore, we intend to investigate a hybrid design that combines input verifiability and computational verification, in the context of being resource-aware and resistant to malicious adversaries and maliciously intended input.

7.4. Hardware Acceleration and Circuit Optimization

IoT devices often operate under constraints, such as limited processing power or critical real-time requirements, and incorporating PETs requires additional resources, while other operational tasks also need resources simultaneously. To address these issues in the context of MPC-based protocols, we can consider two optimization techniques: (i) hardware-dependent optimizations, that parallelize operations based on the specific function, and (ii) circuit-level optimization, by reducing the number of gates or depth depending on the function. We discuss below how these techniques can be applied in our use cases.

When using secure function evaluation (Chap. 5), the function is represented as a circuit which contains multiple gates representing operations. Fast execution of circuit evaluation is provided through hardware acceleration, known as Single Instruction Multiple Data (SIMD),

or by manually minimizing the circuit size by decreasing the number of computationally intensive gates, e.g., AND gates in Boolean circuits and multiplication in arithmetic circuits. SIMD architectures allow us to utilize the vectorization registers AVX2 in Intel Processors and NEON in ARM Processors [171]. The vectorization registers enable evaluation of gates such as multiplication and AND gates of vectorized input, wherein the component-wise *andings/multiplications* are parallelized. In Obscura, we use two *inner products*, where we could leverage the SIMD architecture to speed up the component-wise multiplication. This strategy would reduce the computation overhead for complex circuits, especially when the circuit size cannot be reduced. However, it is hardware-dependent and the transition between AVX2 and NEON is not straightforward [157].

A possible hardware-independent solution is to manually optimize the circuit size by identifying operations that can be combined to reduce the number of gates or by reducing the circuit depth by repositioning them [84]. In Chap. 5, we use ABAC to represent policies and rules to construct complex policies. Each rule requires three AND gates, and the evaluation function requires 10 AND gates. The combination of the rules and the representation of the policy as a tree could allow us to map the policy tree into the circuit to identify the actual number of AND Gates. Additionally, this could show the circuit depth and size based on the number of gates, which would then allow us to reduce the number of gates by a certain combination of the policy rules or to reduce the circuit depth, e.g., by allowing us to reposition gates. Moreover, we could improve the performance of our work by switching from ABY to ABY2.0 [163]. However, as mentioned in Sec. 7.1, the amount of preprocessing values will increase, which in turn may increase the storage size. In practice, IoT devices may need storage for sensor data, e.g., video-feed, heat map, to name a few, rather than being primarily occupied with pre-processed values.

7.5. Replacement of Signature Schemes

As mentioned in the previous section, the IoT devices are limited in their resources. Therefore, we discuss below signature-based schemes that preserve selective disclosure and the challenges of integrating them into the IoT domain based on our use cases.

The solution discussed in Chapter 4 leverages Schnorr signatures for NIZK based on elliptic-curve cryptography. In our protocol, the IMA-log contains the proof to be verified by the partial verifier. However, measurements can change over time. Therefore, an interesting approach is to adapt the scheme for updatable NIZK proofs and succinct proofs for frequently changing measurements. Ideally, we would require a scheme that can batch a subset of proofs associated with a partial verifier to reduce the communication overhead and the storage size of the IMA-log. Additionally, the partial verifier can perform batch verification. However, if any single proof is invalid, the entire batch fails, and the verifier cannot identify which specific entry was tampered with.

The possible candidate that promises to achieve the requirements discussed above is BBS+, which provides selective disclosure and allows the generation of NIZK and updatable tokens for updatable proofs [104, 215]. These features, in combination with the succinctness of the generated proof, could sufficiently reduce both proof-generation overhead and commu-

nication overhead. Zk-SNARKs [143] provide the ability to update the signature, but the generation of proofs is typically very expensive. Further investigations are required to understand how to integrate pairing-based cryptography with the TPM, and if NIZKs can be used in the chain of trust.

7.6. Threshold Calibration

Protocols designed with thresholds allow IoT devices to have control over the threshold configurations with respect to privacy and computation overhead. Depending on the scenario or task, this allows them to trade off privacy against computation overhead. We give a brief discussion below on how such a threshold could be calibrated.

Both ePPTM and PRM are designed based on energy and privacy thresholds, which enable a fallback to plain PSI. In our work, we delegate the responsibility of defining the thresholds to the SPs or the network administrators. However, we do not specify how a threshold could be calibrated. A naive approach to identifying the threshold is to assume the worst-case scenario, where the iterations of the *iterative division algorithm* and the *incremental capsule matching* reach the maximum depth of the binary tree, assuming that all cells or trajectories collide. Instead, by knowing the energy consumption of the worst-case scenario and the percentage of privacy leakage in each round of our algorithms, we could determine a threshold where we can accept a certain percentage of privacy leakage of the cell occupation and trajectories while conserving energy. We could further investigate such trade-offs in realistic use cases as part of our future work.

7.7. Future-proof Schemes

National Institute of Standards and Technology (NIST) recently announced that, by 2030, reference cryptographic algorithms such as RSA, ECDSA, EdDSA, DH, and ECDH will be deprecated, and in 2035, they will be disallowed [179], due to the transition to quantum-safe schemes. The transition from classical cryptography to post-quantum-based cryptography is not straightforward and raises questions about the feasibility on constrained IoT devices. Therefore, we discuss below how we may enable quantum-safe construction for our use cases.

To harden ePPTM and PRM for the quantum era, we need to replace the basic primitive they use, i.e., the integer factorization, with a quantum-safe primitive. A first step in this direction is to integrate the idea of the authors in [164]. They provide private proximity tests based on lattice-based cryptography, which is quantum-safe. However, in general as well as for ePPTM and PRM this replacement is not a trivial plug-and-play exercise, especially in the IoT domain. It requires a fundamental understanding of the post-quantum primitives and further investigation of the computational aspects of such primitives when applied and run on constrained devices. Moreover, we remark that the authors in [164] use quantum-safe primitives which are not currently standardized by the NIST. However, alternatively, NIST has recently recommended a key-encapsulation mechanism for asymmetric encryption known as Kyber, which does not provide malleable features (see [188] for more details)

necessary for ePPTM and PRM.

The schemes discussed in Chapter 5 and Chapter 6 are also *quantumly* vulnerable, specifically in the *setup phase*, since we use OT or HE for the AND gate and the multiplication gate. A strategy to make them quantum-safe is to investigate the existing quantum-safe versions of OT or HE and how they can be integrated into our protocols while still limiting the resulting computational overhead. The work in Chapter 4 is also not quantum-safe, due to the reliance on elliptic-curve cryptography. We believe a possible solution is to investigate whether post-quantum secure signatures [13] provide selective disclosure, and how they can be integrated with IMA and Hardware Security Module (HSM).

Future work should investigate the rising challenges of transitioning privacy-preserving schemes toward *quantum-safe* privacy-preserving schemes while retaining their applicability to the IoT domain.

7.8. Composing Privacy-Enhancing Technologies for IoT Systems

The works presented throughout this thesis consistently incorporated a single PET alongside auxiliary algorithms to solve specific privacy concerns in diverse IoT use cases. As briefly mentioned in Sec. 7.3.3, relying on a single PET is often insufficient against active adversary models. Particularly for *input verifiability*, an additional PET is necessary to ensure that the input is not maliciously altered whilst keeping the input confidential. Such considerations can be generalized beyond *input verifiability* to the design of privacy solutions for IoT use cases by integrating composable PETs.

However, integrating several PETs into a single privacy-preserving solution running on IoT devices is a non-trivial task. The current literature, e.g., the contribution by Li et al. [142], contextualizes the principles of *privacy-by-design* to the IoT domain, but focuses on the data collection and data processing layers. In contrast, our works emphasize the need for safety-critical real-time computations that do not rely on TTPs. In many real-world IoT-based use cases, sensitive data could already be deployed onboard AVs or IoT devices, and must be processed locally. In this context, integrating several PETs could allow us to protect the data during transit and computation on the device, providing enhanced privacy guarantees. We therefore see a clear path for this research to be further integrated with the principles of *privacy-by-design* or *privacy-by-architecture* – creating resource and privacy-aware schemes to be deployed across diverse IoT environments while not sacrificing safety and real-time responsiveness.

7.9. Final Considerations

In conclusion, this thesis paves the way for the integration of PETs in the IoT domain to address real-world privacy, performance, and legislative concerns. This chapter has identified

some limitations of this research and has highlighted areas to improve that require further investigation.

We remark that we first identified the privacy challenge in the IoT domain and then identified the suitable PET to address the problem. However, in some cases, we could use a top-down approach by investigating how a solution with various settings addresses real-world privacy concerns. Our current understanding shows that, in the future, the combination of various PETs will play an important role, as mentioned in Sec. 7.8. However, combining various PETs leads to additional computational, bandwidth, and energy overhead, which could be unsustainable for constrained IoT devices, thus opening the road for future research in the directions laid down by this thesis.

8

Conclusion

This final chapter concludes the thesis. We summarize the results discussed in the previous chapters and we use them to answer the research questions introduced in Chapter 1.

8.1. Summary of Contributions

We recall below the main research question defined in Chapter 1:

Main Research Question (MRQ)
How can we design and deploy Privacy-Enhancing Technologies effectively and efficiently so to be supported on constrained IoT devices and networks?

In the thesis, we have demonstrated that effective and efficient deployment of PETs on constrained IoT devices and networks cannot be executed through a general-purpose *one-size-fits-all* approach, but must instead be carefully customized to fit the specific requirements and constraints of the particular use case.

In Part I, we identified that discovering interferences between two independent networks working within the same spectrum raises confidentiality concerns, due to the sensitivity of the information connected to the channels assigned to various devices. We addressed this issue by presenting PRM, the first scheme to identify potential interferences among multi-tenant wireless networks in advance, without exposing the entire scheduling information of each network to untrusted parties (Chapter 2). The second use case focused on addressing the private matching problem of detecting spatiotemporal collisions among AVs in advance, to improve safety and reduce risks. We proposed ePPTM, a fully-accurate protocol for privacy-preserving trajectory matching among AVs, improving over our preliminary work in [191], and we demonstrated its effectiveness through a proof-of-concept evaluation on real-world datasets (Chapter 3).

We further discovered, in Part II, that traditional binary remote attestation protocols require the disclosure of all details about the executed software from all vendors, e.g., proprietary

code and sensitive configuration details, onboard IoT devices, posing confidentiality concerns. We introduced a privacy-preserving remote attestation approach that allows us to execute traditional binary remote attestation while selectively disclosing software details (Chapter 4).

In Part III, we investigated further real-world IoT use cases characterized by privacy concerns. We first considered the shared ownership of the involved data, in the context of Third-Party UAV services. Third-Party UAV services require the adoption of multi-party access control solutions, possibly leaking confidential information about the access control policies specified by the data owners. We proposed a privacy-preserving multi-party access control solution tailored to the application scenarios of Third-Party UAV Services and provided an extensive analysis of the feasibility of such a solution on drones (Chapter 5). To address the privacy concerns, we integrate SFE. SFE is a secure 2-party computation strategy over a function that evaluates policies, and the sensitive information of the policies is pre-shared through additive secret sharing. We leverage additive secret sharing so that neither the requester nor the drone can access the sensitive information defined in the policy by the DOs. We provide an extensive analysis of the feasibility of our solution regarding the policy size and the request size, so to evaluate the impact of the complexity and granularity of the policy on the overhead of drones.

The second use case for MPC is built on the consideration that on-device processing of sensitive biometric information creates a serious privacy risk, especially when devices can easily be stolen. Indeed, when an adversary seizes a drone, they can extract the stored biometric data, posing privacy risks for the users. We proposed *Obscura*, the first solution for privacy-preserving face verification onboard commercial drones (Chapter 6). To tackle the privacy risk, we securely compute the scaled cosine similarity function to identify whether the distance between two biometric templates is within a threshold. To realize our solution, we integrate the building blocks of ABY2.0 [163], where we leverage the secure addition and secure multiplication based on the *optimized additive secret sharing*. We further reduce the computational overhead by only computing the parts involving the reference template in the scaled cosine similarity function.

We also revisit and answer the sub-research questions laid down in Chapter 1.

SRQ1: *How can we effectively and efficiently identify matches in private input data stored onboard two IoT devices?*

In Chapter 2, we designed the protocol PRM, which identifies collisions in the RST of two independent networks without revealing the entire RST to the other party. Similarly, in Chapter 3, we introduce ePPTM, identifying in advance collision among two IoT devices or AVs. Both schemes identify matches through the underlying concept of PSI. PSI allows us to identify and reveal the actual matches without revealing the non-matching values. This guarantees the privacy of the data onboard the IoT devices, i.e., the RST or the entire trajectory path.

During the design of PRM and ePPTM we realized that using the plain PSI for solving these problems leads to a significant computational and energy overhead on the involved IoT devices. In PRM the RST in the communication technology IEEE 802.15.4 can have 1616

cells, which leads to the same amount of exponentiations onboard the IoT device. Similar considerations apply to ePPTM, depending on the number of trajectory points. Therefore, in both schemes, we proposed auxiliary algorithms to reduce the computational overhead and to identify *effectively* and *efficiently* the collisions. In PRM, we introduce the algorithm *iterative set division*, and we evaluated the approach on simulated data to understand the feasibility of our solution. We discovered that the overhead depends on the distribution of the occupied cells in the RST.

In ePPTM, we designed the algorithm *incremental capsule matching*, for detecting collisions in the trajectory data structure. As a result, our scheme and the auxiliary algorithms are efficient in identifying immediate non-collisions. However, if the relation between the input data is strong, then our solution may introduce more computational overhead. We discussed in more detail in Sec. 2.6, Sec. 3.6, and Sec. 7.6 how to fine-tune the algorithm to trade off between false positives and overhead.

We demonstrate the effectiveness of PRM and ePPTM by identifying private matches through proof-of-concept implementations and through the security analysis of the schemes.

SRQ2: *How can we remotely attest the trustworthiness of the proprietary software running on IoT devices while preserving its confidentiality?*

In Chapter 4, we designed a scheme that allows *remote attestation with constrained disclosure*. Specifically, we adapt the IMA to integrate NIZK, which enables us to blind the measurements, a.k.a. *event hash*. We then use a TPM to extend the event hashes into the TPM PCR. Furthermore, we store the generated NIZK proof along with its associated data in the IMA log. We modify the architecture of the remote attestation process by introducing a partial verifier that allows us to selectively disclose the event hashes from the IMA log to the corresponding partial verifier while maintaining the ability to verify the integrity of the TPM PCR values over the masked values (event hashes). In the end, we provide the results of all partial verifiers and the masked values to the relying party. This allows the relaying party to verify and aggregate results to confirm if the prover's system is trustworthy.

The results, obtained by using a real TPM on the Raspberry Pi, showcased that our solution requires 228 ms for the attestation process, while the traditional approach requires 218 ms. The adaptation of our scheme only introduces an average of 10 ms overhead. Meanwhile, the verification process requires a significant computational overhead. Therefore, we discussed in Sec. 7.5 various approaches to adapt our solution, which may lead to reduced computational overhead. To conclude, our solution preserves the privacy of the measurements while providing integrity and trustworthiness of the prover's system. We demonstrate the effectiveness of our scheme through a proof-of-concept implementation and a security analysis.

SRQ3: *Which design strategies allow reducing the computational overhead while guaranteeing input privacy in IoT devices?*

We have answered this question by designing schemes that leverage the strategy of precomputing the necessary cryptographic materials, so to reduce the computation overhead of various PETs when executed at runtime. As highlighted in Chapter 5, to pre-share the policies, we use additive secret sharing. This allows the drone not to understand the sensitive conditions of the policy while the policy is pre-shared by a trusted dealer, i.e., the DO. More-

over, in Sec. 7.1, we describe the *offline-online* paradigm, which allows the IoT devices to rely on trusted dealers during the *setup phase* (executed offline). As highlighted in our findings in Chapter 6, we pre-compute the function-dependent shares, leading to a significantly improved computational overhead. As stated in Chapter 6, our scheme Obscura requires approx. 27 ms to verify the face of a user while also achieving state-of-the-art accuracy of 97.2% for face recognition.

We highlight that the possibility of applying pre-computations to reduce computational and energy overhead at runtime depends on the function and the scenario. In Obscura the SP provides the pre-computed values, allowing to reduce the computational overhead during the online phase. However, this may lead to an increase in the storage size for multiple sessions. In Sec. 7.1, we discussed possible strategies to compute the necessary shares and values for the online phase with pseudorandom correlation generator (PCG) instantiated with Learning Parity with Noise (LPN) assumptions. To conclude, Obscura is the first work showcasing the strategy for mapping the MPC-setting and terminology for the IoT ecosystem to provide input privacy, while reducing the computational overhead. This solution can be adapted for IoT scenarios with different functions.

SRQ4: *How can we realize privacy-aware operations on IoT devices without relying on trusted third parties?*

To address this sub-research question, in the various chapters of this thesis, we have designed protocols that, when necessary, involve third parties only offline, before the deployment of the IoT devices. At runtime, such protocols always involve only the interacting parties, while not relying on any trusted party to complete the protocol.

In Chapter 2 and Chapter 3, we use *privacy-preserving equality testing* for privacy-aware operations on the IoT devices, which do not require a TTP. In Chapter 4, we use the TPM for providing a chain of trust and integrity. However, we adapt IMA by integrating NIZK to verify only the associated data while not disclosing any non-associated data and guaranteeing integrity over the masked measurements. This preserves the privacy of the proprietary software running of different SPs, without requiring a TTP.

Similarly, in Chapter 5 and Chapter 6, in the *offline phase*, we distribute the shares and necessary cryptographic materials for the 2-Party computation (MPC). During the *online-phase*, both parties do not require a TTP since they evaluate locally over the shares and reconstruct the result.

8.2. Final words

Overall, this thesis provides manifold contributions: (i) we identify many real-world IoT-related use cases generating privacy concerns; (ii) we identify privacy-enhancing technologies suitable for addressing these challenges; (iii) we design new algorithms and strategies to integrate such PETs on real-world constrained IoT devices while dealing effectively and efficiently with the constraints of the IoT ecosystem; and finally, (iv) we extensively evaluate such solutions on relevant real-world IoT devices to showcase their effectiveness and efficiency.

This thesis paves the way for the integration of PETs in the IoT domain to address real-world concerns about privacy, performance, and regulations. Our current understanding indicates that composing multiple PETs plays a significant role in future deployments, as stated in Sec. 7.8. Therefore, through our work, we aim to motivate the IoT security and applied cryptography communities to join forces to develop methods that can reduce the additional computational, bandwidth, and energy overhead while preserving the underlying security and privacy properties of PETs. The contributions presented in this thesis represent an initial step toward *privacy-aware next-generation IoT deployments*. In Chapter 7, we have outlined future key research directions for various strategies, as well as challenges that need to be addressed to deliver truly efficient and privacy-preserving schemes in the IoT domain.

A

Appendices

A. ProVerif

ProVerif assumes that the underlying cryptographic primitives at the roots of the protocols are robust, and that the adversary is aware of the steps and the public values of the protocol [32]. Based on these assumptions, ProVerif formally verifies the security of the cryptographic protocol against the Dolev-Yao attacker model, i.e., an adversary able to access, modify, delete, and forge new messages on the public communication channel. If ProVerif identifies an attack during the formal verification, it also lists the steps to achieve to break the protocol.

In our context, we applied ProVerif to formally verify the secrecy of the elements possessed by the two communicating entities. Recall that ProVerif provides the output *not attacker(elem[]) is true*, when the attacker cannot retrieve the value of *elem*. On the other hand, if the output is *not attacker(elem[]) is false*, then the attacker can retrieve the value of *elem*. Similarly, ProVerif provides the output *weak secret(elem[]) is true* when the attacker cannot execute guessing attacks on the value *elem* or brute-force it, while it provides the outcome *weak secret(elem[]) is false* when the attacker is able to guess or bruteforce the value *elem*. Proverif outputs *non-interference (elem[]) is true* then the adversary cannot distinguish if the input of the protocol is changing; otherwise, ProVerif returns the output *non-interference (elem[]) is false*. Finally, ProVerif verifies message authenticity by defining event sequences such as (event(e_1)==>event(e_0)) to confirm the occurrence of e_0 before e_1.

Chapter 2 ProVerif

Lst. 2.1 in Sec. 2.4.1 shows the output provided by ProVerif. We provide the source code with documentation in Listing A.1 for interested readers to reproduce our results and further extend them in customized use cases. Additionally, we released the code, which is available in [72].

Listing A.1: PRM ProVerif code.

```
(*Dolev-Yao model Open Channel*)
(*Channel between mobile network A and static network B*)
free c:channel.

free a: bitstring [private].
weaksecret a.
free b: bitstring [private].
weaksecret b.


(* RSA modulus *)
type N.


(* Randomness Generated by A *)
free rA: bitstring [private].

(* functions *)


query attacker(a).
query attacker(b).
query attacker(rA).

(*Auxiliary Functions*)
fun hash(bitstring):bitstring.
fun map(bitstring):bitstring.(*secure function of 2H(x)+1*)
fun append(bitstring,bitstring):bitstring.
fun mod(bitstring,N):bitstring.
fun exp_mod(bitstring,bitstring,N):bitstring.
fun inv_mod(bitstring,N):bitstring.
(*fun check(bitstring,bitstring):bool.*)

(*Events*)
event end_STATIC_B(bitstring).
event end_MOBILE_A(bitstring).


(* The process for mobile network A*)
let mobileA(nB:N) =
    (* Preparation *)
    (*new rA:bitstring;*)
    (*let lA = map(lA) in*)
    let cA = exp_mod(rA, map(a), nB) in
    out (c,cA);
    in (c, cB:bitstring);
    if cB = hash(mod(rA,nB)) then (
        let y = hash(mod(append(rA,a),nB)) in
```

```
49          out(c,y);
50
51          event end_MOBILE_A(cB)).
52
53
54  (* The process for static network B*)
55  let staticB(nB:N) =
56      in(c, cA:bitstring);
57      let eB = inv_mod(map(b),nB) in
58      let x = exp_mod(cA,eB,nB) in
59      let cB = hash(x) in
60      out (c, cB);
61      in (c, y:bitstring);
62      if y = hash(mod(append(x,b),nB)) then (
63      event end_STATIC_B(y)).
64
65  process
66      new nB: N;
67      ((!mobileA(nB)) | (!staticB(nB)))
```

Chapter 3 ProVerif

Fig. 3.6 in Sec. 3.4.1 shows the output provided by ProVerif. We include the source code with documentation in Listing A.2 of the first round of ePPTM. This allows interested readers to reproduce our results and extend them for customized use cases. Additionally, we released the code, which is available in [58].

Listing A.2: ePPTM ProVerif code.

```
1   (*Dolev-Yao model Open Channel*)
2   (*Channel between UAV A and UAV B*)
3   free c:channel.
4
5
6   (* RSA modulus *)
7   type N.
8
9   (* The attacker should not know the actual identifier of the
        capsule mapped from f (secret).
10  Although we define the secret as "weak" to verify if offline
        brute-force or dictionary attacks are feasible. *)
11
12  free dA_i: bitstring [private].
13  weaksecret dA_i.
14  free dB_i_k: bitstring [private].
15  weaksecret dB_i_k.
16
17
```

A

```
(* Types, Constants and Variables *)
type radius.
type length.
type angle.
type origin.
type nonce.
type vector.
type index.


query attacker(dA_i).
query attacker(dB_i_k).

(*Verify the non-interference property for the capsule identifiers
    A and B, which means that
the different secrets are indistinguishable for the adversary,
    strong secrecy.*)
noninterf dB_i_k.
noninterf dA_i.

(*Auxiliary Functions*)
fun hash(bitstring):bitstring.
fun map(bitstring):bitstring.(*secure function of 2H(x)+1*)
fun append(bitstring,bitstring):bitstring.
fun mod(bitstring,N):bitstring.
fun exp_mod(bitstring,bitstring,N):bitstring.
fun inv_mod(bitstring,N):bitstring.
fun extract_elem(vector,index):bitstring.
fun add_elem(vector,bitstring,index):vector.
fun map_trajectory(origin,angle,radius,length):vector.
(* Type converter *)
fun nonce_to_bitstring(nonce): bitstring [data,typeConverter].
(*Events*)
event end_UAV_A(vector).
event end_UAV_B(bitstring).



(* The process for UAV A*)
let uavA(nB:N) =

    new O_i:origin;
    new T_i:angle;
    new r_i:radius;
    new h_i:length;
    new xi_i:nonce;


    let cA_i = exp_mod(nonce_to_bitstring(xi_i), dA_i, nB) in
```

```
    out (c,(cA_i,O_i,T_i,r_i,h_i));
    in (c, cB_i:vector);
    new k:index;
    let cB_i_k = extract_elem(cB_i,k) in
    let wA_i_k = hash(mod(nonce_to_bitstring(xi_i),nB)) in
    if wA_i_k = cB_i_k then (
        let wA_i =
            hash(mod(append(nonce_to_bitstring(xi_i),dA_i),nB)) in
        out(c,wA_i);
        event end_UAV_A(cB_i)).



(* The process for UAV B*)
let uavB(nB:N) =

    in(c,
        (cA_i:bitstring,O_i:origin,T_i:angle,r_i:radius,h_i:length));
    let GB_i = map_trajectory(O_i,T_i,r_i,h_i) in
    new k:index;
    let eB_i_k = inv_mod(dB_i_k,nB) in
    let x_i_k = exp_mod(cA_i,eB_i_k,nB) in
    let cB_i_k = hash(x_i_k) in
    new cB_i:vector;
    let cB_i = add_elem(cB_i,cB_i_k,k) in
    out (c, cB_i);
    in (c, wA_i:bitstring);
    if wA_i = hash(mod(append(x_i_k,dB_i_k),nB)) then (
        event end_UAV_B(wA_i)).

process
    new nB: N;
    ((!uavA(nB)) | (!uavB(nB)))
```

Chapter 4 ProVerif

Lst. 4.1 in Sec. 4.7.1 shows the output provided by ProVerif. We include the source code with documentation in Listing A.3 of RACD protocol. This allows interested readers to reproduce our results and extend them for customized use cases. Additionally, we released the source code on GitHub at `https://github.com/DominikRoy/RACD`.

A

Listing A.3: RACD ProVerif code.

```
(*Dolev-Yao model Open Channel*)
free c:channel.
type list.
type tuple.
```

```
type none.
type nonce.
type skey.
type pkey.
type result.
type key.
free x_i: bitstring [private].
weaksecret x_i.


(* Randomness generated by Prover *)
free r_i: bitstring [private].
free v_i: bitstring [private].



(* Elliptic Curve *)
type G.
type L.



(* Auxiliary Functions *)
fun templatehash(bitstring):bitstring.
fun mod(bitstring,L):bitstring.
fun mul(bitstring,bitstring):bitstring.
fun point_mul(G,G):G.
fun hash(G,G,G):bitstring.
fun map(bitstring):bitstring. (*secure function of 2H(x)+1*)
fun append(G,G,G):bitstring.
fun exp(G,bitstring):G.
fun sub(bitstring,bitstring):bitstring.
fun tpm_pcr_extend(bitstring,G):none.
fun ima_cd(bitstring,G,bitstring,bitstring):none.
fun ima_cd_event():G.
fun ima_cd_s():bitstring.
fun ima_cd_c():bitstring.
fun requestTPMQuote():bitstring.
fun hash_chain(G):bitstring.
fun retrieve_all():bitstring.
fun collect_results(G,bool):list.
fun retrieve_results(list):bitstring.
fun retrieve_event(list):G.
(* Public key Cryptography *)
fun pk(skey): pkey.


(* Signatures *)
fun ok () : result .
```

```
54 fun sign ( bitstring , skey ) : bitstring .
55 reduc forall m : bitstring, sk : skey ; getmess(sign(m, sk)) = m .
56 reduc forall m : bitstring, sk : skey ; checksign(sign(m, sk),
       pk(sk)) = ok() .
57
58
59 (*Events*)
60 event secureboot().
61 event requestAttestation(nonce).
62 event acceptAttestationRequest(nonce). (*attester*)
63 event sendAttestationResult(bitstring,list). (*attester*)
64 event requestpartialVerification(nonce,G,
       bitstring,bitstring,bitstring). (*attester*)
65 event verifiedAttestationResult(bitstring, G,bool).
       (*partialverifer*)
66 (*event failedAttestationResult(pkey, bitstring,bool).*)
67 event trustable(). (*verifier*)
68
69 query attacker(x_i).
70 query attacker(r_i).
71 query attacker(v_i).
72
73
74 query pk:pkey, n:nonce, event_hash:G,
       c_i:bitstring,c_i':bitstring, tpmQuote_signed:bitstring,
       result:bool, partialAttestationresults:list;
75 event(trustable()) ==> (event(sendAttestationResult(
       tpmQuote_signed,partialAttestationresults)) ==>
       event(verifiedAttestationResult( tpmQuote_signed,
       event_hash,result))).
76
77
78 noninterf x_i among (r_i,v_i,   ima_cd_c(), ima_cd_s()).
79
80 let verifier(pk:pkey) =
81     new n:nonce;
82     (*event requestAttestation(n);*)
83     out(c,n);
84
85     in (c,(tpmQuote_signed:bitstring,
           partialAttestationresults:list));
86     let result' = retrieve_results(partialAttestationresults) in
87     let event_hash = retrieve_event(partialAttestationresults) in
88     let hash_chained = hash_chain(event_hash) in
89     new valid:bitstring;
90     let (hash_chained':bitstring, n':nonce) =
           getmess(tpmQuote_signed)in
91     if checksign(tpmQuote_signed,pk) = ok() then
92         if n' =n && hash_chained' = hash_chained && result' =
```

A

```
            valid then (
93          event trustable()).
94
95
96  let attester(index:bitstring, pk:pkey, sk:skey, g:G,odr:L) =
97      let event_hash = exp(g,mod(mul(r_i,
            mod(templatehash(x_i),odr)),odr)) in
98      let g_i = exp(g, mod(templatehash(x_i),odr)) in
99      let t_i = exp(g_i,v_i) in
100     let c_i = hash(g_i,t_i,event_hash) in
101     let s_i = mod(sub(v_i,mod(mul(r_i, mod(c_i,odr)),odr)),odr) in
102     (*event secureboot();*)
103     let pcr = tpm_pcr_extend(index,event_hash)in
104     let ima = ima_cd(index,event_hash,c_i,s_i)in
105
106
107     in(c,n:nonce);
108     (*event acceptAttestationRequest(n);*)
109     let tpmQuote = requestTPMQuote() in
110     let tpmQuote_signed = sign(tpmQuote,sk) in
111     let event_hash_ima = ima_cd_event() in
112     let s_i_ima = ima_cd_s() in
113     let c_i_ima= ima_cd_c() in
114     (*event requestpartialVerification(n,
            event_hash_ima,c_i_ima,s_i_ima, tpmQuote_signed);*)
115     out (c,(event_hash_ima,c_i_ima,s_i_ima,n, tpmQuote_signed));
116
117     in (c,(tpmQuote_signed':bitstring,
            event_hash':G,attresult:bool));
118     let partialAttestationresults =
            collect_results(event_hash',attresult) in
119     event sendAttestationResult(tpmQuote_signed',
            partialAttestationresults);
120     out(c,(tpmQuote_signed', partialAttestationresults)).
121
122
123 let partialVerifier(pk:pkey, g:G, odr:L)=
124     in(c, (event_hash_ima:G,c_i_ima:bitstring,
            s_i_ima:bitstring,n:nonce, tpmQuote_signed:bitstring));
125     let g_i = exp(g, mod(templatehash(x_i),odr)) in
126     let g_i_s = exp(g_i,s_i_ima) in
127     let event_hash_c = exp(event_hash_ima,mod(c_i_ima,odr)) in
128     let t_i' = point_mul(g_i_s,event_hash_c) in
129     let c_i' = hash(g_i,t_i',event_hash_ima) in
130     let hash_chained = hash_chain(event_hash_ima) in
131     let (hash_chained':bitstring, n':nonce) =
            getmess(tpmQuote_signed)in
132     if checksign(tpmQuote_signed,pk) = ok() then
133         if c_i' = c_i_ima && n' = n && hash_chained' =
```

A

```
            hash_chained then (
134         event verifiedAttestationResult(
                tpmQuote_signed,event_hash_ima, true);
135         out(c,(tpmQuote_signed, event_hash_ima,true))).
136
137
138
139 process
140     new sk:skey;
141     let pkey = pk(sk) in
142     new index:bitstring;
143     new g: G;
144     new odr:L;
145     ((!verifier(pkey)) | (!attester(index,pkey,sk,g,odr)) |
          (!partialVerifier(pkey,g,odr)) )
```

References

[1] Become a Drone Pilot, Online: https://www.faa.gov/uas/commercial_operators/become_a_drone_pilot/. (Accessed: 2023-03-01).

[2] Drone Services Market, Online: https://www.fortunebusinessinsights.com/drone-services-market-102682. (Accessed: 2023-03-01).

[3] **A. Nikoukar, S. Raza, A. Poole, et al.** Low-Power Wireless for the Internet of Things: Standards and Applications. *IEEE Access PP* (11 2018), 1–1.

[4] **Abdullahi, S. M., Sun, S., Wang, B., Wei, N., and Wang, H.** Biometric template attacks and recent protection mechanisms: A survey. Online: https://www.sciencedirect.com/science/article/pii/S1566253523004608. *Information Fusion 103* (2024), 102144.

[5] **Advanced Micro Devices, Inc.** AMD Secure Encrypted Virtualization (SEV), Online: https://www.amd.com/en/developer/sev.html.

[6] **Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., and Ayyash, M.** Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys& Tutorials 17*, 4 (2015), 2347–2376.

[7] **AllAboutSSL**. Drones is now getting Digital TLS / SSL Certificates. https://www.whatissslcertificate.com/drones-is-now-getting-digital-tls-ssl-certificates/. (Accessed: 2025-Mar-27).

[8] **Alliance, W.** WPA3™ Specification Version 3.4. https://www.wi-fi.org/system/files/WPA3%20Specification%20v3.4.pdf. (Accessed 2025-Aug-22).

[9] **Antonio De Rubertis, et al.** Performance evaluation of end-to-end security protocols in an Internet of Things. In *SoftCOM* (2013), pp. 1–6.

[10] **Aouedi, O., Vu, T.-H., Sacco, A., Nguyen, D. C., Piamrat, K., Marchetto, G., and Pham, Q.-V.** A Survey on Intelligent Internet of Things: Applications, Security, Privacy, and Future Directions. *IEEE Communications Surveys & Tutorials 27*, 2 (2025), 1238–1292.

[11] **Apvrille, L., Tanzi, T., and Dugelay, J.** Autonomous drones for assisting rescue services within the context of natural disasters. In *GASS* (2014), pp. 1–4.

[12] **Archer, D. W., de Balle Pigem, B., Bogdanov, D., Craddock, M., Gascon, A., Jansen, R., Jug, M., Laine, K., McLellan, R., Ohrimenko, O., Raykova, M., Trask, A., and**

Wardley, S. UN Handbook on Privacy-Preserving Computation Techniques, Online: https://arxiv.org/abs/2301.06167.

[13] **Argo, S., Güneysu, T., Jeudy, C., Land, G., Roux-Langlois, A., and Sanders, O.** Practical Post-Quantum Signatures for Privacy. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2024), CCS '24, Association for Computing Machinery, p. 1523–1537.

[14] **Arm Ltd.** mbed TLS, Online: https://tls.mbed.org/.

[15] **ASHTON, K.** That 'Internet of Things' Thing. Online: https://cir.nii.ac.jp/crid/1573950401096572288. *RFID Journal 22* (2009).

[16] **Atzori, L., Iera, A., and Morabito, G.** The Internet of Things: A survey. *Computer Networks 54*, 15 (2010), 2787–2805.

[17] **Badlani, D. K., and Diglio, A.** Microsoft open sources its software bill of materials (SBOM) generation tool, Online: https://devblogs.microsoft.com/engineering-at-microsoft/microsoft-open-sources-software-bill-of-materials-sbom-generation-tool/.

[18] **Baghery, K., Knapen, N., Nicolas, G., and Rahimi, M.** Pre-Constructed Publicly Verifiable Secret Sharing and Applications. Cryptology ePrint Archive, Paper 2025/576, Online: https://eprint.iacr.org/2025/576.

[19] **Barton, T. and Azhar, Hannan**. Forensic Analysis of Popular UAV Systems. *Proc. of the 7th Int. Conf. on Emerging Security Technologies* (September 2017).

[20] **Baumgarten, M. C., Röper, J., Hahnenkamp, K., and Thies, K.** Drones delivering Automated External Defibrillators—Integrating Unmanned Aerial Systems into the Chain of Survival: A Simulation Study in Rural Germany. *Resuscitation 172* (2022), 139–145.

[21] **Beaver, Donald**. Efficient Multiparty Protocols Using Circuit Randomization. In *Advances in Cryptology — CRYPTO '91* (Berlin, Heidelberg, 1992), Feigenbaum, Joan, Ed., Springer Berlin Heidelberg, pp. 420–432.

[22] **Belguechi, R. O., and Rosenberger, C.** Mitigate authentication attack risk on cancelable biometrics by leveraging attacker knowledge. Online: https://doi.org/10.1186/s13635-025-00198-3. *EURASIP Journal on Information Security 2025*, 1 (Apr 2025), 13.

[23] **Belwafi, K., Alkadi, R., Alameri, S. A., Hamadi, H. A., and Shoufan, A.** Unmanned Aerial Vehicles' Remote Identification: A Tutorial and Survey. *IEEE Access 10* (2022), 87577–87601.

[24] **Bernstein, C.** What is drone services (UAV services)?, Online: https://tinyurl.com/jtd8p5sr. (Accessed: 2023-03-01).

[25] **Bernstein, D. J.** Curve25519: New Diffie-Hellman Speed Records. In *Public Key Cryptography - PKC 2006* (Berlin, Heidelberg, 2006), M. Yung, Y. Dodis, A. Kiayias, and T. Malkin, Eds., Springer Berlin Heidelberg, pp. 207–228.

[26] **Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., and Yang, B.-Y.** High-Speed High-Security Signatures. In *CHES* (2011), vol. 6917 of *Lecture Notes in Computer Science*, Springer, pp. 124–142.

[27] **Bernstein, D. J., and Lange, T.** SafeCurves: Choosing safe curves for elliptic-curve cryptography, Online: `https://safecurves.cr.yp.to`.

[28] **Bhati, A. S., Pohle, E., Abidin, A., Andreeva, E., and Preneel, B.** Let's Go Eevee! A Friendly and Suitable Family of AEAD Modes for IoT-to-Cloud Secure Computation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2023), CCS '23, Association for Computing Machinery, p. 2546–2560.

[29] **Bhuiyan, M. N., Rahman, M. M., Billah, M. M., and Saha, D.** Internet of Things (IoT): A Review of Its Enabling Technologies in Healthcare Applications, Standards Protocols, Security, and Market Opportunities. *IEEE Internet of Things Journal 8*, 13 (2021), 10474–10498.

[30] **Birkholz, H., Thaler, D., Richardson, M., and Pan, W.** Remote ATtestation procedureS (RATS) Architecture. RFC 9334, RFC Editor, Jan. 2023.

[31] **Bishop, C. M.** *Pattern Recognition and Machine Learning (Information Science and Statistics)*, 1 ed. Springer, 2007.

[32] **Blanchet, B.** Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security 1*, 1–2 (Oct. 2016), 1–135.

[33] **Blanchet, B.** Symbolic and Computational Mechanized Verification of the ARINC823 Avionic Protocols. In *IEEE CSF'17* (Aug. 2017), pp. 68–82.

[34] **Blum, M., De Santis, A., Micali, S., and Persiano, G.** Noninteractive Zero-Knowledge. Online: `https://epubs.siam.org/doi/10.1137/0220068`. *SIAM Journal on Computing 20*, 6 (1991), 1084–1118.

[35] **Bock, L.** *Identity Management with Biometrics: Explore the latest innovative solutions to provide secure identification and authentication*. Packt Publishing Ltd, 2020.

[36] **Boddeti, V. N.** Secure Face Matching Using Fully Homomorphic Encryption. In *International Conference on Biometrics Theory, Applications and Systems (BTAS)* (2018), IEEE, pp. 1–10.

[37] **Bohling, F., Mueller, T., Eckel, M., and Lindemann, J.** Subverting Linux' Integrity Measurement Architecture. In *Proceedings of the 15th International Conference on Availability, Reliability and Security* (New York, NY, USA, 2020), ARES '20, Association for Computing Machinery.

[38] **Boutros, F., Huber, M., Siebke, P., Rieber, T., and Damer, N.** SFace: Privacy-friendly and Accurate Face Recognition using Synthetic Data. In *2022 IEEE International Joint Conference on Biometrics (IJCB)* (2022), pp. 1–11.

[39] **Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Rindal, P., and Scholl, P.** Efficient Two-Round OT Extension and Silent Non-Interactive Secure Computation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2019), CCS '19, Association for Computing Machinery, p. 291–308.

[40] **Brendel, J., Cremers, C., Jackson, D., and Zhao, M.** The Provable Security of Ed25519: Theory and Practice, Online: `http://eprint.iacr.org/2020/823`.

[41] **Brickell, E., Camenisch, J., and Chen, L.** Direct Anonymous Attestation. In *Proceedings of the 11th ACM Conference on Computer and Communications Security* (New York, NY, USA, 2004), CCS '04, Association for Computing Machinery, p. 132–145.

[42] **Brighente, A., Conti, M., Schotsman, M., and Sciancalepore, S.** Obfuscated Location Disclosure for Remote ID Enabled Drones. *arXiv preprint arXiv:2407.14256* (2024).

[43] **Brighente, A., Conti, M., and Sciancalepore, S.** Hide and Seek: Privacy-Preserving and FAA-Compliant Drones Location Tracing. In *Proc. of the Int. Conf. on Availability, Reliability and Security* (2022), pp. 1–11.

[44] **Buchanan-Wollaston, Joe and Storer, Tim and Glisson, William**. Comparison of the Data Recovery Function of Forensic Tools. In *Advances in Digital Forensics IX* (Berlin, Heidelberg, 2013), Peterson, Gilbert and Shenoi, Sujeet, Ed., Springer Berlin Heidelberg, pp. 331–347.

[45] **Business Wire**. Global Drones as a Service Market - by Applications and Leading Industries, Online: `tinyurl.com/yw29enwt`. (Accessed: 2023-03-01).

[46] **Candal-Ventureira, D., González-Castaño, F. J., Gil-Castiñeira, F., and Fondo-Ferreiro, P.** Is the Edge Really Necessary for Drone Computing Offloading? An Experimental Assessment in Carrier-Grade 5G Operator Networks. *Software: Practice and Experience 53*, 3 (2023), 579–599.

[47] **Cerulli, A., Cristofaro, E. D., and Soriente, C.** Nothing Refreshes Like a RePSI: Reactive Private Set Intersection. Cryptology ePrint Archive, Paper 2018/344.

[48] **Cha, S.-C., Hsu, T.-Y., Xiang, Y., and Yeh, K.-H.** Privacy Enhancing Technologies in the Internet of Things: Perspectives and Challenges. *IEEE Internet of Things Journal 6*, 2 (2019), 2159–2187.

[49] **Chakraborty, D., Hanzlik, L., and Bugiel, S.** simTPM: User-centric TPM for Mobile Devices. In *28th USENIX Security Symposium (USENIX Security 19)* (Santa Clara, CA, Aug. 2019), USENIX Association, pp. 533–550.

[50] **Chase, M.** Multi-authority Attribute Based Encryption. In *Theory of Cryptography* (Berlin, Heidelberg, 2007), Springer, pp. 515–534.

[51] **Chase, M., and Chow, S. S.** Improving Privacy and Security in Multi-Authority Attribute-Based Encryption. In *CCS '09* (2009), p. 121–130.

[52] **Chaum, D. L.** Untraceable electronic mail, return addresses, and digital pseudonyms. Online: https://doi.org/10.1145/358549.358563. *Commun. ACM 24*, 2 (Feb. 1981), 84–90.

[53] **Chen, L., Löhr, H., Manulis, M., and Sadeghi, A.-R.** Property-Based Attestation without a Trusted Third Party. In *Proceedings of the 11th International Conference on Information Security* (Berlin, Heidelberg, 2008), ISC '08, Springer-Verlag, pp. 31–46.

[54] **Chen, Y., Li, K., Wu, Y., Huang, J., and Zhao, L.** Energy Efficient Task Offloading and Resource Allocation in Air-Ground Integrated MEC Systems: A Distributed Online Approach. *IEEE Transactions on Mobile Computing 23*, 8 (2024), 8129–8142.

[55] **Cheng, N., Önen, M., Mitrokotsa, A., Chouchane, O., Todisco, M., and Ibarrondo, A.** Nomadic: Normalising Maliciously-Secure Distance with Cosine Similarity for Two-Party Biometric Authentication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (New York, NY, USA, 2024), ASIA CCS '24, Association for Computing Machinery, p. 257–273.

[56] **Christy Bieber, J.** 93% Have Concerns About Self-Driving Cars According to New Forbes Legal Survey. https://www.forbes.com/advisor/legal/auto-accident/perception-of-self-driving-cars/. (Accessed 2025-Mar-27).

[57] **Coker, G., Guttman, J., Loscocco, P., Herzog, A., Millen, J., O'Hanlon, B., Ramsdell, J., Segall, A., Sheehy, J., and Sniffen, B.** Principles of Remote Attestation. *Int. J. Inf. Secur. 10*, 2 (June 2011), 63–81.

[58] **D. George, S. Sciancalepore**. Open Source Code of the Implementation of ePPTM into ProVerif. https://github.com/DominikRoy/PPTM. (Accessed: 2025-Mar-27).

[59] **D. George, S. Sciancalepore**. Open Source Code of the Implementation of ePPTM. https://github.com/DominikRoy/ePPTM. (Accessed: 2025-Mar-27).

[60] **Damgård, I., Geisler, M., and Krøigaard, M.** Efficient and Secure Comparison for On-Line Auctions. In *Information Security and Privacy* (2007), J. Pieprzyk, H. Ghodosi, and E. Dawson, Eds., Lecture Notes in Computer Science, Springer, pp. 416–430.

[61] **De Cristofaro, E., Kim, J., and Tsudik, G.** Linear-Complexity Private Set Intersection Protocols Secure in Malicious Model. In *Advances in Cryptology - ASIACRYPT 2010* (Berlin, Heidelberg, 2010), M. Abe, Ed., Springer Berlin Heidelberg, pp. 213–231.

[62] **De Cristofaro, E., and Tsudik, G.** Practical Private Set Intersection Protocols with Linear Complexity. In *Financial Cryptography and Data Security* (Berlin, Heidelberg, 2010), R. Sion, Ed., Springer Berlin Heidelberg, pp. 143–159.

[63] **Debes, H. B., and Giannetsos, T.** ZEKRO: Zero-Knowledge Proof of Integrity Conformance. In *Proceedings of the 17th International Conference on Availability, Reliability and Security* (New York, NY, USA, 2022), ARES '22, Association for Computing Machinery.

[64] **Dehghan, M., and Sadeghiyan, B.** Privacy-Preserving Collision Detection of Moving Objects. *Transactions on Emerging Telecommunications Technologies 30*, 3 (2019), e3484.

[65] **Demmler, D., Schneider, T., and Zohner, M.** ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation.

[66] **Denis, F.** Libsodium documentation, Online: `https://libsodium.gitbook.io/doc/`.

[67] **Desai, A., Bautista, O. G., and Akkaya, K.** Privacy-Preserving Collision Detection for Drone-based Aerial Package Delivery using Secure Multi-Party Computation. In *Int. Symp. on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing* (2023), MobiHoc '23, p. 510–515.

[68] **Diglio, A.** Generating Software Bills of Materials (SBOMs) with SPDX at Microsoft, Online: `https://devblogs.microsoft.com/engineering-at-microsoft/generating-software-bills-of-materials-sboms-with-spdx-at-microsoft/`.

[69] **DJI**. DJI Demonstrates Direct Drone-To-Phone Remote Identification. `https://tinyurl.com/y39pft7x`. (Accessed: 2025-Mar-27).

[70] **DJI**. DJI Mini 2 Specs. `https://www.dji.com/nl/mini-2/specs`. (Accessed: 2023-03-01).

[71] **DJI**. T10 - Specifications - DJI — dji.com. `https://www.dji.com/nl/t10/specs`. (Accessed: 2025-Aug-22).

[72] **Dominik Roy George**. PRM ProVerif Code. `https://github.com/DominikRoy/PRM`. (Accessed: 2022-08-10).

[73] **Dominik Roy George**. Open Source Code of the Implementation of Obscura. `https://github.com/DominikRoy/Obscura`. (Accessed: 2025-Aug-21).

[74] **Drone Remote-ID**. Tech specifications for Remote ID range. `https://drone-remote-id.com/#tech_specifications_for_remote_id_range`. (Accessed: 2025-Mar-27).

[75] **DroneDOJO**. Raspberry Pi Drone | The Ultimate Project Drone. `https://dojofordrones.com/raspberry-pi-drone/`. (Accessed: 2025-Mar-27).

[76] **Dwork, C., McSherry, F., Nissim, K., and Smith, A.** Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography* (Berlin, Heidelberg, 2006), S. Halevi and T. Rabin, Eds., Springer Berlin Heidelberg, pp. 265–284.

[77] **EASA**. Data Protection Impact Assessment, Online: `https://www.easa.europa.eu/en/domains/civil-drones/privacy/data-protection-impact-assessment`. (Accessed: 2025-Aug-22).

[78] **Eckel, M., George, D. R., Grohmann, B., and Krauß, C.** Remote Attestation with Constrained Disclosure. In *Proceedings of the 39th Annual Computer Security Applica-*

tions Conference (New York, NY, USA, 2023), ACSAC '23, Association for Computing Machinery, p. 718–731.

[79] **EdalatNejad, K., Lueks, W., Sukaitis, J., Narbel, V. G., Marelli, M., and Troncoso, C.** Janus: Safe Biometric Deduplication for Humanitarian Aid Distribution. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), pp. 655–672.

[80] **Erkin, Z., Franz, M., Guajardo, J., Katzenbeisser, S., Lagendijk, I., and Toft, T.** Privacy-Preserving Face Recognition. In *Privacy Enhancing Technologies* (2009), I. Goldberg and M. J. Atallah, Eds., Lecture Notes in Computer Science, Springer, pp. 235–253.

[81] **Escudero, D.** An Introduction to Secret-Sharing-Based Secure Multiparty Computation. Cryptology ePrint Archive, Paper 2022/062, Online: https://eprint.iacr.org/2022/062.

[82] **ESPcopter**. ESPcopter: Programmable MiniDrone. https://espcopter.com/. (Accessed: 2021-06-17).

[83] **EU**. Cyber Resilience Act. https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act. (Accessed: 2025-Aug-22).

[84] **Evans, D., Kolesnikov, V., and Rosulek, M.** A Pragmatic Introduction to Secure Multi-Party Computation. Online: http://dx.doi.org/10.1561/3300000019. *Foundations and Trends® in Privacy and Security 2*, 2-3 (2018), 70–246.

[85] **Executive Office of the President**. EO 14028: Improving the Nation's Cybersecurity, Online: https://www.federalregister.gov/executive-order/14028.

[86] **Fan, K., Xu, H., Gao, L., Li, H., and Yang, Y.** Efficient and privacy preserving access control scheme for fog-enabled IoT. *Future Generation Computer Systems 99* (2019), 134–142.

[87] **Fraunhofer SIT**. CHARRA: CHAllenge-Response based Remote Attestation with TPM 2.0, Online: https://github.com/Fraunhofer-SIT/charra.

[88] **Frikken, K. B., and Atallah, M. J.** Privacy Preserving Route Planning. In *Proc. of ACM Workshop on Privacy in the Electronic Society* (2004), pp. 8–15.

[89] **Fuchs, A., and Struk, T.** tpm2-software/tpm2-tss, Online: https://github.com/tpm2-software/tpm2-tss. original-date: 2015-06-30T16:21:57Z.

[90] **Funke, J., Brown, M., Erlien, S. M., and Gerdes, J. C.** Collision Avoidance and Stabilization for Autonomous Vehicles in Emergency Scenarios. *IEEE Transactions on Control Systems Technology 25*, 4 (2017), 1204–1216.

[91] **Garrido, G. M., Sedlmeir, J., Ömer Uludağ, Alaoui, I. S., Luckow, A., and Matthes, F.** Revealing the landscape of privacy-enhancing technologies in the context of data markets for the IoT: A systematic literature review.

Online: https://www.sciencedirect.com/science/article/pii/S1084804522001126. *Journal of Network and Computer Applications 207* (2022), 103465.

[92] **George, D. R.** *Privacy-Preserving Remote Attestation Protocol.* Master's thesis, TU Darmstadt/TU Wien, 2021.

[93] **George, D. R., and Sciancalepore, S.** PRM - Private Interference Discovery for IEEE 802.15. 4 Networks. In *2022 IEEE Conference on Communications and Network Security (CNS)* (2022), pp. 136–144.

[94] **George, D. R., and Sciancalepore, S.** ePPTM -Enhanced Privacy-Preserving Trajectory Matching on Autonomous Vehicles. *IEEE Internet of Things Journal* (2025), 1–1.

[95] **George, D. R., Sciancalepore, S., and Zannone, N.** Privacy-Preserving Multi-Party Access Control for Third-Party UAV Services. In *Proc.of the 28th ACM Symposium on Access Control Models and Technologies* (2023), pp. 19–30.

[96] **Globenswire**. Autonomous Vehicle Market Size to Reach USD 888.40 Billion by 2028. https://www.globenewswire.com/news-release/2021/02/01/2167017/0/en/Autonomous-Vehicle-Market-Size-to-Reach-USD-888-40-Billion-by-2028-Emergence-of-Advanced-Driver-Assistance-System-ADAS-will-be-the-Key-Factor-Driving-the-Industry-Growth-States-Eme.html. (Accessed: 2025-Mar-27).

[97] **Goldreich, O., Micali, S., and Wigderson, A.** How to play ANY mental game. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing* (New York, NY, USA, 1987), STOC '87, Association for Computing Machinery, p. 218–229.

[98] **Goldreich, O., and Oren, Y.** Definitions and properties of zero-knowledge proof systems. Online: https://doi.org/10.1007/BF00195207. *Journal of Cryptology 7*, 1 (Dec 1994), 1–32.

[99] **Grand View Research**. *Internet of Things (IoT) Market Size, Share & Trends Analysis Report*. https://www.grandviewresearch.com/industry-analysis/internet-of-things-iot-market. Forecasts global IoT revenue to reach $2.28 trillion by 2030 with an 11.4% CAGR (2024–2030). (Accessed 23-May-2025).

[100] **Greenshpan, M., and List, T.** FA6 Drone - Face-Six - The Face Recognition Software Company — face-six.com. https://www.face-six.com/drone/. (Accessed: 2025-Aug-22).

[101] **Greenwood, Faine and Joseph, Dan**. Aid from the Air: A Review of Drone use in the RCRC Global Network, Online: https://americanredcross.github.io/rcrc-drones/Aid_from_the_Air.pdf. (Accessed: 2025-Aug-22).

[102] **Hamburg, M.** Decaf: Eliminating cofactors through point compression, Online: http://eprint.iacr.org/2015/673.

[103] **Hamburg, M., de Valence, H., Lovecruft, I., and Arcieri, T.** Ristretto - The Ristretto Group, Online: https://ristretto.group/ristretto.html.

[104] **Han, J., Chen, L., Schneider, S., Treharne, H., and Wesemeyer, S.** Privacy-Preserving Electronic Ticket Scheme with Attribute-Based Credentials. *IEEE Transactions on Dependable and Secure Computing 18*, 4 (2021), 1836–1849.

[105] **Hao, F.** Schnorr Non-interactive Zero-Knowledge Proof. RFC 8235, Online: https://rfc-editor.org/rfc/rfc8235.txt.

[106] **Harth-Kitzerow, C., Suresh, A., Wang, Y., Yalame, H., Carle, G., and Annavaram, M.** High-Throughput Secure Multiparty Computation with an Honest Majority in Various Network Settings, Online: https://arxiv.org/abs/2206.03776.

[107] **Hermeto, R. T., Gallais, A., and Theoleyre, F.** Scheduling for IEEE802. 15.4-TSCH and slow channel hopping MAC in low power industrial wireless networks: A survey. *Computer Communications 114* (2017), 84–105.

[108] **Hes, R., and Borking, J.** Privacy-Enhancing Technologies: The Path to Anonymity. Tech. rep., Registratiekamer, 1998.

[109] **Hsu, H.-J., and Chen, K.-T.** DroneFace: An Open Dataset for Drone Research. In *Proceedings of the 8th ACM on Multimedia Systems Conference* (New York, NY, USA, 2017), MMSys'17, Association for Computing Machinery, p. 187–192.

[110] **Huang, G. B., Ramesh, M., Berg, T., and Learned-Miller, E.** Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments. Tech. Rep. 07-49, University of Massachusetts, Amherst, October 2007.

[111] **Huang, H., and Wang, L.** Efficient Privacy-Preserving Face Verification Scheme. Online: https://www.sciencedirect.com/science/article/pii/S2214212621002386. *Journal of Information Security and Applications 63* (2021), 103055.

[112] **Huang, P., Hoang, T., Li, Y., Shi, E., and Suh, G. E.** Efficient Privacy-Preserving Machine Learning with Lightweight Trusted Hardware. Online: https://doi.org/10.56553/popets-2024-0119. *Proc. Priv. Enhancing Technol. 2024*, 4 (2024), 327–348.

[113] **Huda, S. A., and Moh, S.** Survey on computation offloading in UAV-Enabled mobile edge computing. *Journal of Network and Computer Applications 201* (2022), 103341.

[114] **Hussain, R., and Zeadally, S.** Autonomous Cars: Research Results, Issues, and Future Challenges. *IEEE Communications Surveys Tutorials 21*, 2 (2019), 1275–1313.

[115] **Ibarrondo, A., Chabanne, H., and Önen, M.** Funshade: Function Secret Sharing for Two-Party Secure Thresholded Distance Evaluation. *Proc. Priv. Enhancing Technol. 2023*, 4 (oct 2023), 21–34.

[116] **Im, J.-H., Jeon, S.-Y., and Lee, M.-K.** Practical Privacy-Preserving Face Authentication for Smartphones Secure Against Malicious Clients. *IEEE Transactions on Information Forensics and Security 15* (2020), 2386–2401.

[117] **Inside Unmanned Systems**. Amsterdam's Canals Float Fleet of Autonomous Boats for Transport, Waste Collection. https://insideunmannedsystems.com/amsterdams-canals-float-fleet-of-autonomous-boats-for-passenger-transport-waste-collection-environmental-sensing/. (Accessed: 2025-Mar-27).

[118] **Intel**. Intel NUC7i5BNK. https://www.intel.com/content/dam/support/us/en/documents/mini-pcs/nuc-kits/NUC7i5BN_NUC7i7BN_TechProdSpec.pdf. (Accessed: 2025-Aug-22).

[119] **Intel Corporation**. Intel Trusted Execution Technology (Intel TXT) Overview, Online: https://www.intel.com/content/www/us/en/developer/articles/tool/intel-trusted-execution-technology.html.

[120] **Intel Corporation**. Intel Software Guard Extensions (Intel SGX), Online: https://software.intel.com/sgx.

[121] **Intel Corporation**. Intel Trust Domain Extensions (Intel TDX), Online: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-trust-domain-extensions.html.

[122] **Jäger, L., Lorych, D., and Eckel, M.** A Resilient Network Node for the Industrial Internet of Things. In *Proceedings of the 17th International Conference on Availability, Reliability and Security* (New York, NY, USA, 2022), ARES '22, Association for Computing Machinery.

[123] **Jahromi, A. N., Karimipour, H., Dehghantanha, A., and Choo, K.-K. R.** Toward Detection and Attribution of Cyber-Attacks in IoT-Enabled Cyber–Physical Systems. *IEEE Internet of Things Journal 8*, 17 (2021), 13712–13722.

[124] **Jeong, S., Kim, H.-S., Paek, J., and Bahk, S.** OST: On-Demand TSCH Scheduling with Traffic-Awareness. In *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications* (2020), pp. 69–78.

[125] **Jie, L. S., and Ping, H. Y.** A Privacy-Preserving Integrity Measurement Architecture. In *2010 Third International Symposium on Electronic Commerce and Security* (July 2010), IEEE Transactions on Information Forensics and Security, pp. 242–246.

[126] **Jin, H.** Drones in construction 2022: Top Full Guide For You, Online: https://lucidcam.com/drones-in-construction/.

[127] **Jo, M. H., Schneider, P., and Vinel, A.** Driving Towards Safety: Open Challenges in Safeguarding CPS-IoT for Cooperative Intelligent Transportation System*. In *2024 IEEE Intelligent Vehicles Symposium (IV)* (2024), pp. 1707–1714.

[128] **Josefsson, S., and Schaad, J.** Algorithm Identifiers for Ed25519, Ed448, X25519, and X448 for Use in the Internet X.509 Public Key Infrastructure. RFC 8410, Online: https://rfc-editor.org/rfc/rfc8410.txt.

[129] **K. Phung, B. Lemmens, M. Goossens, et al.** Schedule-based multi-channel communication in wireless sensor networks: A complete design and performance evaluation. *Ad Hoc Networks 26* (2015), 88–102.

[130] **Kalra, I., Singh, M., Nagpal, S., Singh, R., Vatsa, M., and Sujit, P. B.** DroneSURF: Benchmark Dataset for Drone-based Face Recognition. In *2019 14th IEEE Int. Conf. on Automatic Face & Gesture Recognition (FG 2019)* (2019), pp. 1–7.

[131] **Kang, B., and Choo, H.** An experimental study of a reliable IoT gateway. *ICT Express 4*, 3 (2018), 130–133.

[132] **Kangunde, V., Jamisola, R. S., and Theophilus, E. K.** A Review on Drones controlled in Real-Time. Online: `https://doi.org/10.1007/s40435-020-00737-5`. *International Journal of Dynamics and Control 9*, 4 (Dec 2021), 1832–1846.

[133] **Kapadia, A., Tsang, P. P., and Smith, S. W.** Attribute-Based Publishing with Hidden Credentials and Hidden Policies. In *NDSS* (2007).

[134] **Karalis, A., Zorbas, D., and Douligeris, C.** Collision-Free Advertisement Scheduling for IEEE 802.15.4-TSCH Networks. *Sensors 19*, 8 (2019).

[135] **Khalid, O., Rolfes, C., and Ibing, A.** On Implementing Trusted Boot for Embedded Systems. In *2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)* (Austin, TX, USA, June 2013), IEEE, pp. 75–80.

[136] **Kobeissi, N., Bhargavan, K., and Blanchet, B.** Automated Verification for Secure Messaging Protocols and their Implementations: A Symbolic and Computational Approach. In *IEEE EuroS&P'17* (Apr. 2017), pp. 435–450.

[137] **Komninos, N., and Junejo, A.** Privacy Preserving Attribute Based Encryption for Multiple Cloud Collaborative Environment. In *UCC* (2015), pp. 595–600.

[138] **Kong, X., Chen, Q., Hou, M., Wang, H., and Xia, F.** Mobility Trajectory Generation: A Survey. *Artificial Intelligence Review 56*, 3 (Dec 2023), 3057–3098.

[139] **Koubâa, A., Ammar, A., Alahdab, M., Kanhouch, A., and Azar, A. T.** Deepbrain: Experimental evaluation of cloud-based computation offloading and edge computing in the internet-of-drones for deep learning applications. *Sensors 20*, 18 (2020), 5240.

[140] **Krishna Prakasha, K., and Sumalatha, U.** Privacy-Preserving Techniques in Biometric Systems: Approaches and Challenges. *IEEE Access 13* (2025), 32584–32616.

[141] **Lauer, H., Sakzad, A., Rudolph, C., and Nepal, S.** A Logic for Secure Stratified Systems and its Application to Containerized Systems. In *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)* (Aug 2019), pp. 562–569.

[142] **Li, C., and Palanisamy, B.** Privacy in Internet of Things: From Principles to Technologies. *IEEE Internet of Things Journal 6*, 1 (2019), 488–505.

[143] **Liang, J., Hu, D., Wu, P., Yang, Y., Shen, Q., and Wu, Z.** SoK: Understanding zk-SNARKs: The Gap Between Research and Practice. Cryptology ePrint Archive, Paper 2025/172, Online: https://eprint.iacr.org/2025/172.

[144] **Lu, H., Li, Y., Mu, S., Wang, D., Kim, H., and Serikawa, S.** Motor Anomaly Detection for Unmanned Aerial Vehicles Using Reinforcement Learning. *IEEE Internet of Things Journal 5*, 4 (2017), 2315–2322.

[145] **Lufadeju, Yemi**. UNICEF expands Network of Drone Testing Corridors, Online: https://www.unicef.org/press-releases/unicef-expands-network-drone-testing-corridors. (Accessed: 2025-Aug-22).

[146] **Lundblade, L.** QCBOR: an implementation of nearly everything in RFC8949, Online: https://github.com/laurencelundblade/QCBOR.

[147] **Luo, W., Shen, Q., Xia, Y., and Wu, Z.** Container-IMA: A privacy-preserving Integrity Measurement Architecture for Containers. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)* (Chaoyang District, Beijing, Sept. 2019), USENIX Association, pp. 487–500.

[148] **M. Palattella, N., Nicola, X. Vilajosana, et al.** Standardized Protocol Stack For The Internet Of (Important) Things. *IEEE Commun. Surveys Tuts. 15* (01 2013).

[149] **Ma, Y., Wu, L., Gu, X., He, J., and Yang, Z.** A Secure Face-Verification Scheme Based on Homomorphic Encryption and Deep Neural Networks. *IEEE Access 5* (2017), 16532–16538.

[150] **Mariani, S., Cabri, G., and Zambonelli, F.** Coordination of Autonomous Vehicles: Taxonomy and Survey. Online: https://doi.org/10.1145/3431231. *ACM Computing Surveys 54*, 1 (Feb. 2021).

[151] **Markets and Markets**. Drone Package Delivery Market by Solution (Platform, Infrastructure, Software, Service), Type, Range (Short (<25 km), Medium (25–50 km), Long (>50 km)), Package Size, Duration, End Use, Operation Mode - Global Forecast to 2030, Online: https://www.marketsandmarkets.com/Market-Reports/drone-package-delivery-market-10580366.html. (Accessed: 2025-Aug-22).

[152] **Mehmood, Y., Ahmad, F., Yaqoob, I., Adnane, A., Imran, M., and Guizani, S.** Internet-of-Things-Based Smart Cities: Recent Advances and Challenges. *IEEE Communications Magazine 55*, 9 (2017), 16–24.

[153] **Microsoft Corp.** Microsoft SBOM Tool (Open Source on GitHub), Online: https://github.com/microsoft/sbom-tool.

[154] **Ming, Y., and Zhang, T.** Efficient Privacy-Preserving Access Control Scheme in Electronic Health Records System. *Sensors 18* (10 2018), 3520.

[155] **Morales, D., Agudo, I., and Lopez, J.** Private set intersection: A systematic literature review. Online: https://www.sciencedirect.com/science/

article/pii/S1574013723000345. *Computer Science Review 49* (2023), 100567.

[156] **Moreira-Matias, L., Gama, J., Ferreira, M., Mendes-Moreira, J., and Damas, L.** Predicting Taxi–Passenger Demand Using Streaming Data. *IEEE Transactions on Intelligent Transportation Systems 14*, 3 (2013), 1393–1402.

[157] **Mustafa, D., Alkhasawneh, R., Obeidat, F., and Shatnawi, A. S.** MIMD Programs Execution Support on SIMD Machines: A Holistic Survey. *IEEE Access 12* (2024), 34354–34377.

[158] **Nishide, T., Yoneyama, K., and Ohta, K.** Attribute-Based Encryption with Partially Hidden Encryptor-Specified Access Structures. In *ACNS '08* (2008), Springer, p. 111–129.

[159] **Nkenyereye, L., Liu, C. H., and Song, J.** Towards Secure and Privacy Preserving Collision Avoidance System in 5G Fog based Internet of Vehicles. *Future Generation Computer Systems 95* (2019), 488–499.

[160] **Osadchy, M., Pinkas, B., Jarrous, A., and Moskovich, B.** SCiFI - A System for Secure Face Identification. In *2010 IEEE Symposium on Security and Privacy* (2010), pp. 239–254. ISSN: 2375-1207.

[161] **P. Kotzanikolaou, C. Patsakis, E. Magkos, et al.** Lightweight private proximity testing for geospatial social networks. *Computer Communications 73* (2016), 263–270.

[162] **Paci, F., Squicciarini, A. C., and Zannone, N.** Survey on Access Control for Community-Centered Collaborative Systems. *ACM Comput. Surv. 51*, 1 (2018), 6:1–6:38.

[163] **Patra, A., Schneider, T., Suresh, A., and Yalame, H.** ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation. In *30th USENIX Security Symposium (USENIX Security 21)* (Aug. 2021), USENIX Association, pp. 2165–2182.

[164] **Patsakis, C., Kotzanikolaou, P., and Bouroche, M.** Private Proximity Testing on Steroids: An NTRU-based Protocol. In *Security and Trust Management* (Cham, 2015), S. Foresti, Ed., Springer International Publishing, pp. 172–184.

[165] **Pere Molina, M. Eulàlia Paré, Ismael Colomina et al.** Drones to the Rescue! Unmanned Aerial Search Missions Based on Thermal Imaging and Reliable Navigation. *InsideGNSS*, 7 (2012), 36–47.

[166] **Perroud, D.** Drones in construction and infrastructure, Online: https://tinyurl.com/mr4b6hmt. (Accessed: 2023-Mar-01).

[167] **Pietro Boccadoro and Domenico Striccoli and Luigi Alfredo Grieco**. An Extensive Survey on the Internet of Drones. Online: https://www.sciencedirect.com/science/article/pii/S1570870521001335. *Ad Hoc Networks 122* (2021), 102600.

[168] **Pillai, G., Suresh, A., Gupta, E., Ganapathy, V., and Patra, A.** Privadome: Delivery Drones and Citizen Privacy. Online: https://doi.org/10.56553/popets-2024-0039. *Proc. Priv. Enhancing Technol. 2024*, 2 (2024), 29–48.

[169] **Pinkas, B., Schneider, T., and Zohner, M.** Faster private set intersection based on {OT} extension. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)* (2014), pp. 797–812.

[170] **Pinto, S., and Santos, N.** Demystifying Arm TrustZone: A Comprehensive Survey. Online: https://doi.org/10.1145/3291047. *ACM Comput. Surv. 51*, 6 (Jan. 2019).

[171] **Pohl, A., Cosenza, B., and Juurlink, B.** Vectorization cost modeling for NEON, AVX and SVE. Online: https://www.sciencedirect.com/science/article/pii/S0166531620300262. *Performance Evaluation 140-141* (2020), 102106.

[172] **Quadri, I.** The Impact of Autonomous Drones on Privacy and Security - U.S. RESIST NEWS. https://www.usresistnews.org/2025/01/28/the-impact-of-autonomous-drones-on-privacy-and-security/. (Accessed 2025-Aug-22).

[173] **R. Morabito, R. Petrolo, V. Loscri, et al.** LEGIoT: a Lightweight Edge Gateway for the Internet of Things. *Fut. Gen. Comp. Systs. 81* (10 2017).

[174] **Rabin, M. O.** How To Exchange Secrets with Oblivious Transfer. Cryptology ePrint Archive, Paper 2005/187, Online: https://eprint.iacr.org/2005/187. https://eprint.iacr.org/2005/187.

[175] **Rao, A.** Rising to the Challenge – Data Security with Intel Confidential Computing, Online: https://community.intel.com/t5/Blogs/Products-and-Solutions/Security/Rising-to-the-Challenge-Data-Security-with-Intel-Confidential/post/1353141.

[176] **Raspberry PI**. Raspberry Pi 3 Model B+. https://www.raspberrypi.org/products/raspberry-pi-3-model-b-plus/. (Accessed: 2025-Mar-27).

[177] **Rathgeb, C., and Uhl, A.** A survey on biometric cryptosystems and cancelable biometrics. vol. 2011, p. 3.

[178] **Reda, M., Onsy, A., Haikal, A. Y., and Ghanbari, A.** Path Planning Algorithms in the Autonomous Driving System: A Comprehensive Review. *Robotics and Autonomous Systems 174* (2024), 104630.

[179] **Regenscheid, A.** Transition to post-quantum cryptography standards. Tech. rep., Gaithersburg, MD, 2024.

[180] **Reimsbach-Kounatze, C., Reynolds, T., and Girot, C.** Emerging privacy-enhancing technologies. Tech. rep., 2023.

[181] **Reuters**. U.S. push for Self-driving Cars faces Union, Lawyers opposition. https://www.reuters.com/business/autos-transportation/us-push-

self-driving-cars-faces-union-lawyers-opposition-2021-06-16/. (Accessed: 2025-Mar-27).

[182] **Ronin, F.** Being a Responsible User and Creator of Open Source, Online: https://blog.developer.adobe.com/being-a-responsible-user-and-creator-of-open-source-bbbcb79857fd.

[183] **Sadeghi, A.-R., Schneider, T., and Wehrenberg, I.** Efficient Privacy-Preserving Face Recognition. In *Information, Security and Cryptology – ICISC 2009* (2010), D. Lee and S. Hong, Eds., Lecture Notes in Computer Science, Springer, pp. 229–244.

[184] **Sadeghi, A.-R., and Stüble, C.** Property-Based Attestation for Computing Platforms: Caring about Properties, Not Mechanisms. In *Proceedings of the 2004 Workshop on New Security Paradigms* (New York, NY, USA, 2004), NSPW '04, Association for Computing Machinery, pp. 67–77.

[185] **Sailer, R., Jaeger, T., Zhang, X., and van Doorn, L.** Attestation-Based Policy Enforcement for Remote Access. In *Proceedings of the 11th ACM Conference on Computer and Communications Security - CCS '04* (Washington DC, USA, 2004), ACM Press, p. 308.

[186] **Sailer, R., Zhang, X., Jaeger, T., and van Doorn, L.** Design and Implementation of a TCG-Based Integrity Measurement Architecture. In *Proceedings of the 13th Conference on USENIX Security Symposium* (San Diego, CA, USA, 2004), vol. 13 of *SSYM'04*, USENIX Association, p. 16.

[187] **Sathya, V., Rochman, M. I., and Ghosh, M.** Hidden-nodes in coexisting LAA & Wi-Fi: a measurement study of real deployments. In *2021 IEEE International Conference on Communications Workshops (ICC Workshops)* (2021), IEEE, pp. 1–7.

[188] **Schwabe, P., and Westerbaan, B.** Kyber Post-Quantum KEM. Internet-Draft draft-cfrg-schwabe-kyber-04, Internet Engineering Task Force, Jan. 2024. Work in Progress.

[189] **Sciancalepore, S.** PARFAIT: Privacy-preserving, secure, and low-delay service access in fog-enabled IoT ecosystems. *Computer Networks 206* (2022), 108799.

[190] **Sciancalepore, S.** Privacy and Confidentiality Issues in Drone Operations: Challenges and Road Ahead. *IEEE Network* (2024), 1–1.

[191] **Sciancalepore, S., and George, D. R.** Privacy-Preserving Trajectory Matching on Autonomous Unmanned Aerial Vehicles. In *Annual Computer Security Applications Conference* (2022), pp. 1–12.

[192] **Sciancalepore, S., Piro, G., Caldarola, D., Boggia, G., and Bianchi, G.** On the Design of a Decentralized and Multiauthority Access Control Scheme in Federated and Cloud-Assisted Cyber-Physical Systems. *IEEE Internet of Things Journal 5*, 6 (2018), 5190–5204.

[193] **Shanthi, K., Sivalakshmi, P., Vidhya, S. S., and Lakshmi, K. S.** Smart Drone with Real Time Face Recognition. Online: https://www.sciencedirect.com/science/article/pii/S2214785321050732. *Materials Today: Proceedings 80* (2023), 3212–3215. SI:5 NANO 2021.

[194] **Sharp, J., Wu, C., and Zeng, Q.** Authentication for Drone Delivery through a Novel Way of using Face Biometrics. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking* (New York, NY, USA, 2022), MobiCom '22, Association for Computing Machinery, p. 609–622.

[195] **Sheikhalishahi, M., Stork, I., and Zannone, N.** Privacy-preserving policy evaluation in multi-party access control. *J. Comput. Secur. 29* (2021), 613–650.

[196] **Sheikhalishahi, M., Tillem, G., Erkin, Z., and Zannone, N.** Privacy-Preserving Multi-Party Access Control. In *WPES* (2019), ACM, p. 1–13.

[197] **Sinha, S.** State of IOT 2024: Number of connected IOT devices growing 13

[198] **Sommers, J., and Barford, P.** Cell vs. WiFi: on the performance of metro area mobile connections. In *Proceedings of the 2012 Internet Measurement Conference* (New York, NY, USA, 2012), IMC '12, Association for Computing Machinery, p. 301–314.

[199] **Squicciarini, A. C., Rajtmajer, S. M., and Zannone, N.** Multi-party access control: requirements, state of the art and open challenges. In *SACMAT* (2018), ACM, pp. 49–49.

[200] **Stadler, T., Oprisanu, B., and Troncoso, C.** Synthetic Data – Anonymisation Groundhog Day. In *31st USENIX Security Symposium (USENIX Security 22)* (Boston, MA, Aug. 2022), USENIX Association, pp. 1451–1468.

[201] **Stallings, W.** *High-speed networks and internets: performance and quality of service*. Pearson Education, 2002.

[202] **Starace, L. L. L.** Porto Taxi Trajectories — figshare.com. `https://figshare.com/articles/dataset/Porto_taxi_trajectories/12302165`. (Accessed 2025-Mar-27).

[203] **Sun, G., He, L., Sun, Z., Wu, Q., Liang, S., Li, J., Niyato, D., and Leung, V. C. M.** Joint Task Offloading and Resource Allocation in Aerial-Terrestrial UAV Networks With Edge and Fog Computing for Post-Disaster Rescue. *IEEE Transactions on Mobile Computing 23*, 9 (2024), 8582–8600.

[204] **Sun, J., Zhang, R., and Zhang, Y.** Privacy-Preserving Spatiotemporal Matching. In *Proc. IEEE INFOCOM* (2013), pp. 800–808.

[205] **Sun, J., Zhang, R., and Zhang, Y.** Privacy-Preserving Spatiotemporal Matching for Secure Device-to-Device Communications. *IEEE Internet of Things Journal 3*, 6 (2016), 1048–1060.

[206] **Sun, S., and Makri, E.** SoK: Multiparty Computation in the Preprocessing Model. Cryptology ePrint Archive, Paper 2025/060, Online: `https://eprint.iacr.org/2025/060`.

[207] **Svaigen, A. R., Boukerche, A., Ruiz, L. B., and Loureiro, A. A. F.** MixDrones: A Mix Zones-Based Location Privacy Protection Mechanism for the Internet of Drones. In *Proc.of the 24th Int. ACM Conf.on Modeling, Analysis and Simulation of Wireless and Mobile Systems* (2021), MSWiM '21, p. 181–188.

[208] **Taigman, Y., Yang, M., Ranzato, M., and Wolf, L.** DeepFace: Closing the Gap to Human-Level Performance in Face Verification. In *2014 IEEE Conference on Computer Vision and Pattern Recognition* (2014), IEEE, pp. 1701–1708.

[209] **Tedeschi, P., Al Nuaimi, F. A., Awad, A. I., and Natalizio, E.** Privacy-Aware Remote Identification for Unmanned Aerial Vehicles: Current Solutions, Potential Threats, and Future Directions. *IEEE Transactions on Industrial Informatics 20*, 2 (2023), 1069–1080.

[210] **Tedeschi, P., Ganti, S. G., and Sciancalepore, S.** Selective Authenticated Pilot Location Disclosure for Remote ID-enabled Drones. In *Proc.of Privacy Enhancing Technology Symposium* (2024).

[211] **Tedeschi, P., Sciancalepore, S., and Di Pietro, R.** ARID: Anonymous Remote IDentification of Unmanned Aerial Vehicles. In *Annual Computer Security Applications Conference* (2021), Association for Computing Machinery, p. 207–218.

[212] **Tedeschi, P., Sciancalepore, S., and Di Pietro, R.** PPCA - Privacy-Preserving Collision Avoidance for Autonomous Unmanned Aerial Vehicles. *IEEE Trans. on Depend. and Secure Comput.* (2022), 1–1.

[213] **Tedeschi, P., Sciancalepore, S., and Di Pietro, R.** PPCA - Privacy-Preserving Collision Avoidance for Autonomous Unmanned Aerial Vehicles. *IEEE Transactions on Dependable and Secure Computing* (2022), 1–1.

[214] **Tedeschi, P., Sciancalepore, S., and Di Pietro, R.** Lightweight Privacy-Preserving Proximity Discovery for Remotely-Controlled Drones. In *Proc.of the 39th Annual Computer Security Applications Conference* (2023), ACSAC '23, p. 178–189.

[215] **Tessaro, S., and Zhu, C.** Revisiting BBS Signatures. Cryptology ePrint Archive, Paper 2023/275, Online: `https://eprint.iacr.org/2023/275`.

[216] **The Economist**. Business is Booming as Regulators relax Drone Laws. `https://www.economist.com/science-and-technology/2021/06/17/business-is-booming-as-regulators-relax-drone-laws`. (Accessed: 2025-Mar-27).

[217] **The United States Department of Commerce**. The Minimum Elements For a Software Bill of Materials (SBOM), Online: `https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf`. (Pursuant to Executive Order 14028 on Improving the Nation's Cybersecurity).

[218] **The World Economic Forum**. How Autonomous Aircraft could make Flying Safer, easier to Access and more Sustainable. `https://www.weforum.org/agenda/2023/08/autonomous-aircraft-could-make-flying-safer-easier-to-access-and-more-sustainable/`. (Accessed: 2025-Mar-27).

[219] **Thornton, G., and Bagheri Zadeh, P.** An Investigation into Unmanned Aerial System (UAS) Forensics: Data Extraction & Analysis. Online:

https://www.sciencedirect.com/science/article/pii/S2666281722000609. *Forensic Science International: Digital Investigation 41* (2022), 301379.

[220] **Toulas, B.** New Intel chips won't play Blu-ray disks due to SGX deprecation, Online: https://www.bleepingcomputer.com/news/security/new-intel-chips-wont-play-blu-ray-disks-due-to-sgx-deprecation/.

[221] **Trask, A., Bluemke, E., Collins, T., Drexler, B. G. E., Cuervas-Mons, C. G., Gabriel, I., Dafoe, A., and Isaac, W.** Beyond Privacy Trade-offs with Structured Transparency, Online: https://arxiv.org/abs/2012.08347.

[222] **Trusted Computing Group**. *TPM 2.0 Mobile Reference Architecture Specification*, family 2.0, level 00, revision 142 ed. Trusted Computing Group, Dec. 2014.

[223] **Trusted Computing Group**. *TPM 2.0 Mobile Common Profile*, family 2.0, level 00, revision 31 ed. Trusted Computing Group, Dec. 2015.

[224] **Trusted Computing Group**. *TCG Trusted Attestation Protocol (TAP) Information Model for TPM Families 1.2 and 2.0 and DICE Family 1.0*, version 1.0 revision 0.36 ed. Trusted Computing Group, Sept. 2019.

[225] **Trusted Computing Group**. *TCG Trusted Attestation Protocol (TAP) Use Cases for TPM Families 1.2 and 2.0 and DICE*, version 1.0 revision 0.35 ed. Trusted Computing Group, Nov. 2019.

[226] **Trusted Computing Group**. TCG TSS 2.0 Overview and Common Structures Specification.

[227] **Trusted Computing Group**. *Trusted Platform Module Library – Part 1: Architecture*, family 2.0, level 00, revision 01.59 ed., Nov. 2019.

[228] **Vilajosana, X., Pister, K., and Watteyne, T.** Minimal IPv6 over the TSCH Mode of IEEE 802.15.4e (6TiSCH) Configuration. RFC 8180, Online: https://www.rfc-editor.org/info/rfc8180.

[229] **Vilajosana, X., Watteyne, T., Chang, T., Vučinić, M., Duquennoy, S., and Thubert, P.** Ietf 6tisch: A tutorial. *IEEE Communications Surveys & Tutorials 22*, 1 (2019), 595–615.

[230] **Voigt, P., and Bussche, A. v. d.** *The EU General Data Protection Regulation (GDPR): A Practical Guide*, 1st ed. Springer Publishing Company, Incorporated, 2017.

[231] **Wang, B., Lueks, W., Sukaitis, J., Narbel, V. G., and Troncoso, C.** Not Yet Another Digital ID: Privacy-Preserving Humanitarian Aid Distribution. In *2023 IEEE Symposium on Security and Privacy (SP)* (2023), pp. 645–663.

[232] **Wang, J., Liu, J., and Kato, N.** Networking and Communications in Autonomous Driving: A Survey. *IEEE Communications Surveys Tutorials 21*, 2 (2019), 1243–1274.

[233] **Wang, J., Zhang, L., Huang, Y., and Zhao, J.** Safety of Autonomous Vehicles. Online: https://onlinelibrary.wiley.com/doi/abs/10.1155/2020/8867757. *Journal of Advanced Transportation 2020*, 1 (2020), 8867757.

[234] **Wei-jiang, X., Wei-wei, J., Liu-sheng, H., and Yi-fei, Y.** Privacy-Preserving Collision Detection of Two Circles. In *Int. Conf. on Scalable Information Systems* (2007), InfoScale '07.

[235] **Wisse, E., Tedeschi, P., Sciancalepore, S., and Di Pietro, R.** A^2RID—Anonymous Direct Authentication and Remote Identification of Commercial Drones. *IEEE Internet of Things Journal 10*, 12 (2023), 10587–10604.

[236] **Woods, D.** 10 Killer Use Cases: What Drones-as-a-Service Can Do For Your Business, Online: https://tinyurl.com/5bhpm44x. (Accessed: 2023-03-01).

[237] **Wu, P., Ning, J., Shen, J., Wang, H., and Chang, E.** Hybrid Trust Multi-party Computation with Trusted Execution Environment. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022* (2022), The Internet Society.

[238] **Xiang, C., Tang, C., Cai, Y., and Xu, Q.** Privacy-Preserving Face Recognition with Outsourced Computation. *Soft Computing 20*, 9 (2016), 3735–3744.

[239] **Xu, Y., Zeng, Q., Wang, G., Zhang, C., Ren, J., and Zhang, Y.** A Privacy-Preserving Attribute-Based Access Control Scheme. In *SpaCCS* (2018).

[240] **Yang, K., Han, Q., Li, H., Zheng, K., Su, Z., and Shen, X.** An Efficient and Fine-Grained Big Data Access Control Scheme With Privacy-Preserving Policy. *IEEE Internet of Things Journal 4*, 2 (2017), 563–571.

[241] **Yao, A. C.-C.** How to generate and exchange Secrets. In *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)* (1986), pp. 162–167.

[242] **Yapp, J., Seker, R., and Babiceanu, R.** UAV as a service: Enabling on-demand access and on-the-fly re-tasking of multi-tenant UAVs using cloud services. In *DASC* (2016), IEEE, pp. 1–8.

[243] **Ye, A., Li, Y., Xu, L., Li, Q., and Lin, H.** A Trajectory Privacy-Preserving Algorithm Based on Road Networks in Continuous Location-Based Services. In *IEEE Trustcom/BigDataSE/ICESS* (2017), pp. 510–516.

[244] **Zachariae, A., Plahl, F., Tang, Y., Mamaev, I., Hein, B., and Wurll, C.** Human-Robot Interactions in Autonomous Hospital Transports. Online: https://www.sciencedirect.com/science/article/pii/S0921889024001398. *Robotics and Autonomous Systems 179* (2024), 104755.

[245] **Zanella, A., Bui, N., Castellani, A., Vangelista, L., and Zorzi, M.** Internet of Things for Smart Cities. *IEEE Internet of Things Journal 1*, 1 (2014), 22–32.

[246] **Zhang, K., Ni, J., Yang, K., Liang, X., Ren, J., and Shen, X. S.** Security and Privacy in Smart City Applications: Challenges and Solutions. *IEEE Communications Magazine 55*, 1 (2017), 122–129.

[247] **Zhang, L., Kan, H., and Wang, Y.** Privacy-Preserving AGV Collision-Resistance at the Edge Using Location-Based Encryption. *IEEE Transactions on Services Computing 16*, 4 (2023), 2868–2878.

[248] **Zhang, Y., Carballo, A., Yang, H., and Takeda, K.** Perception and Sensing for Autonomous Vehicles under Adverse Weather Conditions: A Survey. *ISPRS Journal of Photogrammetry and Remote Sensing 196* (2023), 146–177.

[249] **Zhang, Y., Chen, X., Li, J., Wong, D. S., and Li, H.** Anonymous Attribute-Based Encryption Supporting Efficient Decryption Test. In *ASIA CCS '13* (2013), ACM, p. 511–516.

[250] **Zhou, S., Chan, K., Li, C., and Loy, C. C.** Towards Robust Blind Face Restoration with Codebook Lookup Transformer. In *Advances in Neural Information Processing Systems* (2022), S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., pp. 30599–30611.

[251] **Zohar, M., and Kasatkin, D.** Integrity Measurement Architecture (IMA), Online: `https://sourceforge.net/p/linux-ima/wiki/Home/`.

[252] **Zohar, M., Safford, D., and Sailer, R.** Using IMA for Integrity Measurement and Attestation, Online: `https://blog.linuxplumbersconf.org/2009/slides/David-Stafford-IMA_LPC.pdf`.

Acknowledgements

My four-year PhD journey went by quickly, and I would not have achieved it without the valuable support of many people. I want to take this opportunity to thank everyone who supported and encouraged me throughout the process of finishing this thesis.

First and foremost, I extend my sincere gratitude to my daily supervisor, Savio Sciancalepore. Thank you for your guidance and for the opportunities you provided to help me improve as a researcher, teacher, and reviewer. I appreciated being able to knock on your office door at any time to ask questions or clarify doubts. Thank you for your patience and for helping me realize what I truly want to pursue in research.

A heartfelt thanks also goes to Nicola Zannone for our research discussions, our general conversations, and the laughter we shared during non-Dutch working hours. Thank you for your continuous support and for your valuable feedback on my thesis.

I am deeply grateful to Sandro Etalle for taking time from his busy schedule to listen to my concerns regarding both professional work and private matters. I also appreciate the enjoyable times we shared at your place, including dinner, music and dance.

I sincerely thank my Doctoral Committee -Prof. Dr. Ir. Nirvana Meratnia, Dr. Gabriele Oligeri, and Prof. Dr. Gennaro Boggia for their time in reviewing my thesis and providing insightful feedback.

During my PhD, I was fortunate to meet many wonderful people with whom I shared serious, funny, and light-hearted conversations. I would like to thank them all, in no particular order: Andy, Berry, Jerry, Luca, Emmanuele, Ema, Arne, Mauricio, Marc, Nithish, Bernardo, Alex, Anina, Tommaso, Jesús, Hossain, Mike, Rick, Jorrit, Michele G., Elena, Teresa, Aida, Silvia, Leon, Tom, and many more. I am also grateful to my office mates, Arpan and Toon, for all the enjoyable and enriching discussions, as well as for the crash course in quantum (security). Special thanks to my office neighbor, Boris, for our small talks and for always answering the question:"Where is Basheer?".

Mirjam and Jolande, you were both there from the start to the end of my PhD—a big thanks to you both. When I encountered administrative issues, neither of you backed down until the root problem was identified and resolved (I really appreciated your dedication). Jolande, thanks for listing and for all the laughter. Mirjam, thanks as well for all the funny moments (as well as helping me out with the move).

I also want to thank my friends who created lasting memories with me during this journey. I'll keep this brief. Michele C. (a.k.a Michelle), Cristoffer, Martin, and Pavlo: you were there from the start to the end of my PhD, some physically and some virtually. Thank you for listening to my rants, for the laughter, for the meaningful discussions, and for your support

at every step of my PhD. Gelah, Anna, and Paola: thank you for your kind and funny conversations, many of which made my day. Giuseppe, thank you for taking the initiative to go out for dinners, movies, parties, and board games. Those moments gave me a much-needed break from research. Oleksander (thanks for the crash course in photography!) and Matthias: thank you for the fun and productive gym sessions. Ben and Leonidas: thank you for the entertaining conversations. David, thank you for all the conversations and the joyful laughter we shared in both appropriate and (sometimes) inappropriate places.

Stash, thanks for being there (brother), especially when you drove me to Leuven (it truly felt like my parents taking me to my first day at school). I really appreciated our occasional dinner meetings at Fuso. Basheer, thank you for always having my back, brother. I truly enjoyed our long discussions on both philosophical and (un)ethical questions. Thank you also for introducing me to Hamdi and for the engaging conversations. And to Hamdi, thank you for all your advice. Abhishek, thank you for the chicken soup and for being there at the final stretch of my PhD, for proofreading, and for the countless telcos. I really value our meetings and the fruitful ideas we have shared, which we will continue to develop. Altan, there are no words to thank you enough. *Our genuine laughter*, coming straight from the diaphragm, brightened some of our days (which I will never forget). Thank you for pulling me out of my busy schedule to go for runs and walks to clear my mind, and for helping me out with some applied mathematical questions and clearing my doubts!

Another thanks goes to my Austrian friends, especially Michael and Thomas, for the conversations we had, which meant a lot to me.

Finally, I would like to express my gratitude to my family. First, I want to thank all our family friends and relatives for their prayers and support. Dennis, my brother, thank you for always being there for me. Joby, I appreciate your help with the cover.

My parents have been my strongest pillars of support. Nearly every evening, they called to ask how I was doing, and even at the brink of giving up, they were there to encourage me to finish what I started. I am forever grateful to have you as my parents!

My dear Vellipapa, thank you for being a role model. I truly wish you were here to witness this event and stand next to me with your smile.

Dominik Roy George
Eindhoven, October 2025

Curriculum Vitæ

Dominik Roy George was born in Vienna, Austria. He earned his BSc in Medical Informatics from TU Wien in 2018, and in 2021, he completed his MSc double-degree program, specializing in Software Engineering and Internet Computing (TU Wien), and in IT Security (TU Darmstadt). During his Master thesis, he worked as a research assistant at the Fraunhofer Institute for Secure Information Technology (SIT), Germany.

In 2021, he began his PhD at Eindhoven University of Technology under the supervision of dr. Savio Sciancalepore. His research focuses on integrating Privacy-Enhancing Technologies into the IoT Ecosystem, of which the research outcome is presented in this dissertation. During his PhD he supervised several students and research projects. Furthermore, he led and designed the lab activities for the Advanced Network Security course.

He served as reviewer, external reviewer, sub-reviewer, and artifact-reviewer for several conferences and journals and received the ACSAC 2024 Distinguished Artifact Reviewer Award.